

Università degli Studi di Napoli
Federico II

A.A. 2024-2025

Programmazione

Estratto

Questa dispensa raccoglie, riorganizza e spiega il materiale del corso di Programmazione tenuto dal professor *Francesco Isgrò* per il corso di laurea triennale in Informatica della Federico II di Napoli (slide dell'A.A. 2024-2025). Il linguaggio del corso è il **C**.

È pensata per chi **parte da zero**: non si dà per scontato di aver mai scritto una riga di codice, né di aver mai usato un terminale. Il primo capitolo accompagna passo passo nell'installazione di tutto ciò che serve (l'editor **Visual Studio Code**, il compilatore e le estensioni) su Windows, macOS e Linux; i successivi seguono l'ordine delle lezioni.

L'obiettivo è **capire**, non memorizzare: ogni regola è accompagnata dal suo *perché*, ogni programma è stato compilato ed eseguito e ne viene mostrato l'output reale, e ciò che accade *dentro* il computer (le variabili in memoria, i puntatori, le chiamate di funzione, i nodi di una lista) è disegnato in uno schema. Dove le slide contengono una svista, il testo la corregge e lo segnala.

Sono inclusi **soltanto i contenuti tecnici** presentati a lezione; le informazioni organizzative (orari, modalità d'esame, appelli) restano sulle slide e sui canali del docente, dove vengono aggiornate. Questo documento **non sostituisce** le lezioni, le slide e i libri di testo: ne è un complemento.

Nel testo ricorrono alcuni elementi grafici: riquadri **verdi** con approfondimenti, **gialli** con avvertenze ed errori tipici, **rossi** con errori gravi da evitare, riquadri **Definizione** e riquadri **In sintesi** che riassumono il ragionamento. Il codice è disegnato come una finestra di VS Code e, **accanto**, compare ciò che produce: il **terminale**, dove i comandi digitati sono in bianco e ciò che si scrive mentre il programma è in esecuzione è in **giallo**, oppure uno schema.

Redazione a cura di **Fabio Rosiello**

Indice

1 L'ambiente di lavoro 10

Che cosa ci serve 10
Linux, Windows o macOS? 11
Passo 1: installare Visual Studio Code 12
Scaricare il programma 12
Installazione su Windows 13
Installazione su macOS 13
Installazione su Linux 14
Passo 2: installare il compilatore 14
Windows: WSL e Ubuntu 14
Windows: creare l'utente di Ubuntu 15
Windows: installare il compilatore in Ubuntu 15
macOS: gli strumenti da riga di comando 16
Linux 17
Passo 3: le estensioni di VS Code 17
Windows: collegare VS Code a Ubuntu 18
Passo 4: la cartella del corso 18
Com'è fatta la finestra di VS Code 19
Creare un file 20
Il terminale 20
Il prompt 21
Cartelle e percorsi 21
I comandi essenziali 22
Trucchi che fanno risparmiare tempo 23
Il primo programma 24
Compilare ed eseguire 24
Perché `./`? 25
Gli errori di compilazione 25
E il pulsante "Run"? 26
Quando qualcosa non funziona 27
Riferimenti 27

2 Il calcolatore 28

Che cos'è la programmazione 28
Lo schema di un calcolatore 29
La CPU e il linguaggio macchina 29
Che cosa sa fare un processore 29
Il linguaggio macchina 30
L'assembly 30
Un esempio: contare i bit a 1 31
I limiti dell'assembly 31
I livelli di astrazione 32
Algoritmo e programma 33
La memoria 33
Celle e indirizzi 33
La notazione binaria 34
Quanti numeri in un byte 34
I numeri negativi 35
I tipi di dato 35
Gli stessi bit, significati diversi 35

Il tipo come astrazione 36
La gerarchia di memoria 36
Il software 37
Gli strati del software 37
L'avvio del computer 37
Che cosa fa il sistema operativo 38
I servizi per i programmi 38
Installare e scrivere un sistema operativo 39
Riferimenti 39

3 Dal problema al programma 40
Programmare 40
I linguaggi di alto livello 40
Interprete e compilatore 41
Una via di mezzo: il bytecode 41
Le librerie 42
Come nasce un eseguibile 42
Compilazione e linking 43
La catena completa 44
Librerie statiche e dinamiche 45
Gli algoritmi 46
Che cos'è un algoritmo 46
Quando le risorse non bastano 47
Come si descrive un algoritmo 47
Esempio 1: l'area di un cerchio 48
Esempio 2: da Fahrenheit a Celsius 49
Esempio 3: il segno di un numero 50
Riferimenti 51

4 La programmazione strutturata 52
Il linguaggio C 52
Da dove viene 52
Lo standard 53
La programmazione strutturata 53
Il problema: gli spaghetti 53
Il teorema di Böhm e Jacopini 54
Le tre strutture di controllo 54
La sequenza 55
La selezione a una via 55
La selezione a due vie 56
La selezione a n vie 57
L'iterazione 58
Il ciclo su contatore 59
Il ciclo su condizione 59
Condizione in coda 60
Condizione in testa 61
Combinare le strutture 62
Lo sviluppo top-down 62
Dividere il problema 62
Come si invoca un sotto-programma 63
I tipi di istruzione 64
Riferimenti 65

5 Primi programmi in C 66
Il primo programma, riga per riga 66
Scrivere, compilare, eseguire 66
Il preprocessore e `#include` 67
La funzione `main` 67
`printf`, le stringhe e il punto e virgola 67
`return` e il valore restituito al sistema 68
L'output con `printf` 68
Il cursore 68
Disegnare con i caratteri 69

Variabili, espressioni e assegnazione 69
Il problema della maratona 69
I commenti 69
Le dichiarazioni 70
L'assegnazione 70
Come si calcola un'espressione 70
La trappola della divisione intera 71
Parole chiave e nomi 71
Il preprocessore: `#define` e `#include` 71
Le costanti simboliche 71
I file di intestazione 71
Header e libreria: la prova 72
Un esempio con le costanti: Pacific Sea 73
Il programma 73
Gli `#include` a catena 74
Le costanti dell'header 74
`const` 74
I tipi reali e la notazione scientifica 74
`printf` e `scanf` 75
Funzioni di input e output formattato 75
Le conversioni di `printf` 75
`scanf` e l'operatore `&` 76
Un esempio: echo 77
Il controllo del flusso 78
Vero e falso in C 78
`if` 78
`if-else` 79
`while` 80
Assegnazioni abbreviate e incremento 81
`for` 81
Un esempio completo: `running_sum` 82
Il programma 82
Riga per riga 82
Tre modi di dare l'input 83
Una versione senza la lettura iniziale 84
Quando la compilazione fallisce 85
Le funzioni 86
Dichiarare prima di usare 86
L'esempio maxmin 87
Come funziona 88
Due `x` diverse 88
Che cosa succede senza prototipi 89
Un punto e virgola di troppo 89
Il passaggio dei parametri per valore 90
Riferimenti 91

6 Elementi lessicali e operatori 92

Dai caratteri ai token 93
I token 93
Gli spazi 94
Identificatori della libreria 94
I commenti 95
Le parole chiave 95
Gli identificatori 96
La regola 96
Maiuscole, lunghezza, scelta dei nomi 96
Gli operatori 97
Precedenza e associatività 97
La tabella degli operatori 98
Incremento e decremento 99
Prefisso e postfisso 99
Un esempio da simulare 99
Quando `++` diventa pericoloso 100
La libreria standard e i nomi riservati 101
Riferimenti 102

7 Programmi su più file e make 103

Perché più file 103
Un piccolo progetto 104
Compilare separatamente, collegare insieme 105
Il progetto delle lezioni 106
I file 106
Il problema 106
make 107
L'idea: dipendenze e date 107
Il makefile 107
I tipi di riga 108
Che cosa fa `make`, passo per passo 108
Le dipendenze dall'header 109
Il nome del target 110
Le macro 110
Un makefile completo 111
Riferimenti 112

8 Tipi di dato 113

Dichiarazioni, espressioni, assegnazioni 113
I tipi fondamentali 114
Forma completa e forma abbreviata 114
Tipi interi e tipi reali 114
I tipi derivati 115
Caratteri e il tipo `char` 115
Un carattere è un numero 115
La codifica ASCII 116
Caratteri come interi, interi come caratteri 117
Le sequenze di escape 118
Caratteri che non si possono scrivere 118
Virgolette, apici e campanello 118
Il cursore e i caratteri di controllo 119
Un'animazione con `\r` 120
Il buffer di `printf` 120
Attenzione a `++` dentro `printf` 121
Come è fatto un `char` 122
I bit di un carattere 122
Tre tipi per i caratteri 122
I numeri negativi 122
Lo stesso byte, due letture 123
Il `char` del mio computer ha il segno? 123
Il tipo `int` 123
Quanti valori 123
L'overflow 124
`short`, `long` e `unsigned` 125
Più piccoli, più grandi, senza segno 125
Il tipo delle costanti intere 126
I tipi reali 126
`float`, `double` e `long double` 126
La virgola mobile 127
Ai confini del `float` 128
Ai confini del `double` 129
I reali non sono esatti 130
Dare un nome ai tipi: `typedef` 130
L'operatore `sizeof` 131
Quanti byte 131
Una sorpresa 132
Leggere e scrivere un carattere alla volta 133
`getchar` e `putchar` 133
Il ciclo di lettura 134
`EOF` 134
Perché `int` e non `char` 135
Maiuscole e minuscole 136
Funzioni matematiche 137
La libreria matematica 137

Il valore assoluto 138
Conversioni e cast 138
La promozione a intero 138
Le conversioni aritmetiche 139
Le conversioni nelle assegnazioni 140
Il cast 141
Riferimenti 142

9 Il flusso di controllo 143

Vero e falso in C 143
Le condizioni sono numeri 143
Gli operatori relazionali 144
Confrontare due espressioni 144
Qualche esempio 144
`a < b < c` non è la matematica 145
I reali e la precisione 145
Gli operatori di uguaglianza 146
`==` e `!=` 146
L'errore più comune: `=` al posto di `==` 146
Gli operatori logici 147
La negazione 147
And e *or* 147
Precedenza 148
La valutazione in cortocircuito 149
`if` e `if-else` 150
La forma generale 150
Le graffe, anche per un'istruzione sola 151
`if-else` 151
L'`else` pendente 152
Il `while` 153
La forma generale 153
Attenti ai cicli infiniti 154
Il corpo vuoto 155
Contare i caratteri 156
La catena di `if-else` 157
Il `for` 158
`for` e `while` 158
Cicli annidati 159
Meglio una relazione che un'uguaglianza 159
Il `do-while` 160
La condizione in coda 160
Lo `switch` 161
La selezione a n vie 161
Come funziona 162
Più casi, un'azione 163
`break` e `continue` 164
Uscire o saltare un giro 164
Riferimenti 165

10 Le funzioni 166

Programmi fatti di funzioni 166
La definizione di una funzione 167
Intestazione e corpo 167
`void`: niente risultato, niente parametri 168
Altri esempi 169
Le definizioni nel C tradizionale 169
L'istruzione `return` 170
Terminare e restituire 170
Il valore si può ignorare 170
I prototipi 171
Dichiarare prima di usare 171
Che cosa succede senza prototipo 172
Come il compilatore scopre le funzioni 173
Un parametro senza tipo 173

La definizione ANSI 174
Il passaggio per valore 174
Una copia per ogni argomento 174
La pila delle chiamate 175
Visibilità: variabili locali e globali 176
Locali e globali 176
I blocchi 177
Le classi di memorizzazione 177
`auto` 177
`extern` 178
`register` 179
`static` per le variabili locali 180
`static` per le variabili globali 181
`static` per le funzioni 183
Riassunto 184
Riferimenti 184

11 Array e puntatori 185

Gli array 185
Tante variabili, un solo nome 185
La dichiarazione 186
L'inizializzazione 187
Gli indici e l'errore più comune 188
I puntatori 189
Indirizzi 189
Dichiarare e usare un puntatore 189
Che cosa contiene un puntatore 190
Espressioni con i puntatori 190
Che cosa non si può puntare 191
Il passaggio per indirizzo 192
Modificare le variabili del chiamante 192
Lo scambio di due variabili 193
Array e puntatori 194
Il nome di un array è un indirizzo 194
Quattro modi di sommare un array 195
Un array non si può spostare 195
Gli array come parametri 196
Si passa l'indirizzo 196
`sizeof` su un parametro 197
Ordinare un array 197
Il bubble sort 197
Il selection sort 198
L'allocazione dinamica 198
`malloc`, `calloc` e `free` 198
Un esempio 199
Il programma in esecuzione 200
Lo stack e lo heap 200
Riferimenti 201

12 Stringhe e array a più dimensioni 202

Le stringhe 202
Un array di caratteri con un terminatore 202
Le stringhe costanti sono puntatori 203
Array o puntatore? 203
La stringa vuota 203
Contare le parole 204
La parola chiave `const` 205
Variabili che non cambiano 205
Una promessa verificata dal compilatore 205
Funzioni per caratteri e stringhe 206
I caratteri: `ctype.h` 206
Le stringhe: `string.h` 207
Come sono fatte 208
Lo spazio per la copia 209

Gli array di puntatori 209
Un array di stringhe 209
Ordinare le parole di un testo 210
Come funziona 211
Il programma in esecuzione 211
Gli array a più dimensioni 212
Array di array 212
La storage mapping function 213
L'inizializzazione 213
Le matrici come parametri 214
`int **` non è una matrice 215
Passare l'indirizzo base 216
Le matrici dinamiche 216
Un array di righe 216
Memoria contigua 217
Gli array a lunghezza variabile 218
Le dimensioni decise durante l'esecuzione 218
Pro e contro 219
Riferimenti 219

L'AMBIENTE DI LAVORO

Prima di scrivere il primo programma bisogna preparare il proprio *laboratorio*: qualche programma da installare e un paio di abitudini da prendere. Questo capitolo accompagna passo passo dall'installazione alla prima esecuzione, su **Windows**, **macOS** e **Linux**. Ogni passo è spiegato: non serve sapere nulla di computer oltre ad accenderli e a scaricare un file da Internet.

Leggi solo la parte del tuo sistema operativo

Le istruzioni di installazione sono divise per sistema operativo. Segui quelle del tuo computer e salta le altre: dalla sezione *Il terminale* in poi le istruzioni tornano a essere le stesse per tutti.

CHE COSA CI SERVE

Un computer non capisce il C. Capisce soltanto il proprio **linguaggio macchina**: lunghe sequenze di numeri che il processore sa eseguire, una dopo l'altra. Noi invece scriveremo i programmi in C, un linguaggio fatto di parole inglesi e simboli, comprensibile a una persona. Fra noi e il processore servono quindi tre strumenti, che compaiono nella figura 1.1.

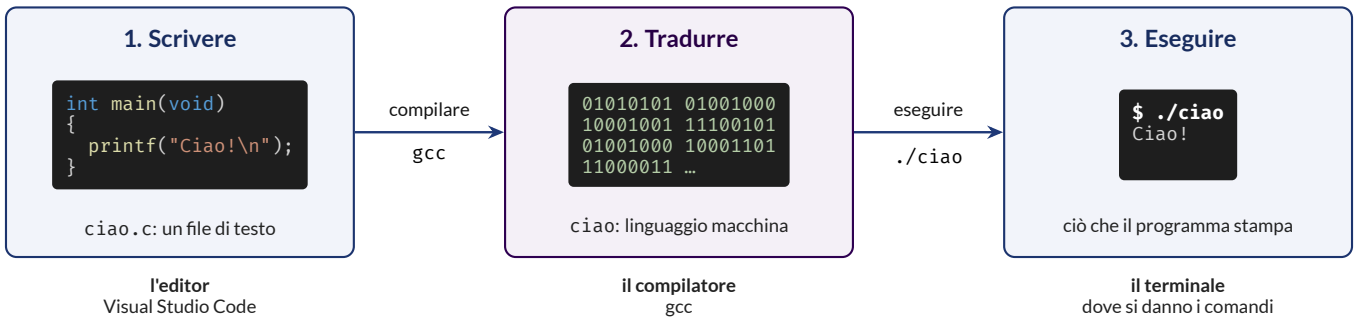


Figura 1.1: *Dal testo scritto da noi al programma che gira: i tre strumenti da installare.*

- L'**editor** è il quaderno su cui scriviamo il programma. Il programma, per ora, è un normale **file di testo** con estensione `.c` (per esempio `ciao.c`). Useremo **Visual Studio Code** (o semplicemente *VS Code*), un editor gratuito che colora il codice, segnala gli errori mentre si scrive e contiene al suo interno anche un terminale.
- Il **compilatore** è il traduttore: legge il file di testo e produce un nuovo file, l'**eseguibile**, che contiene lo stesso programma tradotto in linguaggio macchina. Useremo **gcc**, il compilatore C del progetto GNU, lo stesso usato a lezione.
- Il **terminale** è una finestra in cui si *scrivono* i comandi al computer invece di cliccare su icone. Da lì chiederemo a gcc di compilare e poi lanceremo il programma, che stamperà i suoi risultati nella stessa finestra.

Il capitolo 3 spiegherà in dettaglio che cosa succede durante la traduzione. Per ora basta l'idea: **si scrive, si compila, si esegue**.

Linux, Windows o macOS?

Il corso **assume Linux** come sistema operativo: i comandi visti a lezione (e all'esame) sono quelli di Linux. Non serve però cambiare computer o cancellare Windows: in ogni sistema c'è un modo semplice per avere gli stessi strumenti.

Sistema	Che cosa si installa	Perché
Windows	**WSL** (*Windows Subsystem for Linux*) con **Ubuntu**, e dentro Ubuntu gcc	WSL fa girare un vero Linux dentro Windows, senza riavvii né partizioni
macOS	gli **strumenti da riga di comando** di Apple (*Command Line Tools*)	macOS è un parente di Linux: ha già un terminale molto simile, manca solo il compilatore
Linux	il pacchetto `build-essential` (Ubuntu, Debian, Mint)	è il sistema del corso: basta un comando

VS Code si installa allo stesso modo ovunque (passo 1). Su Windows c'è una particolarità che conviene capire subito, perché spiega diversi passaggi successivi: i file del corso e il compilatore stanno *dentro* Ubuntu, mentre la finestra di VS Code è un normale programma di Windows. VS Code allora si divide in due parti che collaborano (figura 1.2): l'interfaccia, su Windows, e un piccolo *server*, dentro Ubuntu, che legge e salva i file e lancia il compilatore.

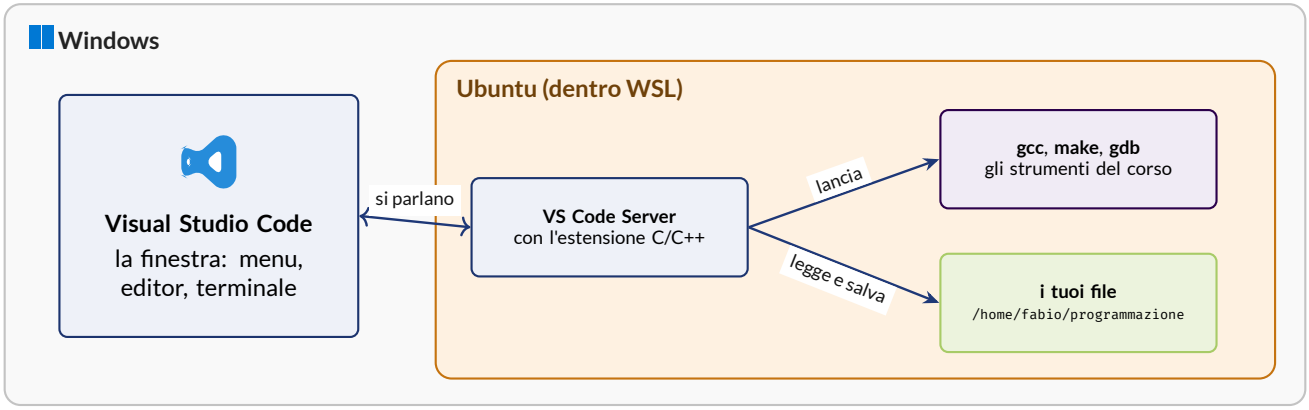


Figura 1.2: *Su Windows, con WSL: l'interfaccia di VS Code resta su Windows, mentre file e compilatore vivono in Ubuntu. Su macOS e su Linux tutto sta nello stesso sistema e questa divisione non c'è.*

Perché non un compilatore "nativo" per Windows?

Esistono versioni di gcc per Windows (MinGW, per esempio) e funzionano. Ma con esse alcuni dettagli cambiano: l'eseguibile si chiama `a.exe` invece di `a.out`, la fine dell'input si segnala con Ctrl + Z invece di Ctrl + D, i comandi del terminale sono diversi. Con WSL si usano **esattamente** i comandi delle lezioni, ed è ciò che conta.

Passo 1: installare Visual Studio Code

Scaricare il programma

Visual Studio Code si scarica gratuitamente dal sito ufficiale, `https://code.visualstudio.com`. La pagina *Download* propone un pulsante per ogni sistema operativo (figura 1.3): si sceglie il proprio.

Visual Studio Code, non Visual Studio

Microsoft produce anche **Visual Studio** (senza "Code"): è un altro programma, molto più grande, pensato per altri linguaggi. Il nome giusto è **Visual Studio Code**, con l'icona blu della figura 1.3.

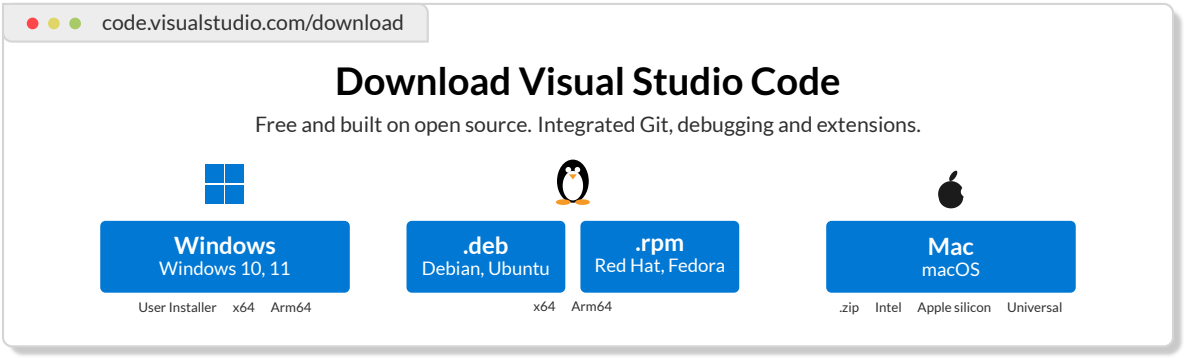


Figura 1.3: *La pagina di download di VS Code. L'aspetto del sito cambia spesso; i tre gruppi di pulsanti, invece, restano.*

Installazione su Windows

Il pulsante *Windows* scarica un file chiamato più o meno `VSCodeUserSetup-x64-1.xx.x.exe` (i numeri sono la versione). È un normale programma di installazione:

1. Apri il file scaricato (doppio clic, oppure clic sulla notifica di download del browser).
2. Nella prima schermata scegli *I accept the agreement* (accetti la licenza) e fai clic su *Next*.
3. Nelle due schermate successive (cartella di installazione e menu Start) lascia tutto com'è e fai clic su *Next*.
4. Nella schermata *Select Additional Tasks* (figura 1.4) spunta le due voci *Add "Open with Code" action…*: aggiungono la voce "Apri con Code" al menu del tasto destro sui file e sulle cartelle, comodissima. La voce **Add to PATH** deve restare spuntata: permette di aprire VS Code scrivendo `code` nel terminale, e la useremo.
5. Fai clic su *Next*, poi su *Install*. Al termine, lascia spuntato *Launch Visual Studio Code* e fai clic su *Finish*.

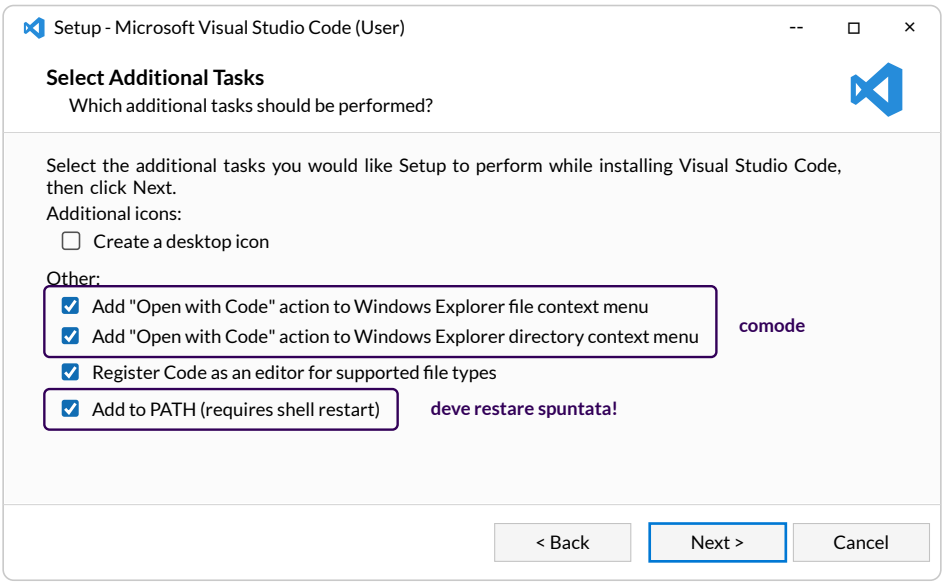


Figura 1.4: *La schermata più importante dell'installazione su Windows.*

Installazione su macOS

Il pulsante *Mac* scarica un archivio `.zip`. Safari di solito lo estrae da solo: nella cartella *Download* compare l'applicazione *Visual Studio Code* (altrimenti basta un doppio clic sul file `.zip`).

1. Trascina *Visual Studio Code* dalla cartella *Download* alla cartella *Applicazioni* (figura 1.5): così è installata davvero e la si trova nel Launchpad.
2. Aprila. Al primo avvio macOS avvisa che è un'applicazione scaricata da Internet: conferma con *Apri*.
3. Per poter aprire VS Code dal terminale con il comando `code`: premi Cmd + Shift + P, scrivi *shell command* e scegli *Shell Command: Install 'code' command in PATH*.

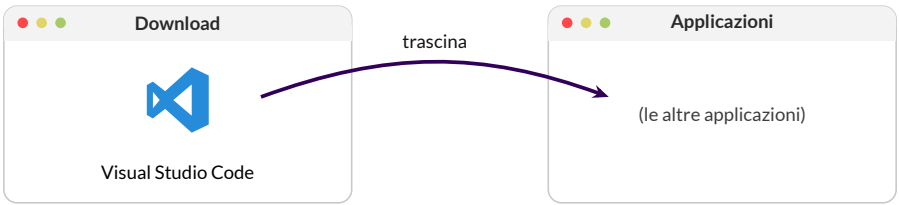


Figura 1.5: *Su macOS si installa un'applicazione trascinandola nella cartella Applicazioni.*

Installazione su Linux

Su Ubuntu, Debian e derivate si scarica il pacchetto `.deb` e lo si installa dal terminale (il terminale si apre con Ctrl + Alt + T; il suo uso è spiegato più avanti in questo capitolo):

Terminale

```
$ cd ~/Scaricati
$ sudo apt install ./code_*.deb
```

Il comando `cd` entra nella cartella dei download (su un sistema in inglese si chiama `Downloads`); `sudo apt install` installa il pacchetto, dopo aver chiesto la password dell'utente. In alternativa, VS Code si trova anche nel negozio di applicazioni della distribuzione (su Ubuntu, *App Center*).

Passo 2: installare il compilatore



Windows: WSL e Ubuntu

Su Windows 10 (aggiornato) e Windows 11 WSL si installa con un unico comando, da scrivere in **PowerShell** aperta **come amministratore**, cioè con il permesso di modificare il sistema.

1. Premi il tasto Windows, scrivi *powershell* e, nel riquadro a destra, scegli *Esegui come amministratore* (figura 1.6). Windows chiede conferma: rispondi *Sì*.
2. Nella finestra blu (o nera) che si apre, scrivi il comando `wsl --install` e premi Invio.
3. Attendi il download (qualche minuto: Ubuntu pesa circa un gigabyte) e, quando richiesto, **riavvia il computer**.

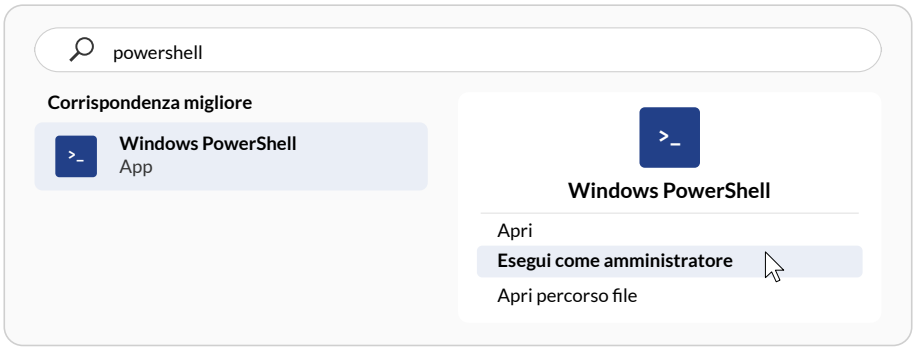


Figura 1.6: *La ricerca nel menu Start: PowerShell va aperta come amministratore.*

Amministratore: Windows PowerShell

```
PS C:\WINDOWS\system32> wsl --install
Installazione: Piattaforma macchina virtuale
Piattaforma macchina virtuale è stato installato.
Installazione: Sottosistema Windows per Linux
Sottosistema Windows per Linux è stato installato.
Installazione: Ubuntu
Ubuntu è stato installato.
L'operazione richiesta è riuscita. Le modifiche saranno effettive al riavvio del sistema.
```

Windows: creare l'utente di Ubuntu

I messaggi possono essere un po' diversi a seconda della versione di Windows: quello che conta è che non compaiano errori. Dopo il riavvio si apre da sola una finestra di Ubuntu (se non si apre, cerca *Ubuntu* nel menu Start) che completa l'installazione e chiede di creare un **utente Linux**: un nome (in minuscolo, senza spazi; non deve essere per forza uguale a quello di Windows) e una password.

Ubuntu

```
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows username.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username: fabio
New password:
Retype new password:
passwd: password updated successfully
Installation successful!
fabio@PC-FABIO:~$
```

Mentre scrivi la password non compare nulla

Nel terminale, mentre si digita una password, **non compaiono nemmeno gli asterischi**: sembra che la tastiera non funzioni, invece i caratteri vengono letti. Scrivi la password e premi Invio. Annotala: servirà ogni volta che installerai qualcosa.

Windows: installare il compilatore in Ubuntu

L'ultima riga stampata da Ubuntu, `fabio@PC-FABIO:~$`, è il **prompt**: Ubuntu è pronta a ricevere comandi (la spiegheremo pezzo per pezzo più avanti). Restano da installare il compilatore e gli altri strumenti del corso. Scrivi questi due comandi, uno alla volta, premendo Invio dopo ciascuno:

Ubuntu

```
$ sudo apt update
[sudo] password for fabio:
...
$ sudo apt install build-essential gdb
...
Do you want to continue? [Y/n] y
...
```

- `sudo` (*superuser do*) esegue il comando che segue con i permessi di amministratore, e per questo chiede la password appena creata.
- `apt` è il **gestore di pacchetti** di Ubuntu: una specie di negozio di applicazioni che si usa dal terminale. Con `update` scarica l'elenco aggiornato dei programmi, con `install` installa quelli indicati (e chiede conferma: si risponde `y`, *yes*).
- `build-essential` è un pacchetto che ne raccoglie altri: il compilatore **gcc**, il programma **make** (che useremo dal capitolo 7) e le *librerie* standard del C. **gdb** è il *debugger*, utile per eseguire un programma un'istruzione alla volta.

Per verificare che tutto sia andato a buon fine, si chiede a gcc la sua versione:

Ubuntu

```
$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

I numeri di versione possono essere diversi: va bene qualunque versione. Se invece compare `gcc: command not found`, l'installazione non è riuscita: ripeti i due comandi `apt` e controlla che non stampino errori.

Se `wsl --install` non funziona

L'errore più comune all'avvio di Ubuntu è un messaggio che parla di *virtualizzazione* (per esempio il codice `0x80370102`): WSL usa una funzione del processore che su alcuni computer è disattivata. Si attiva dal BIOS/UEFI del computer (la voce si chiama *Virtualization Technology*, *Intel VT-x* o *AMD-V*); la procedura cambia da modello a modello, quindi conviene cercare "attivare virtualizzazione" seguito dal modello del proprio PC. Le istruzioni ufficiali di Microsoft sono indicate nei riferimenti a fine capitolo.

macOS: gli strumenti da riga di comando

Su macOS il compilatore fa parte degli *strumenti da riga di comando* di Apple, gratuiti. Non serve installare Xcode (l'ambiente di sviluppo completo, molto più grande): bastano gli strumenti.

1. Apri il **Terminale**: premi Cmd + Spazio (si apre Spotlight), scrivi *terminale* e premi Invio.
2. Scrivi il comando `xcode-select --install` e premi Invio.
3. Compare una finestra come quella della figura 1.7: fai clic su *Installa* (non su *Ottieni Xcode*) e accetta la licenza. Il download richiede qualche minuto.

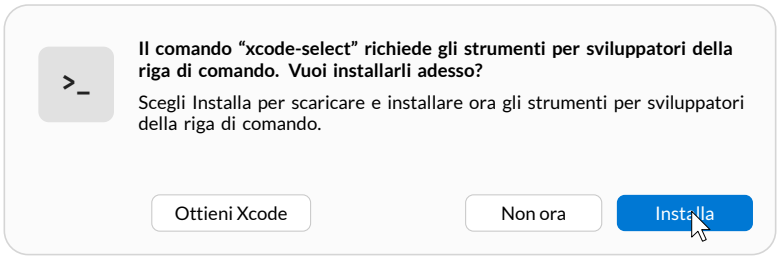


Figura 1.7: *La richiesta di installazione degli strumenti da riga di comando (il testo esatto dipende dalla versione di macOS).*

Al termine, la verifica:

Terminale di macOS

```
$ gcc --version
Apple clang version 16.0.0 (clang-1600.0.26.6)
Target: arm64-apple-darwin24.1.0
Thread model: posix
InstalledDir: /Library/Developer/CommandLineTools/usr/bin
```

Sorpresa: il comando `gcc` risponde *Apple clang*. Su macOS il compilatore è **clang**, un altro compilatore C, che per comodità risponde anche al nome `gcc` e accetta le stesse opzioni. Per tutto ciò che si fa nel corso il comportamento è lo stesso.

Il prompt di macOS

Nel Terminale di macOS il prompt finisce con il simbolo `%` invece che con `$` (per esempio `fabio@MacBook-Air ~ %`). In questa dispensa i comandi sono sempre preceduti da `$`: su Mac si scrivono allo stesso modo.

Linux

Su Ubuntu, Debian e Linux Mint bastano gli stessi due comandi visti per Windows (in Ubuntu, il terminale si apre con Ctrl + Alt + T):

Terminale

```
$ sudo apt update
$ sudo apt install build-essential gdb
```

Su Fedora il comando equivalente è `sudo dnf install gcc make gdb`. La verifica, anche qui, è `gcc --version`.

Passo 3: le estensioni di VS Code

VS Code, appena installato, sa già colorare il codice C. Le **estensioni** sono componenti aggiuntivi gratuiti che gli insegnano a fare di più. Si installano dalla vista *Extensions*: si apre con l'icona dei quattro quadratini nella barra a sinistra (oppure con Ctrl + Shift + X, su Mac Cmd + Shift + X); si scrive il nome nella casella di ricerca e si fa clic su Install (figura 1.8).

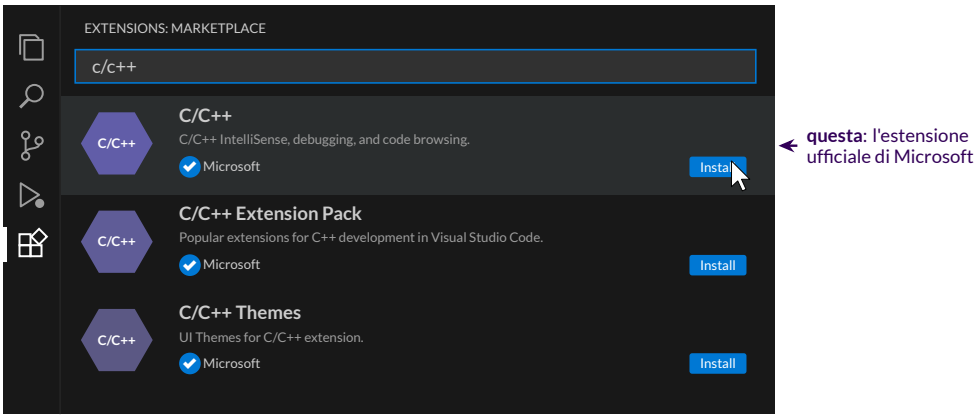


Figura 1.8: *La vista Extensions con la ricerca "c/c++". Basta la prima: C/C++ di Microsoft (si riconosce dal segno blu di "editore verificato").*

Le estensioni da installare sono poche:

Estensione	Per chi	A che cosa serve
C/C++ (Microsoft)	tutti	*IntelliSense*: suggerimenti mentre si scrive (nomi di funzione, parametri), errori sottolineati in rosso ancora prima di compilare, navigazione nel codice (clic destro, *Go to Definition*), debugger.
WSL (Microsoft)	solo Windows	collega VS Code a Ubuntu, come nella figura 1.2.

Spesso non serve nemmeno cercarle: aprendo per la prima volta un file `.c`, VS Code propone da solo, con una notifica in basso a destra, di installare l'estensione consigliata per il linguaggio C.

E la lingua italiana?

VS Code può essere tradotto in italiano con l'estensione *Italian Language Pack*. In questa dispensa però i menu sono citati **in inglese**, come compaiono dopo l'installazione: se scegli l'italiano dovrai tradurre i nomi delle voci (*File* resta *File*, *Terminal* diventa *Terminale*, e così via).

Windows: collegare VS Code a Ubuntu

Su Windows l'ordine conta: prima si installa l'estensione **WSL**, poi si apre VS Code *dentro* Ubuntu, e solo allora si installa **C/C++**. Il modo più semplice per aprire VS Code dentro Ubuntu è farlo dal terminale di Ubuntu, con il comando `code` seguito da un punto (il punto significa "la cartella in cui mi trovo"):

Ubuntu

```
$ code .
Installing VS Code Server for Linux x64 (...)
Downloading: 100%
Unpacking: 100%
```

La prima volta VS Code installa il suo *server* dentro Ubuntu (lo fa da solo, in pochi secondi), poi si apre la finestra. Come si capisce che è collegata a Ubuntu? Dall'**indicatore remoto**, il riquadro blu nell'angolo in basso a sinistra (figura 1.9).



Figura 1.9: *La barra di stato di VS Code collegato a Ubuntu. Se il riquadro in basso a sinistra non dice WSL: Ubuntu, VS Code sta lavorando su Windows e non troverà il compilatore.*

Ora, nella vista *Extensions*, l'estensione C/C++ mostra il pulsante Install in WSL: Ubuntu: è quello giusto, perché l'estensione deve lavorare dove stanno i file e il compilatore. Le volte successive, VS Code ricorda le cartelle aperte: dal menu **File › Open Recent** si riapre direttamente una cartella di Ubuntu, già collegata.

Dove sono i file di Ubuntu?

La cartella personale di Ubuntu (`/home/fabio`) è visibile anche da Esplora file di Windows, alla voce *Linux* nel riquadro di sinistra. Viceversa, da Ubuntu il disco `C:` di Windows si trova in `/mnt/c`. Conviene però tenere i file del corso **dentro Ubuntu**, nella cartella personale: lì compilare è più veloce e non ci sono problemi di permessi.

Passo 4: la cartella del corso

Conviene tenere tutti i programmi del corso in una cartella dedicata. La creiamo dal terminale (su Windows, quello di **Ubuntu**) e la apriamo in VS Code:

Terminale

```
$ mkdir programmazione
$ cd programmazione
$ code .
```

`mkdir` (*make directory*) crea la cartella, `cd` (*change directory*) ci entra, `code .` apre VS Code sulla cartella corrente. In alternativa si può fare tutto con il mouse: si crea la cartella come al solito e la si apre in VS Code da **File › Open Folder...**.

La prima volta che si apre una cartella, VS Code chiede se ci si fida degli autori dei file che contiene (*Do you trust the authors of the files in this folder?*): sono i nostri file, quindi si risponde Yes, I trust the authors. Senza questa conferma VS Code apre la cartella in *modalità con restrizioni* e disattiva le estensioni.

Com'è fatta la finestra di VS Code

La figura 1.10 mostra VS Code con la cartella del corso aperta, un file C nell'editor e il terminale in basso. Vale la pena guardarla con calma: sono le zone che useremo sempre.

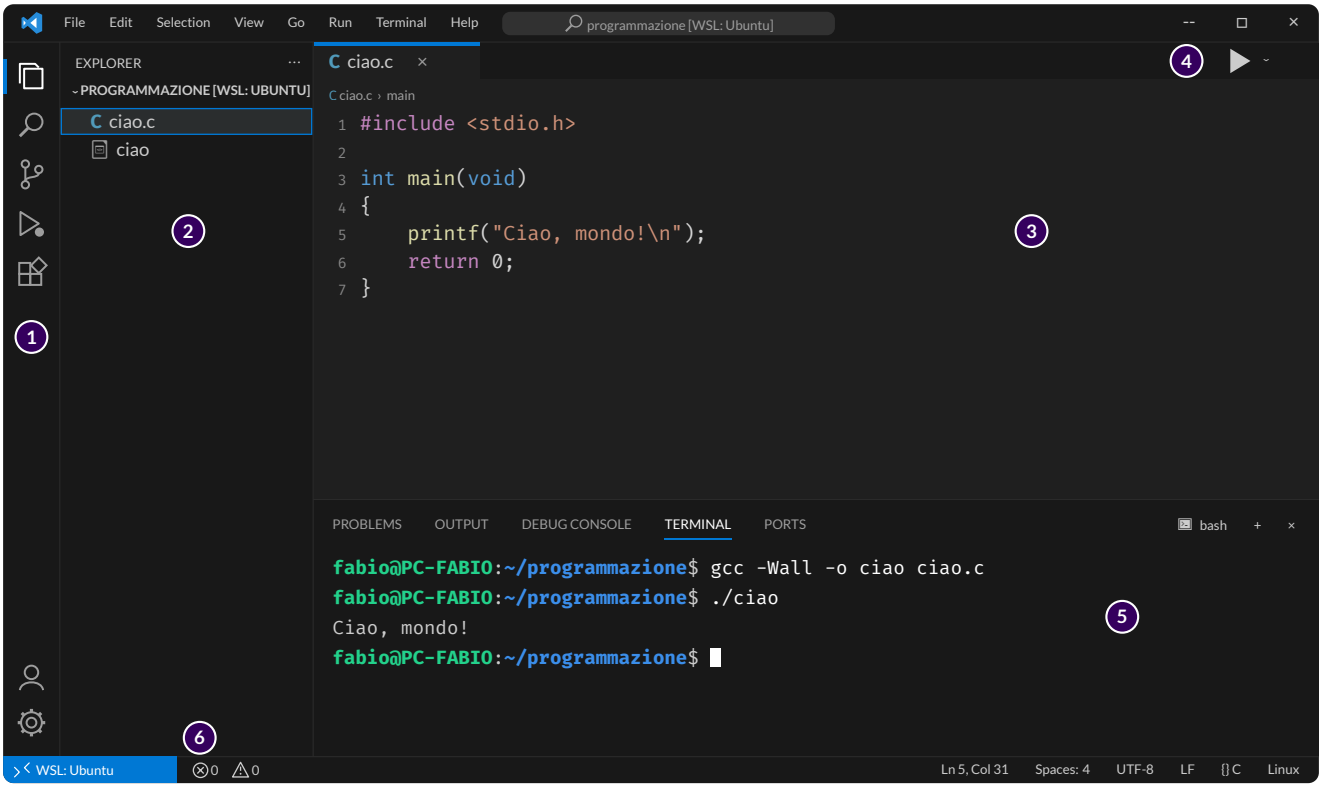


Figura 1.10: *Visual Studio Code con la cartella* `programmazione` *aperta (su Windows, collegato a Ubuntu).*

1. La **barra delle attività**: cambia ciò che compare nella barra laterale. Le icone, dall'alto: *Explorer* (i file), *Search* (cerca nei file), *Source Control* (Git), *Run and Debug* (il debugger), *Extensions*.

2. La **barra laterale**, qui con l'*Explorer*: i file della cartella aperta. Un clic apre un file nell'editor; il pulsante con il foglio e il segno "+", che compare passando con il mouse sul nome della cartella, crea un nuovo file.

3. L'**editor**: il file aperto, con i numeri di riga a sinistra. In alto ci sono le schede dei file aperti e il "percorso" (*ciao.c* › *main*: il cursore è dentro la funzione `main`).

4. Il pulsante *Run*, aggiunto dall'estensione C/C++ (ne parleremo alla fine del capitolo).

5. Il **pannello** in basso, qui con il *terminale* integrato: un vero terminale (su Windows, di Ubuntu) già posizionato nella cartella aperta. Si apre dal menu **Terminal › New Terminal**; accanto alla voce del menu è indicata anche la scorciatoia da tastiera, che dipende dalla tastiera in uso.

6. La **barra di stato**: a sinistra l'indicatore remoto e il numero di errori e avvisi; a destra la posizione del cursore (*Ln* riga, *Col* colonna) e il linguaggio riconosciuto (*C*).

Creare un file

Un nuovo file si crea dall'*Explorer* (pulsante "nuovo file" accanto al nome della cartella) oppure da **File** › **New File...**. Il nome deve finire con `.c`: è l'**estensione** del nome a dire a VS Code (e a gcc) che il file contiene codice C. Con un nome come `ciao.txt`, o senza estensione, niente colori, niente suggerimenti e gcc si rifiuterebbe di compilarlo.

Nomi di file semplici

Usa nomi brevi, in minuscolo, **senza spazi** e senza lettere accentate: `esercizio1.c`, `media_voti.c`. Gli spazi nei nomi costringono a usare le virgolette nei comandi del terminale e sono una fonte continua di errori.

IL TERMINALE

Il terminale è un dialogo scritto con il computer: noi scriviamo un **comando** e premiamo Invio, il computer lo esegue e risponde con del testo. Il programma che legge i comandi si chiama **shell**; in Ubuntu è *bash*, su macOS *zsh*. Per il corso servono pochi comandi, ma vanno capiti bene: sono il modo in cui compileremo ed eseguiremo ogni programma.

Il prompt

Quando la shell è pronta mostra il **prompt**, una riga che finisce con `$` (su Mac `%`) e dopo la quale si scrive. Il prompt di Ubuntu non è decorativo: dice *chi* siamo, *dove* siamo e che la shell *aspetta* un comando (figura 1.11).



Figura 1.11: *Le parti del prompt di Ubuntu.*

La parte più importante è la **cartella corrente** (o *directory di lavoro*): ogni comando lavora *lì*, a meno di indicare diversamente. Se scriviamo `gcc ciao.c` e il file `ciao.c` non sta nella cartella corrente, gcc non lo trova. Moltissimi problemi dei principianti si risolvono guardando il prompt.

Il prompt, nella dispensa

D'ora in poi i comandi sono mostrati con il solo `$` come prompt e in **grassetto bianco**; le righe normali sono ciò che il computer risponde, e il testo in **giallo** è ciò che si scrive mentre un programma è in esecuzione. Il `$` **non va digitato**: si scrive solo ciò che lo segue.

Cartelle e percorsi

I file sono organizzati in cartelle, che contengono altre cartelle, che contengono altri file: un **albero**, che in Linux parte da un'unica radice chiamata `/` (figura 1.12).

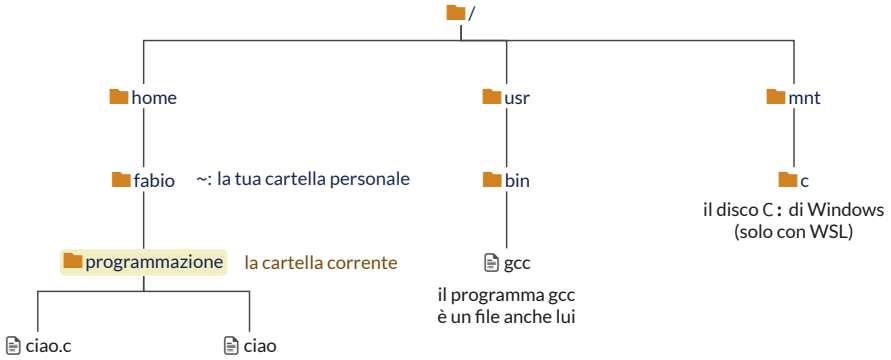


Figura 1.12: *Una parte dell'albero delle cartelle di Ubuntu.*

Un file si indica con il suo **percorso**, cioè la strada per raggiungerlo, con i nomi delle cartelle separati da `/`:

- un percorso **assoluto** parte dalla radice e vale ovunque ci si trovi: `/home/fabio/programmazione/ciao.c`;
- un percorso **relativo** parte dalla cartella corrente: se si è in `/home/fabio`, lo stesso file è `programmazione/ciao.c`; se si è già in `programmazione`, è semplicemente `ciao.c`.

Tre nomi speciali completano il quadro: `.` è la cartella corrente, `..` è la cartella che la contiene ("sali di un livello") e `~` è la cartella personale. Dalla cartella `programmazione`, quindi, `..` è `/home/fabio`.

Comando	Che cosa fa	Da dove viene il nome
`pwd`	stampa il percorso della cartella corrente	*print working directory*
`ls`	elenca il contenuto della cartella corrente	*list*
`ls -l`	come sopra, con dimensione e data dei file	*long*
`cd nome`	entra nella cartella `nome`	*change directory*
`cd ..`	sale nella cartella superiore	
`cd`	torna nella cartella personale	
`mkdir nome`	crea la cartella `nome`	*make directory*
`cp da a`	copia il file `da` nel file `a`	*copy*
`mv da a`	sposta o rinomina un file	*move*
`rm file`	cancella un file (**senza cestino!**)	*remove*
`cat file`	mostra il contenuto di un file di testo	*concatenate*
`clear`	pulisce lo schermo del terminale	

Ecco una breve sessione di lavoro; la spiegazione segue subito dopo.

Terminale

```
$ pwd
/home/fabio
$ ls
programmazione
$ cd programmazione
$ ls
ciao.c
$ mkdir lezione2
$ ls
ciao.c  lezione2
$ cp ciao.c lezione2/prova.c
$ cd lezione2
$ ls
prova.c
$ cd ..
$ pwd
/home/fabio/programmazione
```

Si parte dalla cartella personale (`pwd` lo conferma), si entra in `programmazione`, si crea la sottocartella `lezione2` e ci si copia il file `ciao.c` con il nuovo nome `prova.c` (si noti il percorso relativo `lezione2/prova.c`). Poi si entra in `lezione2`, si controlla che la copia ci sia, e con `cd ..` si torna su di un livello. Molti comandi, quando riescono, **non stampano niente**: è la convenzione di Linux, "nessuna notizia, buona notizia".

`rm` non perdona

Il comando `rm` cancella definitivamente: il file non passa per il cestino e non si recupera. Controlla sempre due volte il nome prima di premere Invio.

Trucchi che fanno risparmiare tempo

- Tab **completa** i nomi: scrivendo `cd prog` e premendo Tab, la shell completa da sola in `cd programmazione/`. Oltre a risparmiare tempo evita gli errori di battitura: se il completamento non funziona, il file non si trova dove pensiamo.
- ↑ e ↓ scorrono i **comandi già dati**: per ricompilare dopo una modifica basta premere ↑ fino a ritrovare il comando `gcc` e poi Invio.
- Ctrl + C **interrompe** il programma in esecuzione. Se un programma non termina mai (capiterà, con i cicli infiniti del capitolo 9), è il modo per fermarlo.
- Ctrl + D, scritto come input a un programma, segnala la **fine dell'input**; lo useremo dal capitolo 5.
- Nel terminale Ctrl + C non copia: per copiare e incollare, nel terminale di VS Code si usano il tasto destro del mouse o Ctrl + Shift + C e Ctrl + Shift + V.

IL PRIMO PROGRAMMA

È il momento di mettere insieme i pezzi. Nella cartella `programmazione` crea il file `ciao.c` e scrivici, esattamente così, il programma seguente.

```c
#include <stdio.h>

int main(void)
{
    printf("Ciao, mondo!\n");
    return 0;
}
```

Il significato di ogni riga sarà spiegato nel capitolo 5; per ora basta sapere che `printf` stampa il testo fra virgolette, e che `\n` significa "vai a capo". Prima di proseguire, **salva** il file con Ctrl + S (su Mac Cmd + S).



Figura 1.13: *La scheda del file dice se ci sono modifiche non salvate.*

gcc legge il file **dal disco**: se le modifiche non sono salvate, compila la versione vecchia. È l'errore più frequente all'inizio ("ho corretto ma non cambia niente!"). Per evitarlo una volta per tutte, si può attivare il salvataggio automatico dal menu **File › Auto Save**.

Compilare ed eseguire

Ora, nel terminale, compiliamo ed eseguiamo:

```
Terminale

$ gcc ciao.c
$ ls
a.out  ciao.c
$ ./a.out
Ciao, mondo!
```

- `gcc ciao.c` compila il file. Non stampa nulla: nessun messaggio significa **nessun errore**. Il risultato è un nuovo file, l'eseguibile, che per tradizione si chiama `a.out`.
- `./a.out` esegue il programma, che stampa la sua riga.

Il nome `a.out` va bene per una prova, ma con più programmi nella stessa cartella ogni compilazione sovrascriverebbe la precedente. Con l'opzione `-o` (*output*) si sceglie il nome dell'eseguibile; e con `-Wall` (*warnings: all*) si chiede a gcc di segnalare anche le cose sospette, che non impediscono la compilazione ma spesso nascondono un errore. È il modo in cui conviene compilare sempre:

```
Terminale

$ gcc -Wall -o ciao ciao.c
$ ./ciao
Ciao, mondo!
```

Ogni volta che si cambia il programma bisogna ripetere **tutto il giro** (figura 1.14): l'eseguibile è una traduzione fatta in un certo momento, e non si aggiorna da solo quando cambia il testo.

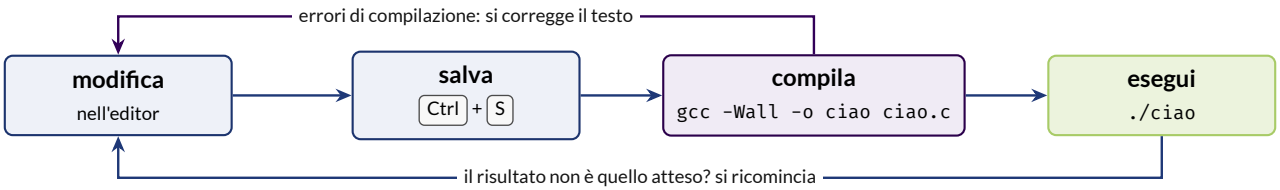


Figura 1.14: *Il ciclo di lavoro: modifica, salva, compila, esegui.*

Perché `./`?

Perché per eseguire il programma si scrive `./ciao` e non solo `ciao`, mentre per gcc basta `gcc`? Quando si scrive il nome di un comando, la shell non lo cerca dappertutto: lo cerca solo in un elenco di cartelle prestabilite, quelle dove stanno i programmi installati (come `/usr/bin`, dove si trova `gcc`). L'elenco è una *variabile d'ambiente* chiamata `PATH`, e per sicurezza non contiene la cartella corrente. Scrivendo `./ciao` si dà il **percorso** del file invece del nome di un comando: "il programma `ciao` che sta *qui*" (`.` è la cartella corrente).

Terminale

```
$ ciao
ciao: command not found
$ ./ciao
Ciao, mondo!
```

Gli errori di compilazione

Che cosa succede se il programma contiene un errore? Togliamo il punto e virgola alla fine della riga di `printf` e ricompiliamo:

errore.c

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Ciao, mondo!\n")
6     return 0;
7 }
```

Terminale

```
$ gcc -Wall -o errore errore.c
errore.c: In function ‘main’:
errore.c:5:29: error: expected ‘;’ before ‘return’
    5 |     printf("Ciao, mondo!\n")
      |                             ^
      |                             ;
    6 |     return 0;
      |     ~~~~~~
```

Il messaggio va letto con calma, perché dice quasi tutto (figura 1.15): **dove** è l'errore (file, riga e colonna), **di che tipo** è e **che cosa** si aspettava il compilatore. Le righe successive ricopiano il codice e indicano con `^` il punto esatto; gcc arriva perfino a suggerire il `;` mancante. Quando c'è un `error` l'eseguibile **non viene prodotto**: bisogna correggere e ricompilare.



Figura 1.15: *Anatomia di un messaggio di errore di gcc.*

Si noti che l'errore è segnalato alla riga **5**, alla fine della riga di `printf`, anche se il compilatore se ne accorge solo leggendo `return` alla riga 6: il punto e virgola poteva mancare solo lì. In generale l'errore sta nella riga indicata **o poco prima**.

Esistono due tipi di messaggi:

- un `error` è un errore vero: il testo non è C valido e l'eseguibile non viene creato;
- un `warning` (avviso) segnala qualcosa di lecito ma sospetto: l'eseguibile viene creato, ma quasi sempre il warning indica un errore di ragionamento. **I warning vanno trattati come errori**: un programma compilato con `-Wall` non deve produrne nessuno.

Leggere solo il primo errore

Un solo errore può confondere il compilatore e generarne a cascata molti altri, alcuni dei quali senza senso. La regola pratica: si corregge **il primo** messaggio dell'elenco, si ricompila, e si ripete.

Con l'estensione C/C++ installata, VS Code si accorge di molti errori **prima** della compilazione e li sottolinea con un'ondina rossa; passando con il mouse sopra la sottolineatura compare la spiegazione (figura 1.16). Il numero di errori compare anche nella barra di stato, accanto a ⊗.



Figura 1.16: *L'estensione C/C++ sottolinea l'errore mentre si scrive.*

E il pulsante "Run"?

L'estensione C/C++ aggiunge in alto a destra nell'editor un pulsante ▶ (*Run C/C++ File*) che compila ed esegue il file aperto con un clic; la prima volta chiede quale compilatore usare (si sceglie *gcc*). È comodo, ma in questa dispensa compileremo **sempre dal terminale**: è ciò che si fa a lezione e all'esame, permette di scegliere le opzioni di gcc, e soprattutto fa vedere che cosa succede davvero, passo per passo. Dal capitolo 7 useremo `make`, che si lancia anch'esso dal terminale.

Quando qualcosa non funziona

Sintomo	Causa probabile e rimedio
`gcc: command not found`	Il compilatore non è installato (ripeti il passo 2). Su Windows, controlla di essere nel terminale di **Ubuntu** (prompt con `$`) e non in PowerShell (prompt `PS C:\...`).
`code: command not found`	Su Windows chiudi e riapri il terminale di Ubuntu dopo aver installato VS Code (il `PATH` si aggiorna solo all'apertura); su Mac esegui *Shell Command: Install 'code' command in PATH*.
`ciao.c: No such file or directory`	Il file non è nella cartella corrente: controlla con `pwd` e `ls` dove ti trovi e che cosa c'è, oppure il file non è stato salvato con quel nome.
`./ciao: No such file or directory`	Il programma non è stato compilato, oppure la compilazione è fallita (c'erano degli `error`), oppure l'eseguibile ha un altro nome (`a.out`?).
Ho corretto ma il programma fa ancora la cosa sbagliata	Hai salvato? Hai ricompilato? Stai eseguendo l'eseguibile giusto?
Il programma non termina	Probabilmente un ciclo infinito, oppure il programma aspetta un input da tastiera. Ctrl + C lo interrompe.
In VS Code `#include <stdio.h>` è sottolineato in rosso	L'estensione C/C++ non trova il compilatore. Su Windows: VS Code non è collegato a Ubuntu (guarda l'indicatore remoto) o l'estensione non è installata *in WSL*. Altrove: verifica che `gcc --version` funzioni.

In sintesi

Per scrivere ed eseguire un programma C servono tre strumenti: un **editor** (VS Code) per scrivere il testo del programma in un file `.c`, un **compilatore** (gcc) che lo traduce in linguaggio macchina, e un **terminale** da cui lanciare il compilatore e il programma. Su Windows tutto ciò vive in Ubuntu, grazie a WSL. Il ciclo di lavoro è sempre lo stesso: *modifica, salva, compila, esegui*; e i messaggi di gcc indicano file, riga e colonna del problema.

Riferimenti

- **Installare Linux in Windows con WSL** — Microsoft Learn.
  `https://learn.microsoft.com/it-it/windows/wsl/install`
- **Using C++ and WSL in VS Code** — documentazione di VS Code (vale anche per il C).
  `https://code.visualstudio.com/docs/cpp/config-wsl`
- **Using Clang in Visual Studio Code** (macOS) e **Using C++ on Linux in VS Code** — documentazione di VS Code.
  `https://code.visualstudio.com/docs/cpp/config-clang-mac`
  `https://code.visualstudio.com/docs/cpp/config-linux`
- **User Interface** — la guida alle zone della finestra di VS Code.
  `https://code.visualstudio.com/docs/getstarted/userinterface`

Il calcolatore

Per imparare a guidare non serve saper costruire un motore, ma aiuta molto sapere che cosa fanno il volante, i pedali e il cambio. Allo stesso modo, per programmare non serve conoscere l'elettronica di un computer, ma bisogna avere un'idea chiara di come è fatto e di come esegue un programma: molte regole del C, che altrimenti sembrerebbero capricci, diventano ovvie. Questo capitolo costruisce quell'idea, partendo da che cosa significa *programmare*.

Che cos'è la programmazione

Il corso si apre con la definizione del dizionario Treccani:

> *"In informatica, la stesura di un programma in una forma interpretabile dal calcolatore elettronico (o direttamente o dopo una rielaborazione opportuna), compiuta dal programmatore sulla base delle specifiche di programma fornite dall'analista; la programmazione comprende la definizione del problema da risolvere, la preparazione del diagramma di flusso del programma, la sua codifica (cioè la trascrizione, in un linguaggio opportuno, delle varie istruzioni), la messa a punto e la preparazione di una documentazione a uso dell'operatore."*

La definizione contiene già tutto il percorso del corso. Scrivere codice (la *codifica*) è solo una delle fasi, e non la prima: prima bisogna capire il problema e progettare la soluzione, dopo bisogna verificare che funzioni (la *messa a punto*) e spiegarla (figura 2.1). La parentesi "o direttamente o dopo una rielaborazione opportuna" è importante: il programma che scriviamo non è ancora ciò che il computer esegue, va *rielaborato*, cioè tradotto. È il compito del compilatore incontrato nel capitolo precedente.

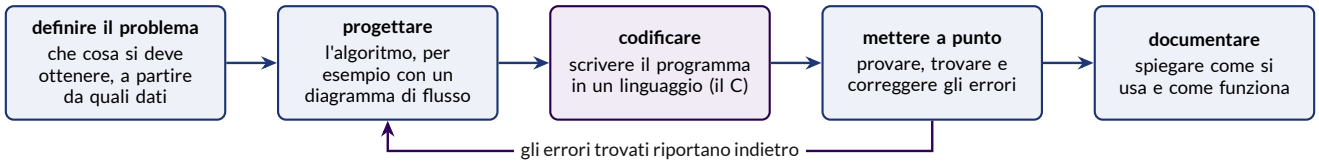


Figura 2.1: *Le fasi della programmazione secondo la definizione: scrivere il codice è solo una di esse.*

Come si impara a programmare

A lezione il docente avverte che il corso parte da zero e all'inizio va piano, ma che velocità e difficoltà crescono presto, e che entro il primo mese quasi tutti incontrano difficoltà serie, soprattutto negli esercizi. È normale e si supera, con alcune abitudini:

- non dare per scontato di saper già programmare, e non saltare le lezioni "facili";
- studiare con costanza: la programmazione si accumula, ogni argomento usa i precedenti;
- **scrivere ed eseguire** ogni esempio, poi modificarlo per vedere che cosa succede: leggere un programma non basta, come non basta guardare qualcuno nuotare;
- chiedere appena qualcosa non è chiaro ("non siete gli unici");
- prendere appunti e fare sintesi: le slide sono, per loro natura, sintetiche; questa dispensa prova a riempire gli spazi, ma non sostituisce le lezioni.

Lo schema di un calcolatore

Un computer moderno è un oggetto complicatissimo, ma il suo funzionamento si capisce con uno schema di tre soli blocchi (figura 2.2):

- la **CPU** (*Central Processing Unit*, il *processore*), che esegue le istruzioni;
- la **memoria centrale** (o *principale*, la RAM), che contiene le istruzioni da eseguire e i dati su cui lavorano;
- i **dispositivi di ingresso e uscita** (*I/O*, *Input/Output*): tastiera, mouse, schermo, disco, rete…, collegati attraverso delle *interfacce*.

I blocchi comunicano attraverso il **bus**, un insieme di collegamenti condiviso su cui viaggiano dati, indirizzi e comandi. Fisicamente, è la *scheda madre* (*motherboard*) a collegare fra loro processore, memoria e periferiche.

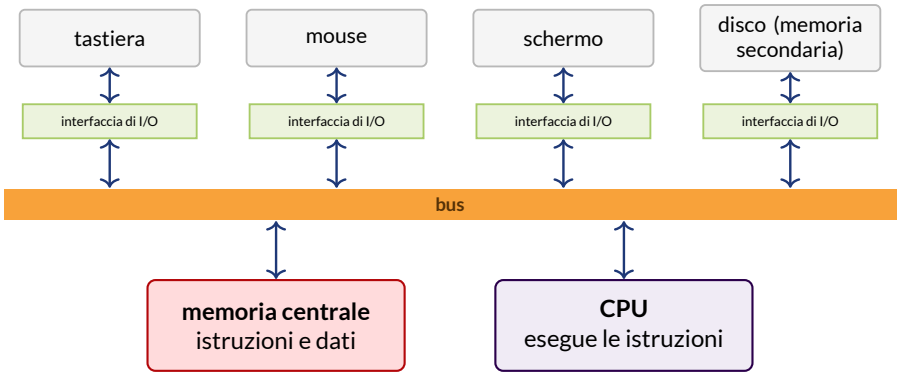


Figura 2.2: *Schema semplificato di un calcolatore (architettura di von Neumann): CPU, memoria e dispositivi di I/O collegati dal bus.*

Questo schema porta il nome di *John von Neumann*, che nel 1945 ne descrisse l'idea centrale: le **istruzioni** del programma stanno nella **stessa memoria** dei **dati**. Un programma non è cablato nei circuiti: è una sequenza di numeri in memoria, che si può caricare, sostituire, copiare. È questo che rende un computer una macchina *generale*, capace di fare cose diversissime a seconda del programma caricato.

La CPU e il linguaggio macchina

Che cosa sa fare un processore

Tutte le elaborazioni di un calcolatore sono svolte dalla CPU, oppure da altri componenti su suo comando. Sorprendentemente, un processore sa compiere solo operazioni **molto semplici**:

- leggere, scrivere o copiare una o più celle di memoria;
- sommare, sottrarre, moltiplicare, dividere il contenuto di celle di memoria;
- leggere e scrivere in zone di memoria "speciali" per pilotare i dispositivi di I/O (per esempio la scheda video);
- poche altre operazioni elementari, come confrontare due numeri e **saltare** a un'altra istruzione a seconda del risultato.

Il segreto è la velocità: un processore moderno esegue **miliardi** di queste operazioni al secondo. Ogni cosa che un computer fa, dal mostrare una pagina web al riprodurre un video, è scomposta in una lunghissima sequenza di passi elementari.

Ogni processore ha un proprio insieme di istruzioni, e ogni istruzione è identificata da una certa configurazione di bit (di zeri e uni). L'insieme delle istruzioni di un processore si chiama **linguaggio macchina** di quel processore. Processori di famiglie diverse hanno linguaggi macchina diversi: un programma tradotto per i processori Intel e AMD dei PC (famiglia *x86-64*) non gira su un Mac con processore Apple (famiglia *ARM*), e viceversa.

Per eseguire un programma, allora, bisogna fare due cose: mettere in memoria la sequenza di configurazioni di bit che rappresentano le sue istruzioni, e dire al processore a quale indirizzo si trova la prima. Da lì in poi il processore ripete sempre lo stesso ciclo (figura 2.3): **preleva** l'istruzione all'indirizzo indicato da un suo registro speciale, il **program counter**; **decodifica** l'istruzione, cioè capisce che cosa chiede; la **esegue**; e passa all'istruzione successiva, a meno che l'istruzione appena eseguita non fosse un *salto*, che cambia il program counter.

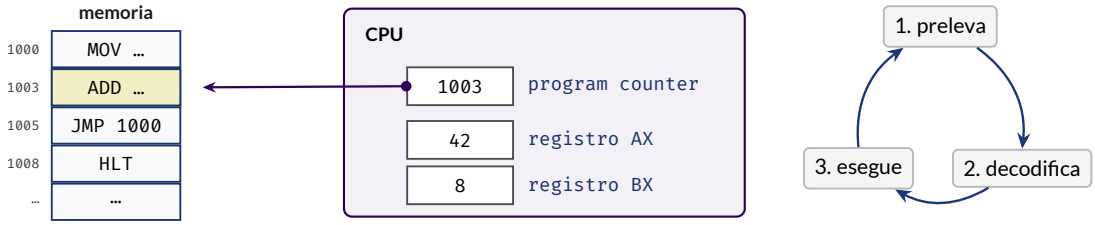


Figura 2.3: *Il program counter indica l'indirizzo della prossima istruzione; la CPU la preleva, la decodifica e la esegue, poi ricomincia. I registri sono piccole celle di memoria interne alla CPU.*

L'ASSEMBLY

Il linguaggio macchina è una sequenza di bit, praticamente illeggibile per una persona. Il primo passo per renderlo comprensibile è l'**assembly**: un linguaggio quasi identico al linguaggio macchina, in cui però ogni codice operativo è sostituito da un'abbreviazione (un *codice mnemonico*): `MOV` per *move* (copia), `ADD` per *add* (somma), `JMP` per *jump* (salta). Un programma, l'**assembler**, traduce l'assembly in linguaggio macchina, un'istruzione alla volta.

Un esempio: contare i bit a 1

Il programma seguente, visto a lezione, conta quanti bit valgono 1 in un dato di 16 bit. A sinistra di ogni istruzione c'è il suo indirizzo in memoria (in esadecimale, cioè in base 16: per ora basta considerarlo un "numero civico"), a destra la spiegazione.

indirizzo	istruzione		significato
`3F0D0100`	`00000000 00000000`		il dato da elaborare (16 bit)
`3F0D0102`	`00000000`		qui andrà il risultato (8 bit)
`40B1A200`	`MOV`	`$0, %AL`	azzera il contatore, il registro AL
`40B1A203`	`MOV`	`(3F0D0100), %DX`	copia nel registro DX il dato da elaborare
`40B1A207`	`CMP`	`%DX, $0`	confronta DX con 0...
`40B1A211`	`JE`	`40B1A232`	...e se sono uguali salta alla fine
`40B1A215`	`SHL`	`%DX`	sposta i bit di DX di un posto a sinistra: il bit che "esce" finisce nel *carry flag* CF
`40B1A217`	`JC`	`40B1A225`	se CF vale 1 (è uscito un 1) salta all'incremento
`40B1A221`	`JMP`	`40B1A207`	altrimenti torna al confronto
`40B1A225`	`ADD`	`$1, %AL`	incrementa il contatore AL
`40B1A228`	`JMP`	`40B1A207`	torna al confronto
`40B1A232`	`MOV`	`%AL, (3F0D0102)`	scrive il risultato in memoria
`40B1A236`	`HLT`		fine (*halt*)

L'idea è questa: a ogni giro si fa "scivolare" il dato di un posto a sinistra, e si guarda il bit che cade fuori dal bordo; se è un 1, si incrementa il contatore. Quando nel dato non resta nessun 1 (cioè vale 0), si è finito. Seguiamo l'esecuzione con un dato che contiene due bit a 1, `1010000000000000`:

giro	DX prima	bit uscito (CF)	DX dopo	AL	che cosa succede
1	`1010000000000000`	`1`	`0100000000000000`	`1`	è uscito un 1: `JC` salta all'`ADD`
2	`0100000000000000`	`0`	`1000000000000000`	`1`	è uscito uno 0: `JMP` torna al confronto
3	`1000000000000000`	`1`	`0000000000000000`	`2`	è uscito un 1: si incrementa
4	`0000000000000000`			`2`	DX vale 0: `JE` salta alla fine

Il risultato, 2, viene scritto in memoria e il programma si ferma. Si noti che in assembly non esiste un'istruzione "ripeti finché": la ripetizione si ottiene **saltando indietro** (`JMP 40B1A207`) e la scelta **saltando avanti** se una condizione è vera (`JE`, `JC`).

I limiti dell'assembly

Con l'assembly si può scrivere qualunque programma, anche un sistema operativo o un'interfaccia grafica, ma è un lavoro enorme: le istruzioni sono di **basso livello**, cioè vicine alla macchina e lontane dal modo in cui ragioniamo. Per "contare gli uni" abbiamo dovuto pensare in termini di registri, flag e indirizzi di salto; e il programma funziona solo sul processore per cui è scritto.

I LIVELLI DI ASTRAZIONE

Servono istruzioni più vicine al modo di ragionare umano. La soluzione è l'**astrazione**: costruire un linguaggio che "nasconde" la macchina sottostante e permette di esprimere gli algoritmi in modo simbolico. Sono i **linguaggi di alto livello**, come il C, che hanno due proprietà fondamentali:

- sono più vicini al linguaggio umano (e matematico) che al linguaggio macchina;
- sono **generali**: non dipendono dal processore. Lo stesso programma C si traduce per un PC, per un Mac o per un telefono.

La figura 2.4 mostra la stessa istruzione ai tre livelli. È la traduzione reale prodotta da gcc su un PC (processore x86-64) per l'istruzione C `c = a + b;`, in cui `a`, `b` e `c` sono tre variabili intere.

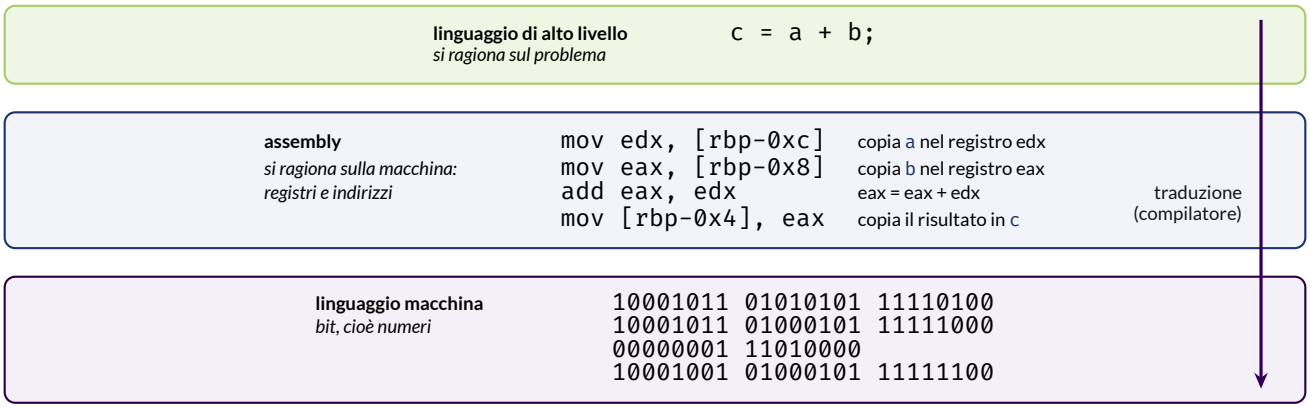


Figura 2.4: *La stessa istruzione a tre livelli di astrazione. Un'istruzione di alto livello diventa più istruzioni macchina; ogni istruzione assembly corrisponde a un'istruzione macchina.*

Ogni livello richiede di conoscere cose diverse:

- per il **linguaggio macchina**, come sono rappresentate le informazioni in bit;
- per l'**assembly**, le caratteristiche della macchina: registri, insieme di istruzioni, dimensione dei dati;
- per un **linguaggio di alto livello**, solo il linguaggio: ci si astrae dai dettagli della macchina.

Il prezzo da pagare è che un programma di alto livello non può essere eseguito direttamente: va **tradotto** in linguaggio macchina. È il compito del **compilatore**, l'argomento del capitolo 3.

Algoritmo e programma

Due parole che useremo continuamente vanno distinte con cura.

Algoritmo

Una sequenza di passi elementari che risolve un problema. La sequenza deve essere **finita**, **non ambigua**, **deterministica** e **generale**.

Programma

La formalizzazione di un algoritmo in un linguaggio di programmazione.

Le quattro proprietà dell'algoritmo non sono pignolerie: ciascuna esclude un modo concreto in cui un procedimento può fallire. Prendiamo una ricetta di cucina, che è quasi un algoritmo:

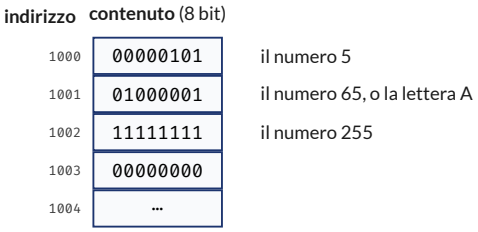
- **finita**: deve terminare. "Mescola finché non è pronto" è accettabile solo se prima o poi "è pronto";
- **non ambigua**: ogni passo deve avere un solo significato. "Sale quanto basta" va bene per un cuoco, non per un computer, che non ha buon senso: bisogna dire quanti grammi;
- **deterministica**: in ogni momento il passo successivo è determinato; con gli stessi dati si ottiene sempre lo stesso risultato;
- **generale**: risolve una *classe* di problemi, non un solo caso. Una ricetta "per 4 persone" è meno generale di una "per n persone"; un algoritmo che calcola l'area di un cerchio di raggio 2 è inutile, ne serve uno che funzioni per qualunque raggio.

La memoria

Celle e indirizzi

La memoria centrale è il contenitore dei dati su cui lavora il processore. Si può immaginare come una lunghissima fila di caselle consecutive, le **locazioni** (o *celle*) di memoria. Ogni cella:

- è l'unità minima di memorizzazione: si legge e si scrive una cella intera, non un pezzo;
- contiene tipicamente **8 bit**, cioè un **byte**;
- è identificata da un numero, il suo **indirizzo**, come le case di una strada dal numero civico.



Un **bit** (*binary digit*, cifra binaria) è la più piccola unità di informazione: vale 0 oppure 1. Elettronicamente corrisponde a qualcosa che ha due stati, come un interruttore acceso o spento. Con un bit solo si distinguono due situazioni; mettendone insieme 8 se ne distinguono molte di più: vediamo quante, e come si usano per rappresentare i numeri.

La notazione binaria

Siamo abituati alla notazione **decimale**: dieci cifre, da 0 a 9, e un sistema **posizionale**, in cui il valore di una cifra dipende dalla posizione. In `352` il 3 vale trecento perché sta nella posizione delle centinaia:

$$352 = 3 \cdot 100 + 5 \cdot 10 + 2 \cdot 1 = 3 \cdot 10^2 + 5 \cdot 10^1 + 2 \cdot 10^0.$$

La notazione **binaria** funziona allo stesso modo, ma con due sole cifre, 0 e 1, e con le potenze di **2** al posto delle potenze di 10 (figura 2.5).

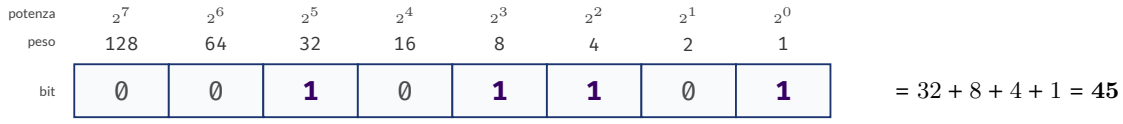


Figura 2.5: *Il byte `00101101`: ogni bit a 1 aggiunge il peso della sua posizione.*

Per passare dal binario al decimale, quindi, si sommano i pesi delle posizioni in cui c'è un 1. Il passaggio inverso si fa con le **divisioni successive per 2**: si divide il numero per 2, si annota il resto (0 o 1), si ripete sul quoziente fino ad arrivare a 0; i resti, letti **dal basso verso l'alto**, sono le cifre binarie.

```
13 : 2 = 6   resto 1
 6 : 2 = 3   resto 0
 3 : 2 = 1   resto 1
 1 : 2 = 0   resto 1
```



Per esempio, 13 in binario è `1101`: lo si verifica sommando i pesi, 8 + 4 + 1 = 13. Perché funziona? Ogni divisione per 2 "stacca" l'ultima cifra binaria del numero, esattamente come dividere per 10 un numero decimale ne stacca l'ultima cifra (il resto di 352 : 10 è 2, l'ultima cifra). Per questo il primo resto trovato è la cifra più a destra, e i resti vanno letti al contrario.

Quanti numeri in un byte

Con n posizioni e 2 simboli possibili per ciascuna si formano 2^n combinazioni diverse: ogni bit in più **raddoppia** le possibilità. Un byte ne ha $2^8 = 256$, quindi rappresenta i numeri naturali da 0 a 255:

bit	valore	bit	valore
`00000000`	0	`00000100`	4
`00000001`	1	`00000101`	5
`00000010`	2	…	…
`00000011`	3	`11111111`	255

I numeri negativi

E i numeri negativi? Bisogna usare una parte dell'informazione per il **segno**. L'idea più semplice, vista a lezione, è la **rappresentazione in modulo e segno**: il primo bit indica il segno (0 positivo, 1 negativo) e gli altri 7 il valore assoluto. Con 7 bit si rappresentano $2^7 = 128$ valori assoluti, da 0 a 127, quindi l'intervallo è $[-127,\ 127]$.

Una svista nelle slide

Le slide indicano l'intervallo $[-127..128]$, ma in modulo e segno il massimo è +127 (`01111111`), non 128: con 7 bit il valore assoluto più grande è $2^7 - 1 = 127$. L'intervallo è simmetrico, $[-127,\ 127]$, e lo zero compare due volte: `00000000` (+0) e `10000000` (−0).

Proprio quel "doppio zero", e il fatto che le somme con numeri negativi richiederebbero circuiti speciali, fanno sì che i computer reali usino un'altra rappresentazione, il **complemento a due**, in cui con un byte si rappresentano i numeri da −128 a +127. L'idea si vede bene con 3 bit (figura 2.6): i numeri sono disposti su una ruota, come le cifre del contachilometri di un'auto; partendo da `000` e andando **indietro** di un passo si arriva a `111`, che quindi vale −1.

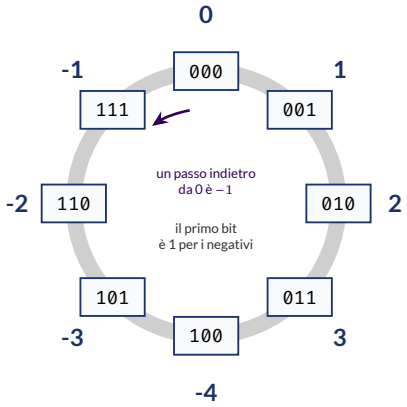


Figura 2.6: *Il complemento a due su 3 bit: 8 combinazioni, da −4 a +3, un solo zero. Su 8 bit si va da −128 a +127.*

Il complemento a due ha conseguenze molto concrete in C (per esempio su che cosa succede quando un numero "trabocca" oltre il massimo): le vedremo nel capitolo 8.

I TIPI DI DATO

Gli stessi bit, significati diversi

Il byte `01000001` contiene semplicemente otto bit. È il numero 65? È la lettera *A*? Dipende da come lo si **interpreta**: la stessa quantità di memoria può rappresentare un numero naturale, un intero con segno, un numero reale o un carattere. La memoria non sa che cosa contiene; lo sa il programma.

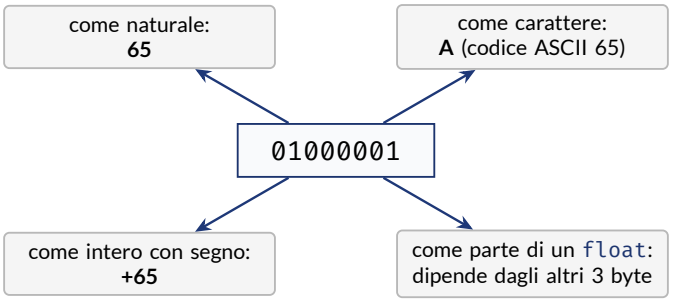


Figura 2.7: *Lo stesso byte, letto con occhiali diversi.*

Il tipo come astrazione

I linguaggi di alto livello risolvono il problema con i **tipi di dato**: si scrive il programma in termini di *che cosa* sono i dati, senza preoccuparsi di come e dove saranno davvero memorizzati. In C si può scrivere

```c
int variabile = 5;
```

e non ci si chiede in quali celle finirà il 5, né come sarà scritto in bit: si usa `int` nel suo significato astratto di "contenitore di numeri interi". Eppure un `int` non sta in una cella sola: ne occupa **quattro consecutive** (32 bit, quindi valori ben oltre 255). È il tipo a stabilire che quelle quattro celle vanno lette insieme, come un unico numero (figura 2.8).

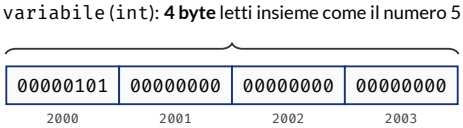


Figura 2.8: *Un `int` occupa 4 celle consecutive (sui PC il byte meno significativo sta all'indirizzo più basso).*

Ogni linguaggio definisce i propri tipi. In C, i principali sono:

tipo	che cosa rappresenta	dimensione tipica
`char`	un carattere (codice ASCII)	8 bit (1 byte)
`int`	un numero intero	32 bit (4 byte)
`float`	un numero reale	32 bit (4 byte)
`double`	un numero reale, più preciso	64 bit (8 byte)

Le dimensioni sono quelle dei computer di oggi; lo standard del C fissa solo dei minimi, come vedremo nel capitolo 8.

La gerarchia di memoria

La memoria centrale può contenere molti programmi e dati, ma ha due difetti:

- è **lenta** rispetto al processore: leggere una cella richiede il tempo di un centinaio di operazioni della CPU;
- è **volatile**: quando si spegne il computer, il contenuto si perde.

Contro la lentezza si costruisce una **gerarchia**: piccole memorie velocissime vicino al processore, memorie via via più grandi e lente man mano che ci si allontana. Le memorie veloci costano molto per ogni bit, quindi sono piccole. Contro la volatilità ci sono le **memorie di massa** (dischi, SSD, chiavette), che conservano i dati anche a computer spento (figura 2.9).

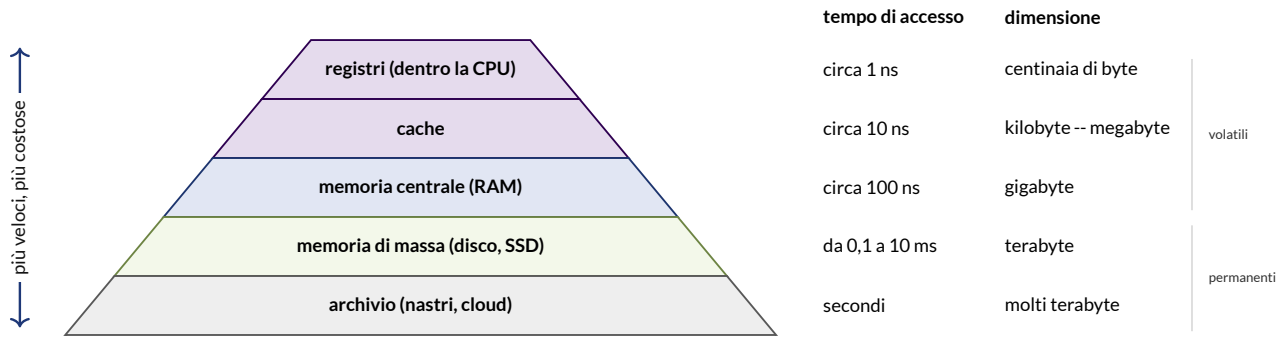


Figura 2.9: *La gerarchia di memoria (1 ns è un miliardesimo di secondo). In alto memorie piccole, veloci, costose e volatili; in basso grandi, lente, economiche e permanenti.*

Un'immagine aiuta: la CPU è un impiegato alla scrivania. I registri sono ciò che tiene in mano; la cache, i fogli sulla scrivania; la RAM, lo scaffale nella stanza; il disco, l'archivio in cantina. L'impiegato lavora solo su ciò che ha in mano, e se un foglio non è sulla scrivania deve alzarsi a prenderlo: per questo i dati usati spesso vengono tenuti vicini.

Gli strati del software

Fra l'hardware e i programmi che usiamo c'è un sistema software a strati (figura 2.10). Ogni strato usa i servizi di quello sotto e ne offre di più comodi a quello sopra: è ancora l'idea di astrazione.



Figura 2.10: *Fra l'hardware e l'ambiente in cui si sviluppano i programmi c'è un sistema software stratificato.*

L'avvio del computer

Il primo programma eseguito all'accensione è il **sistema operativo**: è lui che, quando chiediamo di eseguire un programma, lo carica in memoria e dice al processore dove si trova la sua prima istruzione. Ma chi dice al processore dove si trova la prima istruzione del sistema operativo? È un problema dell'uovo e della gallina, risolto così (figura 2.11): all'accensione il computer esegue un piccolo programma che sta in una memoria permanente della scheda madre (il *firmware*); questo carica in memoria il contenuto di un blocco predefinito di un disco, il **Master Boot Record** (MBR). Il codice del MBR, chiamato **bootloader**, è una sequenza di byte che si assume essere un programma in linguaggio macchina: il suo compito è caricare il sistema operativo. Il MBR è configurabile: se sul disco ci sono più sistemi operativi, si può scegliere quale avviare.

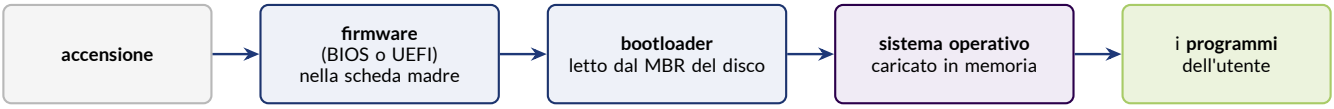


Figura 2.11: *L'avvio: ogni programma carica il successivo, più grande e più capace.*

MBR e UEFI

Il Master Boot Record è il meccanismo storico dei PC. I computer recenti usano il firmware *UEFI*, che invece di un singolo blocco legge il bootloader da una piccola partizione dedicata del disco. L'idea non cambia: un programma minuscolo, che sta in un posto noto in anticipo, avvia un programma più grande.

Che cosa fa il sistema operativo

Il sistema operativo **alza il livello di astrazione** della macchina: è più facile da programmare dell'hardware sottostante e semplifica la scrittura di ogni altro programma. Le sue funzioni principali:

- **gestione dei processi**, cioè dei programmi in esecuzione: li crea e li termina, li sospende e li riattiva, li fa comunicare. Il processore esegue un'istruzione alla volta, ma il sistema operativo lo assegna a turno ai vari programmi per pochi millesimi di secondo ciascuno (la *schedulazione*): così sembrano procedere tutti insieme;
- **gestione della memoria centrale**: assegna la memoria ai processi, li carica e li rimuove, e la **protegge**, cioè impedisce a un programma di scrivere nella memoria di un altro;
- **gestione delle periferiche**: le configura e le pilota, con il *buffering* (tiene i dati in memoria finché non vengono elaborati) e il *caching* (tiene vicini i dati usati spesso);
- **gestione del file system**: nasconde all'utente come sono organizzati i dati sul disco, offrendo il concetto di *file* e di *cartella* (l'albero di cartelle del capitolo 1), con le operazioni di creazione, lettura, scrittura, copia, ricerca e i diritti di accesso;
- **interfaccia utente**: a riga di comando (l'interprete dei comandi, la shell che usiamo nel terminale) o grafica;
- **rilevamento degli errori**: divisione per zero, carta finita, rete caduta…; per ogni errore serve un'azione di recupero appropriata.

L'ultima voce ci riguarderà da vicino. Un programma C che divide un intero per zero chiede al processore un'operazione impossibile; il processore avvisa il sistema operativo, che **termina il programma**:

div0.c

```c
#include <stdio.h>

int main(void)
{
    int a = 7, b = 0;
    printf("%d\n", a / b);
    return 0;
}
```

Terminale

```
$ gcc -o div0 div0.c
$ ./div0
Floating point exception
```

Il messaggio parla di "virgola mobile" anche se la divisione è fra interi: è un nome storico.

I servizi per i programmi

Il sistema operativo offre servizi ai programmi: caricarli in memoria ed eseguirli, compiere le operazioni di I/O (un programma normale non può pilotare direttamente i dispositivi), leggere e scrivere i file, far comunicare processi sullo stesso computer o su computer diversi (con memoria condivisa o scambio di messaggi), rilevare gli errori. Ci sono poi servizi per gestire il sistema nel suo insieme: allocazione delle risorse fra utenti e processi, contabilità del loro uso, protezione e sicurezza.

Anche il più semplice dei nostri programmi passa da questi servizi. Quando il programma del capitolo precedente esegue `printf`, il testo non va direttamente sullo schermo (figura 2.12).

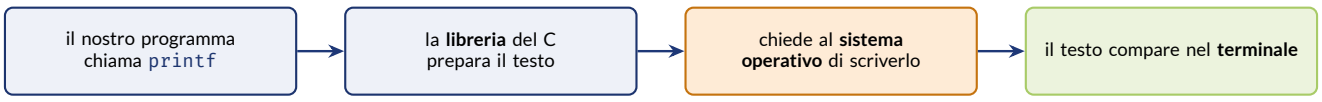


Figura 2.12: *Il percorso di una* `printf`*: dal programma alla libreria, dalla libreria al sistema operativo, e infine al dispositivo.*

Installare e scrivere un sistema operativo

Lo stesso sistema operativo gira su macchine diverse, quindi va **configurato** in fase di installazione: processore, memoria, suddivisione del disco (partizioni), dispositivi presenti, opzioni come l'algoritmo di schedulazione o il numero massimo di processi.

I primi sistemi operativi erano scritti in assembly; oggi si scrivono in linguaggi di alto livello, perché il codice si scrive più in fretta, è più compatto, più facile da capire e da correggere. Un sistema operativo scritto in un linguaggio di alto livello è molto più flessibile e manutenibile. Non a caso, il C nacque proprio per scrivere un sistema operativo, Unix, come racconta il capitolo 4.

> **In sintesi**
>
> Un calcolatore è fatto di CPU, memoria e dispositivi di I/O, collegati dal bus. La memoria è una fila di celle di un byte, ognuna con un indirizzo, che contengono sia i dati sia le istruzioni dei programmi. La CPU sa eseguire solo istruzioni elementari del proprio **linguaggio macchina**, una alla volta, guidata dal program counter; ripetizioni e scelte si ottengono con i salti. I linguaggi di alto livello come il C nascondono questi dettagli (astrazione), ma vanno **tradotti**. I bit di per sé non hanno significato: sono i **tipi** a stabilire come interpretarli. Il sistema operativo, il primo programma caricato all'avvio, gestisce processi, memoria, periferiche e file, e offre i suoi servizi a tutti gli altri programmi.

RIFERIMENTI



- **Deitel & Deitel**, *Il linguaggio C — Fondamenti e tecniche di programmazione*, Pearson: capitolo 1 (computer, hardware e software, linguaggi macchina, assembly e di alto livello).
- **Treccani**, voce *programmazione*.
  https://www.treccani.it/vocabolario/programmazione/

Dal problema al programma

Il capitolo precedente ha mostrato che cosa sa fare la macchina: eseguire, velocissima, istruzioni elementari scritte in linguaggio macchina. Questo capitolo percorre la strada nell'altro senso, partendo da un **problema** da risolvere: come si descrive la soluzione (l'algoritmo), come la si scrive in un linguaggio di alto livello (il programma) e come il programma diventa qualcosa che la CPU può eseguire.

Programmare

La programmazione è l'attività che porta a sviluppare un software. Non ogni procedimento, però, può diventare un programma: deve poter essere **descritto in modo non ambiguo**, deve **raggiungere l'obiettivo in un tempo finito** e deve essere **deterministico**. Un procedimento con queste caratteristiche si chiama **algoritmo**: una ricetta di cucina ben scritta, o il metodo per risolvere un'equazione di secondo grado.

Programmare significa descrivere al calcolatore *come* risolvere un problema. Il processore mette a disposizione molte operazioni di base (somma, sottrazione, operazioni logiche, spostamenti di valori, letture e scritture verso le periferiche), ma, come abbiamo visto, conviene esprimersi in un linguaggio di alto livello e lasciare a un programma il compito di tradurre.

I linguaggi di alto livello

Un linguaggio di alto livello (*HLL*, *High Level Language*) ha tre caratteristiche:

- un **alto livello di astrazione**: le istruzioni hanno un significato più complesso di quelle della macchina, e non serve sapere come il processore le realizza (`A + B` somma due valori, punto);
- viene **tradotto** in linguaggio macchina (o in assembly) da un programma traduttore, ed è quindi indipendente dal processore e dal sistema operativo;
- è più **simile al linguaggio umano**, quindi più facile da scrivere e da ricordare (in molti linguaggi `print X` mostra il valore di `X`).

Comunque avvenga la traduzione, il programmatore scrive le istruzioni in un file di testo: il **codice sorgente** (o *programma sorgente*, o semplicemente *programma*). Per tradurlo ci sono due strategie, l'**interprete** e il **compilatore**.

Interprete e compilatore

Un **interprete** traduce le istruzioni del codice sorgente **una alla volta** in linguaggio macchina, e ciascuna viene eseguita subito dalla CPU, prima di passare alla successiva. Un **compilatore**, invece, traduce **tutto** il codice sorgente in linguaggio macchina e lo salva in un file, il **programma eseguibile**; l'esecuzione avviene dopo, ed è un'operazione separata (figura 3.1).

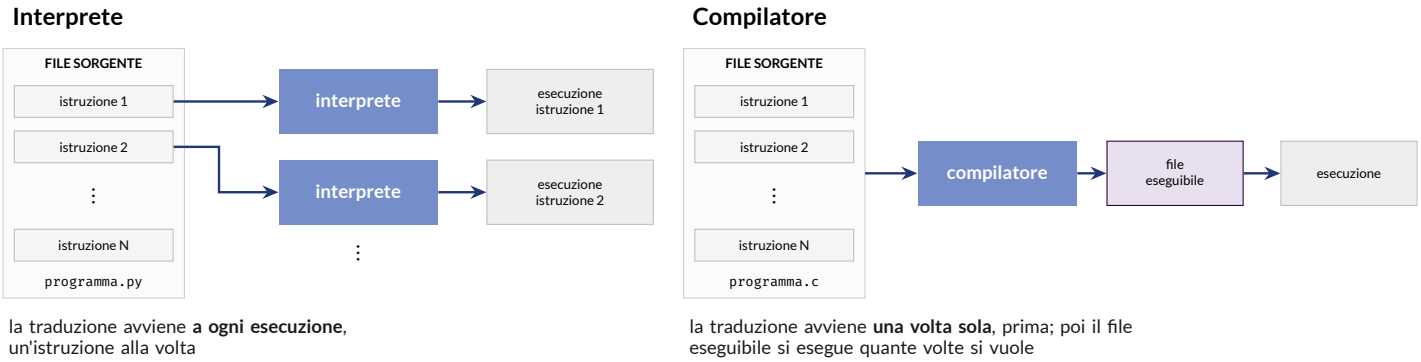


Figura 3.1: *Interprete (a sinistra) e compilatore (a destra).*

Il C è un linguaggio **compilato**; Python e JavaScript, per esempio, sono di solito interpretati. Le due strategie hanno pregi e difetti opposti:

	Programma interpretato	**Programma compilato**
velocità	più lento: a ogni esecuzione l'interprete deve ritradurre le istruzioni	più veloce: è già tradotto; e il compilatore sa *ottimizzare* il codice, per renderlo più veloce o più piccolo
chi deve avere che cosa	l'utente deve avere l'interprete e saperlo usare (si può nascondere, ma c'è)	all'utente basta l'eseguibile (un doppio clic); solo il programmatore ha bisogno del compilatore
codice sorgente	serve all'utente, quindi è visibile e modificabile da chiunque	non serve per eseguire: si può non distribuirlo, proteggendo il lavoro (risalire al sorgente è possibile, ma difficile)

Una via di mezzo: il bytecode

Alcuni linguaggi, come Java e Python, compilano il sorgente in un file intermedio, non eseguibile direttamente, scritto in una sorta di assembly generico chiamato **bytecode**, che si traduce molto rapidamente nel codice macchina di un processore reale. Su ogni computer si installa un interprete del bytecode (per Java, la *Java Virtual Machine*): così lo stesso file gira su macchine con linguaggi macchina diversi (figura 3.2).

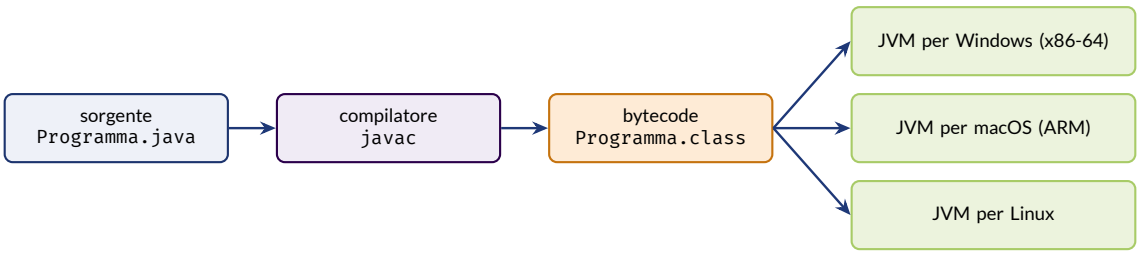


Figura 3.2: *Un solo file di bytecode, un interprete diverso per ogni tipo di macchina.*

Le librerie

In un linguaggio di alto livello il programmatore non deve programmare da sé le funzioni di base: calcolare una radice quadrata o una tangente, stampare un numero sullo schermo, leggere un valore dalla tastiera. Queste operazioni sono state programmate e compilate una volta per tutte da chi produce il compilatore, e sono a disposizione sotto forma di **funzioni** (per esempio `tan(x)` calcola la tangente di `x`, `printf` stampa).

Il loro codice, già tradotto in linguaggio macchina, è raccolto in file chiamati **librerie**: collezioni di funzioni pronte all'uso. Un programma C "vero" è quindi fatto di due parti: il codice scritto da noi e il codice delle librerie che usa.

Come nasce un eseguibile

Torniamo al compilatore. Finora lo abbiamo trattato come una scatola chiusa, che riceve il sorgente e restituisce l'eseguibile; in realtà la traduzione di un programma C avviene in più passaggi, ciascuno svolto da uno strumento diverso. Conoscerli aiuta a capire i messaggi di errore, che cambiano a seconda del passaggio in cui qualcosa è andato storto.

Compilazione e linking

La creazione di un eseguibile (in inglese *build*) avviene in due fasi (figura 3.3):

1. **compilazione**: il sorgente viene tradotto in codice macchina, ma le funzioni di libreria che usa sono ancora **mancanti**. Il risultato è un file intermedio, il **file oggetto** (estensione `.o`), che non si può eseguire;
2. **linking** (collegamento): uno o più file oggetto e una o più librerie (già in codice macchina) vengono **uniti** dal **linker**, che aggiunge le parti mancanti e produce un unico file eseguibile.

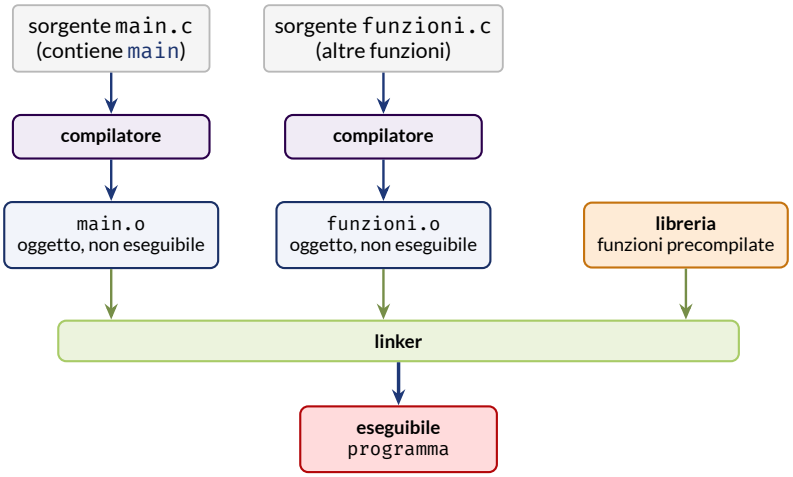


Figura 3.3: *La compilazione produce i file oggetto, il linking li unisce fra loro e con le librerie.*

Quando nel capitolo 1 abbiamo scritto `gcc ciao.c`, gcc ha eseguito entrambe le fasi in un colpo solo. Le si può però separare, ed è istruttivo farlo con il primo programma di esempio del corso, `area_cerchio.c` (lo vedremo per intero fra poco). L'opzione `-c` (*compile*) chiede a gcc di fermarsi dopo la compilazione:

Terminale

```
$ gcc -c area_cerchio.c
$ ls -l area_cerchio.c area_cerchio.o
-rw-r--r-- 1 fabio fabio  255 ott 31 10:09 area_cerchio.c
-rw-r--r-- 1 fabio fabio 1960 ott 31 10:09 area_cerchio.o
$ nm area_cerchio.o
                 U __isoc99_scanf
                 U __stack_chk_fail
0000000000000000 T main
                 U printf
```

Il comando `nm` elenca i nomi che compaiono in un file oggetto. La `T` accanto a `main` significa che il suo codice è **presente** nel file (*text*, cioè codice); la `U` accanto a `printf` e `scanf` (che compare con il nome interno `__isoc99_scanf`) significa **undefined**: il programma le usa, ma il loro codice non c'è. Sono proprio le "funzioni di libreria ancora mancanti". Il linking le aggiunge:

Terminale

```
$ gcc -o area_cerchio area_cerchio.o
$ ./area_cerchio
Inserisci il raggio del cerchio: 5
L'area del cerchio è: 78.54
```

Dato un file `.o` invece di un `.c`, gcc salta la compilazione e fa solo il linking. Separare le due fasi diventa utile quando un programma è diviso in più file: se ne cambia uno, basta ricompilare quello e rifare il linking. Lo vedremo nel capitolo 7.

La catena completa

La figura 3.4 mostra tutto il percorso di un programma C, dall'editor fino alla CPU, con gli strumenti coinvolti.

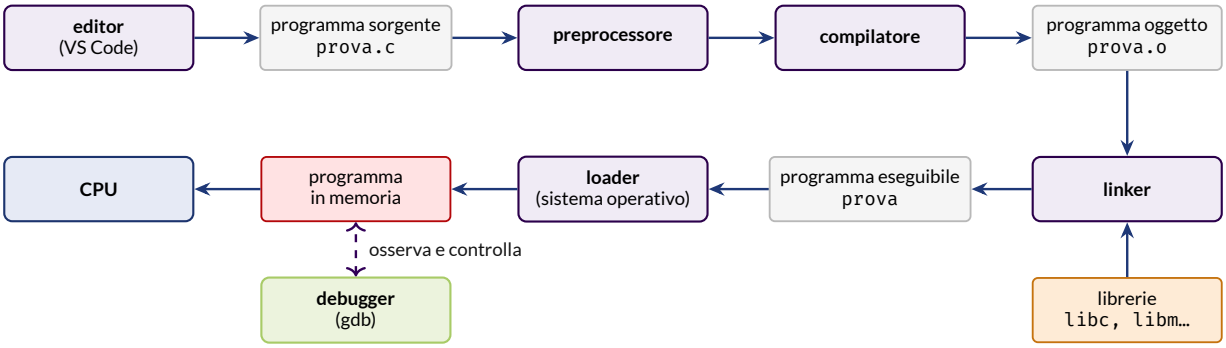


Figura 3.4: *Dal sorgente al processore: gli strumenti (in viola) e i file che producono.*

- L'**editor** serve a scrivere il sorgente.
- Il **preprocessore** lavora sul testo prima della compilazione vera e propria: esegue le righe che cominciano con `#`. `#include <stdio.h>` viene sostituita dall'intero contenuto del file `stdio.h`; `#define PI 3.14159` fa sì che ogni `PI` del programma diventi `3.14159`.
- Il **compilatore** traduce in codice macchina e produce il file oggetto; il **linker** lo unisce alle librerie.
- Il **loader** è la parte del sistema operativo che, quando scriviamo `./prova`, copia l'eseguibile in memoria e fa partire la CPU dalla sua prima istruzione (capitolo 2).
- Il **debugger** permette di eseguire il programma un passo alla volta, fermarlo e guardare il contenuto delle variabili: è lo strumento per la *messa a punto*.

Il lavoro del preprocessore si può vedere con l'opzione `-E`, che ferma gcc subito dopo questa fase e stampa il testo risultante. Il nostro programma di 15 righe diventa di oltre 800, quasi tutte provenienti da `stdio.h`; in fondo c'è il nostro codice, con `PI` già sostituito:

Terminale

```
$ gcc -E area_cerchio.c | wc -l
828
$ gcc -E area_cerchio.c | tail -6
    scanf("%f", &raggio);

    area = 3.14159 * raggio * raggio;

    printf("L'area del cerchio è: %.2f\n", area);
```

(Il simbolo `|` è una *pipe*: passa l'output di un comando a un altro. `wc -l` conta le righe, `tail -6` mostra le ultime 6.)

Librerie statiche e dinamiche

Il linking può collegare una libreria in due modi.

Con una **libreria statica**, il codice delle funzioni usate viene **copiato** dentro il file eseguibile. Le stesse funzioni possono essere usate da più programmi, ma ognuno ne contiene una copia (figura 3.5, a sinistra).

Con una **libreria dinamica**, nell'eseguibile **non** viene inserito il codice delle funzioni, ma solo un **riferimento**: il nome del file della libreria e il nome della funzione. Su Windows sono le *DLL* (*Dynamic Link Libraries*), su Linux e macOS le *shared libraries* (file `.so` e `.dylib`). Quando parte il primo programma che usa una libreria dinamica, il sistema operativo la carica in memoria; se poi parte un altro programma che usa la stessa libreria, la trova già in memoria e la **condivide** (figura 3.5, a destra).

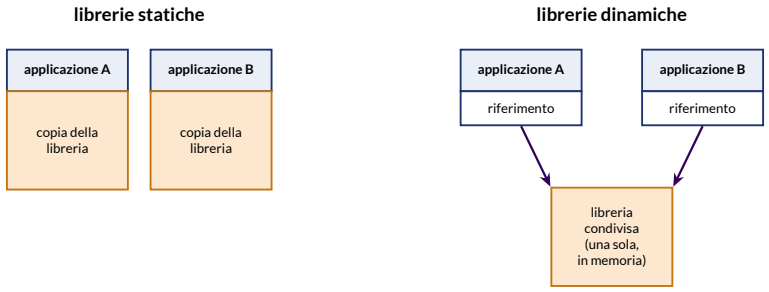


Figura 3.5: *A sinistra ogni eseguibile contiene la sua copia della libreria; a destra gli eseguibili contengono solo un riferimento a un'unica copia, condivisa.*

I vantaggi delle librerie dinamiche sono evidenti: gli eseguibili sono più piccoli, un programma parte più in fretta se la libreria è già in memoria e, se una funzione va aggiornata (una nuova versione, la correzione di un errore), basta sostituire la libreria, senza ritoccare i programmi. Il rovescio della medaglia: se la libreria manca o è danneggiata, il programma non funziona.

gcc usa le librerie dinamiche, se non si chiede diversamente. Il comando `ldd` mostra da quali dipende un eseguibile: il nostro dipende da `libc.so.6`, la libreria standard del C. Con l'opzione `-static` si ottiene invece un collegamento statico, e la differenza di dimensione parla da sola:

Terminale

```
$ ldd area_cerchio
        linux-vdso.so.1 (0x00007f1b90918000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f1b90600000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f1b9091a000)
$ gcc -static -o area_statico area_cerchio.o
$ ls -l area_cerchio area_statico
-rwxr-xr-x 1 fabio fabio  16072 ott 31 10:09 area_cerchio
-rwxr-xr-x 1 fabio fabio 915784 ott 31 10:09 area_statico
```

Lo stesso programma pesa 16 kilobyte con la libreria dinamica e quasi un megabyte con quella statica, che contiene dentro di sé tutto il codice di `printf`, `scanf` e delle funzioni che queste usano. (Su macOS `-static` non è disponibile per i programmi normali, e al posto di `ldd` si usa `otool -L`.)

GLI ALGORITMI

Che cos'è un algoritmo

Possiamo ora precisare la definizione del capitolo precedente. Un algoritmo è la **descrizione operativa** di come risolvere un problema: un **elenco finito di istruzioni elementari**, da eseguire in successione. Le singole istruzioni devono:

- richiedere un **tempo finito**;
- **non essere ambigue**;
- avere un **effetto osservabile**.

Inoltre l'algoritmo deve produrre un **risultato**, sempre lo stesso a partire dalle stesse condizioni iniziali, e le risorse che richiede devono essere **ragionevoli** per la tecnologia attuale: il tempo complessivo di esecuzione, la memoria massima necessaria, eventuali dispositivi periferici particolari.

Il termine nasce in informatica, ma si applica a qualunque procedimento. Cambiano le istruzioni elementari, non le caratteristiche:

Algoritmo per...	**un'istruzione elementare**
risolvere un'equazione di secondo grado	calcola $b^2 - 4ac$
preparare una ricetta	sbatti 4 albumi
montare un armadio IKEA	unisci i pannelli 12 e 13 con una vite C
avviare il motore di un aereo	inserisci i magneti
installare un'app sul cellulare	tocca il pulsante Installa

Sono invece istruzioni **non adeguate**:

- "calcola tutte le cifre di π": le cifre sono infinite, quindi l'istruzione non termina mai (tempo non finito);
- "aggiungi alcuni degli ingredienti": quali? quanti? (ambigua).

E un algoritmo che con gli stessi dati iniziali produce risultati diversi **non è corretto**, perché non è deterministico: un'equazione di secondo grado deve dare sempre le stesse soluzioni per gli stessi valori di a, b e c.

Quando le risorse non bastano

Un algoritmo che richiede un tempo irragionevole per la tecnologia attuale, considerati costi e benefici, **non risolve il problema**, anche se è corretto. L'esempio delle lezioni: aprire un file zip protetto da password senza conoscerla. Un algoritmo c'è, ed è semplicissimo: provare tutte le combinazioni di caratteri finché non si trova quella giusta (il cosiddetto attacco a *forza bruta*).

Supponiamo che la password sia di 8 caratteri, scelti fra i 96 caratteri stampabili del codice ASCII standard. Per il primo carattere ci sono 96 possibilità; per ciascuna, 96 per il secondo, e così via: in tutto $96^8 \approx 7{,}2 \cdot 10^{15}$ password possibili, cioè circa 7,2 milioni di miliardi. Con un computer che ne prova 34 milioni al secondo, senza mai fermarsi:

$$\frac{7{,}2 \cdot 10^{15}}{3{,}4 \cdot 10^{7} \text{ al secondo}} \approx 2{,}1 \cdot 10^{8} \text{ secondi} \approx 6{,}7 \text{ anni.}$$

Un conto da rifare

Le slide parlano di "quasi 5 anni" per provare tutte le password; il calcolo esatto dà circa **6,7 anni**. In media, però, la password giusta si trova dopo averne provate la metà, cioè in circa 3,4 anni. La conclusione non cambia: per chi ha bisogno del file oggi, l'algoritmo è inutile.

La cosa più istruttiva è che cosa succede allungando la password: ogni carattere in più **moltiplica per 96** il numero di tentativi.

lunghezza	password possibili	tempo per provarle tutte
8	$7{,}2 \cdot 10^{15}$	circa 6,7 anni
9	$6{,}9 \cdot 10^{17}$	circa 645 anni
10	$6{,}6 \cdot 10^{19}$	circa 62 000 anni

Una crescita di questo tipo si chiama *esponenziale*, ed è il motivo per cui le password lunghe sono sicure: non serve un algoritmo più furbo per violarle, servirebbe un tempo che nessuno ha.

Come si descrive un algoritmo

Prima di scrivere un programma conviene descrivere l'algoritmo in una forma intermedia, più libera di un linguaggio di programmazione. Le forme più usate sono:

- la **descrizione testuale**, in italiano;
- lo **pseudo-codice**: una descrizione testuale con un certo livello di formalismo, con parole chiave fisse (`LEGGI`, `STAMPA`, `SE`…) e una struttura simile a quella di un programma, ma senza le regole rigide di un linguaggio vero;
- il **diagramma di flusso** (*flow-chart*): una rappresentazione grafica, in cui ogni forma ha un significato (figura 3.6);
- il **linguaggio di alto livello**, cioè il programma vero e proprio.

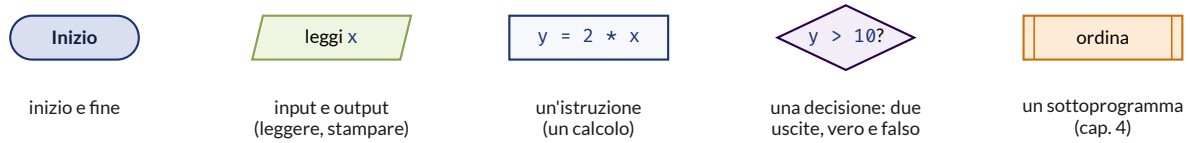


Figura 3.6: *Le forme dei diagrammi di flusso.*

Conviene anche organizzare la descrizione in tre parti: l'**input** (i dati di partenza), l'**elaborazione** (i calcoli) e l'**output** (i risultati). Vediamo i tre esempi delle lezioni, in tutte le forme. Il codice C è commentato solo a grandi linee: ogni dettaglio sarà spiegato nel capitolo 5. Qui conta vedere come ogni riga di pseudo-codice diventa una riga di programma.

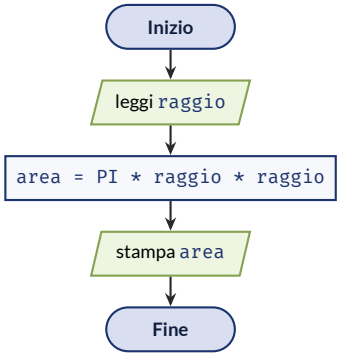
Esempio 1: l'area di un cerchio

- **Input**: il raggio del cerchio r, un numero reale positivo.
- **Elaborazione**: l'area si calcola con la formula $A = \pi \cdot r^2$, dove π è la costante che rappresenta il rapporto fra la circonferenza di un cerchio e il suo diametro.
- **Output**: il valore numerico dell'area.

In forma testuale: *acquisisci dall'utente il valore del raggio; calcola l'area con la formula $\pi \cdot r \cdot r$; visualizza il valore ottenuto.* In pseudo-codice e come diagramma di flusso:

pseudocodice

```
BEGIN
  LEGGI raggio
  AREA = PI * raggio * raggio
  STAMPA AREA
END
```



Ed ecco il programma C, con l'esecuzione accanto:

area_cerchio.c

```c
#include <stdio.h>
#define PI 3.14159

int main() {
    float raggio, area;

    printf("Inserisci il raggio del cerchio: ");
    scanf("%f", &raggio);

    area = PI * raggio * raggio;

    printf("L'area del cerchio è: %.2f\n", area);

    return 0;
}
```

Terminale

```
$ gcc -Wall -o area_cerchio area_cerchio.c
$ ./area_cerchio
Inserisci il raggio del cerchio: 5
L'area del cerchio è: 78.54
```

La corrispondenza con lo pseudo-codice è diretta:

pseudo-codice	**C**	**nota**
	`float raggio, area;`	in C bisogna dichiarare le variabili prima di usarle, dicendone il tipo (`float`: numero reale)
`LEGGI raggio`	`scanf("%f", &raggio);`	prima si stampa un messaggio che dice all'utente che cosa scrivere
`AREA = PI * raggio * raggio`	`area = PI * raggio * raggio;`	identico; `PI` è definito dalla riga `#define`
`STAMPA AREA`	`printf("…%.2f\n", area);`	`%.2f` significa: un numero reale con 2 decimali

Esempio 2: da Fahrenheit a Celsius

- **Input**: una temperatura in gradi Fahrenheit (un numero reale).
- **Elaborazione**: la formula di conversione è $Celsius = (Fahrenheit - 32) \cdot 5/9$.
- **Output**: la temperatura in gradi Celsius.

Lo pseudo-codice ha la stessa forma del primo esempio, e così il programma:

pseudocodice

```
BEGIN
  LEGGI fahrenheit
  CELSIUS = (fahrenheit - 32) * 5 / 9
  STAMPA CELSIUS
END
```

LEGGI diventa `scanf`, il calcolo resta identico, STAMPA diventa `printf`.

conversione.c

```c
#include <stdio.h>

int main() {
    float fahrenheit, celsius;

    printf("Inserisci la temperatura in Fahrenheit: ");
    scanf("%f", &fahrenheit);

    celsius = (fahrenheit - 32) * 5 / 9;

    printf("La temperatura in Celsius è: %.2f\n", celsius);

    return 0;
}
```

Terminale

```
$ gcc -Wall -o conversione conversione.c
$ ./conversione
Inserisci la temperatura in Fahrenheit: 98.6
La temperatura in Celsius è: 37.00
```

L'ordine delle operazioni conta

Matematicamente $(F - 32) \cdot 5/9$ e $5/9 \cdot (F - 32)$ sono la stessa cosa. In C no: scrivendo `celsius = 5 / 9 * (fahrenheit - 32);` il risultato è sempre **0**. Il motivo è che `5` e `9` sono numeri interi, e in C la divisione fra interi dà un intero, troncando i decimali: `5 / 9` vale 0, e 0 per qualunque cosa fa 0. Nella versione del programma, invece, `(fahrenheit - 32)` è un `float`, quindi anche le operazioni successive avvengono fra reali. È una delle trappole più comuni del C, e la ritroveremo nel capitolo 5.

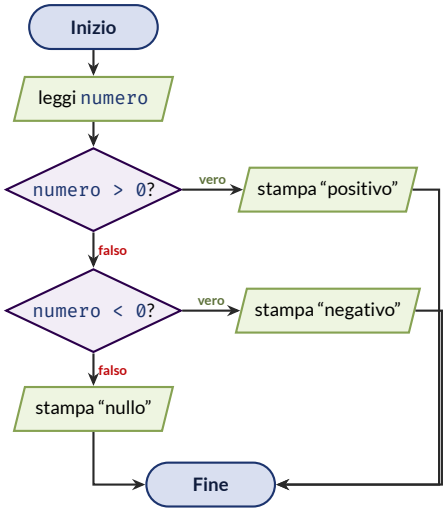
Esempio 3: il segno di un numero

Il terzo esempio introduce una **scelta**: il programma deve fare cose diverse a seconda del dato.

- **Input**: un numero intero.
- **Elaborazione**: se il numero è maggiore di zero è positivo; se è minore di zero è negativo; altrimenti è nullo.
- **Output**: un messaggio che indica se il numero è positivo, negativo o nullo.

pseudocodice

```
BEGIN
  LEGGI numero
  SE numero > 0 ALLORA
    STAMPA "Il numero è positivo"
  ALTRIMENTI SE numero < 0 ALLORA
    STAMPA "Il numero è negativo"
  ALTRIMENTI
    STAMPA "Il numero è nullo"
  END SE
END
```



Nel diagramma di flusso si vede bene che, qualunque sia il numero, viene eseguita **una sola** delle tre stampe: le strade si separano ai rombi e si ricongiungono prima della fine. In C la scelta si scrive con `if` ("se") ed `else` ("altrimenti"):

positivo.c

```c
#include <stdio.h>

int main() {
    int numero;

    printf("Inserisci un numero: ");
    scanf("%d", &numero);

    if (numero > 0) {
        printf("Il numero è positivo.\n");
    } else if (numero < 0) {
        printf("Il numero è negativo.\n");
    } else {
        printf("Il numero è zero.\n");
    }

    return 0;
}
```

Terminale

```
$ ./positivo
Inserisci un numero: 7
Il numero è positivo.
$ ./positivo
Inserisci un numero: -3
Il numero è negativo.
$ ./positivo
Inserisci un numero: 0
Il numero è zero.
```

Per verificare un programma con una scelta, lo si prova con un dato per ogni strada possibile.

Si noti l'ultima nota: un programma con tre strade va provato **tre volte**, una per strada. Un errore nascosto in un ramo che non si prova mai resta lì, in attesa del giorno sbagliato.

In sintesi

Un algoritmo è un elenco finito di istruzioni elementari, non ambigue, ciascuna eseguibile in un tempo finito, che produce sempre lo stesso risultato dagli stessi dati con risorse ragionevoli. Lo si descrive in italiano, in pseudo-codice, con un diagramma di flusso e infine in un linguaggio di alto livello. Il programma sorgente va tradotto: un **interprete** lo fa un'istruzione alla volta a ogni esecuzione, un **compilatore** una volta sola. Per il C la traduzione passa per il preprocessore, il compilatore (che produce i file oggetto) e il linker (che li unisce alle librerie, statiche o dinamiche); al momento dell'esecuzione il loader carica il programma in memoria.

Riferimenti

- **Deitel & Deitel**, *Il linguaggio C*: capitoli 1 e 3 (ambiente di sviluppo, algoritmi, pseudo-codice).
- **GCC — Overall Options**: le opzioni `-E`, `-S`, `-c` e `-o`.
  `https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html`

La programmazione strutturata

Prima di scrivere programmi in C conviene sapere da dove viene il linguaggio e, soprattutto, quale **disciplina** segue: la programmazione strutturata. È un'idea semplice e potentissima: *ogni* programma si può costruire combinando tre soli schemi, la sequenza, la selezione e l'iterazione. Questo capitolo li presenta con i diagrammi di flusso, prima con esempi della vita di tutti i giorni e poi con problemi di informatica; nei capitoli successivi ciascuno diventerà un'istruzione del C.

Il linguaggio C

Da dove viene

Il C è stato creato da **Dennis Ritchie**, ai Bell Labs, nel 1972, come linguaggio per scrivere il sistema operativo **Unix**. La prima versione di Unix, opera di **Ken Thompson**, era scritta in assembly e in un linguaggio chiamato **B**. Il B aveva dei limiti seri:

- non era **tipizzato**: conosceva un solo tipo di dato, la *parola macchina* (il gruppo di bit su cui lavora il processore), e non distingueva fra numeri interi, caratteri o indirizzi;
- faceva un uso pesante di puntatori e indirizzi di memoria.

Il C nasce per superare questi limiti: è un'evoluzione del B che incorpora i **tipi** (ricordate gli "occhiali" del capitolo 2: è il tipo a dire come interpretare i bit), e rispetta i canoni della programmazione strutturata. Nel tempo il linguaggio si è evoluto (figura 4.1):

- il **C tradizionale** (*Traditional C*), descritto nel 1978 nel libro di Kernighan e Ritchie e diffuso all'inizio degli anni '80, introduce fra l'altro i tipi `void` ed `enum`;
- l'**ANSI C**, della fine degli anni '80, aggiunge il tipo `void *`, i **prototipi** delle funzioni e una definizione del linguaggio molto più precisa.

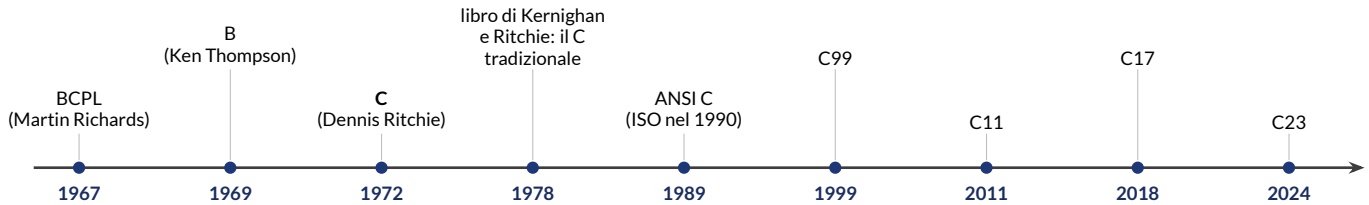


Figura 4.1: *Le tappe del C, dai suoi antenati alle versioni dello standard.*

Una precisazione storica

Le slide datano il B al 1967. Il 1967 è l'anno del suo antenato, il *BCPL* di Martin Richards; il B fu sviluppato da Ken Thompson intorno al 1969, partendo proprio dal BCPL.

Lo standard

L'**ANSI** (*American National Standards Institute*) è l'ente statunitense che definisce gli standard per molti tipi di sistemi, compresi i linguaggi di programmazione. Lo standard ANSI del C (1989) fu approvato nel 1990 dall'**ISO** (*International Organization for Standardization*), e l'ANSI C divenne così lo standard internazionale.

Uno standard specifica il **formalismo** del linguaggio (quali testi sono programmi C corretti, cioè la sua *sintassi*) e la sua **semantica** (che cosa significa ciascun costrutto, cioè che cosa deve fare il programma). Lo scopo è triplice:

- **portabilità**: lo stesso programma si compila ed esegue allo stesso modo con compilatori diversi, su macchine diverse;
- **affidabilità**: il comportamento è definito, non lasciato all'interpretazione di chi scrive il compilatore;
- **manutenibilità**: chiunque conosca lo standard può leggere e modificare il programma.

Dopo il 1990 lo standard è stato aggiornato più volte (C99, C11, C17, C23). Le versioni recenti di gcc seguono per default una di queste, con alcune estensioni proprie; i programmi di questa dispensa funzionano con tutte.

LA PROGRAMMAZIONE STRUTTURATA

Il problema: gli spaghetti

Nel capitolo 2 abbiamo visto che in assembly ripetizioni e scelte si ottengono con i **salti**: "vai all'istruzione all'indirizzo tale". I primi linguaggi di alto livello ereditarono questa abitudine con l'istruzione `GOTO` ("vai a"), che permette di saltare in qualunque punto del programma. In un programma grande, pieno di salti in avanti e all'indietro, seguire l'esecuzione diventa come seguire uno spaghetto nel piatto: è il cosiddetto **spaghetti code** (figura 4.2, a sinistra), di bassa qualità e leggibilità.

Nel 1968–1969 **Edsger W. Dijkstra** criticò duramente l'uso del `GOTO` e propose di regolamentare e standardizzare le metodologie di programmazione: è la **programmazione strutturata**. L'obiettivo è rendere più facile la **lettura**, la **modifica** e la **manutenzione** dei programmi.

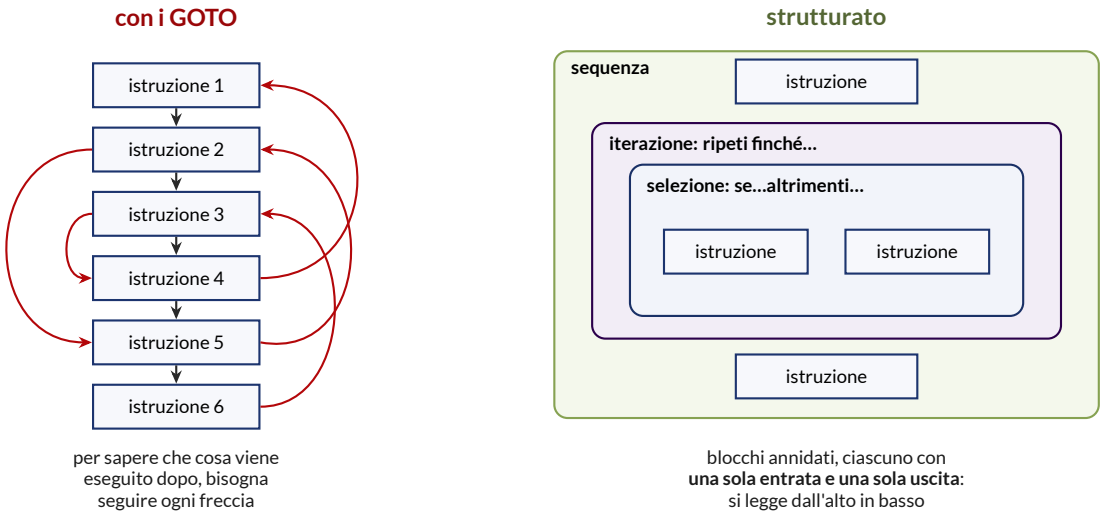


Figura 4.2: *Lo stesso tipo di logica scritto con salti arbitrari (a sinistra) e con le strutture di controllo (a destra).*

Si può davvero fare a meno dei salti arbitrari? La risposta è un teorema, dimostrato nel 1966 da due informatici italiani, **Corrado Böhm** e **Giuseppe Jacopini**.

Teorema di Böhm-Jacopini

Qualsiasi programma scritto usando il `GOTO` può essere riscritto senza, a patto di avere a disposizione tre tipi di **strutture di controllo**: la **sequenza**, la **selezione** e l'**iterazione**.

Le tre strutture bastano, quindi, per *qualunque* algoritmo. Hanno una caratteristica comune decisiva: ciascuna ha **una sola entrata e una sola uscita**. Per questo si possono incastrare l'una dentro l'altra come scatole (figura 4.2, a destra), e un programma si legge dall'alto in basso, capendo ogni blocco senza dover guardare il resto.

Che fine ha fatto il GOTO?

Il C ha ancora un'istruzione `goto`, per ragioni storiche e per pochi casi molto particolari. In questo corso **non si usa**: tutto ciò che serve si scrive con le strutture di controllo.

LE TRE STRUTTURE DI CONTROLLO

Le strutture di controllo del flusso sono dunque tre (figura 4.3). Vediamole nel dettaglio, ciascuna con un problema del mondo reale e un problema di informatica.

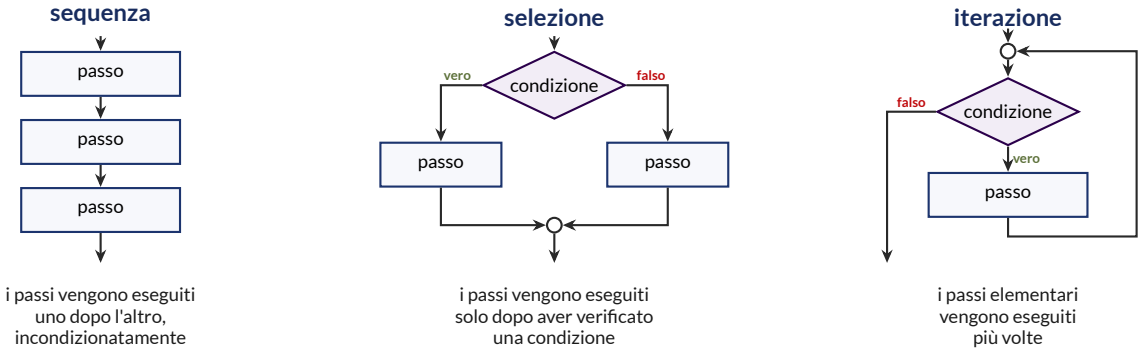


Figura 4.3: *Le tre strutture di controllo del flusso.*

La sequenza

La **sequenza** è la struttura più banale: i passi vengono eseguiti **uno dopo l'altro**, incondizionatamente, nell'ordine in cui sono scritti. Ma l'ordine conta: per alzarsi dal letto bisogna prima mettersi a sedere, e per stampare un risultato bisogna prima averlo calcolato (figura 4.4).

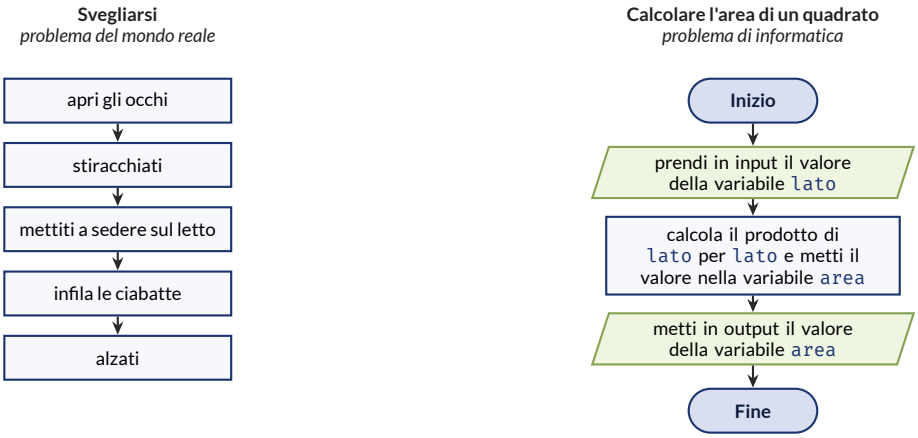


Figura 4.4: *Due sequenze.*

Nel quadrato compare per la prima volta la parola **variabile**: un contenitore con un nome (`lato`, `area`) in cui si mette un valore. È una cella di memoria (o un gruppo di celle) a cui diamo un nome, così da non doverci occupare del suo indirizzo.

La selezione a una via

La **selezione** esegue dei passi **solo dopo aver verificato una condizione**. Ne esistono tre tipi; il più semplice è la **selezione a una via**: se la condizione è vera si eseguono dei passi, se è falsa non si fa niente di particolare e si prosegue (figura 4.5).

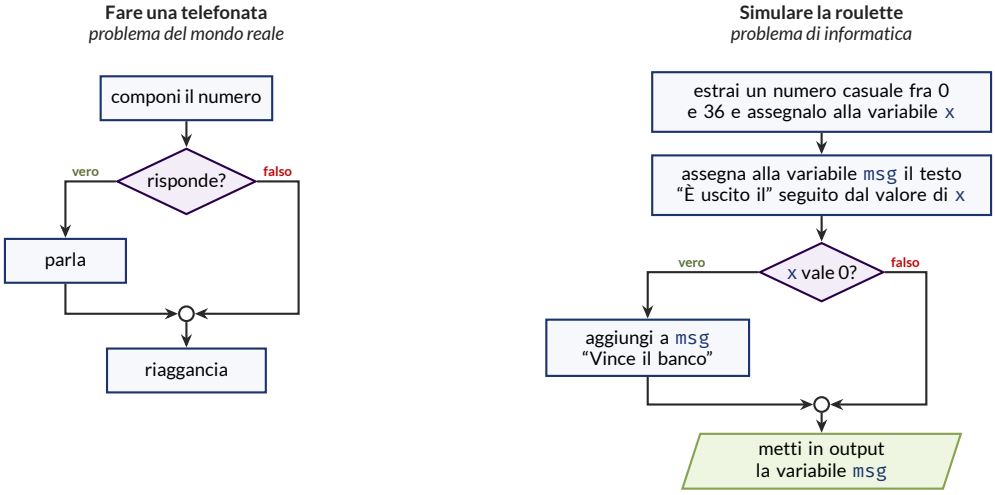


Figura 4.5: *Due selezioni a una via: il ramo "falso" non contiene passi.*

Il pallino dove le strade si ricongiungono è importante: qualunque cosa succeda alla domanda, dopo si riaggancia. La selezione ha una sola entrata (la domanda) e una sola uscita (il pallino), proprio come una singola istruzione.

La selezione a due vie

La **selezione a due vie** prevede delle operazioni da svolgere sia nel caso in cui la condizione risulti **vera**, sia nel caso in cui risulti **falsa**: viene eseguito sempre *uno* dei due rami, mai entrambi, mai nessuno (figura 4.6).

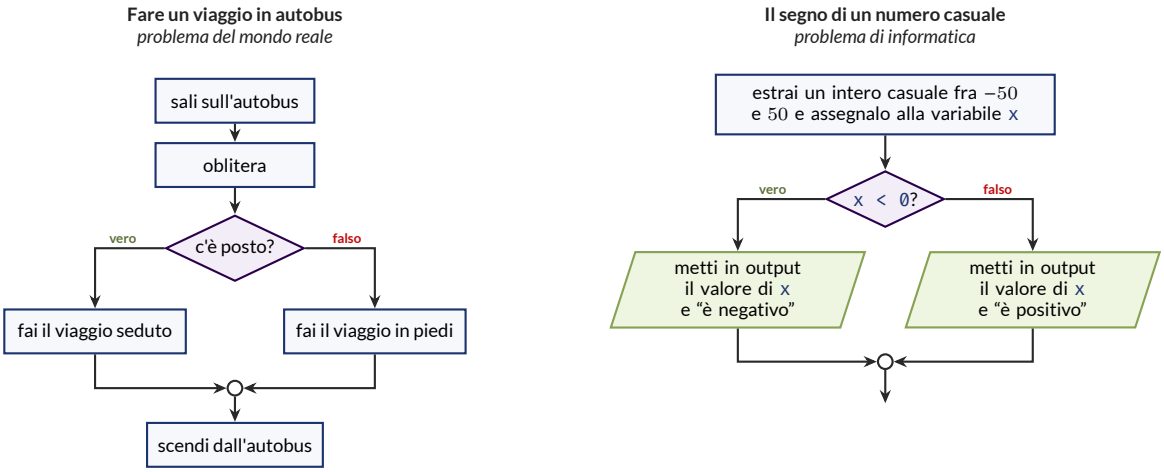


Figura 4.6: *Due selezioni a due vie.*

E se esce lo zero?

L'algoritmo del segno, così com'è, ha un caso particolare: se il numero estratto è 0, la condizione `x < 0` è falsa e viene stampato "0 è positivo". Lo zero però non è né positivo né negativo. Per trattarlo correttamente servono **tre** strade, come nell'esempio 3 del capitolo 3. È un buon esempio di come si verifica un algoritmo: si prova ogni ramo, e si cercano i **casi limite** (qui, il confine fra le due strade).

La **selezione a n vie** prevede operazioni diverse in **diversi casi**, a seconda del valore di qualcosa: invece di una domanda con risposta vero o falso, c'è una domanda con molte risposte possibili, ciascuna con la sua strada (figura 4.7).

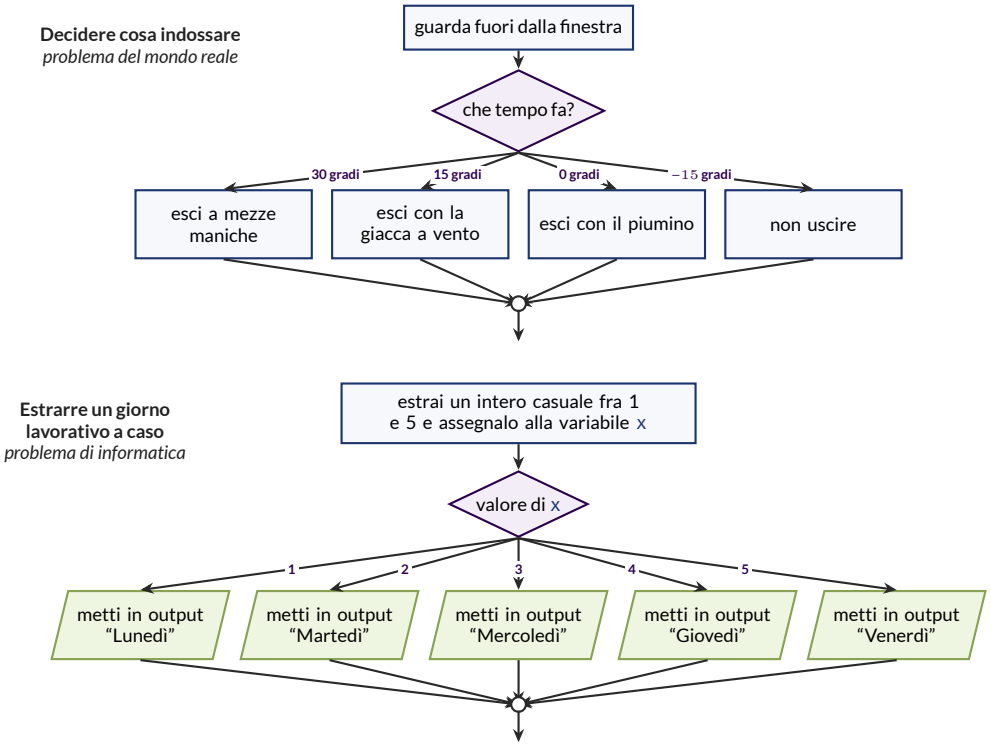


Figura 4.7: *Due selezioni a n vie: le etichette sulle frecce sono i valori possibili.*

Si potrebbe ottenere lo stesso risultato con una catena di selezioni a due vie ("x vale 1? altrimenti, vale 2? altrimenti…"), ma la selezione a n vie dice in modo più diretto che **una sola** domanda decide fra molte strade. In C esistono entrambe le forme (capitolo 9).

Il concetto di **ciclo** (o *iterazione*) è stato introdotto per alleggerire situazioni come questa: dipingere le statuette dei sette nani. Scritto come sequenza, l'algoritmo ripete sette volte lo stesso passo (figura 4.8, a sinistra). Una volta isolata l'istruzione da eseguire **più volte**, si può scrivere un algoritmo che la esegue semplicemente quante volte serve (a destra).

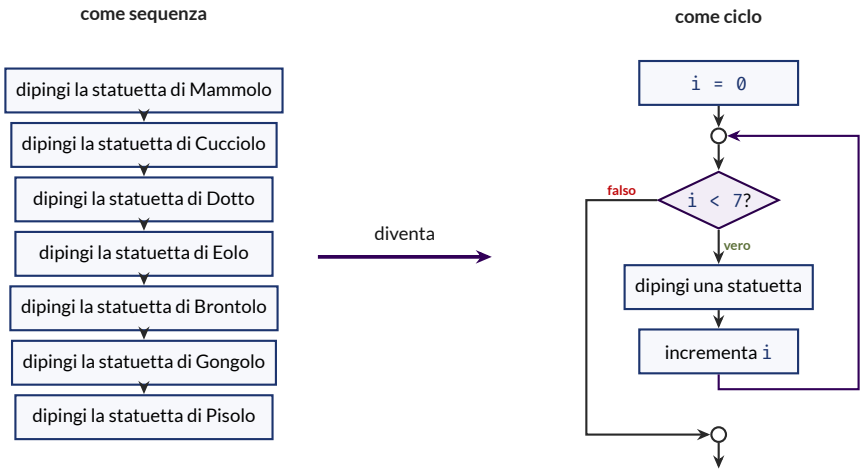


Figura 4.8: *Dipingere le statuette dei sette nani: la ripetizione viene isolata in un ciclo.*

Per sapere quando fermarsi, il ciclo usa una variabile, `i`, che **conta** quante volte il corpo del ciclo è già stato eseguito. Una variabile di questo tipo si chiama **contatore** e per tradizione si chiama `i`. Esistono però due grandi famiglie di cicli, e la difficoltà sta nel capire in quale circostanza conviene usare l'una o l'altra:

- il **ciclo su contatore**, quando si sa *quante volte* ripetere;
- il **ciclo su condizione**, quando non lo si sa, ma si sa *quando* continuare e quando smettere.

Il ciclo su contatore

Nel ciclo su contatore la variabile contatore viene **inizializzata** prima di fare il ciclo, **testata** prima di ripetere il ciclo e **modificata** ogni volta che il ciclo è stato eseguito (figura 4.9, a sinistra). Dimenticare uno dei tre passi è un errore classico: senza inizializzazione il conto parte da un valore a caso, senza modifica il test resta sempre vero e il ciclo non finisce mai.

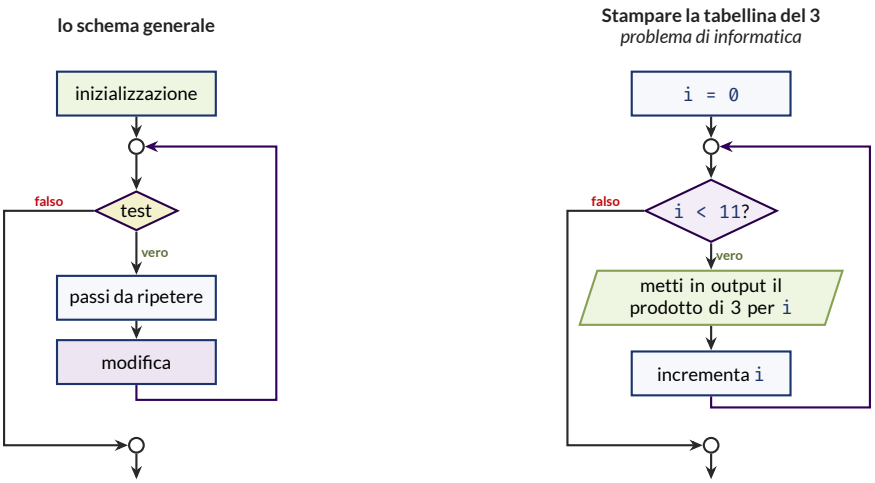


Figura 4.9: *Il ciclo su contatore: lo schema e un esempio.*

Seguiamo l'esecuzione della tabellina, scrivendo a ogni giro il valore di `i`, l'esito del test e l'output. È la tecnica più utile per capire un ciclo (e per trovarne gli errori): **simularlo a mano**, una riga per giro.

valore di `i`	0	1	2	3	4	5	6	7	8	9	10	11
`i < 11`?	vero	vero	vero	vero	vero	vero	vero	vero	vero	vero	vero	**falso**
output	0	3	6	9	12	15	18	21	24	27	30	—

Il corpo viene eseguito 11 volte (per `i` da 0 a 10), e il test viene valutato 12 volte: l'ultima, con `i` che vale 11, è quella che fa uscire dal ciclo.

Il ciclo su condizione

Se non sappiamo a priori **quante** volte dobbiamo eseguire il ciclo, sicuramente sappiamo **quando** eseguirlo e quando no: serve allora un **ciclo su condizione**, che si ripete finché una condizione è vera. Anche qui ci sono due casi, a seconda di quando si fa il test:

- prima eseguiamo il ciclo e poi facciamo il test (**condizione in coda**);
- oppure prima facciamo il test e poi eseguiamo il ciclo (**condizione in testa**).

La differenza sembra piccola, ma non lo è: con la condizione in coda il corpo del ciclo viene eseguito **almeno una volta**; con la condizione in testa può essere eseguito **zero volte**.

Condizione in coda

Si pensi a mangiare un piatto di spaghetti: si mangia una forchettata, poi ci si chiede se ce n'è ancora; se sì, si mangia un'altra forchettata, e così via. Chiedersi "ce n'è ancora?" ha senso solo **dopo** aver cominciato (figura 4.10, a sinistra).

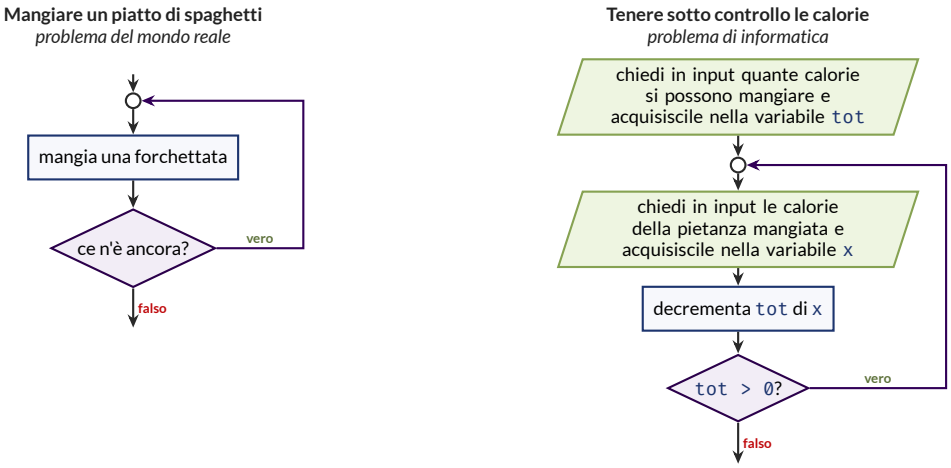


Figura 4.10: *Cicli con la condizione in coda: il corpo viene eseguito almeno una volta.*

Il ciclo su condizione con verifica in coda si presta a due casi:

- quando ha senso porsi la domanda **solo dopo** aver già eseguito una prima volta il ciclo, come per gli spaghetti e le calorie;
- quando c'è un obiettivo da raggiungere mediante uno o più **tentativi**: si tenta, e si ripete finché il tentativo non riesce (figura 4.11).

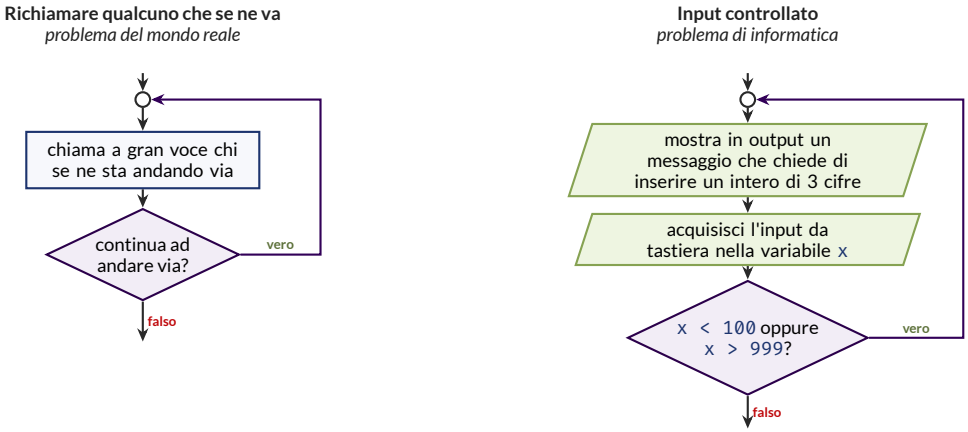


Figura 4.11: *Cicli con la condizione in coda per raggiungere un obiettivo con più tentativi.*

L'input controllato è un caso che ritroveremo spesso: un programma non può fidarsi di ciò che scrive l'utente. Chiede un numero di tre cifre e, finché il numero ricevuto è fuori dall'intervallo da 100 a 999, lo richiede.

Quando invece ha senso porsi la domanda **prima** di eseguire la prima volta il ciclo, allora si pone la verifica della condizione in testa: è il **ciclo su condizione con verifica in testa**. Per alzare tutte le tapparelle di casa ci si chiede prima se ce ne sono ancora da alzare: potrebbero essere già tutte aperte, e allora non si fa nulla (figura 4.12). Si noti che il ciclo su contatore visto prima è un caso particolare di questo schema, con il test in testa.

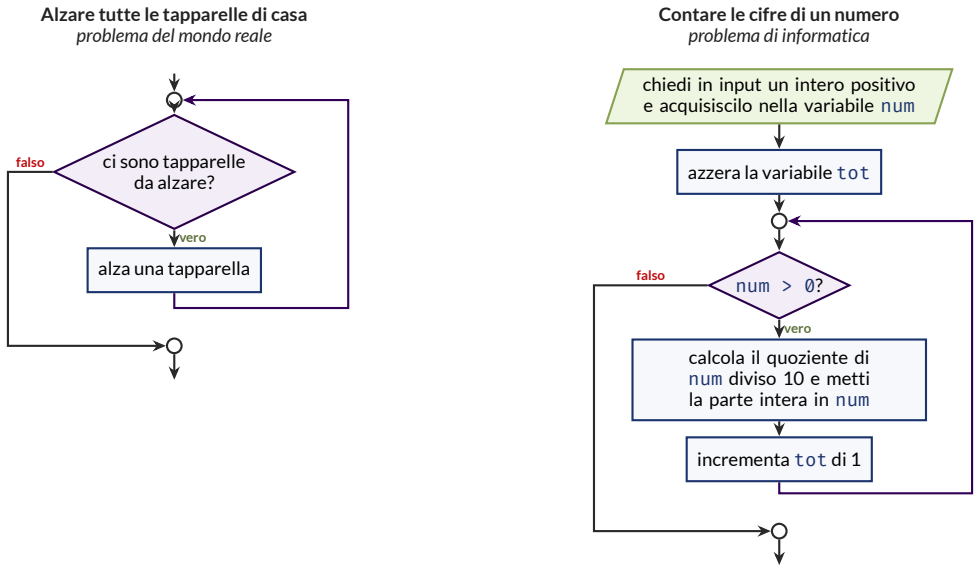


Figura 4.12: *Cicli con la condizione in testa: il corpo può non essere mai eseguito.*

Perché il secondo algoritmo conta le cifre? Dividere per 10 un numero intero, tenendo solo la parte intera, **toglie l'ultima cifra**: da 4072 si passa a 407. Quindi le cifre sono tante quante le divisioni che servono per arrivare a 0. Simuliamo con `num` che vale 4072:

giro	`num` prima	`num > 0`?	dopo il giro
1	4072	vero	`num` = 407, `tot` = 1
2	407	vero	`num` = 40, `tot` = 2
3	40	vero	`num` = 4, `tot` = 3
4	4	vero	`num` = 0, `tot` = 4
5	0	**falso**	si esce: le cifre sono **4**

L'algoritmo chiede un intero **positivo**, ed è importante: con `num` uguale a 0 il test sarebbe subito falso e `tot` resterebbe 0, mentre il numero 0 ha una cifra. Ancora una volta, i casi limite.

Combinare le strutture

Le strutture di controllo possono essere **combinate** tra loro: ovunque un diagramma contiene un passo, quel passo può essere sostituito da un'intera struttura, perché ogni struttura ha una sola entrata e una sola uscita, esattamente come un passo.

Riprendiamo la telefonata. In realtà, il passo "componi il numero" non è esattamente un passo elementare, in quanto è scomponibile: si digita una cifra, e si continua finché ci sono ancora cifre (un ciclo con la condizione in coda). Sostituendo il passo con il suo ciclo si ottiene il diagramma della figura 4.13.

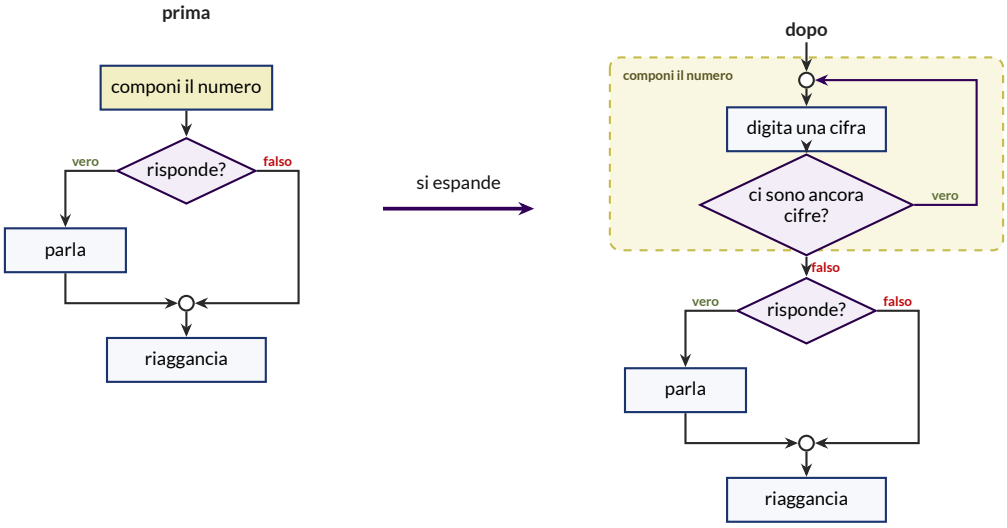


Figura 4.13: *Il passo "componi il numero" sostituito dal ciclo che lo realizza: una selezione che segue un ciclo.*

Questo principio, sostituire un passo con la struttura che lo realizza, è alla base dell'approccio **top-down**.

Lo sviluppo top-down

Dividere il problema

Per **sviluppo top-down** ("dall'alto verso il basso") si intende un approccio alla soluzione di un problema che prevede la sua **scomposizione in sotto-problemi**, più piccoli e quindi più semplici da risolvere. Ogni sotto-problema, a sua volta, si può scomporre ancora, finché non si arriva a passi abbastanza semplici da scrivere direttamente.

Ogni sotto-problema sarà risolto con la realizzazione di un **sotto-programma** (in C, una *funzione*), e uno dei passi elementari del nostro algoritmo potrebbe essere proprio l'esecuzione di uno dei sotto-programmi. Lo svolgimento di un compito ben preciso (anche piccolo) può avvenire in autonomia, ma spesso il sotto-programma ha bisogno di scambiare dei dati con chi lo usa. Per esempio, "componi il numero" richiede che gli si fornisca il numero da comporre.

- I dati che un sotto-programma riceve da chi lo invoca si chiamano **parametri**.
- Il dato che un sotto-programma fornisce in output a chi lo ha invocato si chiama **valore restituito**.

Un'analogia: in un libro di cucina, la ricetta delle lasagne dice "prepara un litro di besciamella (vedi pagina 112)". La ricetta della besciamella è un sotto-programma; "un litro" è il parametro, e la besciamella pronta è il valore restituito. La ricetta delle lasagne non deve sapere come si fa la besciamella: le basta sapere che cosa darle e che cosa riceverà.

Come si invoca un sotto-programma

Supponiamo di avere un programma (che questo sia il programma principale o un sotto-programma non ha alcuna importanza) e un sotto-programma qualsiasi. Quando progettiamo il sotto-programma dobbiamo, fra l'altro, indicare i parametri di cui necessita: sono i **parametri formali**, i nomi con cui il sotto-programma si riferisce ai dati ricevuti. Supponiamo che siano `a`, `b`, `c` e `d`.

L'**invocazione** (da parte del programma chiamante) avviene indicando il nome del sotto-programma, fornendo i parametri con cui lavorare (i **parametri attuali**) e indicando che cosa fare del valore restituito. La figura 4.14 segue l'invocazione passo per passo.

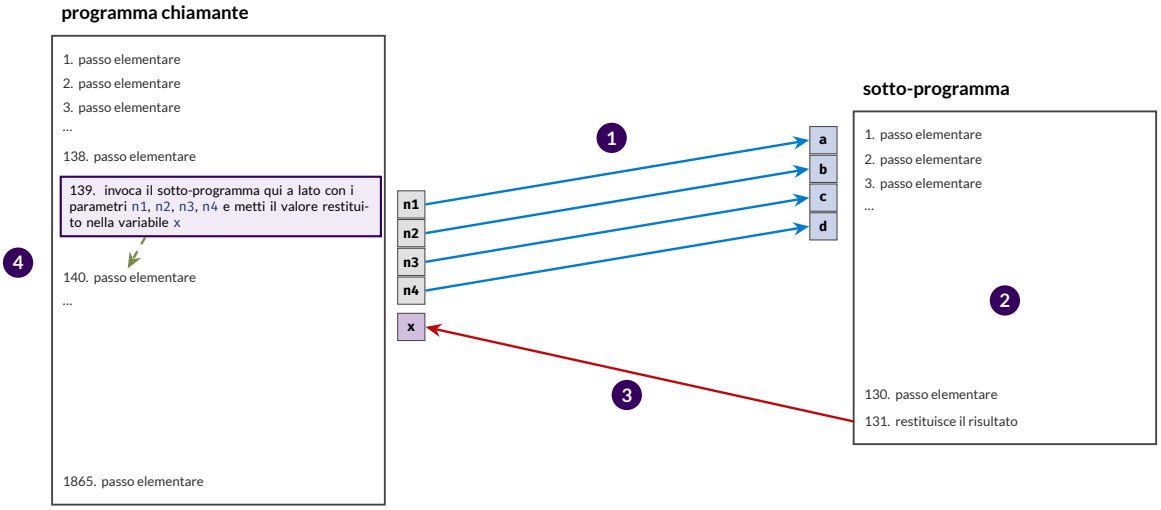


Figura 4.14: *L'invocazione di un sotto-programma con quattro parametri.*

1. All'invocazione, il controllo del flusso passa al sotto-programma, e ogni parametro formale riceve una **copia** del valore del parametro attuale corrispondente: in `a` viene copiato il valore di `n1`, in `b` quello di `n2`, e così via.
2. Il sotto-programma esegue i suoi passi, lavorando con `a`, `b`, `c` e `d`.
3. Quando ha eseguito la sua ultima istruzione, fornisce al programma chiamante il suo **valore di restituzione**, che finisce in `x`…
4. …e l'esecuzione riprende dall'istruzione successiva a quella dell'invocazione (la 140) del programma chiamante.

Due osservazioni importanti. I nomi dei parametri attuali e formali **possono anche non coincidere** (qui `n1` e `a`): il sotto-programma è scritto una volta sola e deve funzionare con qualunque dato gli venga passato, quindi usa nomi suoi. Invece devono **assolutamente coincidere** il numero, il **tipo** di ogni parametro e l'**ordine**: il primo parametro attuale va nel primo formale, il secondo nel secondo. Scambiare l'ordine non produce un errore visibile, produce un risultato sbagliato.

I TIPI DI ISTRUZIONE

Riassumendo, ogni passo elementare di un algoritmo sarà un'istruzione di uno di questi tipi: **input**, **output**, **assegnazione** (mettere un valore in una variabile), **selezione**, **iterazione**, **invocazione di funzione**. Selezione e iterazione si dividono a loro volta come nella figura 4.15.

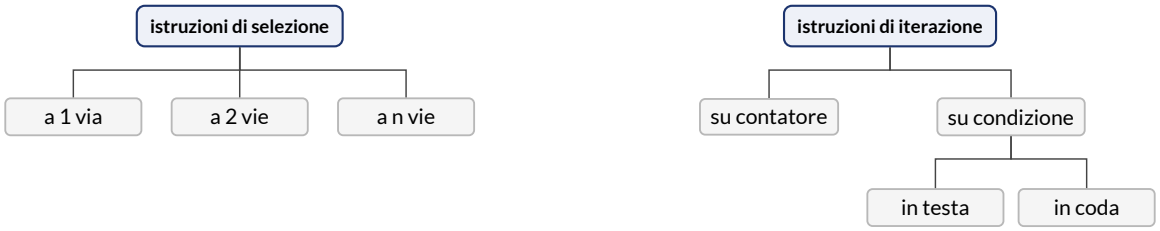


Figura 4.15: *I tipi di istruzioni di selezione e di iterazione.*

Ognuno di questi concetti diventerà un costrutto preciso del C. La tabella seguente anticipa la corrispondenza, per avere fin d'ora una mappa del percorso: non serve ricordarla, la ritroveremo pezzo per pezzo.

istruzione		**in C**	**capitolo**
input		`scanf, getchar`	5, 8
output		`printf, putchar`	5, 8
assegnazione		`x = valore;`	5
selezione	a 1 via	`if`	5, 9
	a 2 vie	`if … else`	5, 9
	a n vie	`switch`	9
iterazione	su contatore	`for`	5, 9
	su condizione, in testa	`while`	5, 9
	su condizione, in coda	`do … while`	9
invocazione di funzione		`nome(parametri)`	10

In sintesi

Il C nasce nel 1972 per scrivere Unix, come evoluzione tipizzata del B; lo standard (ANSI e ISO, dal 1989–90) ne garantisce portabilità, affidabilità e manutenibilità. La programmazione strutturata rinuncia ai salti arbitrari: per il teorema di Böhm-Jacopini bastano **sequenza**, **selezione** (a una, due o n vie) e **iterazione** (su contatore, o su condizione con il test in testa o in coda). Ogni struttura ha una sola entrata e una sola uscita, quindi le strutture si annidano come scatole; sostituire un passo con la struttura che lo realizza è l'idea dello sviluppo **top-down**, in cui i sotto-problemi diventano sotto-programmi che ricevono **parametri** (copiati, nell'ordine) e restituiscono un **valore**.

RIFERIMENTI

- **Kernighan & Ritchie**, *Il linguaggio C*: prefazione e introduzione (storia del linguaggio).
- **E. W. Dijkstra**, *Go To Statement Considered Harmful*, Communications of the ACM, 1968.
- **C. Böhm, G. Jacopini**, *Flow diagrams, Turing machines and languages with only two formation rules*, Communications of the ACM, 1966.

Primi programmi in C

Questo capitolo è una panoramica del C attraverso una serie di programmi di esempio, ciascuno spiegato elemento per elemento. Si introducono brevemente i costrutti principali (l'output, le variabili, l'input, le scelte, i cicli, le funzioni) per avere subito un'idea di come è fatto un programma vero; ogni argomento verrà poi approfondito nei capitoli successivi. Il consiglio è uno solo: **scrivere ed eseguire ogni esempio**, poi modificarlo e osservare che cosa cambia.

Il primo programma, riga per riga

Scrivere, compilare, eseguire

Un programma deve comunicare per essere utile: produce sempre un output di qualche tipo. Il primo esempio stampa una frase sullo schermo. Lo scriviamo nel file `sea.c`, lo compiliamo e lo eseguiamo, come abbiamo imparato nel capitolo 1:

sea.c

```c
#include <stdio.h>

int main(void)
{
    printf("from sea to shining C\n");
    return 0;
}
```

Terminale

```
$ gcc sea.c
$ ./a.out
from sea to shining C
```

Se non ci sono errori, gcc crea il file eseguibile `a.out`.

Sono sette righe, ma ognuna ha un ruolo preciso. La figura 5.1 le numera, e l'elenco che la segue le spiega.

```c
#include <stdio.h>                      1
int main(void)                          2
{                                       3
    printf("from sea to shining C\n");   4
    return 0;                           5
}                                       3
```

Figura 5.1: *Anatomia del primo programma: i numeri rimandano all'elenco seguente.*

1. **Direttiva per il preprocessore**: copia qui il contenuto del file di intestazione `stdio.h`, che descrive `printf`.
2. **Testa della funzione** `main`, da cui comincia l'esecuzione: restituisce un `int` e non riceve parametri (`void`).
3. Le **graffe** delimitano il **corpo** della funzione.
4. Un'**istruzione**: chiama `printf` con un argomento, una **stringa costante**; `\n` va a capo; il `;` chiude l'istruzione.
5. Restituisce 0 al sistema operativo: "tutto bene".

Il preprocessore e `#include`

Prima della compilazione vera e propria viene eseguito, automaticamente, il **preprocessore** (capitolo 3). Le righe che cominciano con `#` sono **direttive** per il preprocessore. La direttiva `#include <stdio.h>` chiede di includere in quel punto una copia del file di intestazione (*header file*) `stdio.h`, che contiene le informazioni necessarie per usare le funzioni di input e output, come `printf`.

`#include` non include la libreria

A lezione il docente è categorico: guai a dire che `#include` "include la libreria". Include un **file di testo**, l'header, che **descrive** le funzioni (come si chiamano, che parametri vogliono, che cosa restituiscono). Il **codice** delle funzioni sta nella libreria, che viene collegata dal **linker**, dopo. Sono due cose diverse, che intervengono in due fasi diverse: lo vedremo in modo molto concreto con la funzione `pow` più avanti in questo capitolo.

La funzione `main`

`int main(void)` è la prima riga della definizione di una **funzione** chiamata `main`. Le parentesi dopo il nome dicono che `main` è una funzione; `int` e `void` sono parole speciali che danno informazioni al compilatore: `void` fra le parentesi significa che la funzione **non ha parametri**, `int` davanti al nome significa che **restituisce un intero**.

Ogni programma C ha **una** funzione `main`, e l'esecuzione comincia da lei. Il corpo della funzione è racchiuso fra le graffe `{` e `}`, che in C servono in generale a raggruppare istruzioni.

`printf`, le stringhe e il punto e virgola

`printf` è una funzione che stampa sullo schermo (più precisamente sullo *standard output*, come vedremo). Non fa parte del linguaggio: appartiene alla **libreria standard**, un insieme di librerie che forniscono al programmatore funzioni pronte; gli header file permettono di usarle, e `stdio.h` fornisce le informazioni per `printf`.

Il testo fra virgolette, `"from sea to shining C\n"`, è una **stringa costante**: è l'**argomento** di `printf` e stabilisce che cosa viene stampato. Al suo interno, `\n` (una barra rovesciata seguita da `n`) è un **singolo carattere**, il *newline*: fa avanzare il cursore all'inizio della riga successiva. L'intera riga è un'**istruzione**, e in C le istruzioni **terminano con un punto e virgola**.

`return` e il valore restituito al sistema

`return 0;` termina la funzione `main` e restituisce il valore 0 al sistema operativo. Per convenzione 0 significa "tutto è andato bene", un valore diverso da 0 segnala un problema. Il valore non si vede nell'output, ma la shell lo conserva, e lo si può leggere con il comando `echo $?` subito dopo l'esecuzione:

Terminale

```
$ ./a.out
from sea to shining C
$ echo $?
0
```

`return` **termina** la funzione: ciò che viene dopo non è eseguito. Il file `sea.c` distribuito con le lezioni ne dà una prova, perché dopo il `return` contiene un'altra `printf`:

sea.c

```c
#include <stdio.h>

int main()
{
    printf("from sea to shining C\n");

    return 0;
    printf("Pippo\n");
}
```

Terminale

```
$ gcc -o sea sea.c
$ ./sea
from sea to shining C
```

"Pippo" non viene mai stampato: l'esecuzione esce da `main` al `return`.

(Qui `main` è scritto con le parentesi vuote, `main()`: per una definizione di funzione è equivalente a `main(void)`; in questa dispensa useremo la forma con `void`, più esplicita.)

L'OUTPUT CON `printf`

Il cursore

`printf` scrive sullo schermo da sinistra a destra e dall'alto verso il basso, come si scrive su un quaderno; va a capo **solo** quando incontra il carattere `\n`. Ogni chiamata di `printf` comincia da dove la precedente ha lasciato il cursore. Per questo il programma seguente, diverso dal primo, stampa **lo stesso** output:

```c
printf("from sea to ");
printf("shining ");
printf("C\n");
```

Terminale

```
from sea to shining C
```

Se vogliamo scrivere su tre righe dobbiamo inserire dei `\n`; e se ne possono mettere più di uno nella stessa stringa:

```c
printf("from sea to\n");
printf("shining\nC\n");
```

Terminale

```
from sea to
shining
C
```

Disegnare con i caratteri

Per essere leggibile, l'output va spaziato opportunamente sullo schermo. Il programma seguente stampa la stessa frase in una cornice di asterischi, preceduta e seguita da dieci righe vuote (ciascun `\n` iniziale produce una riga vuota):

sea_box.c

```c
#include <stdio.h>

int main(void)
{
    printf("\n\n\n\n\n\n\n\n\n\n");
    printf("      ************************\n");
    printf("      *  from sea            *\n");
    printf("      *  to shining C        *\n");
    printf("      ************************\n");
    printf("\n\n\n\n\n\n\n\n\n\n");
    return 0;
}
```

Terminale

```
$ ./sea_box


      ************************
      *  from sea            *
      *  to shining C        *
      ************************


$
```

Qui sono mostrate solo due delle dieci righe vuote sopra e sotto.

VARIABILI, ESPRESSIONI E ASSEGNAZIONE

Il problema della maratona

Il secondo esempio converte una distanza: la lunghezza di una maratona è di 26 miglia e 385 iarde, e vogliamo sapere quanti chilometri sono. Servono due regole: in un miglio ci sono 1760 iarde, e un miglio è lungo 1,609 km.

marathon.c

```c
/* The distance of a marathon in kilometers. */

#include <stdio.h>

int main(void)
{
    int     miles, yards;
    float   kilometers;

    miles = 26;
    yards = 385;
    kilometers = 1.609 * (miles + yards / 1760.0);
    printf("\nA marathon is %f kilometers.\n\n", kilometers);
    return 0;
}
```

Terminale

```
$ gcc -o marathon marathon.c
$ ./marathon

A marathon is 42.185970 kilometers.
```

L'opzione `-o` sceglie il nome del file eseguibile.

I commenti

Tutto ciò che è compreso fra `/*` e `*/` è un **commento**: viene ignorato dal compilatore (anzi, eliminato già dal preprocessore) e serve solo a chi legge. Si possono anche scrivere commenti su una sola riga, che cominciano con `//` e arrivano fino alla fine della riga. Un commento all'inizio del file spiega lo scopo del programma; vi si possono aggiungere informazioni sugli autori e sulle versioni.

Le dichiarazioni

```c
int     miles, yards;
float   kilometers;
```

Queste righe sono **dichiarazioni**. `int` e `float` sono **parole chiave** del C che indicano due **tipi di dato fondamentali**: la prima riga dice al compilatore che `miles` e `yards` sono **variabili** di tipo intero, che assumono valori interi; la seconda che `kilometers` è una variabile di tipo `float`, che assume valori reali. Anche le dichiarazioni terminano con un punto e virgola.

Dichiarare una variabile significa chiedere al compilatore di riservarle dello spazio in memoria e di ricordarne il tipo. In C **ogni variabile va dichiarata prima di essere usata**.

L'assegnazione

`miles = 26;` è un'**istruzione di assegnazione**: il simbolo `=` è l'**operatore di assegnazione**, `26` è una costante intera, e l'effetto è che il valore 26 viene scritto nella variabile `miles` (figura 5.2). Allo stesso modo 385 viene assegnato a `yards`.

= non è l'uguale della matematica

In matematica $x = x + 1$ è un'equazione impossibile. In C `x = x + 1;` è perfettamente sensato e significa: **calcola** il valore dell'espressione a destra (il valore attuale di `x` più 1) e **scrivilo** nella variabile a sinistra. L'assegnazione va letta da destra a sinistra: "`x` riceve…".

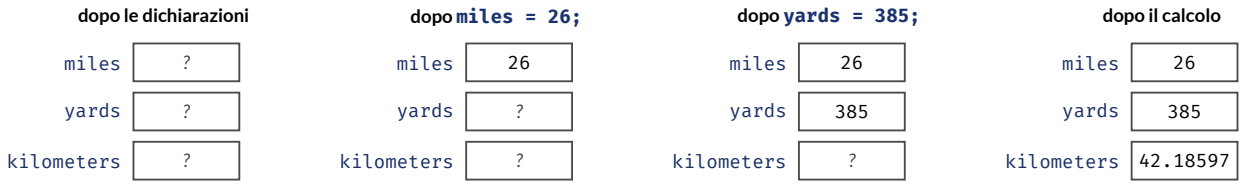


Figura 5.2: *Le variabili della maratona, istruzione dopo istruzione. Appena dichiarata, una variabile contiene un valore a caso ("?"): finché non le si assegna qualcosa, non va usata.*

Come si calcola un'espressione

`kilometers = 1.609 * (miles + yards / 1760.0);` assegna un valore calcolato da un'**espressione**. Le regole sono quelle dell'aritmetica: si calcola prima ciò che sta fra parentesi, e dentro le parentesi la divisione ha la precedenza sulla somma. Quindi prima `yards / 1760.0`, poi il risultato viene sommato a `miles`, e infine moltiplicato per `1.609` (figura 5.3).

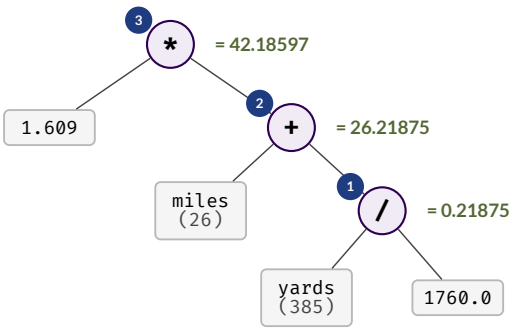


Figura 5.3: *L'albero dell'espressione: si calcola dal basso verso l'alto, nell'ordine indicato dai numeri.*

`1.609` e `1760.0` sono **costanti reali** (in C, di tipo `double`): per questo `yards / 1760.0` è una divisione fra reali, con risultato reale. Infine `printf("…%f…", kilometers)` stampa il valore: la stringa di controllo contiene una **specifica di conversione**, `%f`, che dice "qui va stampato un numero reale"; la variabile corrispondente, il secondo argomento, deve essere di quel tipo.

La trappola della divisione intera

Basta togliere il `.0` alla costante per cambiare completamente il risultato:

```
kilometers = 1.609 * (miles + yards / 1760);
```

```
Terminale
A marathon is 41.834000 kilometers.
```

Ora `1760` è una costante **intera**, e `yards / 1760` è una divisione fra interi: in C il risultato di una divisione fra interi è un intero, e il resto si perde. 385 diviso 1760 fa 0 con resto 385, quindi `yards / 1760` vale **0** e i 385 metri e rotti… spariscono: $1{,}609 \cdot 26 = 41{,}834$. Il compilatore non segnala niente, perché non c'è nessun errore di linguaggio: c'è un errore di ragionamento.

Parole chiave e nomi

Parole come `int` e `float` sono **parole chiave** (*keywords*): sono **riservate** e non possono essere usate come nomi di variabili. Una variabile non può chiamarsi `int`, né `return`. Per lo stesso motivo di chiarezza, non si usano come nomi di variabili i nomi delle funzioni: niente variabili chiamate `printf` o `main`. L'elenco completo delle parole chiave e le regole per i nomi sono nel capitolo 6.

Il preprocessore: `#define` e `#include`

Le costanti simboliche

Il preprocessore scandisce tutto il file e individua le direttive, cioè le righe che cominciano con `#`. Due delle più usate sono `#define` e `#include`. Consideriamo le righe

```
#define LIMIT 100
#define PI    3.14159
```

Il preprocessore sostituirà nel file sorgente tutte le occorrenze di `LIMIT` con `100` e tutte quelle di `PI` con `3.14159`, tranne quelle che compaiono fra virgolette (nelle stringhe) e nei commenti. Un `#define` può stare ovunque nel file e ha effetto solo sulle righe successive; di solito si mette all'inizio, e per convenzione il nome si scrive in maiuscolo.

Le **costanti simboliche** rendono il programma più **leggibile** (`LIMIT` dice che cosa rappresenta, `100` no) e più facile e sicuro da **modificare**. Se la velocità della luce, circa 299 792,458 km/s, compare in cento punti di un programma, scrivendo

```
#define C 299792.458   /* velocità della luce in km/s */
```

e usando `C` ovunque, una stima più accurata si inserisce cambiando una sola riga, senza rischiare di dimenticare qualche occorrenza.

I file di intestazione

Una riga come

```
#include "my_file.h"
```

copia il contenuto di `my_file.h` in quel punto del sorgente. Anche `#include` può stare ovunque, ma tipicamente si mette all'inizio. I file che si includono sono gli **header file** e per convenzione hanno estensione `.h`. Il sistema C ne fornisce alcuni, come `stdio.h`, `stdlib.h`, `string.h`, `math.h`. La differenza fra le due forme:

- `#include <stdio.h>`, con le parentesi angolari, cerca il file fra gli header del sistema;
- `#include "my_file.h"`, con le virgolette, lo cerca prima nella cartella del sorgente: si usa per i nostri header.

Ogni volta che si usa una funzione di input/output come `printf` si deve includere `stdio.h`, che ne contiene il **prototipo**: la descrizione che serve al compilatore per controllare che la chiamata sia corretta. L'header serve alla **compilazione**; il codice della funzione arriva dalla libreria, durante il **linking**.

Header e libreria: la prova

La funzione `pow(x, y)` calcola x^y; il suo prototipo sta in `math.h`, il suo codice nella **libreria matematica**, che su Linux è separata dalla libreria standard e va chiesta esplicitamente al linker con l'opzione `-lm` (*library m*):

potenza.c

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    double base, esponente;

    printf("Base ed esponente: ");
    scanf("%lf%lf", &base, &esponente);
    printf("%g elevato a %g fa %g\n",
           base, esponente, pow(base, esponente));
    return 0;
}
```

Terminale

```
$ gcc -Wall -o potenza potenza.c
/usr/bin/ld: /tmp/ccVCbC9C.o: in function `main':
potenza.c:(.text+0x61): undefined reference to `pow'
collect2: error: ld returned 1 exit status
$ gcc -Wall -o potenza potenza.c -lm
$ ./potenza
Base ed esponente: 2 10
2 elevato a 10 fa 1024
```

Il primo tentativo fallisce, ma si guardi **chi** protesta: `ld`, il linker, non il compilatore. Grazie a `math.h` il compilatore sapeva tutto ciò che gli serviva e ha prodotto il file oggetto; è il linker che non ha trovato il **codice** di `pow` (*undefined reference*). Con `-lm` gli si indica la libreria, e tutto funziona. (Su macOS la libreria matematica è già compresa in quella di sistema e `-lm` non serve, ma non fa danni.)

Un esempio con le costanti: Pacific Sea

Il programma

Pacific Sea è il nome dato alla parte di oceano Pacifico visibile dal campus di Santa Cruz, sulla baia di Monterey. Il programma converte la sua area da chilometri quadrati in acri, miglia quadrate, piedi quadrati e pollici quadrati. Le costanti stanno in un header scritto da noi:

pacific_sea.h

```c
#include <stdio.h>

#define   AREA                        2337
#define   SQ_MILES_PER_SQ_KILOMETER   0.3861021585424458
#define   SQ_FEET_PER_SQ_MILE         (5280 * 5280)
#define   SQ_INCHES_PER_SQ_FOOT       144
#define   ACRES_PER_SQ_MILE           640
```

pacific_sea.c

```c
/* Measuring the Pacific Sea. */

#include "pacific_sea.h"

int main(void)
{
    const int   pacific_sea = AREA;     /* in sq kilometers */
    double      acres, sq_miles, sq_feet, sq_inches;

    printf("\nThe Pacific Sea covers an area");
    printf(" of %d square kilometers.\n", pacific_sea);
    sq_miles = SQ_MILES_PER_SQ_KILOMETER * pacific_sea;
    sq_feet = SQ_FEET_PER_SQ_MILE * sq_miles;
    sq_inches = SQ_INCHES_PER_SQ_FOOT * sq_feet;
    acres = ACRES_PER_SQ_MILE * sq_miles;
    printf("In other units of measure this is:\n\n");
    printf("%22.7e acres\n", acres);
    printf("%22.7e square miles\n", sq_miles);
    printf("%22.7e square feet\n", sq_feet);
    printf("%22.7e square inches\n\n", sq_inches);
    return 0;
}
```

Terminale

```
$ ./pacific_sea

The Pacific Sea covers an area of 2337 square kilometers.
In other units of measure this is:

         5.7748528e+05 acres
         9.0232074e+02 square miles
         2.5155259e+10 square feet
         3.6223572e+12 square inches
```

Gli `#include` a catena

La prima riga include una copia di `pacific_sea.h`, e siccome questo contiene a sua volta `#include <stdio.h>`, il preprocessore include anche una copia di `stdio.h`, che serve per `printf` (figura 5.4). Il risultato è un unico testo, che è quello che il compilatore riceve.

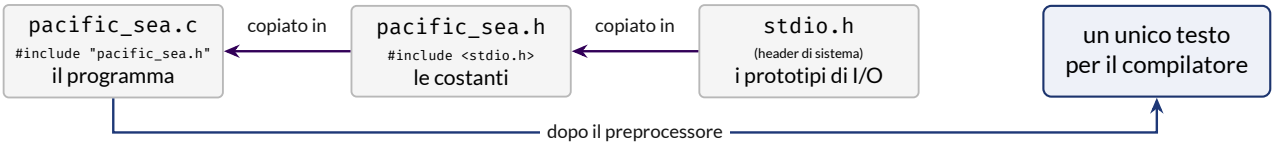


Figura 5.4: *Il preprocessore risolve gli* `#include` *uno dentro l'altro.*

Le costanti dell'header

Le costanti simboliche dell'header (una reale, le altre intere) rendono il codice più chiaro: se ci fosse una nuova stima dell'area basterebbe cambiare `AREA`, e le altre danno un nome significativo ai numeri. Su `SQ_FEET_PER_SQ_MILE` vale la pena fermarsi:

- scrivere `(5280 * 5280)` invece di `27878400` è più chiaro: dice che in un miglio ci sono 5280 piedi, e che un miglio quadrato ne contiene il quadrato. Non si perde efficienza: il compilatore calcola da sé le espressioni fatte solo di costanti;
- le parentesi non sono sintatticamente necessarie, ma sono un'ottima abitudine. Il preprocessore fa una sostituzione di **testo**, senza capire nulla: se fosse `#define SQ_FEET_PER_SQ_MILE 5280 * 5280`, l'espressione `x / SQ_FEET_PER_SQ_MILE` diventerebbe `x / 5280 * 5280`, cioè (divisione e moltiplicazione si eseguono da sinistra a destra) $(x/5280) \cdot 5280$, che vale di nuovo circa x. Con le parentesi diventa `x / (5280 * 5280)`, come si voleva.

`const`

`const int pacific_sea = AREA;` è la dichiarazione di una variabile `pacific_sea` di tipo `int`, **inizializzata** (cioè con un valore assegnato già nella dichiarazione) a 2337: prima della compilazione il preprocessore ha sostituito `AREA` con il suo valore. La parola chiave `const` qualifica il tipo: la variabile riceve il suo valore all'inizializzazione, ma **non può essere modificata**. Il compilatore lo fa rispettare:

costante.c

```c
#include <stdio.h>

int main(void)
{
    const int pacific_sea = 2337;

    pacific_sea = 3000;
    printf("%d\n", pacific_sea);
    return 0;
}
```

Terminale

```
$ gcc -Wall -o costante costante.c
costante.c: In function ‘main’:
costante.c:7:17: error: assignment of read-only
    variable ‘pacific_sea’
    7 |     pacific_sea = 3000;
      |                 ^
```

I tipi reali e la notazione scientifica

`acres`, `sq_miles` e gli altri sono dichiarati `double`. In C i tipi reali (detti *floating point*, "a virgola mobile") sono tre: `float` (tipicamente 6 cifre significative), `double` (tipicamente 15) e `long double` (almeno quanto `double`; sui PC di solito 18). I valori dipendono dall'implementazione; il capitolo 8 li approfondisce.

L'istruzione `printf("%22.7e acres\n", acres);` stampa la riga `5.7748528e+05 acres`. Il numero è in **notazione scientifica**: `e+05` significa "per 10^5", quindi il valore è $5{,}7748528 \cdot 10^5 = 577\,485{,}28$ acri. La conversione `%e` chiede proprio questo formato (detto *e-format*), e la forma `%`m`.`n`e`, con m e n interi positivi, indica che il numero occupa in tutto m caratteri (allineato a destra) con n cifre dopo il punto: qui 22 caratteri e 7 cifre decimali. Per questo nell'output i numeri sono preceduti da spazi e allineati in colonna.

printf E scanf

Funzioni di input e output formattato

`printf` si usa per l'output, `scanf` per l'input; la `f` finale sta per *formatted* (formattato). Non fanno parte del linguaggio, ma della libreria standard, che viene collegata implicitamente alla fine della compilazione. Il programmatore deve comunque dichiarare al compilatore le funzioni che usa: il meccanismo dell'ANSI C è il **prototipo**, e i prototipi delle funzioni della libreria standard sono negli header standard (per `printf` e `scanf`, `stdio.h`).

Entrambe accettano una lista di argomenti:

- la **stringa di controllo**, una stringa che contiene le **specifiche di conversione**: ciascuna è il carattere `%` seguito da un **carattere di conversione** (per esempio `%d` per un intero);
- gli altri argomenti, **uno per ogni conversione** della stringa di controllo, nello stesso ordine.

Le conversioni di `printf`

Per stampare le prime tre lettere dell'alfabeto ci sono molti modi: senza conversioni, con tre conversioni per caratteri, con una conversione per stringhe. Si può anche decidere quanto spazio dare a ciascuna conversione: `%3c` significa "un carattere, in un campo largo 3", allineato a destra.

```c
printf("abc\n");
printf("%c%c%c\n", 'a', 'b', 'c');
printf("%s\n", "abc");
printf("%c%3c%5c\n", 'A', 'B', 'C');
```

Terminale

```
abc
abc
abc
A  B    C
```

Nel codice, `'a'` fra apici singoli è una **costante carattere** (un solo carattere), mentre `"abc"` fra virgolette è una **stringa**. I caratteri di conversione principali di `printf` sono:

conversione	stampa	esempio
`%c`	un carattere	`printf("%c", 'A')` → `A`
`%d`	un intero, in decimale	`printf("%d", 42)` → `42`
`%e`	un reale, in notazione scientifica	`printf("%e", 314159.0)` → `3.141590e+05`
`%f`	un reale	`printf("%f", 3.14159)` → `3.141590`
`%g`	un reale, nel più corto fra `%e` e `%f`	`printf("%g", 314159.0)` → `314159`
`%s`	una stringa	`printf("%s", "abc")` → `abc`

Fra `%` e il carattere di conversione si possono indicare l'ampiezza del campo e, per i reali, il numero di cifre decimali (la *precisione*); un segno meno allinea a sinistra. Le parentesi quadre nell'esempio servono solo a mostrare dove comincia e finisce il campo:

```c
printf("[%d] [%5d] [%-5d]\n", 42, 42, 42);
printf("[%f] [%.2f] [%10.3f]\n", 3.14159, 3.14159, 3.14159);
printf("[%e] [%g] [%g]\n", 314159.0, 314159.0, 0.0000314);
```

Terminale

```
[42] [   42] [42   ]
[3.141590] [3.14] [     3.142]
[3.141590e+05] [314159] [3.14e-05]
```

Si noti che `%f` stampa sempre 6 cifre decimali, se non si indica altro, e che la precisione **arrotonda**: 3.14159 con 3 decimali diventa 3.142.

`scanf` e l'operatore &

`scanf` è l'analoga di `printf` per l'input. Il primo parametro è la stringa di controllo; l'input è sempre una sequenza di caratteri (ciò che si digita sulla tastiera), e le conversioni specificano come i caratteri devono essere interpretati. I parametri successivi, però, non sono variabili ma **indirizzi**: gli indirizzi delle variabili in cui scrivere i valori letti.

```
1 scanf("%d", &x);
```

`&` è l'**operatore di indirizzo**: `&x` è l'indirizzo della variabile `x`, il "numero civico" della sua cella di memoria (figura 5.5). Perché serve? Perché `scanf` deve **scrivere** nella variabile, e per farlo deve sapere dove si trova. Scrivendo `scanf("%d", x)` le si passerebbe il *valore* di `x` (un numero qualunque, ancora indefinito), e `scanf` non saprebbe dove mettere il dato letto; il motivo profondo lo vedremo alla fine del capitolo, con il passaggio dei parametri per valore.



Figura 5.5: `scanf` *riceve l'indirizzo di* `x` *e scrive lì il valore letto (l'indirizzo 1000 è solo un esempio).*

Le conversioni principali di `scanf`:

conversione	legge e scrive in una variabile di tipo
`%c`	`char` (un carattere)
`%d`	`int` (un intero decimale)
`%f`	`float`
`%lf`	`double`
`%s`	una stringa (capitolo 12)

`%f` o `%lf`?

Con `printf`, `%f` va bene sia per i `float` sia per i `double`. Con `scanf` no: per un `double` serve `%lf`. Il motivo: `scanf` riceve solo un indirizzo, e dalla conversione deve capire quanti byte scrivere a quell'indirizzo (4 per un `float`, 8 per un `double`). Con la conversione sbagliata scrive un numero di byte sbagliato, e il valore letto è senza senso.

Un esempio: echo

Il programma seguente legge tre caratteri, un intero, un `float` e un `double`, e li ristampa (fa da "eco"):

```c
#include <stdio.h>

int main(void)
{
    char     c1, c2, c3;
    int      i;
    float    x;
    double   y;

    printf("\n%s\n%s", "Input three characters ", "an int, a float, and a double \n");
    scanf("%c%c%c%d%f%lf", &c1, &c2, &c3, &i, &x, &y);
    printf("\nHere is the data that you typed in:\n");
    printf("%3c%3c%3c\n%17d%17f%17f\n\n", c1, c2, c3, i, x, y);
    return 0;
}
```

Proviamolo due volte, con input quasi uguali: nel secondo c'è solo uno spazio fra `B` e `C`.

Terminale

```
$ ./echo

Input three characters
an int, a float, and a double
ABC 3 55 77.7

Here is the data that you typed in:
  A  B  C
              3        55.000000        77.700000
```

Terminale

```
$ ./echo

Input three characters
an int, a float, and a double
AB C 3 55 77.7

Here is the data that you typed in:
  A  B
     1386527472         0.000000         0.000000
```

Il secondo risultato è disastroso, e il motivo è una regola di `scanf`: quando legge **numeri** salta gli spazi che li precedono, ma quando legge **caratteri** no, perché lo spazio *è* un carattere. La figura 5.6 mostra quali caratteri "mangia" ciascuna conversione.

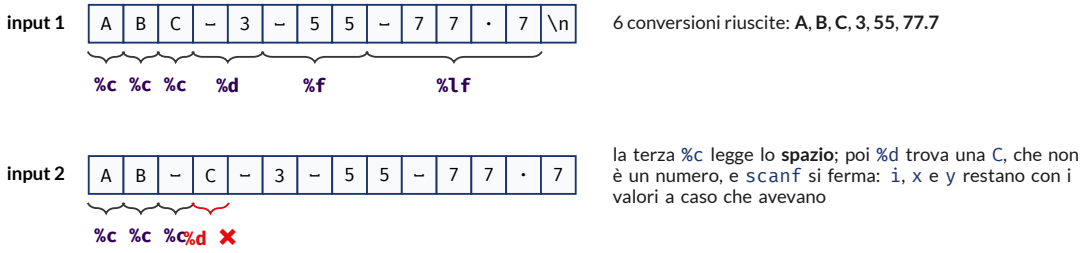


Figura 5.6: *I caratteri dell'input (␣ indica uno spazio) e la conversione che li consuma.*

Nel secondo caso `i`, `x` e `y` non ricevono nulla, e `printf` stampa ciò che contenevano: valori casuali, diversi da un computer all'altro (e da un'esecuzione all'altra). Come accorgersene dal programma? `scanf` **restituisce il numero di conversioni riuscite**. Salvandolo in una variabile e stampandolo (è il programma `echo_conversions.c` delle lezioni) si ottiene 6 nel primo caso e 3 nel secondo:

```c
int conversions;
...
conversions = scanf("%c%c%c%d%f%lf", &c1, &c2, &c3, &i, &x, &y)
    ;
...
printf("Correct conversions. %d\n", conversions);
```

Terminale

```
Correct conversions. 6
...
Correct conversions. 3
```

Controllare il valore restituito da `scanf` è il modo corretto di verificare che l'input sia stato letto: lo useremo fra poco.

Il controllo del flusso

Vero e falso in C

Le istruzioni sono normalmente eseguite in sequenza, ma la maggior parte degli algoritmi richiede di alterare il flusso sequenziale (capitolo 4). Il C offre `if` e `if-else` per scegliere fra azioni alternative, `while` e `for` per ripetere. Tutti richiedono di valutare un'espressione **logica**, cioè vera o falsa, e in C la regola è semplicissima:

- il **falso** è rappresentato dal valore **0**;
- **ogni valore diverso da zero** è interpretato come **vero**.

`if`

La forma generale è

```c
if (espressione) {
    istruzioni_if;
}
istruzione_successiva;
```

Se `espressione` è diversa da zero (vera) si eseguono `istruzioni_if` e poi `istruzione_successiva`; se vale 0 (falsa) si salta direttamente a `istruzione_successiva`. È la selezione a una via del capitolo precedente. Un esempio:

```c
a = 1;
if (b == 3) {
    a = 5;
}
printf("%d", a);
```

Il simbolo `==` è l'operatore di **uguaglianza**: verifica se `b` è uguale a 3. Se è vero, si assegna 5 ad `a` e viene stampato **5**; se è falso si esegue subito la stampa, e viene stampato **1**.

`==` e `=` sono due cose diverse

`==` **confronta**, `=` **assegna**. Scrivere `if (b = 3)` non è un errore per il compilatore: assegna 3 a `b`, e siccome il valore dell'assegnazione è 3 (diverso da zero, cioè vero), il ramo viene eseguito **sempre**. È uno degli errori più frequenti in C; il capitolo 9 ci torna in dettaglio. Compilando con `-Wall`, gcc avvisa.

Fra le graffe può esserci una sequenza di istruzioni; con una sola istruzione le graffe si possono omettere, ma conviene **metterle sempre**: se in seguito si aggiunge un'istruzione al blocco, senza graffe resterebbe fuori dall'`if`, con un errore difficile da vedere.

if-else

```c
if (espressione) {
    istruzioni_if;
} else {
    istruzioni_else;
}
```

Se `espressione` è diversa da zero si esegue il primo blocco, se è zero il secondo: è la selezione a due vie. Uno dei due blocchi viene sempre eseguito, mai entrambi.

Nell'esempio seguente, se `cnt` vale 0 il programma stampa $2 + 3 + 5 = 10$, altrimenti $-1 - 2 - 3 = -6$. A destra la variante con `if (cnt)`: non c'è nessun confronto, la decisione si basa solo sul valore di `cnt` (0 è falso, il resto vero), e i risultati si invertono.

```c
if (cnt == 0) {
    a = 2;
    b = 3;
    c = 5;
} else {
    a = -1;
    b = -2;
    c = -3;
}
printf("%d", a + b + c);
```

```c
if (cnt) {
    a = 2;
    b = 3;
    c = 5;
} else {
    a = -1;
    b = -2;
    c = -3;
}
printf("%d", a + b + c);
```

	con `cnt == 0`	con `cnt`
se `cnt` vale 0	stampa 10	stampa -6
se `cnt` vale, per esempio, 4	stampa -6	stampa 10

sum.c

```c
#include <stdio.h>

int main(void)
{
    int   i = 1, sum = 0;

    while (i <= 5) {
        sum = sum + i;
        i = i + 1;
    }
    printf("sum = %d\n", sum);
    return 0;
}
```

Terminale

```
$ ./sum
sum = 15
```

Il programma somma i numeri da 1 a 5. Nella dichiarazione, `int i = 1, sum = 0;` le due variabili vengono anche **inizializzate**. `<=` significa "minore o uguale".

Il `while` verifica se `i <= 5`: se la condizione è vera esegue le istruzioni del blocco fino alla `}`, poi il controllo **torna all'inizio** del ciclo per ripetere il test. L'iterazione continua finché il test non fallisce. È il ciclo con la condizione in testa del capitolo 4. Simuliamolo:

giro	`i <= 5`?	`sum` dopo il giro	`i` dopo il giro
1	$1 \le 5$, vero	$0 + 1 = 1$	2
2	$2 \le 5$, vero	$1 + 2 = 3$	3
3	$3 \le 5$, vero	$3 + 3 = 6$	4
4	$4 \le 5$, vero	$6 + 4 = 10$	5
5	$5 \le 5$, vero	$10 + 5 = 15$	6
6	$6 \le 5$, **falso**	si esce: `sum` vale 15	

La forma generale è

```
while (espressione) {
    istruzioni_while;
}
istruzioni;
```

Se `espressione` è diversa da zero si eseguono `istruzioni_while` e poi si verifica di nuovo `espressione`; il processo continua finché `espressione` vale 0. Perché il ciclo finisca, è **necessaria** un'istruzione dentro il ciclo che possa cambiare il valore dell'espressione (qui `i = i + 1`): senza, il test resterebbe vero per sempre.

Assegnazioni abbreviate e incremento

Istruzioni come `sum = sum + i;` sono così frequenti che il C ha una forma abbreviata: `sum += i;`. In generale, un'istruzione della forma `variabile op= espressione;` (con `op` uno fra `+`, `-`, `*`, `/`) equivale a `variabile = variabile op espressione;`.

Per aggiungere o togliere 1 ci sono gli **operatori di incremento e decremento**:

forma	come istruzione a sé, equivale a
`++i` e `i++`	`i = i + 1;`
`--i` e `i--`	`i = i - 1;`

Il ciclo di `sum.c` si scrive quindi anche così:

```
while (i <= 5) {
    sum += i;
    ++i;
}
```

++ dentro le espressioni

Come istruzioni isolate, `++i` e `i++` fanno la stessa cosa. Usati **dentro** un'espressione più grande (per esempio `a = i++;`) si comportano in modo diverso, e il loro funzionamento è più complicato: lo vedremo con calma nel capitolo 6. Per ora li useremo solo da soli.

`for`

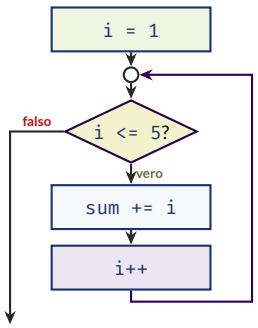
Il ciclo `for` ha la forma

```
for (expr1; expr2; expr3) {
    istruzioni_for;
}
```

```
expr1;
while (expr2) {
    istruzioni_for;
    expr3;
}
```

ed è equivalente al `while` a destra. Tipicamente `expr1` fa un'assegnazione iniziale, `expr2` un test, `expr3` incrementa una variabile: sono i tre passi del ciclo su contatore del capitolo 4 (inizializzazione, test, modifica), riuniti in un'unica riga. La somma da 1 a 5 diventa:

```
for (i = 1; i <= 5; i++) {
    sum += i;
}
```



UN ESEMPIO COMPLETO: `running_sum`

Il programma

Il programma seguente legge una sequenza di numeri e, per ciascuno, stampa una riga di tabella con il conteggio, il numero letto, il minimo, il massimo, la somma e la media dei numeri letti fino a quel momento.

running_sum.c

```c
/* Compute the minimum, maximum, sum, and average. */

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int      i;
    double   x, min, max, sum, avg;

    if (scanf("%lf", &x) != 1) {
        printf("No data found - bye!\n");
        exit(1);
    }
    min = max = sum = avg = x;
    printf("%5s%9s%9s%9s%12s%12s\n%5s%9s%9s%9s%12s%12s\n\n",
           "Count", "Item", "Min", "Max", "Sum", "Average",
           "-----", "----", "---", "---", "---", "-------");
    printf("%5d%9.1f%9.1f%9.1f%12.3f%12.3f\n",
           1, x, min, max, sum, avg);
    for (i = 2; scanf("%lf", &x) == 1; ++i) {
        if (x < min)
            min = x;
        else if (x > max)
            max = x;
        sum += x;
        avg = sum / i;
        printf("%5d%9.1f%9.1f%9.1f%12.3f%12.3f\n",
               i, x, min, max, sum, avg);
    }
    return 0;
}
```

Riga per riga

- `if (scanf("%lf", &x) != 1)`: `scanf` restituisce il numero di conversioni riuscite; qui ne è richiesta una sola, quindi un valore diverso da 1 (`!=` significa "diverso") vuol dire che non è stato letto niente. In quel caso il programma stampa un messaggio ed esce con `exit(1)`: `exit`, il cui prototipo sta in `stdlib.h`, termina il programma restituendo al sistema operativo il valore indicato (qui 1, "qualcosa è andato storto").
- `min = max = sum = avg = x;`: un'assegnazione in C è anche un'espressione, che **vale** il valore assegnato. Le assegnazioni a catena si eseguono da destra a sinistra: `avg = x`, poi `sum = (avg = x)`, e così via. Tutte e quattro le variabili ricevono il primo numero letto.
- Le due `printf` con `%5s`, `%9s`… stampano l'intestazione della tabella: stringhe costanti in campi di larghezza fissa, così le colonne restano allineate.
- `for (i = 2; scanf("%lf", &x) == 1; ++i)`: un ciclo `for` in cui il **test è la lettura stessa**. Il ciclo continua finché `scanf` riesce a leggere un numero, cioè finché c'è input; non si sa in anticipo quante volte girerà. `i` conta i numeri letti e parte da 2 perché il primo è già stato letto prima del ciclo.
- `if (x < min) min = x; else if (x > max) max = x;`: aggiorna, se necessario, il minimo o il massimo. L'`else` contiene a sua volta un `if`. (Si poteva anche scrivere con due `if` indipendenti.)
- `avg = sum / i;`: qui la divisione è fra un `double` e un `int`, quindi è reale: nessuna trappola.

Tre modi di dare l'input

Il programma legge i numeri dallo **standard input**, il "canale" da cui un programma riceve i dati. Normalmente è la tastiera, e allora i numeri si possono scrivere uno alla volta, premendo Invio dopo ciascuno, oppure tutti su una riga; ma come fa il programma a sapere che l'input è finito? Bisogna segnalarlo con Ctrl + D all'inizio di una riga: `scanf` non riesce più a leggere, restituisce un valore diverso da 1, e il ciclo finisce.

Terminale

```
$ ./running_sum
3
Count     Item      Min       Max         Sum     Average
-----     ----      ---       ---         ---     -------

    1      3.0      3.0       3.0       3.000       3.000
-5 7
    2     -5.0     -5.0       3.0      -2.000      -1.000
    3      7.0     -5.0       7.0       5.000       1.667
Ctrl+D
$
```

(Ctrl + D in realtà non compare sullo schermo: qui è indicato per chiarezza.) Il terzo modo è il più comodo per provare un programma: scrivere i dati in un file e usarlo **al posto** della tastiera, con la **redirezione** `<` (figura 5.7). Il file `running_sum.in` delle lezioni contiene i dieci numeri `3 -5 7 -9 11 -13 15 -17 19 -21`.

Terminale

```
$ ./running_sum < running_sum.in
Count     Item      Min       Max         Sum     Average
-----     ----      ---       ---         ---     -------

    1      3.0      3.0       3.0       3.000       3.000
    2     -5.0     -5.0       3.0      -2.000      -1.000
    3      7.0     -5.0       7.0       5.000       1.667
    4     -9.0     -9.0       7.0      -4.000      -1.000
    5     11.0     -9.0      11.0       7.000       1.400
    6    -13.0    -13.0      11.0      -6.000      -1.000
    7     15.0    -13.0      15.0       9.000       1.286
    8    -17.0    -17.0      15.0      -8.000      -1.000
    9     19.0    -17.0      19.0      11.000       1.222
   10    -21.0    -21.0      19.0     -10.000      -1.000
```

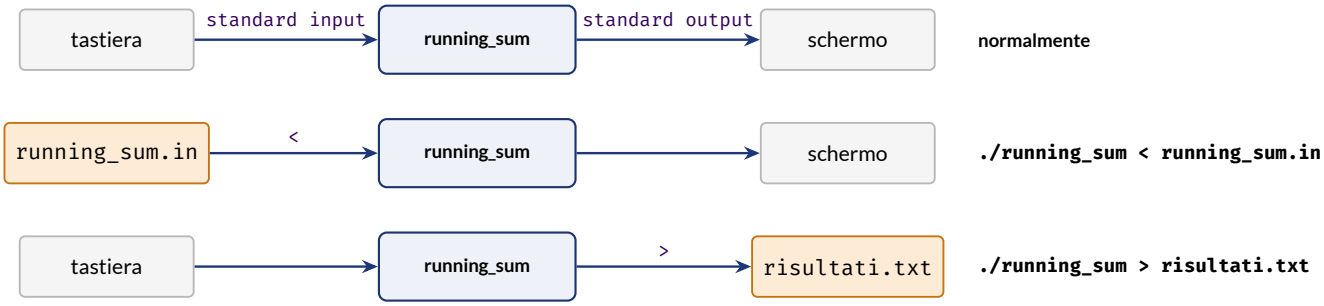


Figura 5.7: *La redirezione: con `<` un file prende il posto della tastiera, con `>` un file prende il posto dello schermo.*

È la shell a collegare il file al programma: il programma non se ne accorge nemmeno, continua a leggere dallo standard input. Simmetricamente, con `>` si può mandare l'output in un file invece che sullo schermo, e i due si possono combinare: `./running_sum < running_sum.in > risultati.txt`. Infine, se non c'è nessun dato, il programma esce con il suo messaggio, e il valore restituito al sistema è 1:

Terminale

```
$ ./running_sum < /dev/null
No data found - bye!
$ echo $?
1
```

(`/dev/null` è un file speciale di Linux, sempre vuoto.)

Una versione senza la lettura iniziale

Nel programma si fa una prima lettura **prima** del ciclo, poi le altre **nel** ciclo: lo stesso codice di lettura compare due volte. Quando è possibile, è più elegante mettere tutto dentro il ciclo. Ecco la versione delle lezioni con un `while`, in cui `cont` conta i numeri letti e il caso "nessun dato" si controlla alla fine:

running_sum2.c

```c
/* Compute the minimum, maximum, sum, and average. */

#include <stdio.h>

int main(void)
{
    int      cont = 0;
    double   x, min, max, sum, avg;

    while (scanf("%lf", &x) == 1) {
        cont = cont + 1;
        if (cont == 1) {
            sum = avg = min = max = x;
            printf("%5s%9s%9s%9s%12s%12s\n%5s%9s%9s%9s%12s%12s\n\n",
                   "Count", "Item", "Min", "Max", "Sum", "Average",
                   "-----", "----", "---", "---", "---", "-------");
        } else {
            if (x < min) min = x;
            else if (x > max) max = x;
            sum += x;
            avg = sum / cont;
        }
        printf("%5d%9.1f%9.1f%9.1f%12.3f%12.3f\n",
               cont, x, min, max, sum, avg);
    }
    if (cont == 0) {
        printf("No data found - bye!\n");
    }
    return 0;
}
```

L'output è identico. Di solito una variabile come `i` è la variabile di un ciclo su contatore; qui `cont` conta gli elementi letti, e il primo elemento è trattato a parte dentro il ciclo (`cont == 1`). Non serve più `stdlib.h`, perché non si usa `exit`.

Quando la compilazione fallisce

I messaggi del compilatore sono importanti e vanno letti; a volte però possono creare confusione. A lezione si compila il file `running_sum2Errore.c`, identico a quello appena visto tranne che per un dettaglio: nella prima riga manca la barra iniziale del commento.

```c
* Compute the minimum, maximum, sum, and average. */

#include <stdio.h>
...
```

Il risultato è una lunga, deprimente lista di **51 errori**, molti dei quali addirittura negli header file di sistema:

```
$ gcc -Wall -o running_sum2 running_sum2Errore.c
running_sum2Errore.c:1:11: error: expected '=', ',', ';', 'asm' or '__attribute__' before 'the'
    1 | * Compute the minimum, maximum, sum, and average. */
      |           ^~~
running_sum2Errore.c:1:11: error: unknown type name 'the'
In file included from /usr/include/stdio.h:44,
                 from running_sum2Errore.c:3:
/usr/include/x86_64-linux-gnu/bits/types/struct_FILE.h:95:3: error: unknown type name 'size_t'
   95 |   size_t __pad5;
      |   ^~~~~~
...
```

Che cosa è successo? Senza `/*` la prima riga non è più un commento: il compilatore la legge come codice C e cerca di darle un senso (`* Compute the…`), fallendo. Ma non sa dove finisce l'errore, e continua a leggere il resto del file "fuori fase": anche il contenuto di `stdio.h`, perfettamente corretto, viene letto nel contesto sbagliato e produce errori a cascata. La regola del capitolo 1 vale più che mai: si guarda **il primo** errore, che qui indica correttamente la riga 1, e si ignorano gli altri finché non lo si è corretto.

Una curiosità

Il file `running_sum2.c` distribuito con le lezioni ha lo stesso difetto di `running_sum2Errore.c` (i due file sono identici): per compilarlo bisogna aggiungere la `/` mancante all'inizio.

LE FUNZIONI

Dichiarare prima di usare

Una **funzione** è un pezzo di codice che rappresenta un blocco del processo di soluzione del problema: è il sotto-programma del capitolo 4. Un programma C è costituito da una o più funzioni, in uno o più file sorgente. Una di esse è necessariamente `main`, da cui comincia l'esecuzione; le altre sono richiamate da `main` o da altre funzioni.

In C le funzioni devono essere **dichiarate prima di usarle**. Supponiamo di voler usare la funzione `pow` della libreria matematica: la chiamata `pow(x, y)` restituisce x^y. La sua dichiarazione è

```
1 double pow(double x, double y);
```

Questo tipo di dichiarazione si chiama **prototipo** della funzione: dice il tipo del valore restituito (`double`), il nome e il tipo dei parametri. I nomi dei parametri, nel prototipo, sono ignorati dal compilatore: `double pow(double, double);` è equivalente, perché al compilatore interessano solo i tipi. È comunque buona norma metterli, soprattutto se sono significativi.

La forma generale di un prototipo è `tipo nome_funzione(lista dei tipi dei parametri);`. La parola chiave `void` si usa quando la funzione non ha parametri (`int f(void);`) o non restituisce niente (`void f(int);`). Se necessario, al momento della chiamata gli argomenti vengono convertiti al tipo indicato nel prototipo: chiamando `pow(2, 10)` con due interi, questi diventano `2.0` e `10.0`.

L'esempio maxmin

Il programma seguente stampa delle informazioni su se stesso, legge un intero `n`, legge `n` numeri reali e ne trova il massimo e il minimo.

maxmin.c

```c
#include <stdio.h>

float   maximum(float x, float y);
float   minimum(float x, float y);
void    prn_info(void);

int main(void)
{
    int     i, n;
    float   max, min, x;

    prn_info();
    printf("Input n:  ");
    scanf("%d", &n);
    printf("\nInput %d real numbers:  ", n);
    scanf("%f", &x);
    max = min = x;
    for (i = 2; i <= n; ++i) {
        scanf("%f", &x);
        max = maximum(max, x);
        min = minimum(min, x);
    }
    printf("\n%s%11.3f\n%s%11.3f\n\n",
           "Maximum value:", max,
           "Minimum value:", min);
    return 0;
}

float maximum(float x, float y)
{
    if (x > y)
        return x;
    else
        return y;
}

float minimum(float x, float y)
{
    if (x < y)
        return x;
    else
        return y;
}

void prn_info(void)
{
    printf("\n%s\n%s\n\n",
           "This program reads an integer value for n, and then",
           "processes n real numbers to find max and min values.");
}
```

Terminale

```
$ ./maxmin

This program reads an integer value for n, and then
processes n real numbers to find max and min values.

Input n:  5

Input 5 real numbers:  737.7799  -11.2e+3  -777  0.001  3.14159

Maximum value:    737.780
Minimum value: -11200.000
```

Si noti che `scanf` con `%f` accetta anche la notazione scientifica: `-11.2e+3` è $-11{,}2 \cdot 10^3 = -11\,200$.

Come funziona

- In cima al file ci sono i **prototipi** di `maximum`, `minimum` e `prn_info`. I primi due dicono che le funzioni hanno due parametri di tipo `float` e restituiscono un `float`; il terzo che `prn_info` non ha parametri e non restituisce alcun valore.
- `prn_info();` chiama la funzione, che a sua volta chiama `printf` per stampare le informazioni sul programma e sul suo uso.
- Il programma chiede e legge `n` (`%d`: un intero decimale), poi legge il primo numero reale in `x`, e `max = min = x;` inizializza massimo e minimo con quel valore.
- A ogni giro del ciclo si legge un nuovo valore di `x`; `max = maximum(max, x);` calcola il massimo fra `max` e `x` e lo assegna a `max`, e lo stesso per il minimo.

La **definizione** di `maximum` specifica che cosa la funzione fa quando viene chiamata. È fatta di una **testa**, la parte prima della prima graffa (`float maximum(float x, float y)`: il primo `float` è il tipo del valore restituito, gli altri due i tipi dei parametri), e di un **corpo**, la parte fra le graffe.

L'istruzione `return` **interrompe** la funzione e restituisce al chiamante il valore della sua espressione; la forma generale è `return espressione;`. In una funzione `void` si può scrivere `return;` senza espressione, o lasciare che la funzione arrivi da sola alla graffa finale, come fa `prn_info`.

Due x diverse

Le funzioni `maximum` e `minimum` hanno entrambe i parametri `x` e `y`, e una variabile `x` c'è anche in `main`. Non c'è nessuna relazione fra loro: sono variabili **logicamente e fisicamente diverse**, in celle di memoria diverse. Usare lo stesso nome informa solo chi legge che il loro uso è simile. La figura 5.8 mostra la memoria durante la chiamata `maximum(max, x)` al secondo giro.

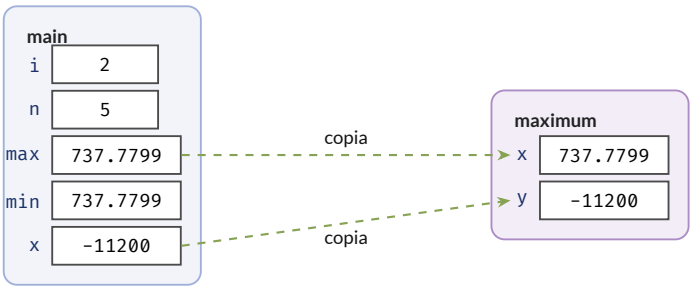


Figura 5.8: *La* `x` *di* `main` *e la* `x` *di* `maximum` *sono due variabili distinte:* `maximum` *riceve una* ***copia*** *dei valori.*

Che cosa succede senza prototipi

Le slide mostrano anche una versione di `maxmin` senza i prototipi in cima al file. Con gcc il risultato è questo:

Terminale

```
$ gcc -Wall -o maxmin maxmin.c
maxmin.c:9:4: warning: implicit declaration of function ‘prn_info’
maxmin.c:17:13: warning: implicit declaration of function ‘maximum’
maxmin.c:18:13: warning: implicit declaration of function ‘minimum’
maxmin.c:26:7: error: conflicting types for ‘maximum’; have ‘float(float,  float)’
maxmin.c:34:7: error: conflicting types for ‘minimum’; have ‘float(float,  float)’
```

Il compilatore legge il file dall'alto in basso. Quando incontra `maximum(max, x)` non sa ancora nulla di `maximum`: per una vecchia regola del C, "immagina" che sia una funzione che restituisce un `int` (è la *dichiarazione implicita* di cui avvisa il warning). Più avanti trova la definizione vera, che restituisce un `float`, e la contraddizione diventa un errore. Le versioni più recenti di gcc (dalla 14) non accettano più le dichiarazioni implicite nemmeno come avviso: la chiamata di una funzione non dichiarata è direttamente un errore. La lezione è semplice: **ogni funzione va dichiarata prima dell'uso**.

Un modo alternativo, che va bene per i file piccoli, è mettere le **definizioni** delle funzioni **prima** di `main`: quando il compilatore arriva alla chiamata ha già letto la definizione, e la testa della funzione fa da prototipo.

Un punto e virgola di troppo

Nelle prime slide su `maxmin` la funzione `maximum` contiene un errore di battitura istruttivo:

```c
float maximum(float x, float y)
{
    if (x > y) ;{
        return x;
    }
    else {
        return y;
    }
}
```

Terminale

```
$ gcc -Wall -o maxmin maxmin.c
maxmin.c: In function ‘maximum’:
maxmin.c:31:4: warning: this ‘if’ clause does not guard... [-
    Wmisleading-indentation]
   31 |    if (x > y) ;{
      |    ^~
maxmin.c:34:4: error: ‘else’ without a previous ‘if’
   34 |    else {
      |    ^~~~
```

Il `;` dopo la condizione è un'**istruzione vuota**: l'`if` controlla quella, cioè nulla. Il blocco `{ return x; }` che segue non fa più parte dell'`if`: è un blocco qualunque, eseguito sempre. E quando arriva l'`else`, l'`if` a cui dovrebbe appartenere è già concluso: "`else` senza un `if` precedente". gcc avvisa anche che l'indentazione è *ingannevole*: il codice sembra dire una cosa e ne fa un'altra.

IL PASSAGGIO DEI PARAMETRI PER VALORE

Esistono due modi di passare i parametri a una funzione:

- il **passaggio per valore**: viene creata una **copia** della variabile passata come parametro;
- il **passaggio per riferimento**: viene passata la variabile stessa.

In C i parametri sono passati solo per valore. Il programma seguente lo dimostra: la funzione prova a cambiare il valore di `a`, ma `main` non se ne accorge.

try_to.c

```c
#include <stdio.h>

int main(void)
{
    int    a = 1;
    void   try_to_change_it(int);

    printf("%d\n", a);      /* 1 is printed */
    try_to_change_it(a);
    printf("%d\n", a);      /* 1 is printed again! */
    return 0;
}

void try_to_change_it(int a)
{
    a = 777;
}
```

Terminale

```
$ ./try_to
1
1
```

Il prototipo è scritto dentro `main`: è lecito, e vale solo lì. Di solito lo si mette in cima al file.

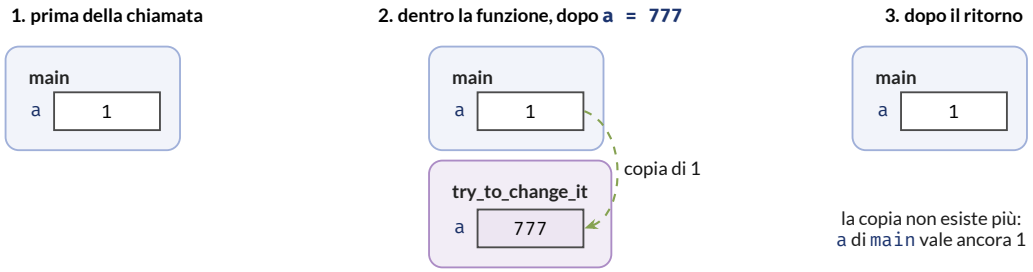


Figura 5.9: *Il passaggio per valore: la funzione lavora su una copia, che scompare quando la funzione termina.*

Alla chiamata, il parametro `a` di `try_to_change_it` (una variabile diversa da quella di `main`, anche se ha lo stesso nome) riceve una **copia** del valore 1. L'assegnazione `a = 777` modifica la copia; quando la funzione termina la copia scompare, e `main` ritrova la sua `a` intatta (figura 5.9).

Ecco perché `scanf` ha bisogno di `&x`: se ricevesse `x`, riceverebbe solo una copia del suo valore e non potrebbe modificare la variabile. Ricevendo l'**indirizzo**, sa dove andare a scrivere. Come una funzione possa usare un indirizzo per modificare una variabile del chiamante (i *puntatori*) è l'argomento del capitolo 11.

In sintesi

Un programma C è fatto di funzioni, e l'esecuzione comincia da `main`. Le righe che cominciano con `#` sono per il preprocessore: `#include` copia un header (che **descrive** le funzioni; il loro codice arriva dalla libreria al momento del linking), `#define` definisce costanti simboliche. Le variabili si dichiarano con un tipo prima di usarle; `=` assegna (si legge da destra a sinistra); la divisione fra interi tronca. `printf` e `scanf` usano una stringa di controllo con le conversioni (`%d`, `%f`, `%lf`…); `scanf` vuole gli **indirizzi** delle variabili e restituisce il numero di conversioni riuscite. In C 0 è falso e ogni altro valore è vero; `if`, `while` e `for` realizzano selezione e iterazione. Le funzioni vanno dichiarate con un prototipo prima dell'uso, e ricevono sempre una **copia** dei parametri.

Riferimenti

- **A. Kelley, I. Pohl**, *A Book on C*, Addison-Wesley: capitolo 1, da cui provengono gli esempi di questo capitolo.
- **Deitel & Deitel**, *Il linguaggio C*: capitoli 2–5.
- **printf** e **scanf** — cppreference.com.
  `https://en.cppreference.com/w/c/io/fprintf`
  `https://en.cppreference.com/w/c/io/fscanf`

Elementi lessicali e operatori

Il C è un linguaggio, e come ogni linguaggio ha un **alfabeto**, delle regole per formare le **parole** e delle regole per comporre le parole in **frasi** corrette, la **sintassi**. Il compilatore, per prima cosa, verifica che il programma sia sintatticamente corretto: la correttezza sintattica è necessaria per la traduzione (un programma con un errore di sintassi non viene tradotto). Non è però sufficiente: un programma sintatticamente perfetto può comunque fare la cosa sbagliata, come la divisione intera della maratona nel capitolo 5.

Questo capitolo guarda il C "al microscopio": come il compilatore legge il testo, quali parole sono riservate, come si scelgono i nomi, in che ordine si calcolano le espressioni.

Dai caratteri ai token

I token

Un programma è una sequenza di caratteri leciti (lettere, cifre, simboli, spazi). Il compilatore, per prima cosa, raggruppa i caratteri in unità dotate di significato, i **token**, come noi raggruppiamo le lettere in parole. Prendiamo un programma che legge due interi e ne stampa la somma:

somma.c

```c
/* Read in two integers and print their sum. */

#include <stdio.h>

int main(void)
{
    int   a, b, sum;

    printf("Input two integers:  ");
    scanf("%d%d", &a, &b);
    sum = a + b;
    printf("%d + %d = %d\n", a, b, sum);
    return 0;
}
```

Terminale

```
$ ./somma
Input two integers:  7 35
7 + 35 = 42
```

I token sono di alcune categorie (figura 6.1):

- gli **identificatori**, i nomi: `main`, `a`, `b`, `sum`, `printf`, `scanf`;
- le **parole chiave**: `int`, `void`, `return`;
- gli **operatori**: `=`, `+`, `&`, e le parentesi `( )` dopo `main`, che dicono che `main` è una funzione;
- la **punteggiatura**: `{ }`, `;` e `,`;
- le **costanti**: i numeri come `0` e le stringhe come `"Input two integers: "`, che il compilatore tratta come un **unico** token, spazi compresi.

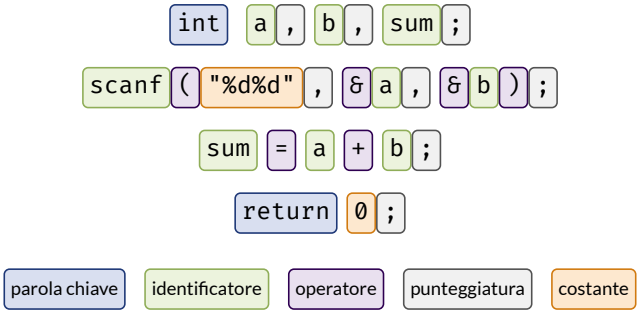


Figura 6.1: *Alcune righe di* `somma.c` *scomposte in token, colorati per categoria.*

Gli spazi

Il compilatore usa gli spazi bianchi (spazi, tabulazioni, a capo) per distinguere i token. Dove due token si "toccherebbero" formando una parola diversa, lo spazio è **necessario**: non si può scrivere `inta, b, sum;`, perché il compilatore non riesce a staccare `int` da `a` e vede un unico identificatore, `inta`, mai dichiarato:

Terminale

```
$ gcc -Wall -c somma.c
somma.c: In function ‘main’:
somma.c:7:5: error: ‘inta’ undeclared (first use in this function); did you mean ‘int’?
    7 |     inta, b, sum;
      |     ^~~~
      |     int
```

Non tutti gli spazi però sono necessari: `int a,b,sum;` va benissimo, perché le virgole separano già i token. E dove servono, servono davvero: `int absum;` dichiara un'unica variabile chiamata `absum`.

Al contrario, gli spazi **in più** sono ignorati: `sum=a+b;` e `sum = a + b ;` sono la stessa istruzione. Ma uno spazio **dentro** un token lo spezza: `s u m = a + b;` non è lecito, perché il compilatore tratta `s`, `u` e `m` come tre token diversi.

Terminale

```
$ gcc -Wall -c somma.c
somma.c:11:5: error: unknown type name ‘s’
   11 |     s u m = a + b;
      |     ^
```

Lo stesso vale per l'operatore di indirizzo: in `&a` il carattere `&` è un token a sé, separato dalla variabile, quindi `& a` o `& b` sono scritture lecite (anche se insolite). Però i due argomenti devono comunque essere separati dalla virgola: `& a & b` non è lecito, perché viene letto come `&a & b`, cioè l'operatore `&` (che fra due operandi ha un altro significato) applicato all'indirizzo di `a` e a `b`, e il compilatore protesta:

Terminale

```
$ gcc -Wall -c somma.c
somma.c:10:23: error: invalid operands to binary & (have ‘int *’ and ‘int’)
   10 |     scanf("%d%d", & a & b);
      |                   ~~~ ^
```

Identificatori della libreria

`printf` e `scanf` sono identificatori come gli altri; le parentesi che li seguono indicano che sono funzioni. Il compilatore non ne conosce il codice: dopo la compilazione, il linker crea l'eseguibile prendendo la loro definizione dalla libreria standard. Non sono parole chiave, ma andrebbero considerati **riservati**: usarli per altri scopi è una pessima idea, come mostra l'ultimo esempio di questo capitolo.

I COMMENTI

I **commenti** sono sequenze arbitrarie di testo fra i delimitatori `/*` e `*/`, oppure da `//` alla fine della riga. I commenti **non sono token**: il compilatore trasforma ogni commento in uno spazio. Lo si può verificare con un esempio un po' estremo, in cui un commento separa `int` da `x`:

commenti.c

```c
#include <stdio.h>

int main(void)
{
    int/* un commento vale come uno spazio */x = 5;

    printf("%d\n", x);   // un commento su una riga
    return 0;
}
```

Terminale

```
$ ./commenti
5
```

I commenti `/* */` **non si annidano**: il primo `*/` chiude il commento, qualunque cosa ci sia prima. Con `/* esterno /* interno */ ancora esterno? */` il commento finisce dopo `interno`, e "ancora esterno?" viene letto come codice (gcc avvisa: *"/*" within comment*).

I commenti sono la documentazione del codice. Andrebbero scritti **insieme** al codice: una volta che il programma funziona si tende a scriverne di brevissimi, o a non scriverne affatto. Un buon commento spiega il *perché*, non ripete il *che cosa*: `i = i + 1; /* incrementa i */` non aiuta nessuno, `/* il primo elemento è già stato letto */` sì.

LE PAROLE CHIAVE

Le **parole chiave** sono parole riservate, che non possono essere usate in altri contesti (per esempio come nomi di variabili). Il C ne ha, nel complesso, poche; l'ANSI C ne definisce 32:

`auto`	`break`	`case`	`char`	`const`	`continue`	`default`	`do`
`double`	`else`	`enum`	`extern`	`float`	`for`	`goto`	`if`
`int`	`long`	`register`	`return`	`short`	`signed`	`sizeof`	`static`
`struct`	`switch`	`typedef`	`union`	`unsigned`	`void`	`volatile`	`while`

Le parole chiave sono minuscole

Nella tabella delle slide compaiono `Volatile` e `Unsigned` con l'iniziale maiuscola: è un effetto della correzione automatica. Le parole chiave sono tutte **minuscole**, e siccome il C distingue maiuscole e minuscole, `Unsigned` non sarebbe una parola chiave ma un identificatore qualunque.

Alcune implementazioni hanno parole chiave aggiuntive (gcc, per esempio, riconosce `asm` e `typeof`), e le versioni successive dello standard ne hanno aggiunte poche altre (fra cui `inline`, e nel C23 `bool`, `true` e `false`). Non serve impararle a memoria: VS Code le colora in blu o in viola, e l'errore di usarne una come nome si vede subito.

Gli identificatori

La regola

Un **identificatore** (un nome di variabile, di funzione, di tipo…) è una sequenza di caratteri composta da **lettere**, **cifre** e dal carattere speciale `_` (*underscore*), che **non comincia con una cifra**. Formalmente, nella notazione delle slide:

$$\texttt{identifier ::= \{letter | underscore\}}_1\ \texttt{\{letter | underscore | digit\}}_{0+}$$

dove `|` significa "oppure", il pedice $_1$ "esattamente una volta" e il pedice $_{0+}$ "zero o più volte". La stessa regola si può disegnare come un percorso da seguire, un *diagramma sintattico* (figura 6.2): un nome è lecito se lo si può ottenere percorrendo le frecce dall'inizio alla fine e raccogliendo un carattere a ogni casella attraversata.

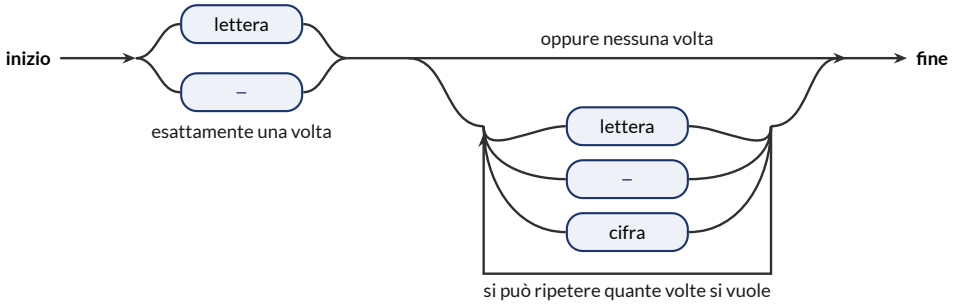


Figura 6.2: *Il diagramma sintattico di un identificatore.*

identificatori leciti		**identificatori non leciti**	
`K`	una lettera sola	`not#me`	`#` non è ammesso
`_id`	comincia con `_`	`101_south`	comincia con una cifra
`iamanidentifier2`	cifre dopo l'inizio	`-plus`	`-` è un operatore
`so_am_i`		`int`	è una parola chiave

Maiuscole, lunghezza, scelta dei nomi

Il C è *case sensitive*: le lettere maiuscole e minuscole sono considerate **diverse**, quindi `sum`, `Sum` e `SUM` sono tre identificatori distinti.

Sulla lunghezza: nei sistemi più vecchi contavano solo i primi 8 caratteri, per cui `i_am_an_identifier` e `i_am_an_elephant` sarebbero stati lo stesso nome (i primi 8 caratteri sono `i_am_an_` in entrambi). L'ANSI C garantisce che siano considerati almeno i primi 31 caratteri, e i compilatori moderni ne considerano molti di più: in pratica, la lunghezza non è un problema.

Gli identificatori vanno scelti in modo che contribuiscano alla **leggibilità** e alla **documentazione** del programma. Per calcolare l'IVA di un prodotto,

```
1 float tax_iva, price, tax_iva_rate;
2
3 tax_iva = price * tax_iva_rate;
```

si capisce da sé, mentre `t = p * r;` richiede un commento per ogni variabile. Infine, per convenzione gli identificatori di sistema (della libreria, del compilatore) cominciano con uno o due underscore: per evitare conflitti, conviene non creare identificatori che cominciano con `_`.

Precedenza e associatività

Gli operatori hanno regole di **precedenza** e di **associatività** che determinano come vengono calcolate le espressioni. Le espressioni fra parentesi sono calcolate per prime; le parentesi si possono usare sia per **chiarire** l'ordine delle operazioni, sia per **modificarlo**.

- In `1 + 2 * 3` l'operatore `*` ha una precedenza più alta di `+`: il risultato è 7, come in `1 + (2 * 3)`.
- In `(1 + 2) * 3` le parentesi hanno la priorità: il risultato è 9.
- In `1 + 2 - 3 + 4 - 5` gli operatori binari `+` e `-` hanno la **stessa** precedenza: l'ordine lo stabilisce la regola di associatività, che per loro è **da sinistra a destra**. L'espressione equivale a `(((1 + 2) - 3) + 4) - 5`, cioè −1.

La figura 6.3 mostra gli alberi delle tre espressioni: precedenza e associatività servono proprio a stabilire, senza ambiguità, quale albero corrisponde a una sequenza di token.

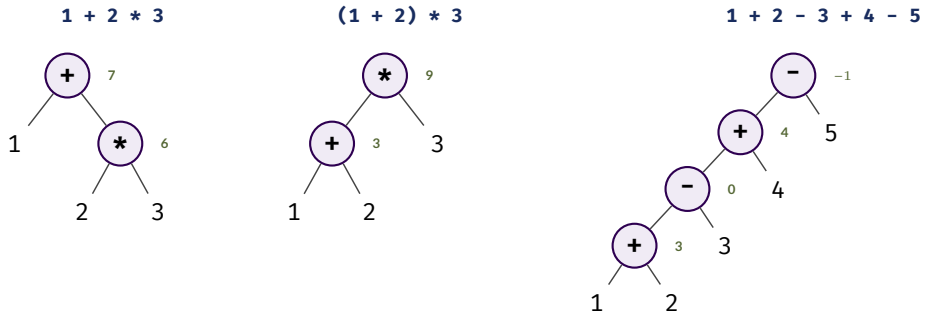


Figura 6.3: *Gli alberi delle tre espressioni, con il valore di ogni operazione: si calcola dal basso verso l'alto.*

La tabella degli operatori

La tabella seguente riassume precedenza e associatività degli operatori visti finora, dalla precedenza più alta (in alto) alla più bassa. Gli operatori sulla stessa riga hanno la stessa precedenza.

operatori	significato	associatività
`()  ++  --`	parentesi; incremento e decremento **postfissi**	da sinistra a destra
`+  -  ++  --`	più e meno **unari**; incremento e decremento **prefissi**	da destra a sinistra
`*  /  %`	moltiplicazione, divisione, resto	da sinistra a destra
`+  -`	somma e sottrazione	da sinistra a destra
`=  +=  -=  *=  /=`	assegnazioni	da destra a sinistra

Qualche commento:

- gli operatori **unari** hanno un solo operando: in `-x` il meno è unario (cambia segno), in `a - x` è binario (sottrae). Gli unari hanno precedenza più alta dei binari: `-a * b` è `(-a) * b`;
- `%` è il **resto** della divisione fra interi (detto anche *modulo*): `7 / 2` vale 3 e `7 % 2` vale 1, perché $7 = 3 \cdot 2 + 1$. Si usa spesso per sapere se un numero è pari (`n % 2 == 0`) o divisibile per un altro;
- le assegnazioni hanno la precedenza più bassa (si calcola prima tutto il lato destro) e associano da **destra** a sinistra: è il motivo per cui `min = max = sum = avg = x;` del capitolo 5 funziona, perché equivale a `min = (max = (sum = (avg = x)));`.

```c
printf("%d %d %d\n", 1 + 2 * 3, (1 + 2) * 3, 1 + 2 - 3 + 4 - 5);
printf("%d %d %d\n", 7 / 2, 7 % 2, -7 / 2);
```

Terminale

```
7 9 -1
3 1 -3
```

Si noti l'ultimo valore: la divisione fra interi **tronca verso lo zero**, quindi $-7/2 = -3{,}5$ diventa -3 (non -4).

Nel dubbio, parentesi

Nessuno ricorda a memoria tutta la tabella delle precedenze del C (che conta una quindicina di livelli). Quando l'ordine non è ovvio, le parentesi lo rendono esplicito per il compilatore e, soprattutto, per chi legge: `(a * b) + c` non costa nulla e non lascia dubbi.

Incremento e decremento

Prefisso e postfisso

Gli operatori di incremento `++` e decremento `--` sono **unari** e si possono usare in due modi: come operatore **prefisso** (`++i`, prima della variabile) o **postfisso** (`i++`, dopo). Si applicano a **variabili**, non a costanti o a espressioni: `++i` e `cnt--` sono usi leciti, `777++` e `++(a * b - 1)` no, perché non c'è una variabile da modificare. Il compilatore lo dice così (*lvalue*, "valore a sinistra", è qualcosa che può stare a sinistra di un'assegnazione, cioè una variabile):

Terminale

```
$ gcc -Wall -c costanti.c
costanti.c:4:8: error: lvalue required as increment operand
costanti.c:5:5: error: lvalue required as increment operand
```

Le due forme non hanno lo stesso comportamento, e per capire la differenza bisogna ricordare che in C **ogni espressione ha un valore**:

- `++i`: la variabile `i` viene incrementata di 1, e l'espressione vale il **nuovo** valore di `i` ("prima incrementa, poi usa");
- `i++`: l'espressione vale il **vecchio** valore di `i`, e poi la variabile viene incrementata ("prima usa, poi incrementa").

Allo stesso modo `--i` decrementa e vale il nuovo valore, `i--` vale il vecchio valore e poi decrementa. Come istruzioni a sé (`i++;`) le due forme sono equivalenti, perché il valore dell'espressione non viene usato da nessuno.

Un esempio da simulare

incr.c

```c
#include <stdio.h>

int main(void)
{
    int a, b, c = 0;

    a = ++c;
    b = c++;
    printf("%d %d %d\n", a, b, ++c);
    return 0;
}
```

Terminale

```
$ ./incr
1 1 3
```

Nelle slide la dichiarazione è scritta `int a, b, c==0;`: è un refuso, `==` è il confronto; serve `=`.

Simuliamo l'esecuzione, istruzione per istruzione:

istruzione	che cosa succede	a	b	c
`int a, b, c = 0;`	dichiarazioni, solo `c` è inizializzata	?	?	0
`a = ++c;`	`c` diventa 1; `++c` vale 1, che va in `a`	1	?	1
`b = c++;`	`c++` vale 1 (il vecchio valore), che va in `b`; poi `c` diventa 2	1	1	2
`printf(…, ++c);`	`c` diventa 3; `++c` vale 3 e viene stampato	1	1	3

Quando ++ diventa pericoloso

`++` e `--` **modificano** la variabile su cui operano. Usati da soli non danno problemi; in espressioni più complesse bisogna prestare molta attenzione. Le slide mostrano questo programma, con l'intento di calcolare $a \cdot (a + 1)$ in tre modi:

ub.c

```c
#include <stdio.h>

int main(void)
{
    int a = 1;

    printf("a*(a+1) = %d\n", a * (a + 1));
    printf("a*(a+1) = %d\n", a * a++);
    a = 1;
    printf("a*(a+1) = %d\n", a * ++a);
    return 0;
}
```

Ecco che cosa succede compilandolo con gcc (su Linux) e con clang (il compilatore di macOS):

gcc

```
$ gcc -Wall -o ub ub.c
ub.c:8:35: warning: operation on ‘a’ may be undefined
     [-Wsequence-point]
ub.c:10:34: warning: operation on ‘a’ may be
    undefined [-Wsequence-point]
$ ./ub
a*(a+1) = 2
a*(a+1) = 2
a*(a+1) = 4
```

clang

```
$ clang -Wall -o ub ub.c
ub.c:8:35: warning: unsequenced modification and
    access to 'a' [-Wunsequenced]
ub.c:10:34: warning: unsequenced modification and
    access to 'a' [-Wunsequenced]
$ ./ub
a*(a+1) = 2
a*(a+1) = 1
a*(a+1) = 2
```

Lo stesso programma dà risultati diversi con compilatori diversi, e nessuno dei due ha torto. In `a * a++` il C non stabilisce se il primo `a` vada letto prima o dopo l'incremento: il programma legge e modifica la stessa variabile nella stessa espressione, senza un ordine definito, e lo standard dice che in questo caso il comportamento è **indefinito** (*undefined behavior*): il compilatore può fare qualunque cosa. Le slide mostrano i risultati di gcc (2, 2, 4) come se fossero "quelli giusti"; in realtà non esistono risultati giusti, e infatti entrambi i compilatori avvisano.

Una variabile modificata con `++` non va usata altrove nella stessa espressione

Espressioni come `a * a++`, `i = i++` o `printf("%d %d", n, n++)` hanno comportamento indefinito. La regola pratica delle slide è la migliore: **meglio evitarle** e scrivere esplicitamente ciò che si intende, per esempio `a * (a + 1)`. E i warning di `-Wall` vanno sempre letti.

La libreria standard e i nomi riservati

Che cosa succede se si **ridefinisce** una funzione della libreria standard? Le slide lo provano con `scanf`, scrivendo una funzione con lo stesso nome che stampa la stringa ricevuta. Per non avere conflitti con il prototipo vero, il file **non** include `stdio.h`:

main.c

```c
void scanf(const char *s)
{
    printf(s);
}

int main()
{
    scanf("The printing scanf\n");
    return 0;
}
```

Terminale

```
$ gcc -o ridef main.c
main.c:1:6: warning: conflicting types for built-in function ‘scanf’; expected ‘int(const char *, ...)’ [-
    Wbuiltin-declaration-mismatch]
main.c:3:5: warning: implicit declaration of function ‘printf’ [-Wimplicit-function-declaration]
main.c:3:5: warning: incompatible implicit declaration of built-in function ‘printf’
main.c:3:5: warning: format not a string literal and no format arguments [-Wformat-security]
$ ./ridef
The printing scanf
```

Il risultato è istruttivo:

- anche senza `stdio.h`, il compilatore si accorge che si sta usando il nome di una funzione “predefinita” (*built-in*) con un tipo diverso da quello che conosce;
- dà però solo dei **warning**, non degli errori: l’eseguibile viene creato;
- e viene usata la **nuova** definizione: la “`scanf`” stampa invece di leggere.

Le slide mostrano anche la variante con la funzione in un file separato (`scanf.c`) e il solo prototipo in `main.c`: il risultato è lo stesso, perché il linker, trovando `scanf` già definita nei nostri file oggetto, non va a cercarla nella libreria. (Si notino anche i warning su `printf`: senza `stdio.h` il compilatore non ne conosce il prototipo. Le versioni di gcc dalla 14 in poi considerano questa dichiarazione implicita un errore, e il programma non si compila più.)

La morale: i nomi delle funzioni della libreria standard **non sono parole chiave**, quindi il compilatore non ne impedisce l’uso, ma vanno trattati come se lo fossero. Un programma che ridefinisce `scanf` fa qualcosa di diverso da ciò che chiunque si aspetta leggendolo.

In sintesi

Il compilatore legge il programma come una sequenza di **token**: identificatori, parole chiave, operatori, punteggiatura, costanti; gli spazi separano i token quando serve e altrimenti sono ignorati, i commenti valgono come spazi. Le 32 parole chiave dell’ANSI C sono riservate e minuscole; un identificatore è fatto di lettere, cifre e `_` e non comincia con una cifra, e maiuscole e minuscole contano. **Precedenza** e **associatività** stabiliscono l’albero di un’espressione (le parentesi lo rendono esplicito). `++i` vale il nuovo valore, `i++` il vecchio; leggere e modificare la stessa variabile nella stessa espressione dà un comportamento **indefinito**. I nomi della libreria standard vanno trattati come riservati.

Riferimenti

- **A. Kelley, I. Pohl**, *A Book on C*: capitolo 2 (elementi lessicali, operatori e il sistema C).
- **C Operator Precedence** — cppreference.com.
  `https://en.cppreference.com/w/c/language/operator_precedence`
- **Order of evaluation** (comportamento indefinito di `i++ + i++` e simili) — cppreference.com.
  `https://en.cppreference.com/w/c/language/eval_order`

Programmi su più file e make

Finora ogni programma stava in un solo file. I programmi veri sono fatti di molti file, a volte migliaia, e compilarli tutti a ogni modifica sarebbe lentissimo. Questo capitolo mostra come si divide un programma in più file, come si compilano separatamente e si collegano (riprendendo il capitolo 3), e come il programma `make` automatizza il lavoro, ricompilando solo ciò che serve.

Perché più file

Dividere un programma in più file sorgente ha diversi vantaggi:

- **organizzazione**: ogni file raccoglie le funzioni che riguardano un aspetto del problema (l'input, i calcoli, l'output), come i capitoli di un libro;
- **riuso**: un file con funzioni utili (per esempio di geometria) si può usare in più programmi;
- **lavoro di gruppo**: persone diverse lavorano su file diversi;
- **compilazione più rapida**: se si modifica un solo file, si ricompila solo quello e si rifà il collegamento.

Un piccolo progetto

Prendiamo un programma che calcola l'area di un cerchio e di un quadrato, e dividiamolo in tre file: le funzioni di calcolo in `aree.c`, il programma principale in `main.c`, e un **file di intestazione** `aree.h` con ciò che serve a entrambi.

aree.h

```h
/* aree.h: costanti e prototipi delle funzioni sulle aree */

#define PI 3.14159

double area_cerchio(double raggio);
double area_quadrato(double lato);
```

aree.c

```c
/* aree.c: definizioni delle funzioni sulle aree */

#include "aree.h"

double area_cerchio(double raggio)
{
    return PI * raggio * raggio;
}

double area_quadrato(double lato)
{
    return lato * lato;
}
```

main.c

```c
#include <stdio.h>
#include "aree.h"

int main(void)
{
    double x;

    printf("Inserisci una misura: ");
    scanf("%lf", &x);
    printf("cerchio di raggio %g: area %g\n",
           x, area_cerchio(x));
    printf("quadrato di lato %g: area %g\n",
           x, area_quadrato(x));
    return 0;
}
```

Il ruolo di ciascun file:

- `aree.h` contiene le **dichiarazioni**: la costante e i prototipi. Chi lo include sa *che* le funzioni esistono e come si chiamano;
- `aree.c` contiene le **definizioni**: il codice delle funzioni, scritto una volta sola. Include anche lui `aree.h`, così il compilatore controlla che le definizioni corrispondano ai prototipi;
- `main.c` include `aree.h` e può quindi chiamare le funzioni, anche se il loro codice sta in un altro file.

Compilare separatamente, collegare insieme

Ogni file `.c` si compila per conto suo con l'opzione `-c`, che produce il file oggetto; poi il linker unisce i file oggetto nell'eseguibile:

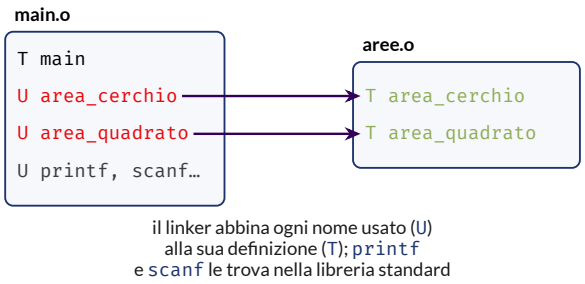
Terminale

```
$ gcc -Wall -c main.c
$ gcc -Wall -c aree.c
$ gcc -o aree main.o aree.o
$ ./aree
Inserisci una misura: 2
cerchio di raggio 2: area 12.5664
quadrato di lato 2: area 4
```

Che cosa contengono i due file oggetto, e come fa il linker a metterli insieme? Lo mostra `nm` (capitolo 3): in `main.o` le due funzioni sono *undefined* (`U`), cioè usate ma senza codice; in `aree.o` sono presenti (`T`). Il linker abbina ogni nome mancante con la sua definizione (schema a destra).

Terminale

```
$ nm main.o
                 U __isoc99_scanf
                 U __stack_chk_fail
                 U area_cerchio
                 U area_quadrato
0000000000000000 T main
                 U printf
$ nm aree.o
0000000000000000 T area_cerchio
0000000000000025 T area_quadrato
```



il linker abbina ogni nome usato (`U`) alla sua definizione (`T`); `printf` e `scanf` le trova nella libreria standard

Che cosa succede se si dimentica `aree.o` al momento del linking? La compilazione di `main.c` era già riuscita (grazie a `aree.h` il compilatore sapeva tutto ciò che serviva), ma il linker non trova il codice delle due funzioni:

Terminale

```
$ gcc -o aree main.o
/usr/bin/ld: main.o: in function `main':
main.c:(.text+0x54): undefined reference to `area_cerchio'
/usr/bin/ld: main.c:(.text+0x89): undefined reference to `area_quadrato'
collect2: error: ld returned 1 exit status
```

È lo stesso errore di `pow` senza `-lm` del capitolo 5: *undefined reference* significa sempre "il linker non trova il codice di questa funzione".

IL PROGETTO DELLE LEZIONI

I file

A lezione si usa un progetto su tre file, che gestisce un elenco di esami sostenuti da alcuni studenti (letti da un file di testo): `main.c`, `funzioni.c` e `studente.h` (figura 7.1). Il codice usa strumenti che vedremo più avanti (strutture, liste, file), e verrà spiegato per intero nel capitolo **??**; per capire `make` basta sapere chi dipende da chi.

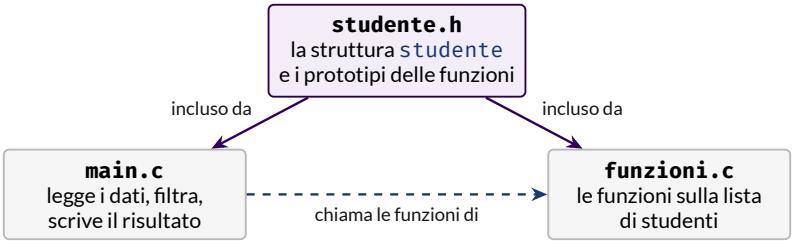


Figura 7.1: *Il progetto delle lezioni: due sorgenti e un header condiviso.*

L'eseguibile si crea come abbiamo visto: la compilazione (`gcc -c main.c funzioni.c`, che si può fare anche con un solo comando per più file) produce `main.o` e `funzioni.o`; il linking (`gcc -o esercizio main.o funzioni.o`) produce l'eseguibile `esercizio`.

Il problema

Con molti file la compilazione può essere lunga, e non sempre è necessario ricompilare tutti i sorgenti: se si modifica solo `funzioni.c`, `main.o` è ancora valido. Vorremmo rendere le operazioni più semplici e automatiche.

Un primo passo è uno **script**: un file di testo che contiene i comandi, da eseguire tutti in fila.

compila.sh

```
1 #!/bin/bash
2 gcc -c main.c
3 gcc -c funzioni.c
4 gcc -o esercizio main.o funzioni.o
```

Terminale

```
$ chmod +x compila.sh
$ ./compila.sh
```

La prima riga (`#!/bin/bash`, detta *shebang*) dice al sistema quale programma deve eseguire lo script: la shell bash. `chmod +x` rende il file eseguibile, una volta per tutte; da lì in poi `./compila.sh` esegue i tre comandi. Il procedimento è automatico, ma i sorgenti vengono comunque compilati **tutti**, ogni volta. Vorremmo un meccanismo che compila solo i file che sono stati modificati.

L'idea: dipendenze e date

Il comando `make` esegue la compilazione in modo automatico seguendo, fondamentalmente, due regole:

- la **dipendenza** fra i file: `esercizio` si costruisce da `main.o` e `funzioni.o`; `main.o` si costruisce da `main.c` (e da `studente.h`, che `main.c` include), e così via (figura 7.2);
- l'**ordine temporale** in cui sono stati creati o modificati i file sorgente, oggetto ed eseguibile: ogni file ha una data e ora di ultima modifica, che `ls -l` mostra.

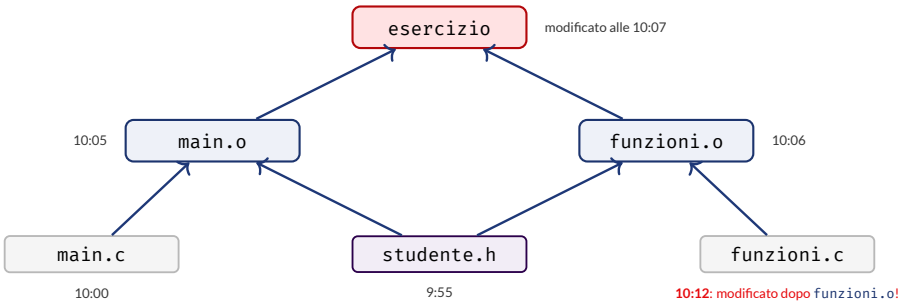


Figura 7.2: *Il grafo delle dipendenze del progetto: le frecce vanno da ciò che serve a ciò che si costruisce.* `funzioni.c` *è più recente di* `funzioni.o`*, quindi* `funzioni.o` *va ricostruito, e di conseguenza anche* `esercizio`*.*

Il ragionamento di `make` è quello che faremmo noi: per costruire un file, prima si assicura che siano aggiornati i file da cui dipende (e così via, all'indietro); poi, se il file **non esiste** oppure è **più vecchio** di almeno uno dei file da cui dipende, lo ricostruisce eseguendo i comandi indicati. Altrimenti non fa niente. Nella situazione della figura 7.2, `make` ricompila solo `funzioni.c` e rifà il linking; `main.c` non viene toccato.

Il makefile

`make` esegue le operazioni descritte in un file, chiamato di solito `makefile` o `Makefile`, che contiene la descrizione delle dipendenze, i comandi da eseguire e, eventualmente, delle definizioni di *macro*. Per usarlo basta scrivere `make` nella cartella che lo contiene; se il file ha un altro nome, per esempio `makeEsercizio`, lo si indica con l'opzione `-f`:

Terminale

```
$ make
$ make -f makeEsercizio
```

Ecco il primo makefile delle lezioni. Nei listati di questo capitolo il carattere di tabulazione (TAB) è disegnato come una freccia ⟶, come fa VS Code quando si chiede di mostrare gli spazi bianchi: il motivo sarà chiaro fra poco.

makefile1

```
1  # makefile di esempio
2  # versione 1
3  #
4
5  eseguibile: main.o funzioni.o
6  ⟶ gcc -o esercizio main.o funzioni.o
7
8  main.o: main.c
9  ⟶ gcc -c main.c
10
11 funzioni.o: funzioni.c
12 ⟶ gcc -c funzioni.c
```

I tipi di riga

In un makefile si distinguono quattro tipi di righe:

- le righe di **dipendenza**: definiscono una relazione fra un **target** (l'obiettivo, ciò che si costruisce) e uno o più file da cui dipende, separati dal carattere `:`. Per esempio `main.o: main.c`;
- le righe di **comando**: definiscono l'operazione da eseguire per ottenere il target, e seguono la sua riga di dipendenza. **Devono cominciare con un TAB**;
- le righe di **commento**: cominciano con `#`;
- le righe di definizione di **macro**, che vedremo fra poco.

Il TAB non è uno spazio

Le righe di comando devono cominciare con un carattere di **tabulazione**, non con degli spazi, anche se a occhio sono identici. Il file `makefile4` delle lezioni contiene proprio questo errore (quattro spazi al posto del TAB), e `make` si rifiuta di proseguire con un messaggio poco chiaro: *makefile4:9: *** missing separator. Stop.* VS Code inserisce davvero un TAB solo se riconosce il file come makefile, cioè se il file si chiama `makefile` o `Makefile`: in basso a destra, nella barra di stato, compare allora *Tab Size* invece di *Spaces*.

Che cosa fa `make`, passo per passo

Proviamo il primo makefile. La prima volta mancano tutti i file oggetto e l'eseguibile, quindi `make` esegue tutto, cominciando dai file da cui dipende il primo target:

Terminale

```
$ make -f makefile1
gcc -c main.c
gcc -c funzioni.c
gcc -o esercizio main.o funzioni.o
```

`make` stampa ogni comando prima di eseguirlo. Si noti l'ordine: per costruire `eseguibile` servono `main.o` e `funzioni.o`, quindi prima vengono costruiti loro. Se ora modifichiamo soltanto `funzioni.c` (il comando `touch` cambia la data di un file, come se lo avessimo modificato e salvato) e rilanciamo `make`, viene ricompilato solo quello:

Terminale

```
$ touch funzioni.c
$ make -f makefile1
gcc -c funzioni.c
gcc -o esercizio main.o funzioni.o
```

Le dipendenze dall'header

Che cosa succede se modifichiamo `studente.h`? Con il primo makefile, niente di buono:

Terminale

```
$ touch studente.h
$ make -f makefile1
gcc -o esercizio main.o funzioni.o
```

Nessun file viene ricompilato, perché nel makefile i file oggetto dipendono solo dai rispettivi `.c`. Ma entrambi i `.c` **includono** `studente.h`: se nell'header cambia, per esempio, la struttura `studente`, i due file oggetto sono stati compilati con la versione vecchia e il programma risultante è incoerente, con errori imprevedibili. La soluzione è dichiarare la dipendenza:

makefile2

```
# makefile di esempio
# versione 2

eseguibile: main.o funzioni.o
	gcc -o esercizio main.o funzioni.o

main.o: main.c studente.h
	gcc -c main.c

funzioni.o: funzioni.c studente.h
	gcc -c funzioni.c
```

Terminale

```
$ make -f makefile2
gcc -c main.c
gcc -c funzioni.c
gcc -o esercizio main.o funzioni.o
```

Ora, dopo una modifica all'header, vengono ricompilati entrambi i sorgenti. La regola generale: **un file oggetto dipende dal suo `.c` e da tutti gli header (nostri) che questo include**.

Il nome del target

Un'altra stranezza: con i primi due makefile il linking viene eseguito **ogni volta**, anche se nulla è cambiato. Possiamo evitarlo? Il motivo è il nome del target: `eseguibile`. `make` controlla se esiste un file con quel nome e se è aggiornato; ma il comando produce un file chiamato `esercizio`, quindi un file `eseguibile` non esiste mai, e `make` rifà il linking per crearlo. Basta che il target abbia lo stesso nome del file prodotto:

makefile3

```
# makefile di esempio
# versione 3

esercizio: main.o funzioni.o
	gcc -o esercizio main.o funzioni.o

main.o: main.c studente.h
	gcc -c main.c

funzioni.o: funzioni.c studente.h
	gcc -c funzioni.c
```

Terminale

```
$ make -f makefile3
make: 'esercizio' is up to date.
```

Tutto è aggiornato, e `make` lo dice senza eseguire nulla.

Le macro

Una riga di definizione di **macro** definisce un nome simbolico per un insieme di oggetti, con la sintassi `NOME = valore`; il nome si usa poi scrivendo `$(NOME)`. È l'equivalente del `#define` del C: si scrive l'elenco una volta sola, e se cambia si modifica in un solo punto.

makefile4

```
# makefile di esempio
# versione 4

OBJ = funzioni.o main.o

esercizio: $(OBJ)
	gcc -o esercizio $(OBJ)

main.o: main.c studente.h
	gcc -c main.c

funzioni.o: funzioni.c studente.h
	gcc -c funzioni.c
```

(Questa è la versione corretta, con il TAB; nel file distribuito con le lezioni la riga del linking comincia con quattro spazi e produce l'errore *missing separator*.)

UN MAKEFILE COMPLETO

Nella pratica i makefile usano quasi sempre qualche macro in più e un target di servizio per fare pulizia. Ecco un makefile per il progetto `aree` di inizio capitolo:

Makefile

```
CC = gcc
CFLAGS = -Wall
OBJ = main.o aree.o

aree: $(OBJ)
	$(CC) $(CFLAGS) -o aree $(OBJ)

main.o: main.c aree.h
	$(CC) $(CFLAGS) -c main.c

aree.o: aree.c aree.h
	$(CC) $(CFLAGS) -c aree.c

clean:
	rm -f aree $(OBJ)
```

Terminale

```
$ make
gcc -Wall -c main.c
gcc -Wall -c aree.c
gcc -Wall -o aree main.o aree.o
$ make
make: 'aree' is up to date.
$ touch main.c
$ make
gcc -Wall -c main.c
gcc -Wall -o aree main.o aree.o
$ make clean
rm -f aree main.o aree.o
```

- `CC` e `CFLAGS` sono i nomi tradizionali delle macro per il compilatore e le sue opzioni: per aggiungere un'opzione a tutte le compilazioni (per esempio `-g`, che serve al debugger) basta cambiare una riga;
- senza argomenti, `make` costruisce il **primo** target del file (qui `aree`); con `make main.o` si costruisce solo quello;
- `clean` non corrisponde a nessun file e non ha dipendenze: è un target "di servizio", che con `make clean` cancella l'eseguibile e i file oggetto, per ripartire da zero.

make non è solo per il C

`make` non sa nulla di C: sa solo che certi file si costruiscono da altri file eseguendo dei comandi, se sono più vecchi. Per questo si usa anche per compilare documenti, generare siti, elaborare dati: ovunque ci sia una catena di file da ricostruire quando qualcosa cambia.

In sintesi

Un programma si divide in più file `.c`, ciascuno compilato separatamente in un file oggetto (`gcc -c`), e in header `.h` con le dichiarazioni condivise; il linker unisce gli oggetti, abbinando ogni funzione usata alla sua definizione (*undefined reference* vuol dire che una definizione manca). `make` legge un **makefile** fatto di righe di dipendenza (`target: file`), righe di comando (che cominciano con un **TAB**), commenti e macro, e ricostruisce un target solo se non esiste o è più vecchio di qualcosa da cui dipende. Gli header vanno elencati fra le dipendenze, e il nome del target deve essere quello del file prodotto.

Riferimenti

- **GNU make — An Introduction to Makefiles**, il manuale ufficiale.
  https://www.gnu.org/software/make/manual/html_node/Introduction.html
- **Deitel & Deitel**, *Il linguaggio C*: appendice sulla compilazione di programmi su più file.

Tipi di dato

Nel capitolo 2 abbiamo visto che la memoria contiene soltanto bit, e che lo stesso byte può essere letto come un numero, come una lettera o come un pezzo di qualcos'altro: è il **tipo** a stabilire quanti byte leggere e come interpretarli. Questo capitolo guarda da vicino i tipi fondamentali del C: i caratteri, gli interi e i reali. Per ciascuno vedremo quanta memoria occupa, quali valori può contenere e che cosa succede quando si superano i suoi limiti; poi come si leggono e si scrivono i caratteri uno alla volta, come si usano le funzioni matematiche e con quali regole il C converte un tipo in un altro.

Dichiarazioni, espressioni, assegnazioni

Tre concetti del capitolo 5 torneranno di continuo, quindi conviene richiamarli (figura 8.1):

- una **dichiarazione**, come `int i;`, riserva in memoria lo spazio per una variabile del tipo indicato e le dà un nome; finché non le si assegna nulla, la variabile contiene un valore a caso;
- un'**assegnazione**, come `i = 7;`, scrive un valore nella variabile;
- un'**espressione**, come `i * 2 + 1`, viene calcolata e produce un risultato che ha un **valore** (qui 15) e un **tipo** (qui `int`). Anche un'assegnazione è un'espressione: `i = 7` vale 7.

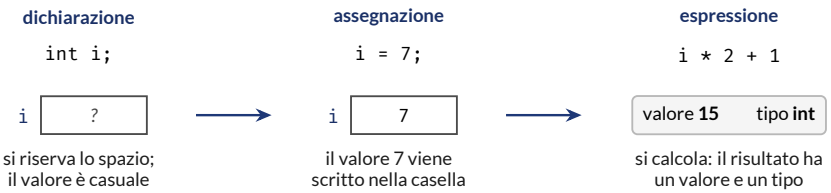


Figura 8.1: *Dichiarazione, assegnazione ed espressione.*

I TIPI FONDAMENTALI

Forma completa e forma abbreviata

Il C ha dodici tipi **fondamentali**. Ciascuno ha una forma completa, che dice tutto esplicitamente, e una forma abbreviata, quella che si usa nella pratica:

forma completa	si scrive	forma completa	si scrive
`char`	`char`	`unsigned short int`	`unsigned short`
`signed char`	`signed char`	`unsigned int`	`unsigned`
`unsigned char`	`unsigned char`	`unsigned long int`	`unsigned long`
`signed short int`	`short`	`float`	`float`
`signed int`	`int`	`double`	`double`
`signed long int`	`long`	`long double`	`long double`

- Sono tutte **parole chiave** (capitolo 6): non si possono usare come nomi di variabili.
- Per `short`, `int` e `long` il segno è sottinteso: `signed int` è la stessa cosa di `int`, quindi la parola `signed` di solito non si scrive. Anche `signed` da solo equivale a `int`, ma si usa raramente.
- `short int`, `long int` e `unsigned int` si abbreviano in `short`, `long` e `unsigned`.
- Per i caratteri, invece, `char`, `signed char` e `unsigned char` sono tre tipi distinti: vedremo fra poco perché.

Tipi interi e tipi reali

In base a ciò che rappresentano, i tipi fondamentali formano due famiglie (figura 8.2). I **tipi interi** (*integral types*) contengono numeri interi e, come vedremo, caratteri; i **tipi reali** (*floating types*) contengono numeri con la virgola. Insieme formano i **tipi aritmetici**, quelli su cui si fanno i calcoli.

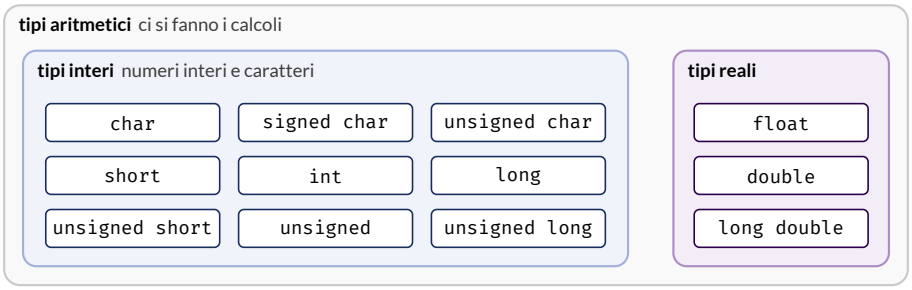


Figura 8.2: *I tipi fondamentali raggruppati per famiglia.*

I tipi derivati

A partire dai tipi fondamentali se ne costruiscono altri, i **tipi derivati** (figura 8.3): gli **array** e i **puntatori** (capitolo 11), le **strutture** (capitolo **??**), le enumerazioni e le unioni. In questo capitolo ci occupiamo solo dei fondamentali.

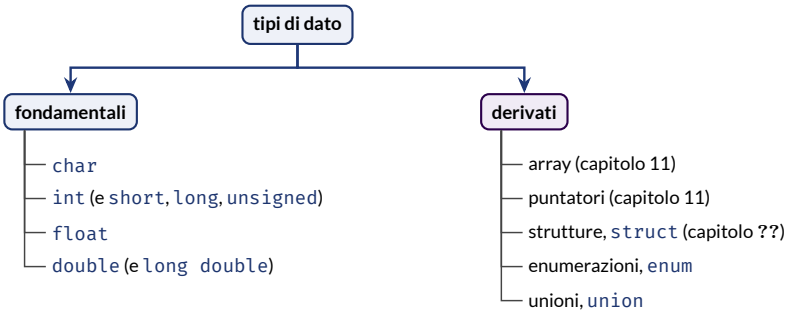


Figura 8.3: *Tipi fondamentali e tipi derivati.*

CARATTERI E IL TIPO `char`

Cominciamo dal tipo più piccolo, `char`, che serve a memorizzare i caratteri: lettere, cifre, simboli.

Un carattere è un numero

Un computer non conosce le lettere: conosce solo numeri. Per memorizzare un carattere lo si sostituisce con un numero, secondo una tabella stabilita una volta per tutte, la **codifica**. Un `char` occupa **1 byte**, quindi può assumere $2^8 = 256$ valori diversi: abbastanza per le lettere maiuscole e minuscole, le cifre, la punteggiatura e alcuni caratteri speciali come lo spazio, la tabulazione e l'a capo. Non tutti i 256 valori, però, corrispondono a caratteri stampabili.

In C ogni tipo intero può rappresentare un carattere; in pratica si usano `char` e `int`. Le funzioni `getchar` e `putchar`, che leggono e scrivono un carattere, lavorano con degli `int`; e una costante carattere come `'a'` è di tipo `int`, non `char` (in C++ invece è un `char`). Lo verificheremo con l'operatore `sizeof`.

La codifica ASCII

Quasi tutti i computer usano la codifica **ASCII** (*American Standard Code for Information Interchange*), che assegna un numero da 0 a 127 a ogni carattere (figura 8.4). Esistono altre codifiche, come la EBCDIC di IBM, usata sui grandi calcolatori (*mainframe*), ma sono rare.

0	\0	16	DLE	32	␣	48	0	64	@	80	P	96	`	112	p
1	SOH	17	DC1	33	!	49	1	65	A	81	Q	97	a	113	q
2	STX	18	DC2	34	"	50	2	66	B	82	R	98	b	114	r
3	ETX	19	DC3	35	#	51	3	67	C	83	S	99	c	115	s
4	EOT	20	DC4	36	$	52	4	68	D	84	T	100	d	116	t
5	ENQ	21	NAK	37	%	53	5	69	E	85	U	101	e	117	u
6	ACK	22	SYN	38	&	54	6	70	F	86	V	102	f	118	v
7	\a	23	ETB	39	'	55	7	71	G	87	W	103	g	119	w
8	\b	24	CAN	40	(	56	8	72	H	88	X	104	h	120	x
9	\t	25	EM	41	)	57	9	73	I	89	Y	105	i	121	y
10	\n	26	SUB	42	*	58	:	74	J	90	Z	106	j	122	z
11	\v	27	ESC	43	+	59	;	75	K	91	[	107	k	123	{
12	\f	28	FS	44	,	60	<	76	L	92	\	108	l	124	|
13	\r	29	GS	45	-	61	=	77	M	93	]	109	m	125	}
14	SO	30	RS	46	.	62	>	78	N	94	^	110	n	126	~
15	SI	31	US	47	/	63	?	79	O	95	_	111	o	127	DEL

caratteri di controllo (non stampabili) · spazio e simboli · cifre · maiuscole · minuscole

Figura 8.4: *La tabella ASCII: accanto a ogni codice (da 0 a 127) il carattere corrispondente. I caratteri di controllo hanno un nome convenzionale, o la sequenza di escape con cui si scrivono in C; ␣ è lo spazio.*

La tabella non va imparata a memoria: in C si scrive `'A'` e il compilatore lo sostituisce con 65. Conviene però capirne la struttura:

- i codici da 0 a 31 e il 127 sono **caratteri di controllo**: non si stampano, ma comandano il dispositivo (il 10, per esempio, è l'a capo); il 32 è lo spazio;
- la scelta dei numeri è una **convenzione**: nella lettera *A* non c'è nulla che la leghi a 65;
- però le cifre, le maiuscole e le minuscole occupano ciascuna dei codici **consecutivi** e in ordine: `'0'` < `'1'` < … < `'9'`, `'a'` < `'b'` < … < `'z'`. Confrontare i codici equivale quindi a confrontare nell'ordine alfabetico, e questo servirà per ordinare i testi;
- le maiuscole vengono prima delle minuscole, e ogni minuscola dista dalla sua maiuscola esattamente 32 posizioni: `'a'` vale 97, `'A'` vale 65;
- il carattere `'7'` non è il numero 7: vale 55. Il numero si ottiene con la differenza `'7' - '0'`, cioè 55 − 48 = 7.

E le lettere accentate?

La tabella ASCII non contiene *è*, *à*, *ù*. I sistemi di oggi (Linux, macOS, VS Code) usano la codifica **UTF-8**, che estende l'ASCII: i primi 128 codici sono gli stessi, mentre gli altri caratteri occupano da 2 a 4 byte. Una lettera accentata, quindi, **non sta in un `char`**: `char c = 'è';` produce due avvisi (*multi-character character constant* e *overflow in conversion*) e un valore senza senso. Dentro una stringa, invece, non c'è problema: `printf("Perché?\n");` stampa correttamente, perché la stringa contiene semplicemente i due byte della *é*.

Caratteri come interi, interi come caratteri

Poiché un `char` contiene un numero, lo si può stampare in due modi: con `%c` `printf` disegna il carattere che ha quel codice, con `%d` scrive il numero. E siccome è un numero, lo si può usare nelle espressioni aritmetiche: `c + 1` è il codice del carattere successivo.

```c
char c = 'a';

printf("%c\n", c);
printf("%d\n", c);
printf("%c%c%c\n", c, c + 1, c + 2);
```

Terminale

```
a
97
abc
```

Con la stessa idea si scorrono i caratteri con un ciclo: la variabile di controllo parte da un codice e arriva a un altro. Nel programma seguente il primo ciclo usa un `int` che va da `'a'` a `'z'`, il secondo un `char` che va da 65 a 90 (i codici di `'A'` e `'Z'`), il terzo stampa con `%d` i codici delle dieci cifre.

caratteri.c

```c
#include <stdio.h>

int main(void)
{
   int i;
   char c;

   /* stampa abc...z */
   for (i = 'a'; i <= 'z'; i++)
      printf("%c", i);
   printf("\n");

   /* stampa ABC...Z */
   for (c = 65; c <= 90; c++)
      printf("%c", c);
   printf("\n");

   /* stampa 48 49 ... 57 */
   for (c = '0'; c <= '9'; c++)
      printf("%d ", c);
   printf("\n");
   return 0;
}
```

Terminale

```
$ ./caratteri
abcdefghijklmnopqrstuvwxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZ
48 49 50 51 52 53 54 55 56 57
```

Una svista nelle slide

Nelle slide il secondo ciclo è scritto `for (c=65; i<=90; c++)`: la condizione controlla `i` invece di `c`. Alla fine del primo ciclo `i` vale 123 (il primo valore per cui `i <= 'z'` è falsa), quindi `i <= 90` è falsa fin dall'inizio e il ciclo non stampa nulla. Il compilatore non può accorgersene: il codice è corretto dal punto di vista della sintassi, e fa semplicemente un'altra cosa.

LE SEQUENZE DI ESCAPE

Caratteri che non si possono scrivere

Alcuni caratteri non si possono scrivere direttamente fra apici o fra virgolette: o perché non sono stampabili (l'a capo, la tabulazione), o perché hanno già un significato (le virgolette chiudono una stringa, l'apice chiude una costante carattere). Per questi il C usa le **sequenze di escape**: un backslash seguito da un carattere. Sono **due simboli** nel codice, ma **un solo carattere** in memoria: `'\n'` è il carattere di codice 10.

carattere	in C	codice	che cosa fa
a capo (*newline*)	`\n`	10	porta il cursore all'inizio della riga successiva
tabulazione (*horizontal tab*)	`\t`	9	sposta il cursore alla prossima tabulazione (ogni 8 colonne)
ritorno carrello (*carriage return*)	`\r`	13	riporta il cursore all'inizio della riga
tabulazione verticale (*vertical tab*)	`\v`	11	scende di una riga, restando nella stessa colonna
salto pagina (*formfeed*)	`\f`	12	sulla stampante cambia pagina; nel terminale fa come `\v`
indietro (*backspace*)	`\b`	8	sposta il cursore indietro di una posizione
campanello (*alert*)	`\a`	7	emette un segnale acustico (o visivo)
carattere nullo (*null*)	`\0`	0	il carattere di codice 0, che chiude le stringhe
backslash	`\\`	92	il carattere `\`
virgolette (*double quote*)	`\"`	34	il carattere `"`
apice (*single quote*)	`\'`	39	il carattere `'`
punto interrogativo	`\?`	63	il carattere `?` (serve di rado)

Virgolette, apici e campanello

Per stampare delle virgolette dentro una stringa bisogna scriverle con il backslash, altrimenti chiuderebbero la stringa. Il programma delle lezioni lo fa in un modo un po' tortuoso, stampando con `%c` il carattere `'\"'`; il modo diretto è la seconda riga:

```c
printf("Di nome %cNemo%c\n", '\"', '\"');
printf("Di nome \"Nemo\"\n");
```

```
Terminale
Di nome "Nemo"
Di nome "Nemo"
```

Fra apici le virgolette non danno fastidio, quindi anche `'"'` va bene; viceversa, l'apice va protetto fra apici (`'\''`) ma non in una stringa (`"l'una"`).

`printf("%c", '\a');` fa suonare il **campanello** del terminale: un'eredità delle telescriventi, che ne avevano uno vero. Nel terminale di VS Code, a seconda delle impostazioni, si sente un suono, la finestra lampeggia o non succede nulla.

Il cursore e i caratteri di controllo

Il programma seguente stampa la stessa stringa con in mezzo, a turno, diversi caratteri di controllo. Ecco che cosa compare sul terminale:

escape.c

```c
#include <stdio.h>

int main(void)
{
  printf("abc \t abc\n");
  printf("abc \v abc\n");
  printf("abc \f abc\n");
  printf("abc \r abc\n");
  printf("abc \r \v abc\n");
  return 0;
}
```

Terminale

```
$ ./escape
abc     abc
abc
    abc
abc
    abc
 abc
 bc
  abc
```

Per capirlo basta seguire il cursore (figura 8.5). Ogni carattere stampabile viene scritto dove si trova il cursore, che poi avanza di una colonna; i caratteri di controllo, invece, spostano soltanto il cursore.

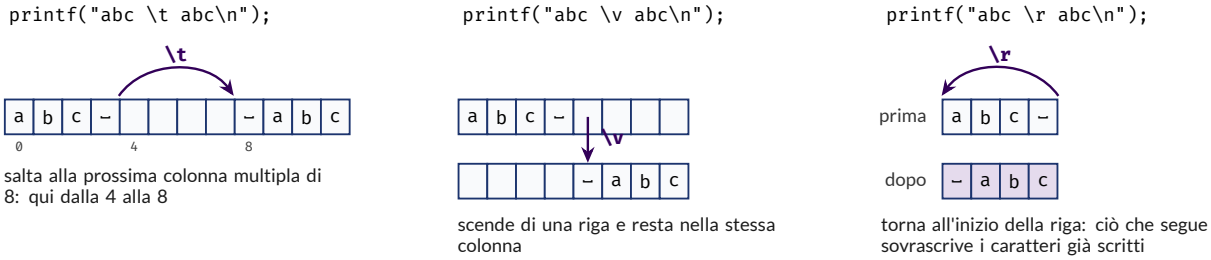


Figura 8.5: *Il percorso del cursore con* `\t`*,* `\v` *e* `\r` *(␣ è uno spazio).*

- `\t` sposta il cursore alla prossima **tabulazione**; le tabulazioni sono ogni 8 colonne (0, 8, 16…), e per questo servono ad allineare i dati in colonna.
- `\v` e `\f` fanno scendere il cursore di una riga, ma senza riportarlo a sinistra.
- `\r` riporta il cursore all'inizio della riga **senza** scendere: ciò che si stampa dopo sovrascrive. Così `"abc "` diventa `" abc"`.
- L'ultima riga combina le due cose: `\r` torna all'inizio, lo spazio sovrascrive la `a` (si legge `" bc"`), `\v` scende di una riga restando nella colonna 1, e lì viene scritto `" abc"`.

E `\n`? Nel terminale fa due cose insieme: scende di una riga **e** torna all'inizio. Sulle macchine da scrivere erano due movimenti distinti, riportare il carrello all'inizio (*carriage return*) e far scorrere il foglio di una riga (*line feed*): per questo i file di testo di Windows terminano ogni riga con due caratteri, `\r\n`, mentre Linux e macOS usano solo `\n`.

Un'animazione con `\r`

Il ritorno carrello permette di ridisegnare più volte lo stesso punto dello schermo. Il programma seguente stampa a ripetizione i caratteri `|`, `/`, `-` e `\`, ciascuno seguito da `\r`, con una pausa di un secondo fra l'uno e l'altro: ogni simbolo prende il posto del precedente e si vede una stanghetta che ruota.

spinner.c

```c
#include <stdio.h>
#include <unistd.h>

#define PAUSA 1

void pausa(int l)
{
    sleep(l);
}

int main()
{
    setbuf(stdout, NULL);
    while (1) {
        printf("|\r");  pausa(PAUSA);
        printf("/\r");  pausa(PAUSA);
        printf("-\r");  pausa(PAUSA);
        printf("\\\r"); pausa(PAUSA);
    }
}
```

- `"\\\r"` contiene due caratteri: il backslash (scritto `\\`) e il ritorno carrello.
- `while (1)` è un ciclo infinito, perché la condizione è sempre vera: il programma si ferma con Ctrl + C.
- `sleep(l)`, dichiarata in `unistd.h`, sospende il programma per `l` secondi. È disponibile su Linux, su macOS e in WSL, non su Windows senza WSL.

Nella cartella degli esempi c'è anche una versione senza `sleep`, in cui `pausa` perde tempo con un ciclo vuoto: `for (i = 0; i < l; i++);` (il `;` da solo è l'istruzione vuota). Funziona, ma la durata della pausa dipende dalla velocità del computer; e compilando con l'ottimizzazione (`gcc -O2`) il compilatore si accorge che il ciclo non fa nulla e lo elimina: in due secondi la versione ottimizzata ha stampato oltre 2,8 milioni di caratteri, quella normale 70.

Il buffer di `printf`

Resta da capire `setbuf(stdout, NULL)`. Per efficienza, `printf` non manda ogni carattere allo schermo appena lo riceve: lo accumula in una zona di memoria, il **buffer**, e la svuota tutta insieme quando incontra un `\n`, quando è piena o quando il programma termina (figura 8.6). Il nostro programma non stampa mai un `\n` e non termina mai: senza `setbuf` il buffer non verrebbe mai svuotato e sullo schermo **non comparirebbe nulla**. L'abbiamo verificato: senza quella riga, in tre secondi il programma non mostra niente. `setbuf(stdout, NULL)` elimina il buffer dell'output standard (`stdout`), e ogni carattere arriva subito allo schermo.

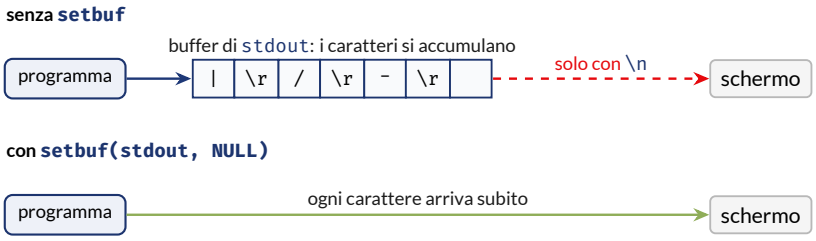


Figura 8.6: *Con il buffer l'output arriva allo schermo a righe intere; senza, un carattere alla volta.*

ATTENZIONE A ++ DENTRO `printf`

Le slide mettono in guardia: con gli operatori `++` e `--` a volte i risultati **non sono prevedibili**. Il programma seguente stampa più volte `c` e `a` incrementandoli dentro la stessa chiamata di `printf`. Compilato con `gcc` dà i risultati delle slide; compilato con `clang` dà risultati del tutto diversi:

printf++.c

```c
#include <stdio.h>

int main()
{
  char c;  int a;

  c = 'a'; a=10;
  printf("%c%c%c\n", c, c, c++);
  printf ("%d %d %d \n", a , a , a++);

  c = 'a'; a=10;
  printf("%c%c%c\n", c, c, ++c);
  printf("%d %d %d \n", a , a , ++a);

  c = 'a'; a=10;
  printf("%c%c%c\n", c, ++c, ++c);
  printf("%d %d %d \n", a , ++a , ++a);

  c = 'a'; a=10;
  printf("%c%c%c\n", c, c++, c++);
  printf("%d %d %d \n", a , a++ , a++);

  c = 'a'; a=10;
  printf("%c%c%c \n", ++c , c++ , c);
  printf("%d %d %d \n", ++a , a++ , a);
}
```

gcc

```
bba
11 11 10
bbb
11 11 11
ccb
12 12 12
cba
12 11 10
caa
12 10 12
```

clang

```
aaa
10 10 10
aab
10 10 11
abc
10 11 12
aab
10 10 11
bbc
11 11 12
```

Nessuno dei due compilatori sbaglia. Il C non stabilisce in che ordine vengano calcolati gli argomenti di una funzione (qui `gcc` li ha calcolati da destra a sinistra, `clang` da sinistra a destra); e modificare una variabile con `++` e usarla di nuovo nella stessa chiamata dà un comportamento **indefinito**, come `a * a++` nel capitolo 6. Con `-Wall`, `gcc` lo segnala per ogni riga:

Terminale

```
$ gcc -Wall -o printf++ printf++.c
printf++.c: In function ‘main’:
printf++.c:8:29: warning: operation on ‘c’ may be undefined [-Wsequence-point]
    8 |   printf("%c%c%c\n", c, c, c++);
      |                             ~^~
```

La regola pratica: una variabile incrementata con `++` o decrementata con `--` **non va usata altrove** nella stessa espressione o nella stessa chiamata. Si incrementa prima, in un'istruzione a parte.

COME È FATTO UN `char`

Vediamo ora che cosa c'è davvero nel byte di un `char`, e perché esistono tre tipi diversi per i caratteri.

I bit di un carattere

La notazione binaria (capitolo 2) è **posizionale** come quella decimale. In base 10 il numero $d_n d_{n-1} \dots d_1 d_0$ vale $d_n \cdot 10^n + \dots + d_1 \cdot 10^1 + d_0 \cdot 10^0$ (per esempio $20753 = 2 \cdot 10^4 + 0 \cdot 10^3 + 7 \cdot 10^2 + 5 \cdot 10^1 + 3 \cdot 10^0$: il 3 è la cifra meno significativa, il 2 la più significativa). In base 2 ogni cifra b_i è un bit, 0 o 1, e il numero vale $b_n \cdot 2^n + \dots + b_1 \cdot 2^1 + b_0 \cdot 2^0$. Dopo la dichiarazione `char c = 'a';` il byte di `c` contiene `01100001` (figura 8.7).

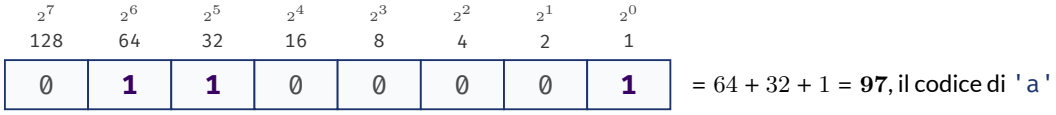


Figura 8.7: *Il byte del carattere `'a'`: ogni bit a 1 aggiunge il peso della sua posizione.*

Tre tipi per i caratteri

Il C ha tre tipi da un byte: `char`, `signed char` e `unsigned char`. Ognuno ha 256 valori distinti, ma non gli stessi:

- `signed char` va da −128 a 127;
- `unsigned char` va da 0 a 255;
- `char` è uno dei due, a seconda dell'**implementazione** (cioè del compilatore e del processore).

Per i caratteri ASCII (da 0 a 127) la differenza non conta. Conta quando il byte si usa come numero: per capirla bisogna vedere come si rappresentano i numeri negativi.

I numeri negativi

L'idea più semplice è il **bit di segno**. Limitiamoci a 4 bit: il bit più significativo indica il segno e gli altri tre il valore, quindi 5 = `0101` e −5 = `1101`. Lo zero però ha due rappresentazioni, `0000` e `1000`, e così un valore va sprecato.

Per questo i computer usano il **complemento a due**: i numeri positivi si rappresentano normalmente, e per un negativo si **invertono** i bit del corrispondente positivo e si **somma 1**. Così `1000` non è più uno zero, ma −8 (figura 8.8).

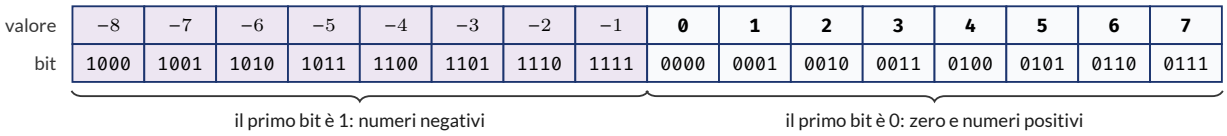


Figura 8.8: *I 16 valori di 4 bit in complemento a due: da −8 a +7, con un solo zero.*

5	`0101`
bit invertiti	`1010`
sommo 1	`+ 0001`
−5	`1011`

Per esempio, −5 si ottiene da `0101`: invertendo i bit si ha `1010`, e sommando 1 si arriva a `1011`. Perché funziona? Un numero più il suo inverso dà sempre `1111` (in ogni colonna c'è un 1 e uno 0, senza riporti), cioè −1. Quindi l'inverso di x vale $-1 - x$, e aggiungendo 1 si ottiene proprio $-x$.

Lo stesso byte, due letture

Su 8 bit funziona allo stesso modo. I byte da `00000000` a `01111111` hanno lo stesso valore, da 0 a 127, per `signed char` e per `unsigned char`. Quelli che cominciano con un 1 no: per `unsigned char` vanno da 128 a 255, per `signed char` da −128 a −1 (figura 8.9). Il byte `11111111`, per esempio, vale 255 oppure −1.

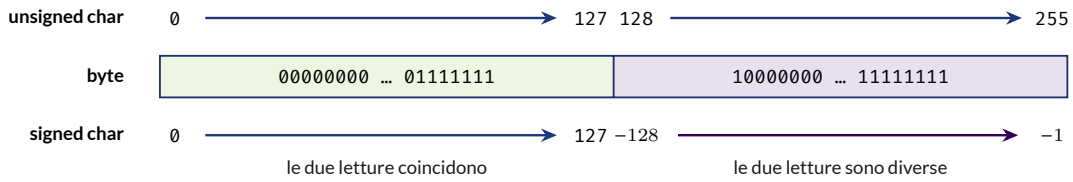


Figura 8.9: *I 256 byte letti come* `unsigned char` *(sopra) e come* `signed char` *(sotto).*

Il `char` del mio computer ha il segno?

Come si scopre se `char` è `signed` o `unsigned`? Si assegna −1 ai tre tipi e si stampano i valori con `%d`:

```c
#include <stdio.h>

int main()
{
    char c = -1;
    unsigned char u = -1;
    signed char s = -1;

    printf("c = %d  u = %d   s = %d\n", c, u, s);

    return 0;
}
```

```
$ gcc -Wall -o char char.c
$ ./char
c = -1  u = 255   s = -1
$ gcc -Wall -funsigned-char -o char char.c
$ ./char
c = 255  u = 255   s = -1
```

In tutte e tre le variabili il byte è `11111111`: −1 assegnato a un `unsigned char`, che non ha negativi, diventa −1 + 256 = 255. La differenza sta nella lettura: per stampare, il valore viene convertito in `int` e conserva il suo significato, quindi −1 per il tipo con segno e 255 per quello senza. Sui PC (processori Intel e AMD) `char` ha il segno, e infatti `c` vale −1. Su altri sistemi, per esempio Linux sui processori ARM come quelli del Raspberry Pi, `char` è senza segno; lo si può simulare con l'opzione `-funsigned-char` di `gcc`, ed ecco `c = 255`.

La conclusione pratica: per i caratteri si usa `char`; quando un byte serve come **numero** piccolo e il segno conta, si scrive esplicitamente `signed char` o `unsigned char`.

Il tipo `int`

`int` è il tipo principale del C, quello che si usa per qualunque numero intero quando non ci sono motivi per sceglierne un altro.

Quanti valori

Un `int` occupa 2 byte (16 bit) sui sistemi vecchi e 4 byte (32 bit) su quasi tutti i sistemi moderni: il numero di valori distinti, quindi, dipende dal sistema. In complemento a due, con n bit si rappresentano gli interi da -2^{n-1} a $2^{n-1}-1$:

- con 16 bit: da $-2^{15} = -32\,768$ a $2^{15}-1 = 32\,767$;
- con 32 bit: da $-2^{31} = -2\,147\,483\,648$ a $2^{31}-1 = 2\,147\,483\,647$, circa più o meno 2 miliardi.

Il massimo è $2^{n-1}-1$ e non 2^{n-1} perché fra i valori con il primo bit a 0 c'è anche lo zero. Le slide chiamano questi estremi N_{min_int} e N_{max_int}: ogni variabile `int i` soddisfa $N_{min_int} \le i \le N_{max_int}$. In C sono disponibili come costanti simboliche, `INT_MIN` e `INT_MAX`, definite nell'header `limits.h`.

L'overflow

Che cosa succede se un calcolo supera `INT_MAX`? Il programma seguente somma due miliardi a due miliardi:

```c
#include <stdio.h>

#define BIG 2000000000   /* 2 miliardi */

int main(void)
{
    int a, b = BIG, c = BIG;

    a = b + c;
    printf("a = %d\n", a);
    return 0;
}
```

```
$ gcc -Wall -o overflow overflow.c
$ ./overflow
a = -294967296
```

Il codice è sintatticamente corretto e il compilatore non segnala nulla; ma in esecuzione $b + c$ vale 4 miliardi, più di `INT_MAX`: è un **integer overflow** (traboccamento). Il programma prosegue come se niente fosse, con un risultato sbagliato. La figura 8.10 mostra da dove viene proprio quel numero: superato il massimo si ricomincia dal minimo, come il contachilometri di un'auto che dopo 99999 torna a 00000 (la ruota del capitolo 2). Contando, si arriva a $4\,000\,000\,000 - 2^{32} = -294\,967\,296$.

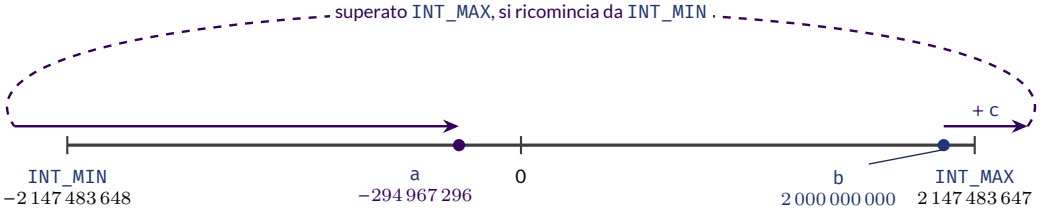


Figura 8.10: `b + c` *su 32 bit: la somma esce a destra e rientra da sinistra.*

Il risultato si spiega così: la somma viene fatta su 32 bit, e i bit che non ci stanno si perdono. Per i tipi `unsigned` questo "giro" è garantito dallo standard (i calcoli sono fatti modulo 2^n); per i tipi con segno lo standard parla invece di comportamento **indefinito**, e il valore ottenuto qui è solo ciò che succede in pratica sui nostri computer. In ogni caso tocca al programmatore assicurarsi che i valori restino nell'intervallo, per esempio usando un tipo più grande.

È quello che fa la versione di `overflow.c` nella cartella degli esempi: usa `unsigned long` (8 byte) e numeri ancora più grandi, 40 miliardi, e la somma, 80 miliardi, viene giusta.

```c
#include <stdio.h>

#define BIG 40000000000

int main()
{
    unsigned long int a, b, c;

    b = c = BIG;

    a = b + c;

    printf("a = %ld\n", a);

    return 0;
}
```

```
$ ./overflow
a = 80000000000
```

(Per un `unsigned long` la conversione esatta di `printf` sarebbe `%lu`; qui `%ld` funziona perché il valore sta anche in un `long`.)

`short`, `long` E `unsigned`

Più piccoli, più grandi, senza segno

Oltre a `char` e `int`, gli altri tipi interi servono per usi più specifici:

- `short` occupa di solito meno memoria di un `int` (2 byte): si usa quando lo spazio conta, per esempio in tabelle con milioni di numeri piccoli;
- `long` si usa per valori grandi (8 byte su Linux e macOS a 64 bit);
- `unsigned` occupa quanto un `int`, ma non ha negativi: con 4 byte va da 0 a $2^{32} - 1 = 4\,294\,967\,295$.

I limiti esatti del proprio sistema si leggono in `limits.h`. Questo programma li stampa (l'output è quello di Linux su un PC a 64 bit):

limiti.c

```c
#include <stdio.h>
#include <limits.h>

int main(void)
{
    printf("signed char:   %d .. %d\n", SCHAR_MIN, SCHAR_MAX);
    printf("unsigned char: %d .. %d\n", 0, UCHAR_MAX);
    printf("char:          %d .. %d\n", CHAR_MIN, CHAR_MAX);
    printf("short:         %d .. %d\n", SHRT_MIN, SHRT_MAX);
    printf("int:           %d .. %d\n", INT_MIN, INT_MAX);
    printf("unsigned:      %u .. %u\n", 0u, UINT_MAX);
    printf("long:          %ld .. %ld\n", LONG_MIN, LONG_MAX);
    printf("unsigned long: %lu .. %lu\n", 0ul, ULONG_MAX);
    return 0;
}
```

Terminale

```
$ ./limiti
signed char:   -128 .. 127
unsigned char: 0 .. 255
char:          -128 .. 127
short:         -32768 .. 32767
int:           -2147483648 .. 2147483647
unsigned:      0 .. 4294967295
long:          -9223372036854775808 .. 9223372036854775807
unsigned long: 0 .. 18446744073709551615
```

Che cosa garantisce lo standard

Lo standard del C non fissa le dimensioni dei tipi, ma solo dei **minimi**: `char` ha almeno 8 bit, `short` e `int` almeno 16, `long` almeno 32; inoltre un `short` non è più grande di un `int`, che non è più grande di un `long`. Le dimensioni reali le sceglie l'implementazione: su Windows, per esempio, `long` ha 4 byte come `int`, mentre su Linux e macOS a 64 bit ne ha 8. Chi ha bisogno di dimensioni esatte trova nell'header `stdint.h` (C99) tipi come `int32_t` e `int64_t`.

Il tipo delle costanti intere

Anche le costanti hanno un tipo. A una costante intera scritta in decimale viene dato il primo tipo in grado di contenerla fra `int` e `long` (e, se non basta, `long long`, un tipo introdotto dal C99). Il tipo si può anche imporre con un **suffisso**: `U` per `unsigned`, `L` per `long`, `UL` per `unsigned long` (vanno bene anche le minuscole, ma la `l` si confonde facilmente con un 1). Per i reali, come vedremo, i suffissi sono `F` e `L`. Ecco i tipi assegnati dal compilatore ad alcune costanti:

costante	tipo	perché
`37`	`int`	sta in un `int`
`37U`	`unsigned`	suffisso `U`
`37L`	`long`	suffisso `L`
`37UL`	`unsigned long`	suffissi `U` e `L`
`2147483647`	`int`	è `INT_MAX`: sta ancora in un `int`
`2147483648`	`long`	non sta più in un `int`
`3.7`	`double`	le costanti reali sono `double`
`3.7F`	`float`	suffisso `F`
`3.7L`	`long double`	suffisso `L`
`'a'`	`int`	le costanti carattere sono `int`

(Le slide indicano come candidati `int`, `long` e `unsigned`: era all'incirca la regola del C più vecchio, il C89. Nel C99 una costante decimale senza suffisso non diventa mai `unsigned`; possono diventarlo quelle scritte in esadecimale o in ottale.)

I TIPI REALI

`float`, `double` e `long double`

L'ANSI C fornisce tre tipi reali: `float`, `double` e `long double`. Le costanti reali, come `3.7`, sono di tipo `double`; il tipo si può imporre con un suffisso: `3.7F` è un `float`, `3.7L` un `long double`.

Di solito il compilatore dà più memoria ai `double` che ai `float`, ma non è obbligatorio; su molte macchine un `float` occupa 4 byte e un `double` 8. Allo stesso modo un `long double` può avere più memoria di un `double` (sui PC ne occupa 16), ma non è obbligatorio.

I valori che un tipo reale può contenere si descrivono con due grandezze:

- la **precisione**: quante cifre significative, in notazione decimale, si possono rappresentare;
- l'**intervallo** (*range*): il valore più grande e il più piccolo che si possono rappresentare.

Su molte macchine un `float` ha una precisione di 6 cifre significative e un intervallo che va da circa 10^{-38} a 10^{38}. Si può pensarlo come un numero della forma

$$0.d_1d_2d_3d_4d_5d_6 \times 10^n, \qquad -38 \le n \le 38,$$

dove ogni d_i è una cifra decimale (figura 8.11). Le cifre stabiliscono la precisione, l'esponente n dice dove va la virgola e i suoi limiti stabiliscono l'intervallo. La virgola "galleggia" a seconda dell'esponente: con le stesse sei cifre si rappresenta 123.451 (esponente 3), 0.000123451 (esponente −3) o 123 451 000 (esponente 9). Da qui il nome *floating point*, virgola mobile.

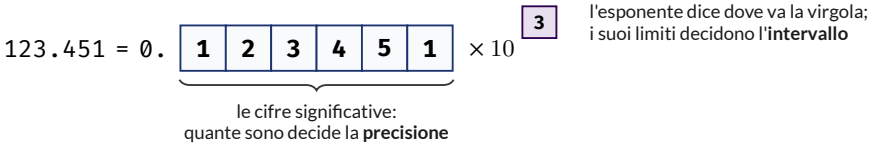


Figura 8.11: *Un* `float` *come sei cifre più un esponente.*

In realtà la macchina usa la base 2, non la base 10: le cifre sono binarie (24 per un `float`), e "6 cifre decimali" è il modo di dire quanta precisione garantiscono. I limiti esatti sono nell'header `float.h`:

reali.c

```c
#include <stdio.h>
#include <float.h>

int main(void)
{
    printf("float:  min %e  max %e  cifre %d\n",
           FLT_MIN, FLT_MAX, FLT_DIG);
    printf("double: min %e  max %e  cifre %d\n",
           DBL_MIN, DBL_MAX, DBL_DIG);
    return 0;
}
```

Terminale

```
$ ./reali
float:  min 1.175494e-38  max 3.402823e+38  cifre 6
double: min 2.225074e-308  max 1.797693e+308  cifre 15
```

Un `double`, su molte macchine, ha quindi circa 15 cifre significative e un intervallo da circa 10^{-308} a 10^{308}: si può pensare come $0.d_1d_2\ldots d_{15} \times 10^n$ con $-308 \le n \le 308$.

Ai confini del `float`

Il programma delle lezioni assegna a un `float` valori sempre più piccoli, poi due valori molto grandi e infine un numero con molte cifre:

```c
#include <stdio.h>

int main()
{
   float x;

   x = 1e-37; printf("x = 1e-37    x = %e\n", x);
   x = 1e-38; printf("x = 1e-38    x = %e\n", x);
   x = 1e-39; printf("x = 1e-39    x = %e\n", x);
   x = 1e-40; printf("x = 1e-40    x = %e\n", x);
   x = 1e-41; printf("x = 1e-41    x = %e\n", x);
   x = 1e-42; printf("x = 1e-42    x = %e\n", x);
   x = 1e-43; printf("x = 1e-43    x = %e\n", x);
   x = 1e-44; printf("x = 1e-44    x = %e\n", x);
   x = 1e-45; printf("x = 1e-45    x = %e\n", x);
   x = 1e-46; printf("x = 1e-46    x = %e\n", x);

   x = 1e38; printf("x = 1e38    x = %e\n", x);
   x = 1e39; printf("x = 1e39    x = %e\n", x);

   x = 123.45123451234512345;
   printf("x = 123.45123451234512345     x = %3.17f\n", x);
}
```

```
$ ./float
x = 1e-37    x = 1.000000e-37
x = 1e-38    x = 9.999999e-39
x = 1e-39    x = 1.000000e-39
x = 1e-40    x = 9.999946e-41
x = 1e-41    x = 9.999666e-42
x = 1e-42    x = 1.000527e-42
x = 1e-43    x = 9.949219e-44
x = 1e-44    x = 9.809089e-45
x = 1e-45    x = 1.401298e-45
x = 1e-46    x = 0.000000e+00
x = 1e38    x = 1.000000e+38
x = 1e39    x = inf
x = 123.45123451234512345
    x = 123.45123291015625000
```

(L'ultima riga dell'output è stata spezzata in due per farla stare nella pagina.) Che cosa è successo? La figura 8.12 riassume i valori positivi di un `float`:

- fino a `FLT_MIN`, circa 1.18×10^{-38}, i valori sono rappresentati con tutta la precisione: `1e-38` diventa `9.999999e-39`, cioè il `float` più vicino;
- sotto `FLT_MIN` i numeri si possono ancora rappresentare, ma con **sempre meno cifre esatte** (sono detti *subnormali*): `9.999946e-41`, `9.949219e-44`, `9.809089e-45`;
- il più piccolo `float` positivo è circa 1.4×10^{-45}: `1e-45` diventa `1.401298e-45`, e `1e-46`, ancora più piccolo, diventa **0**;
- sopra `FLT_MAX`, circa 3.4×10^{38}, non c'è più posto: `1e39` diventa `inf`, **infinito**.

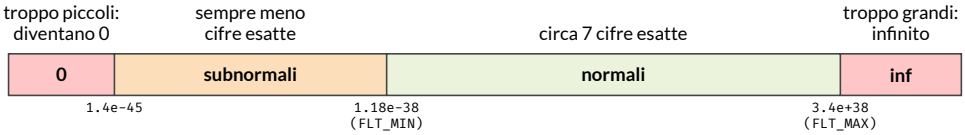


Figura 8.12: *I valori positivi di un* `float` *(la scala non è proporzionale). Per i negativi vale lo stesso, con il segno meno.*

L'ultima riga mostra la **precisione**: del numero `123.45123451234512345` il `float` conserva le prime cifre, poi inventa. La figura 8.13 confronta le cifre con quelle di un `double`, che di cifre giuste ne conserva più del doppio.

nel programma	`123.45123451234512345`	
float	`123.45123291015625000`	8 cifre giuste (ne sono garantite 6)
double	`123.45123451234512402`	17 cifre giuste (ne sono garantite 15)

Figura 8.13: *Lo stesso numero in un* `float` *e in un* `double`*: in verde le cifre esatte, in rosso quelle sbagliate.*

Ai confini del `double`

La stessa prova con un `double` (nell'ultima riga le cifre stampate sono 17, con `%.17e`). Questa volta alcune costanti sono così estreme che `gcc` protesta già in compilazione:

double.c

```c
#include <stdio.h>

int main()
{
   double x;

   x = 1e-307; printf("x = 1e-307    x = %e\n", x);
   x = 1e-308; printf("x = 1e-308    x = %e\n", x);
   x = 1e-309; printf("x = 1e-309    x = %e\n", x);
   x = 1e-400; printf("x = 1e-400    x = %e\n", x);
   x = 1e-401; printf("x = 1e-401    x = %e\n", x);
   x = 1e-402; printf("x = 1e-402    x = %e\n", x);
   x = 1e-403; printf("x = 1e-403    x = %e\n", x);
   x = 1e-404; printf("x = 1e-404    x = %e\n", x);
   x = 1e-405; printf("x = 1e-405    x = %e\n", x);
   x = 1e-406; printf("x = 1e-406    x = %e\n", x);

   x = 1e308; printf("x = 1e308    x = %e\n", x);
   x = 1e309; printf("x = 1e309    x = %e\n", x);

   x = 123.45123451234512345;
   printf("x = 123.45123451234512345     x = %.17e\n", x);
}
```

Terminale

```
$ ./double
x = 1e-307    x = 1.000000e-307
x = 1e-308    x = 1.000000e-308
x = 1e-309    x = 1.000000e-309
x = 1e-400    x = 0.000000e+00
x = 1e-401    x = 0.000000e+00
x = 1e-402    x = 0.000000e+00
x = 1e-403    x = 0.000000e+00
x = 1e-404    x = 0.000000e+00
x = 1e-405    x = 0.000000e+00
x = 1e-406    x = 0.000000e+00
x = 1e308    x = 1.000000e+308
x = 1e309    x = inf
x = 123.45123451234512345
    x = 1.23451234512345124e+02
```

Le costanti da `1e-400` a `1e-406` sono troppo piccole anche per un `double`, e `1e309` è troppo grande: il compilatore lo dice con un avviso per ciascuna riga (qui ne mostriamo due su otto) e le sostituisce con 0 e con infinito.

Terminale

```
$ gcc -Wall -o double double.c
double.c: In function ‘main’:
double.c:10:4: warning: floating constant truncated to zero [-Woverflow]
   10 |    x = 1e-400; printf("x = 1e-400    x = %e\n", x);
      |    ^
double.c:19:4: warning: floating constant exceeds range of ‘double’ [-Woverflow]
   19 |    x = 1e309; printf("x = 1e309    x = %e\n", x);
      |    ^
```

I reali non sono esatti

Con un numero finito di bit si rappresenta un numero finito di valori: la maggior parte dei numeri reali **non è rappresentabile**, e viene sostituita dal valore rappresentabile più vicino. Succede anche a numeri semplicissimi come 0.1: in base 2 è un numero periodico ($0.000110011001100\ldots$), esattamente come $1/3 = 0.333\ldots$ in base 10, e da qualche parte va troncato. Di conseguenza le operazioni con i reali, a differenza di quelle con gli interi, non sono necessariamente esatte:

decimali.c

```c
#include <stdio.h>

int main(void)
{
   double a = 0.1, b = 0.2;

   printf("0.1       = %.20f\n", a);
   printf("0.1 + 0.2 = %.20f\n", a + b);
   printf("0.3       = %.20f\n", 0.3);
   if (a + b == 0.3)
      printf("uguali\n");
   else
      printf("diversi!\n");
   return 0;
}
```

Terminale

```
$ ./decimali
0.1       = 0.10000000000000000555
0.1 + 0.2 = 0.30000000000000004441
0.3       = 0.29999999999999998890
diversi!
```

0.1 + 0.2 e 0.3 differiscono nella diciassettesima cifra, ma per `==` sono diversi. Per questo due reali **non si confrontano con `==`**: si controlla che la loro differenza sia piccola, per esempio con `fabs(a + b - 0.3) < 1e-9` (`fabs`, il valore assoluto, è descritta più avanti). Per i calcoli di piccola taglia questi errori sono trascurabili; nei problemi grandi (la risoluzione di grandi sistemi di equazioni differenziali, la minimizzazione di funzioni complicate) si accumulano, e controllarli è il compito dell'analisi numerica.

DARE UN NOME AI TIPI: `typedef`

Il meccanismo del `typedef` associa un identificatore a un tipo; da quel momento l'identificatore si usa per dichiarare variabili (o funzioni) esattamente come i tipi standard:

```c
typedef char          uppercase;
typedef int           inches;
typedef unsigned long size_t;
typedef unsigned char u_char;

uppercase u;
u_char    c;
inches    length, width;
```

A che cosa serve?

- ad **abbreviare** le dichiarazioni lunghe (`u_char` al posto di `unsigned char`);
- a scrivere nomi che dicono **l'uso** delle variabili: `length` e `width` sono lunghezze in pollici (*inches*);
- a gestire le **differenze fra sistemi**.

Il terzo punto si capisce con un esempio. Un programma scritto per un sistema con `int` di 4 byte va portato su un sistema dove gli `int` hanno 2 byte, troppo pochi per i suoi dati. Se il programma dichiara le sue variabili con un tipo proprio, basta cambiare **una riga**:

```c
/* sul primo sistema */
typedef int tipoInt;
```

```c
/* sul secondo sistema */
typedef long int tipoInt;
```

La libreria standard fa proprio così: `size_t`, il tipo dei risultati di `sizeof`, è definito con un `typedef` e cambia da un sistema all'altro. Il `typedef` sarà molto comodo con le strutture (capitolo **??**).

L'OPERATORE `sizeof`

Quanti byte

Il C fornisce l'operatore unario `sizeof`, che calcola il numero di byte necessari per rappresentare un oggetto. L'oggetto, fra parentesi, può essere un tipo (come `int` o `float`), un'espressione (come `a + b`), un array o una struttura. Ha la stessa priorità degli altri operatori unari. Il programma delle lezioni lo applica ai tipi fondamentali:

sizeofTypes.c

```c
/* Compute the size of some fundamental types. */

#include <stdio.h>

int main(void)
{
   printf("The size of some fundamental types is computed.\n\n");
   printf("       char:%3ld byte \n", sizeof(char));
   printf("      short:%3ld bytes\n", sizeof(short));
   printf("        int:%3ld bytes\n", sizeof(int));
   printf("       long:%3ld bytes\n", sizeof(long));
   printf("   unsigned:%3ld bytes\n", sizeof(unsigned));
   printf("      float:%3ld bytes\n", sizeof(float));
   printf("     double:%3ld bytes\n", sizeof(double));
   printf("long double:%3ld bytes\n", sizeof(long double));
   return 0;
}
```

Terminale

```
$ ./sizeofTypes
The size of some fundamental types is computed.

       char:  1 byte
      short:  2 bytes
        int:  4 bytes
       long:  8 bytes
   unsigned:  4 bytes
      float:  4 bytes
     double:  8 bytes
long double: 16 bytes
```

`sizeof(char)` vale 1 per definizione; gli altri valori sono quelli di Linux su un PC a 64 bit e possono cambiare altrove. Il risultato di `sizeof` è di tipo `size_t`, un intero senza segno; la conversione di `printf` pensata apposta è `%zu`, ma su Linux a 64 bit `size_t` è un `unsigned long` e `%ld` funziona.

Una sorpresa

Con `sizeof` verifichiamo quello che avevamo anticipato sulle costanti carattere:

dimensioni.c

```c
#include <stdio.h>

int main(void)
{
    char c = 'a';

    printf("sizeof(c)     = %zu\n", sizeof(c));
    printf("sizeof('a')   = %zu\n", sizeof('a'));
    printf("sizeof(c + c) = %zu\n", sizeof(c + c));
    return 0;
}
```

Terminale

```
$ ./dimensioni
sizeof(c)     = 1
sizeof('a')   = 4
sizeof(c + c) = 4
```

Una variabile `char` occupa 1 byte, ma la costante `'a'` ne occupa 4: è un `int`. E anche `c + c` è un `int`, per una regola che vedremo alla fine del capitolo, la *promozione a intero*. Si noti infine che `sizeof` guarda solo il **tipo** dell'espressione, senza calcolarla: `sizeof(i++)` non incrementa `i`.

LEGGERE E SCRIVERE UN CARATTERE ALLA VOLTA

`getchar` e `putchar`

Per leggere e scrivere un carattere alla volta la libreria standard offre due funzioni, dichiarate in `stdio.h`:

- `getchar()` legge il prossimo carattere dallo **standard input** (la tastiera, o un file se l'input è rediretto con `<`) e ne restituisce il codice;
- `putchar(c)` scrive sullo **standard output** il carattere di codice `c`.

Di solito sono realizzate come macro (direttive per il preprocessore), ma si usano come funzioni. E lavorano con gli `int`, perché un `int` può rappresentare qualsiasi carattere e, in più, un valore speciale che segnala la fine dell'input.

Il programma seguente legge una sequenza di caratteri e scrive ciascuno **due volte**. Lo proviamo redirigendo sullo standard input il file `input.txt`:

double_out.c

```c
#include <stdio.h>

int main(void)
{
    int c;

    while ((c = getchar()) != EOF) {
        putchar(c);
        putchar(c);
    }
    return 0;
}
```

Terminale

```
$ cat input.txt
abcdefghijklmno
$ ./double_out < input.txt
aabbccddeeffgghhiijjkkllmmnnoo

$
```

Da dove viene la riga vuota? Il file termina con un a capo, e anche `'\n'` è un carattere: raddoppiato, produce una riga vuota. Se invece si lancia il programma senza redirezione, i caratteri arrivano dalla tastiera:

Terminale

```
$ ./double_out
ciao
cciiaaoo

Ctrl+D
$
```

Il programma riceve i caratteri solo dopo Invio, tutti insieme: il terminale raccoglie la riga (così la si può correggere) e la consegna quando la si conferma. Poi `getchar` li restituisce uno alla volta. Per finire, come nel capitolo 5, si preme Ctrl + D a inizio riga.

La condizione `(c = getchar()) != EOF` fa tre cose in una (figura 8.14): legge un carattere, lo assegna a `c` e controlla che l'input non sia finito.

1	`while ((c = getchar()) != EOF)`	`getchar()` legge un carattere, per esempio `a`, e restituisce il suo codice, 97
2	`while ((c = 97) != EOF)`	il valore viene assegnato a `c`; l'assegnazione vale 97
3	`while (97 != EOF)`	97 è diverso da `EOF`: la condizione è vera
4	`while (1)`	si esegue il corpo, poi si ricomincia dal passo 1

Figura 8.14: *Il calcolo della condizione, passo per passo. Quando l'input finisce,* `getchar()` *restituisce* `EOF`*, il confronto* `EOF != EOF` *è falso e il ciclo termina.*

Tutte le parentesi sono necessarie. L'operatore `!=` ha la precedenza su `=` (capitolo 6), quindi senza le parentesi interne `c = getchar() != EOF` verrebbe letto come `c = (getchar() != EOF)`: a `c` andrebbe il risultato del confronto, 1 o 0, e il programma stamperebbe, al posto di ogni carattere, due caratteri di codice 1, invisibili. `gcc` con `-Wall` intuisce l'errore e suggerisce le parentesi:

Terminale

```
$ gcc -Wall -o senza_parentesi senza_parentesi.c
senza_parentesi.c: In function ‘main’:
senza_parentesi.c:7:11: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
    7 |     while (c = getchar() != EOF) {
      |            ^
```

EOF

`EOF` (*end of file*, fine del file) è il valore che `getchar` restituisce quando l'input è finito: alla fine di un file rediretto, o quando si preme Ctrl + D a inizio riga. È una costante simbolica definita in `stdio.h`, tipicamente come

```
#define EOF (-1)
```

ma su un altro sistema potrebbe valere diversamente: usare il nome `EOF` invece di -1 garantisce la **portabilità**. Stampandolo:

eof.c

```
#include <stdio.h>

int main()
{
    printf("EOF = %d\n", EOF);
    printf("EOF = %c\n", EOF);
}
```

Terminale

```
$ ./eof
EOF = -1
EOF = ◆
```

Con `%d` si vede -1. Con `%c` viene scritto un byte che vale 255 (`11111111`, la lettura senza segno di -1), che non corrisponde a nessun carattere: il terminale mostra un simbolo di carattere non valido.

Perché `int` e non `char`

Nel programma, `c` è dichiarata `int` e non `char`. Il motivo è che `getchar` può restituire **257 valori** diversi: i 256 possibili byte, da 0 a 255, più `EOF`. Un `char` ne contiene solo 256, e due valori finirebbero per confondersi (figura 8.15): `char` può essere `signed` o `unsigned`, e `EOF` potrebbe non valere -1; un `int` va bene in ogni caso.

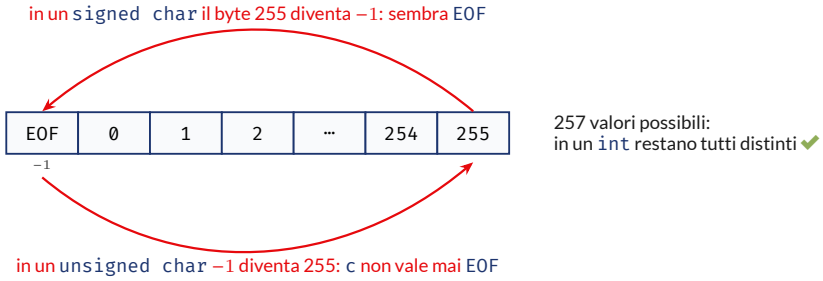


Figura 8.15: *I valori che* `getchar` *può restituire, e che cosa succede se li si mette in un* `char`.

Con `unsigned char` il guasto è evidente:

uchar.c

```c
#include <stdio.h>

int main(void)
{
    unsigned char c;

    while ((c = getchar()) != EOF) {
        putchar(c);
        putchar(c);
    }
    return 0;
}
```

Terminale

```
$ ./uchar < input.txt
aabbccddeeffgghhiijjkkllmmnnoo

^C
```

Dopo il contenuto del file il programma scrive all'infinito il byte 255 (mostrato come carattere non valido), finché non lo si ferma con Ctrl+C. Il motivo: quando l'input finisce, `getchar` restituisce -1, che in `c` diventa 255; il confronto `255 != EOF` è sempre vero, e il ciclo non termina più. Il compilatore lo segnala solo con l'opzione `-Wextra`, che attiva altri avvisi oltre a quelli di `-Wall`:

Terminale

```
$ gcc -Wall -Wextra -o uchar uchar.c
uchar.c: In function ‘main’:
uchar.c:7:27: warning: comparison is always true due to limited range of data type [-Wtype-limits]
    7 |     while ((c = getchar()) != EOF) {
      |                           ^~
```

Con un `signed char` il guasto è più subdolo: il programma funziona, salvo fermarsi in anticipo se l'input contiene un byte che vale 255.

Maiuscole e minuscole

I caratteri sono ordinati come gli interi che li rappresentano, e si comportano come tali:

- `'a' + 1` vale `'b'`, e `'b' + 1` vale `'c'`;
- `'z' - 'a'` vale 25;
- `'A' < 'a'`, perché 65 è minore di 97;
- `'A' - 'a'` ha lo stesso valore di `'B' - 'b'`: la distanza fra maiuscola e minuscola è la stessa per tutte le lettere (figura 8.16).

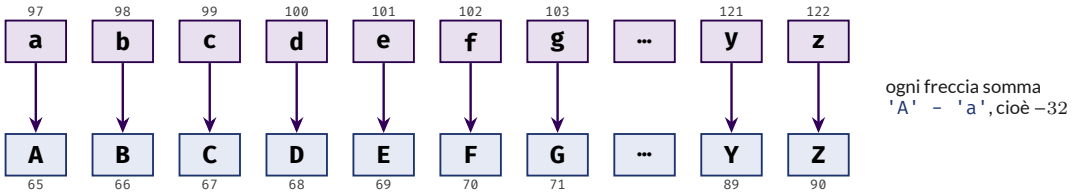


Figura 8.16: *Minuscole e maiuscole (sotto ciascuna, il codice): ogni lettera dista 32 posizioni dalla sua maiuscola.*

Su questa regolarità si basa `capitalize.c`, che trasforma in maiuscolo le lettere minuscole e copia senza modifiche tutti gli altri caratteri:

capitalize.c

```c
#include <stdio.h>

int main(void)
{
   int   c;

   while ((c = getchar()) != EOF)
      if (c >= 'a' && c <= 'z')
         putchar(c + 'A' - 'a');
      else
         putchar(c);
   return 0;
}
```

Terminale

```
$ cat input.txt
abcDE6ygTr56
$ ./capitalize < input.txt
ABCDE6YGTR56
```

Per la `g`, per esempio: $103 + 65 - 97 = 71$, il codice di `'G'`. Scrivere `'A' - 'a'` invece di -32 rende il programma più leggibile e non obbliga a ricordare la tabella. Il corpo del `while` è un'unica istruzione, l'`if-else`, quindi le graffe non servono. (La libreria standard ha una funzione che fa la stessa cosa, `toupper`, dichiarata in `ctype.h`.)

FUNZIONI MATEMATICHE

La libreria matematica

Il linguaggio C non contiene funzioni matematiche predefinite: le funzioni matematiche si trovano nella **libreria matematica**, che fa parte della libreria standard. Per esempio `sqrt` (radice quadrata), `pow` (potenza), `log`, `sin` e `cos`. In genere ricevono parametri di tipo `double` e restituiscono un risultato `double`; i prototipi sono in `math.h`, e su Linux la libreria va collegata con l'opzione `-lm` (capitolo 5). Il programma seguente usa `sqrt`, che ha un parametro, e `pow`, che ne ha due:

radice.c

```c
#include <math.h>
#include <stdio.h>

int main(void)
{
   double x;

   printf("\n%s\n%s\n%s\n\n",
      "The square root of x and x raised",
      "to the x power will be computed.",
      "---");
   while (1) {                  /* do it forever */
      printf("Input x:  ");
      scanf("%lf", &x);
      if (x >= 0.0)
         printf("\n%15s%22.15e\n%15s%22.15e\n%15s%22.15e\n\n",
            "x = ", x,
            "sqrt(x) = ", sqrt(x),
            "pow(x, x) = ", pow(x, x));
      else
         printf("\nSorry, your number must be nonnegative.\n\n");
   }
   return 0;
}
```

Terminale

```
$ gcc -Wall -o radice radice.c -lm
$ ./radice

The square root of x and x raised
to the x power will be computed.
---

Input x:  2

              x =  2.000000000000000e+00
      sqrt(x) =  1.414213562373095e+00
   pow(x, x) =  4.000000000000000e+00

Input x:  -1

Sorry, your number must be nonnegative.

Input x:  ^C
```

- La prima `printf` stampa tre stringhe passate come argomenti, con la conversione `%s`: è un modo per spezzare un testo lungo.
- `%15s` scrive una stringa in 15 caratteri, allineata a destra; `%22.15e` scrive un numero in notazione scientifica con 15 cifre dopo il punto, in 22 caratteri (capitolo 5). Per questo i risultati sono incolonnati.
- `while (1)` non termina mai: si esce con Ctrl + C. Qui Ctrl + D non basta: `scanf` non legge più nulla, `x` conserva l'ultimo valore e il ciclo si ripete all'infinito.
- Nelle slide l'esponente è scritto con tre cifre, `e+000`: è il formato usato su Windows; su Linux le cifre sono due.

Il valore assoluto

Per il valore assoluto la libreria standard ha due funzioni: `abs`, per gli `int`, con il prototipo in `stdlib.h`, e `fabs`, per i `double`, con il prototipo in `math.h`. Scambiarle è un errore insidioso:

assoluto.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(void)
{
    printf("abs(-7)    = %d\n", abs(-7));
    printf("fabs(-2.7) = %f\n", fabs(-2.7));
    printf("abs(-2.7)  = %d\n", abs(-2.7));
    return 0;
}
```

Terminale

```
$ ./assoluto
abs(-7)    = 7
fabs(-2.7) = 2.700000
abs(-2.7)  = 2
```

`abs(-2.7)` vale 2: il prototipo di `abs` dice che il parametro è un `int`, quindi -2.7 viene convertito in intero (-2) prima della chiamata. Con `-Wall` non c'è alcun avviso; con `-Wextra` sì:

Terminale

```
$ gcc -Wall -Wextra -o assoluto assoluto.c -lm
assoluto.c: In function ‘main’:
assoluto.c:9:32: warning: using integer absolute value function ‘abs’ when argument is of floating-point type
    ‘double’ [-Wabsolute-value]
    9 |     printf("abs(-2.7)  = %d\n", abs(-2.7));
      |                                  ^~~
```

Conversioni e cast

Un'espressione aritmetica come `x + y` ha un valore e un tipo. Se `x` e `y` sono entrambi `int`, anche `x + y` è un `int`. Ma che succede se i tipi sono diversi, o più piccoli di un `int`? Il C applica delle **conversioni implicite**, secondo regole precise; e il programmatore può chiederne di **esplicite**, con il *cast*.

La promozione a intero

Un `char` o uno `short`, con o senza segno, si possono usare in qualunque espressione in cui si può usare un `int` o un `unsigned int`. Nelle espressioni vengono infatti **promossi**: se tutti i valori del tipo originale sono rappresentabili da un `int` (ed è il caso dei PC), il valore viene convertito in `int`; altrimenti in `unsigned int`. Il processore, del resto, lavora in modo naturale su valori della dimensione di un `int`.

Così, se `x` e `y` sono `short`, `x + y` è un `int`; e in

```c
char c = 'A';
printf("%c\n", c);
```

il parametro passato a `printf` è un'espressione, `c`, che per la promozione a intero è di tipo `int`, con valore 65. Lo stesso motivo rende `sizeof(c + c)` uguale a 4.

Le conversioni aritmetiche

Quando un operatore binario (`+`, `-`, `*`, `/`, un confronto…) ha operandi di tipo diverso, prima del calcolo uno dei due viene convertito nel tipo dell'altro. Con `int i;` e `float f;`, in `i + f` il valore di `i` viene convertito in `float`, e il risultato è un `float`. La regola generale è che l'operando di tipo **"più piccolo"** viene convertito nel tipo **"più grande"** (figura 8.17). Nell'ordine:

1. se uno dei due operandi è `long double`, l'altro viene convertito in `long double`;
2. altrimenti, se uno è `double`, l'altro diventa `double`;
3. altrimenti, se uno è `float`, l'altro diventa `float`;
4. altrimenti si applica la promozione a intero a entrambi, e poi:
   - se uno è `unsigned long`, l'altro diventa `unsigned long`;
   - altrimenti, se uno è `long` e l'altro `unsigned`: se un `long` può rappresentare tutti i valori di un `unsigned`, l'`unsigned` diventa `long`, altrimenti entrambi diventano `unsigned long`;
   - altrimenti, se uno è `long`, l'altro diventa `long`;
   - altrimenti, se uno è `unsigned`, l'altro diventa `unsigned`;
   - altrimenti entrambi sono `int`.

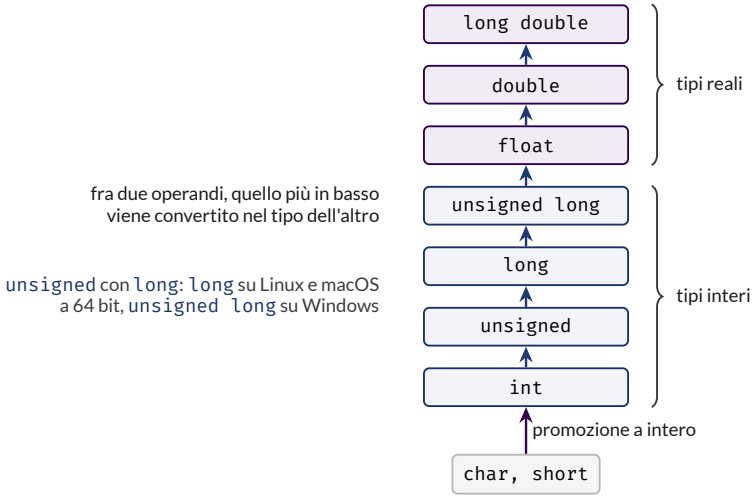


Figura 8.17: *La scala delle conversioni aritmetiche (come su Linux a 64 bit).*

Ecco l'esempio delle slide. Con le dichiarazioni

```
char c;  short s;  int i;  long l;  unsigned u;  unsigned long ul;
float f;  double d;  long double ld;
```

le espressioni seguenti hanno questi tipi (verificati con il compilatore, su Linux a 64 bit):

espressione	tipo	espressione	tipo
`c - s / i`	`int`	`u * 7 - i`	`unsigned`
`u * 2.0 + 1`	`double`	`f * 7 - i`	`float`
`c + 3`	`int`	`7 * s * ul`	`unsigned long`
`c + 5.0`	`double`	`ld + c`	`long double`
`d + s`	`double`	`u - ul`	`unsigned long`
`2 * i / l`	`long`	`u - l`	dipende dal sistema

Qualche caso commentato: in `c - s / i`, `c` e `s` vengono promossi a `int` e tutto il calcolo è fra `int`; in `u * 7 - i` la costante 7 e poi `i` vengono convertiti in `unsigned`; in `u * 2.0 + 1` la costante `2.0` è un `double`, e trascina tutto in `double`. L'ultimo dipende davvero dal sistema: su Linux e macOS a 64 bit un `long` (8 byte) contiene tutti i valori di un `unsigned` (4 byte), quindi `u - l` è un `long`; su Windows, dove `long` ha 4 byte, è un `unsigned long`.

Interi con e senza segno

Mescolare `int` e `unsigned` può dare risultati sorprendenti. In `-1 < 1u`, per la regola vista, -1 viene convertito in `unsigned` e diventa 4 294 967 295: il confronto è **falso**. Un programma con `int i = -1; unsigned u = 1;` e `if (i < u)` sceglie quindi il ramo `else`. Anche qui l'avviso arriva solo con `-Wextra`: *comparison of integer expressions of different signedness*. Conviene compilare sempre con `-Wall -Wextra`.

Le conversioni nelle assegnazioni

Anche l'assegnazione converte: il valore viene trasformato nel tipo della variabile che lo riceve. Da reale a intero la parte decimale viene **troncata**, non arrotondata (cioè si va verso lo zero); da intero a reale non si perde nulla.

tronca.c

```c
#include <stdio.h>

int main(void)
{
    int i = 3.9;
    int j = -3.9;
    double x = 7;

    printf("i = %d   j = %d   x = %f\n", i, j, x);
    return 0;
}
```

Terminale

```
$ ./tronca
i = 3   j = -3   x = 7.000000
```

Il cast

Le conversioni viste finora sono **implicite**: le fa il compilatore da solo. Esistono anche le conversioni **esplicite**, chiamate **cast**: si scrive il tipo desiderato fra parentesi davanti a un'espressione.

```c
int i;
double f;

f = (double) i;
```

La variabile `i` **non viene modificata**: cambia il tipo del valore prodotto dall'espressione `i`. L'uso tipico è evitare la divisione intera, per esempio nel calcolo di una media:

media.c

```c
#include <stdio.h>

int main(void)
{
    int somma = 7, n = 2;
    double media;

    media = somma / n;
    printf("somma / n            = %f\n", media);
    media = (double) somma / n;
    printf("(double) somma / n   = %f\n", media);
    media = (double) (somma / n);
    printf("(double) (somma / n) = %f\n", media);
    printf("somma vale ancora %d\n", somma);
    return 0;
}
```

Terminale

```
$ ./media
somma / n            = 3.000000
(double) somma / n   = 3.500000
(double) (somma / n) = 3.000000
somma vale ancora 7
```

Nel primo caso la divisione è fra due `int` e vale 3; il risultato viene convertito in `double` solo dopo, nell'assegnazione. Il cast è un operatore unario e ha la precedenza su `/`: in `(double) somma / n` si converte prima `somma` in 7.0, poi la divisione è fra un `double` e un `int`, quindi anche `n` viene convertito e il risultato è 3.5. Nel terzo caso le parentesi fanno eseguire prima la divisione intera: il cast arriva quando il danno è fatto (figura 8.18).



Figura 8.18: *Dove sta il cast cambia tutto: a sinistra converte* `somma` *prima della divisione, a destra il risultato già troncato.*

In sintesi

Il C ha dodici tipi fondamentali: i tipi **interi** (`char`, `short`, `int`, `long`, con e senza segno) e i tipi **reali** (`float`, `double`, `long double`). Un `char` è un intero di 1 byte che contiene il **codice** di un carattere (ASCII: le cifre, le maiuscole e le minuscole hanno codici consecutivi); le **sequenze di escape** scrivono i caratteri speciali. Ogni tipo ha un intervallo di valori (`limits.h`, `float.h`): superarlo con gli interi provoca un **overflow** silenzioso, i reali hanno una **precisione** limitata e non sono esatti, quindi non si confrontano con `==`. `sizeof` dà i byte di un tipo o di un'espressione, `typedef` dà un nuovo nome a un tipo. `getchar` e `putchar` leggono e scrivono un carattere; la variabile che riceve `getchar` deve essere `int`, per distinguere `EOF`. Le funzioni matematiche sono in `math.h` (`-lm`). Nelle espressioni `char` e `short` diventano `int`, e l'operando di tipo più piccolo viene convertito nel tipo dell'altro; il **cast** `(tipo)` converte esplicitamente.

Riferimenti

- **A. Kelley, I. Pohl**, *A Book on C*: capitolo 3 (i tipi di dato fondamentali).
- **Arithmetic types** — cppreference.com.
  https://en.cppreference.com/w/c/language/arithmetic_types
- **Implicit conversions** — cppreference.com.
  https://en.cppreference.com/w/c/language/conversion
- **D. Goldberg**, *What Every Computer Scientist Should Know About Floating-Point Arithmetic* (il classico sugli errori dei reali).
  https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

Il flusso di controllo

Le istruzioni di un programma vengono eseguite normalmente in sequenza, una dopo l'altra. Quasi sempre, però, bisogna alterare questo flusso: per **scegliere** fra azioni diverse (con `if`, `if-else` e `switch`) o per **ripetere** un'azione (con `while`, `do-while` e `for`). Nel capitolo 4 abbiamo visto queste strutture nei diagrammi di flusso, e nel capitolo 5 le prime istruzioni del C; qui le vediamo tutte, a partire da ciò che le governa: le **condizioni**.

Vero e falso in C

Le condizioni sono numeri

Il flusso di controllo è regolato da espressioni logiche, che possono essere vere (*TRUE*) o false (*FALSE*). Il C non ha un tipo apposito per i valori logici (il C99 ha aggiunto `_Bool`, ma la regola resta quella che segue): vero e falso sono **numeri**.

- **Falso** è un valore nullo: `0`, `0.0`, il carattere `'\0'`.
- **Vero** è qualunque valore diverso da zero: 1, ma anche 7, −3 o `'A'`.
- Gli operatori che producono un valore logico restituiscono sempre un `int`: **1** se il risultato è vero, **0** se è falso.

Le condizioni si costruiscono con tre famiglie di operatori:

famiglia	operatore	in C	famiglia	operatore	in C
relazionali	minore	`<`	di uguaglianza	uguale a	`==`
	maggiore	`>`		diverso da	`!=`
	minore o uguale	`<=`	logici	negazione (unario)	`!`
	maggiore o uguale	`>=`		e (*and*)	`&&`
				o (*or*)	`||`

Sono tutti operatori binari (con due operandi), tranne la negazione `!`, che è unaria.

GLI OPERATORI RELAZIONALI

Confrontare due espressioni

Gli operatori relazionali prendono due espressioni come operandi: `a < 3`, `a > b`, `-1.3 >= (2.0 * x + 3.3)`. Sono scritti con uno o due caratteri, sempre in quest'ordine e senza spazi in mezzo: per questo `a =< b` è sbagliato (l'ordine è `<=`) e `a < = b` pure (lo spazio spezza il token, capitolo 6). Attenzione anche ad `a >> b`: è corretto, ma `>>` è un altro operatore (lo spostamento dei bit), con tutt'altro significato.

Come li calcola la macchina? Su molte macchine `a < b` viene valutato come `a - b < 0`, e lo stesso per gli altri: si calcola la differenza e se ne guarda il segno.

a - b	a > b	a >= b	a < b	a <= b
positivo	1	1	0	0
zero	0	1	0	1
negativo	0	0	1	1

Qualche esempio

Con le dichiarazioni

```
1 char   c = 'w';
2 int    i = 1, j = 2, k = -7;
3 double x = 7e+33, y = 0.001;
```

ecco alcune espressioni, l'espressione equivalente con tutte le parentesi (che mostra in che ordine si calcola, capitolo 6) e il valore, verificato con il compilatore:

espressione	**equivalente**	**valore**
`'a' + 1 < c`	`('a' + 1) < c`	1
`-i - 5 * j >= k + 1`	`((-i) - (5 * j)) >= (k + 1)`	0
`3 < j < 5`	`(3 < j) < 5`	1
`x - 3.333 <= x + y`	`(x - 3.333) <= (x + y)`	1
`x < x + y`	`x < (x + y)`	0

`'a' + 1` vale 98 e `c` vale 119 (il codice di `'w'`): quindi 1. Nella seconda, $-1-10=-11$ non è maggiore o uguale a -6: quindi 0. Le ultime tre meritano una spiegazione a parte.

`a < b < c` non è la matematica

In matematica $a < b < c$ significa "$a < b$ **e** $b < c$". In C l'espressione è sintatticamente corretta, ma è interpretata come `(a < b) < c`: prima si calcola `a < b`, che vale 0 o 1, e poi si confronta **quel** risultato con `c`. Per questo `3 < j < 5` vale 1: `3 < j` è falso (0), e `0 < 5` è vero. Con numeri scelti apposta, il risultato è chiaramente sbagliato:

catena.c

```c
#include <stdio.h>

int main(void)
{
    int a = 5, b = 2, c = 3;

    if (a < b < c)
        printf("5 < 2 < 3 è vero?!\n");
    if (a < b && b < c)
        printf("questo non viene stampato\n");
    return 0;
}
```

Terminale

```
$ ./catena
5 < 2 < 3 è vero?!
```

Terminale

```
$ gcc -Wall -o catena catena.c
catena.c: In function ‘main’:
catena.c:7:10: warning: comparisons like ‘X<=Y<=Z’ do not have their mathematical meaning [-Wparentheses]
    7 |     if (a < b < c)
      |         ~~^~~
```

Il modo giusto di scrivere la condizione matematica è il secondo `if`: `a < b && b < c`, con l'operatore logico *and* che vedremo fra poco. Come si vede, `gcc` con `-Wall` si accorge dell'errore.

I reali e la precisione

In matematica $x < x + y$ equivale a $y > 0$. Con i reali del computer non sempre: se `x` è molto grande e `y` molto piccolo, l'incremento può andare perso. È il caso di $x = 7 \cdot 10^{33}$ e $y = 0.001$:

precisione.c

```c
#include <stdio.h>

int main()
{
    double x = 7e+33;
    double y = 0.001;

    if (x < x + y) printf("x < x + y\n");
    if (x > x + y) printf("x > x + y\n");
    if (x == x + y) printf("x = x + y\n");
}
```

Terminale

```
$ ./precisione
x = x + y
```

Per scrivere esattamente $7 \cdot 10^{33} + 0.001$ servirebbero 37 cifre significative; un `double` ne ha circa 16 (capitolo 8). L'incremento dato da `y` è troppo piccolo rispetto a `x`: la precisione non basta a registrarlo, e la rappresentazione di `x + y` è identica a quella di `x`.

Gli operatori di uguaglianza

`==` e `!=`

Gli operatori di uguaglianza `==` e `!=` sono binari e agiscono su espressioni: `c == 'A'`, `k != -2`, `x + y == 3 * z - 7`. Non sono invece esempi corretti:

- `a = b`, che è un'**assegnazione**;
- `a = = b - 1`, perché lo spazio spezza l'operatore;
- `(x + y) =! 44`, che il compilatore legge come `(x + y) = (!44)`: un'assegnazione a `x + y`, che non è una variabile. Il messaggio di errore lo dice a modo suo:

Terminale

```
$ gcc -Wall -o errori errori.c
errori.c: In function ‘main’:
errori.c:7:12: error: lvalue required as left operand of assignment
    7 |    (x + y) =! 44;
      |            ^
```

(Un *lvalue* è qualcosa che può stare a sinistra di un'assegnazione, come una variabile.) Anche l'uguaglianza si calcola con una differenza: `expr1 == expr2` equivale a `expr1 - expr2 == 0`, e `expr1 != expr2` a `expr1 - expr2 != 0`.

`expr1 - expr2`	`==`	`!=`
zero	`1`	`0`
non zero	`0`	`1`

con `int i = 1, j = 2, k = 3;`

`i == j`	`i - j == 0`	0
`i != j`	`i - j != 0`	1
`i+j+k == -2*-k`	`((i+j)+k) == ((-2)*(-k))`	1

L'errore più comune: `=` al posto di `==`

Un errore frequentissimo è confondere `a == b` con `a = b`, scrivendo `if (a = b)` invece di `if (a == b)`. Il compilatore lo accetta, perché un'assegnazione è un'espressione con un valore: il valore di `b`. Così la condizione è vera o falsa **a seconda del solo valore di `b`**, e per giunta `a` viene modificata:

uguale.c

```c
#include <stdio.h>

int main(void)
{
   int a = 5, b = 7;

   if (a = b)
      printf("uguali?! ora a vale %d\n", a);
   return 0;
}
```

Terminale

```
$ ./uguale
uguali?! ora a vale 7
```

Per fortuna `gcc` con `-Wall` sospetta l'errore, e suggerisce di mettere l'assegnazione fra parentesi se è davvero voluta:

Terminale

```
$ gcc -Wall -o uguale uguale.c
uguale.c: In function ‘main’:
uguale.c:7:8: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
    7 |    if (a = b)
      |        ^
```

Gli operatori logici

La negazione

L'operatore logico di negazione `!` è unario: se un'espressione vale 0, la sua negata vale 1, e viceversa (se vale qualunque cosa diversa da 0, la negata vale 0). Sono corretti `!a`, `!(x + 7.7)`, `!(a < b || c < d)`; non lo sono `a!` (l'ordine è sbagliato: il `!` va prima) e `a != b` se si voleva una negazione, perché `!=` è un altro operatore.

In logica, negare due volte un'affermazione la lascia invariata: $not(not\ A) = A$. In C non è sempre così: `!!5` vale **1**, non 5. La prima negazione trasforma 5 in 0, la seconda trasforma 0 in 1: la doppia negazione "normalizza" qualunque valore vero a 1.

```
char   c = 'A';
int    i = 7, j = 7;
double x = 0.0, y = 2.3;
```

espressione	equivalente	valore
`!c`	`!c`	0
`!(i - j)`	`!(i - j)`	1
`!i - j`	`(!i) - j`	−7
`!!(x + y)`	`!(!(x + y))`	1
`!x * !!y`	`(!x) * (!(!y))`	1

`!i - j` vale −7 perché la negazione, come tutti gli operatori unari, ha la precedenza sulla sottrazione: `!i` vale 0, e $0 - 7 = -7$.

And e *or*

Gli operatori `&&` (*and*, "e") e `||` (*or*, "o") sono binari, agiscono su espressioni e restituiscono 1 (vero) o 0 (falso): `a && b`, `a || b`, `!(a < b) && c`, `3 && (-2 * a + 7)`. Non sono corretti `a &&` (manca un operando), `a | | b` (gli spazi spezzano l'operatore) e `&b`, che è l'indirizzo di `b` (capitolo 5). Il loro comportamento è descritto dalla **tabella di verità**:

expr1	**expr2**	**expr1 && expr2**	**expr1 || expr2**
zero	zero	0	0
zero	non zero	0	1
non zero	zero	0	1
non zero	non zero	1	1

`&&` è vero solo se **entrambi** gli operandi sono veri; `||` è vero se **almeno uno** lo è.

Precedenza

`&&` ha la precedenza su `||`, ed entrambi hanno priorità più bassa degli operatori unari, aritmetici, relazionali e di uguaglianza; l'associatività va da sinistra a destra. La figura 9.1 riassume l'ordine degli operatori che si usano nelle condizioni (la tabella completa è nel capitolo 6).

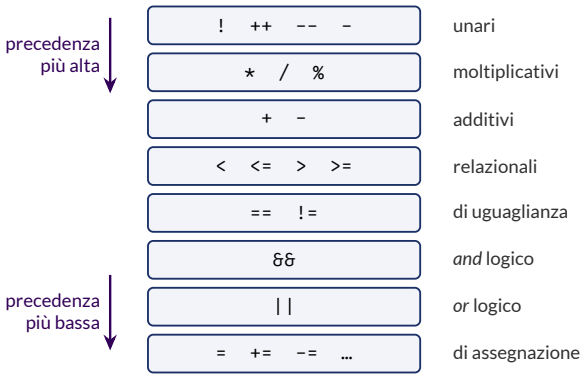


Figura 9.1: *Gli operatori delle condizioni, dalla precedenza più alta alla più bassa.*

Con le dichiarazioni `char c = 'B';`, `int i = 3, j = 3, k = 3;` e `double x = 0.0, y = 2.3;`:

espressione	equivalente	valore
`i && j && k`	`(i && j) && k`	1
`x || i && j - 3`	`x || (i && (j - 3))`	0
`i < j && x < y`	`(i < j) && (x < y)`	0
`i < j || x < y`	`(i < j) || (x < y)`	1
`'A' <= c && c <= 'Z'`	`('A' <= c) && (c <= 'Z')`	1
`c - 1 == 'A' || c + 1 == 'Z'`	`((c - 1) == 'A') || ((c + 1) == 'Z')`	1

Per esempio `x || i && j - 3`: `x` vale 0.0, falso; `j - 3` vale 0, quindi `i && 0` è falso; e "falso o falso" è falso, 0. La quinta riga è il modo corretto di scrivere la condizione matematica 'A' ≤ c ≤ 'Z': "`c` è una lettera maiuscola".

La valutazione in cortocircuito

La valutazione delle espressioni con `&&` e `||` si **ferma appena il risultato è noto** (*short circuit*, cortocircuito):

- in `expr1 && expr2`, se `expr1` è falsa il risultato è falso comunque, e `expr2` **non viene valutata**; solo se `expr1` è vera si valuta anche `expr2`;
- in `expr1 || expr2`, se `expr1` è vera il risultato è vero comunque, e `expr2` non viene valutata; solo se `expr1` è falsa si valuta anche `expr2`.

La differenza si vede quando `expr2` ha un effetto, come un incremento. I due programmi seguenti differiscono solo per l'ordine degli operandi di `&&`, eppure stampano risultati diversi:

sc1.c

```c
#include <stdio.h>

int main()
{
   int a, b;

   a = 0;
   b = 0;
   while (a < 3 && ++b) {
      a++;
   }
   printf("a = %d b = %d\n", a, b);
}
```

Terminale

```
$ ./sc1
a = 3 b = 3
```

sc2.c

```c
#include <stdio.h>

int main()
{
   int a, b;

   a = 0;
   b = 0;
   while (++b && a < 3) {
      a++;
   }
   printf("a = %d b = %d\n", a, b);
}
```

Terminale

```
$ ./sc2
a = 3 b = 4
```

Seguiamo i controlli della condizione (figura 9.2). Nei primi tre entrambi i programmi valutano tutte e due le parti, e `b` cresce insieme ad `a`. Al quarto controllo `a` vale 3: in `sc1.c` la prima parte, `a < 3`, è già falsa, e `++b` non viene eseguito; in `sc2.c` la prima parte è `++b`, che viene eseguita comunque, e `b` arriva a 4.

`while (a < 3 && ++b)`

controllo	a	a < 3	++b	risultato
1	0	vero	b = 1	vero
2	1	vero	b = 2	vero
3	2	vero	b = 3	vero
4	3	falso	non valutato	falso

`while (++b && a < 3)`

controllo	++b	a	a < 3	risultato
1	b = 1	0	vero	vero
2	b = 2	1	vero	vero
3	b = 3	2	vero	vero
4	b = 4	3	falso	falso

Figura 9.2: *I controlli della condizione nei due programmi: al quarto, `++b` viene eseguito solo nel secondo.*

Il cortocircuito è anche una protezione utile. In `if (n != 0 && somma / n > 10)` la divisione viene eseguita solo se `n` è diverso da zero: se `n` vale 0, la prima parte è falsa e la seconda, che dividerebbe per zero, non viene neppure calcolata.

La forma generale

La forma generale dell'`if` è

```
if (expr)
    statement
statementSucc
```

Se `expr` è diversa da zero (vera) viene eseguito `statement`; se è zero (falsa), `statement` viene saltato. In entrambi i casi poi si prosegue con l'istruzione successiva, `statementSucc`: è la selezione a una via della figura 4.5. `expr` può essere qualsiasi espressione, ma di solito è relazionale, di uguaglianza o logica.

voto.c

```c
#include <stdio.h>

int main(void)
{
   int grade;

   printf("Grade: ");
   scanf("%d", &grade);
   if (grade >= 90)
      printf("Congratulations!\n");
   printf("Your grade is %d.\n", grade);
   return 0;
}
```

Terminale

```
$ ./voto
Grade: 95
Congratulations!
Your grade is 95.
$ ./voto
Grade: 70
Your grade is 70.
```

Le congratulazioni dipendono dalla condizione; la riga con il voto viene stampata sempre, perché è l'istruzione successiva all'`if`.

Le graffe, anche per un'istruzione sola

Il corpo di un `if` è **una** istruzione. Per eseguirne più di una si usa un'istruzione composta, cioè un blocco fra graffe:

```
if (c == ' ') {
    ++blank_cnt;
    printf("found another blank\n");
}
```

Le slide consigliano di usare le graffe **sempre**, anche quando l'istruzione è una sola: `if (j < k) { min = j; }` invece di `if (j < k) min = j;`. Il motivo si vede quando, più avanti, si aggiunge una seconda istruzione e ci si dimentica delle graffe: l'indentazione inganna chi legge, ma non il compilatore.

graffe.c

```
#include <stdio.h>

int main(void)
{
   int j = 5, k = 3, min = k;

   if (j < k)
      min = j;
      printf("min aggiornato\n");
   printf("min = %d\n", min);
   return 0;
}
```

Terminale

```
$ ./graffe
min aggiornato
min = 3
```

La condizione è falsa (5 non è minore di 3), eppure `"min aggiornato"` viene stampato: solo `min = j;` dipende dall'`if`, la `printf` è l'istruzione successiva. Per fortuna `gcc` con `-Wall` lo nota:

Terminale

```
$ gcc -Wall -o graffe graffe.c
graffe.c: In function ‘main’:
graffe.c:7:4: warning: this ‘if’ clause does not guard... [-Wmisleading-indentation]
    7 |    if (j < k)
      |    ^~
graffe.c:9:7: note: ...this statement, but the latter is misleadingly indented as if it were guarded by the ‘if
’
    9 |       printf("min aggiornato\n");
      |       ^~~~~~
```

`if-else`

La forma generale dell'`if-else` è

```
if (expr)
    statement1
else
    statement2
```

Se `expr` è diversa da zero viene eseguito `statement1`, se è zero `statement2`: è la selezione a due vie della figura 4.6. Esattamente una delle due istruzioni viene eseguita.

L'`else` pendente

Un `if` può contenere a sua volta un `if` (e lo stesso vale per l'`if-else`). Nel blocco

```
if (a == 1)
    if (b == 2)
        printf("***\n");
```

l'istruzione controllata da `if (a == 1)` è proprio `if (b == 2) printf("***\n");`: le tre stelle vengono stampate solo se `a` vale 1 **e** `b` vale 2. Aggiungiamo un `else`. C'è differenza fra i due blocchi seguenti?

Esempio 1

```
if (a == 1)
    if (b == 2)
        printf("***\n");
    else
        printf("###\n");
```

Esempio 2

```
if (a == 1)
    if (b == 2)
        printf("***\n");
else
    printf("###\n");
```

L'indentazione suggerisce due significati diversi: nel primo l'`else` sembra legato a `if (b == 2)`, nel secondo a `if (a == 1)`. Ma per il compilatore l'indentazione **non ha alcun significato**, e la regola è una sola: **l'`else` si lega all'`if` più vicino** che non ne abbia già uno. I due blocchi sono identici, ed è il secondo a essere ingannevole (figura 9.3).

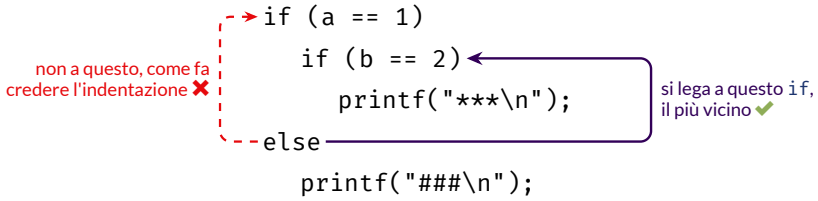


Figura 9.3: *L'`else` dell'Esempio 2 appartiene all'`if` interno, nonostante l'indentazione.*

Lo verifichiamo eseguendo il blocco con valori diversi di `a` e `b` (con `-Wall`, `gcc` suggerisce di aggiungere le graffe: *suggest explicit braces to avoid ambiguous 'else'*):

dangling.c

```
#include <stdio.h>

int main(void)
{
   int a, b;

   printf("a e b: ");
   scanf("%d%d", &a, &b);
   if (a == 1)
      if (b == 2)
         printf("***\n");
   else
      printf("###\n");
   return 0;
}
```

Terminale

```
$ ./dangling
a e b: 1 2
***
$ ./dangling
a e b: 1 3
###
$ ./dangling
a e b: 2 2
$
```

Con `a` uguale a 2 non viene stampato nulla: se l'`else` fosse dell'`if` esterno, comparirebbe `###`. Per ottenere davvero quel significato servono le graffe, che chiudono l'`if` interno:

```
if (a == 1) {
    if (b == 2)
        printf("***\n");
} else
    printf("###\n");
```

Per chiarezza, la formattazione del testo deve sempre rispecchiare il significato.

Il `while`

La forma generale

La forma generale del costrutto `while` è

```
while (expr)
    statement1
statement2
```

1. viene valutata `expr`;
2. se è diversa da zero (vera), viene eseguito `statement1` e il controllo torna all'inizio del ciclo, al punto 1;
3. se è zero (falsa), il controllo passa a `statement2`.

In sostanza `statement1` viene eseguito ripetutamente finché `expr` non diventa falsa: è il ciclo con la condizione in testa (figura 9.4, da confrontare con la figura 4.12). Se `expr` è falsa fin dall'inizio, `statement1` non viene eseguito neppure una volta.

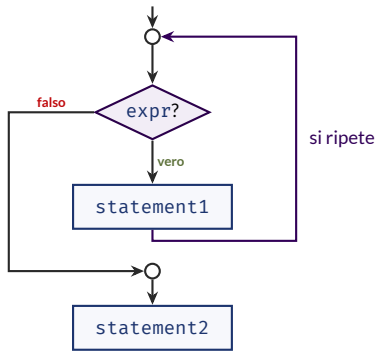


Figura 9.4: *Il* `while`*: la condizione si controlla prima di ogni giro.*

Attenti ai cicli infiniti

Il frammento delle slide

```c
printf("Input a positive integer: ");
scanf("%d", &n);
while (--n) {
    ...   /* do something */
}
```

dovrebbe eseguire il corpo del ciclo finché `--n` diventa 0, cioè $n-1$ volte. Si assume però che l'utente inserisca un intero positivo. Che cosa succede se inserisce 0, o un numero negativo? Proviamo, stampando `n` a ogni giro:

meno.c

```c
#include <stdio.h>

int main(void)
{
   int n;

   printf("Input a positive integer: ");
   scanf("%d", &n);
   while (--n) {
      printf("%d ", n);
   }
   printf("\n");
   return 0;
}
```

Terminale

```
$ ./meno
Input a positive integer: 4
3 2 1
$ ./meno
Input a positive integer: 0
-1 -2 -3 -4 -5 -6 -7 -8 -9 -10 -11 -12 -13 -14 -15
     -16 -17 -18 -19 -20 -21 -22 -23
^C
```

Partendo da 0, `--n` vale -1, poi -2, -3…: non diventa **mai** 0 (o meglio, ci arriverebbe solo dopo più di 4 miliardi di giri e un overflow, che è un comportamento indefinito: in una prova senza stampe, il programma compilato normalmente ha fatto 4 294 967 295 giri prima di fermarsi; compilato con `-O2` non si è fermato affatto). Il ciclo va controllato meglio, in due modi:

```c
while (--n > 0) {
    ...   /* do something */
}
```

```c
if (n <= 0) {
    errore();
}
while (--n > 0) {
    ...   /* do something */
}
```

A sinistra la condizione relazionale `--n > 0` ferma il ciclo anche se `n` parte da un valore non positivo; a destra, in più, si controlla l'input e si segnala il problema (`errore` sta per una funzione che lo gestisce). Un programma non deve mai fidarsi di ciò che scrive l'utente.

Il corpo vuoto

A volte il corpo del `while` è un'istruzione vuota: tutto il lavoro lo fa la condizione. Per esempio

```
while ((c = getchar()) == ' ');
```

salta tutti i caratteri spazio consecutivi: a ogni giro la condizione legge un carattere, e il ciclo continua finché è uno spazio. Il `;` attaccato alla parentesi però si nota poco, e potrebbe sembrare un errore di battitura. Per rendere evidente che è intenzionale si suggerisce di metterlo da solo su una riga:

salta.c

```c
#include <stdio.h>

int main(void)
{
    int c;

    while ((c = getchar()) == ' ')
        ;
    printf("primo carattere non bianco: %c\n", c);
    return 0;
}
```

Terminale

```
$ ./salta
     ciao
primo carattere non bianco: c
```

Contare i caratteri

Il programma seguente legge un testo e conta gli spazi, le cifre, le lettere, le righe e gli altri caratteri. Il corpo del `while` è un'unica istruzione, una catena di `if-else` (per questo le graffe non sono necessarie).

cnt_char.c

```c
/* Count blanks, digits, letters, newlines, and others. */

#include <stdio.h>

int main(void)
{
   int   blank_cnt = 0, c, digit_cnt = 0,
         letter_cnt = 0, nl_cnt = 0, other_cnt = 0;

   while ((c = getchar()) != EOF)     /* braces not necessary */
      if (c == ' ')
         ++blank_cnt;
      else if (c >= '0' && c <= '9')
         ++digit_cnt;
      else if (c >= 'a' && c <= 'z' || c >= 'A' && c <= 'Z')
         ++letter_cnt;
      else if (c == '\n')
         ++nl_cnt;
      else
         ++other_cnt;
   printf("%10s%10s%10s%10s%10s%10s\n\n",
      "blanks", "digits", "letters", "lines", "others", "total");
   printf("%10d%10d%10d%10d%10d%10d\n\n",
      blank_cnt, digit_cnt, letter_cnt, nl_cnt, other_cnt,
      blank_cnt + digit_cnt + letter_cnt + nl_cnt + other_cnt);
   return 0;
}
```

Eseguendolo sul suo stesso sorgente:

Terminale

```
$ ./cnt_char < cnt_char.c
    blanks    digits   letters     lines    others     total

       208        32       361        27       180       808
```

- La condizione delle lettere, `c >= 'a' && c <= 'z' || c >= 'A' && c <= 'Z'`, sfrutta la precedenza di `&&` su `||`: "minuscola oppure maiuscola". È corretta così, ma `gcc -Wall` suggerisce comunque di aggiungere le parentesi, per chiarezza.
- `%10s` e `%10d` scrivono ogni stringa e ogni numero in 10 caratteri, allineati a destra: per questo i numeri stanno in colonna sotto i titoli.
- I numeri dipendono dal file letto: nelle slide il sorgente aveva una riga in più, e il risultato era 28 righe e 809 caratteri.

La catena di `if-else`

In `cnt_char.c` ogni `else` contiene a sua volta un `if`: le domande si fanno una dopo l'altra, e alla prima risposta vera si esegue l'azione corrispondente e si salta tutto il resto. Si forma così una selezione a più vie (figura 9.5). Per questo la catena si scrive con gli `else if` allineati, invece di indentare ogni volta di più: è un'unica scelta fra cinque possibilità.

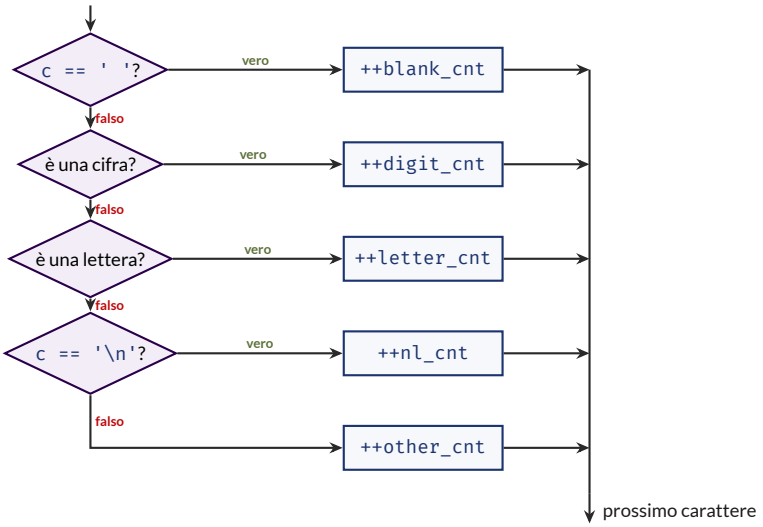


Figura 9.5: *La catena di* `if-else` *di* `cnt_char.c`*: le domande si fanno in ordine, e alla prima risposta vera si incrementa un contatore.*

Il `for`

`for` e `while`

La forma del `for`, a sinistra, è equivalente al `while` a destra:

```
for (expr1; expr2; expr3)
    statementFor
statementSucc
```

```
expr1;
while (expr2) {
    statementFor
    expr3;
}
statementSucc
```

`expr1` è l'**inizializzazione**, eseguita una volta sola; `expr2` è la **condizione**, controllata prima di ogni giro; `expr3` è l'**aggiornamento**, eseguito alla fine di ogni giro (figura 9.6). Il `for` raccoglie in una riga le tre parti di un ciclo su contatore, che con il `while` sarebbero sparse.

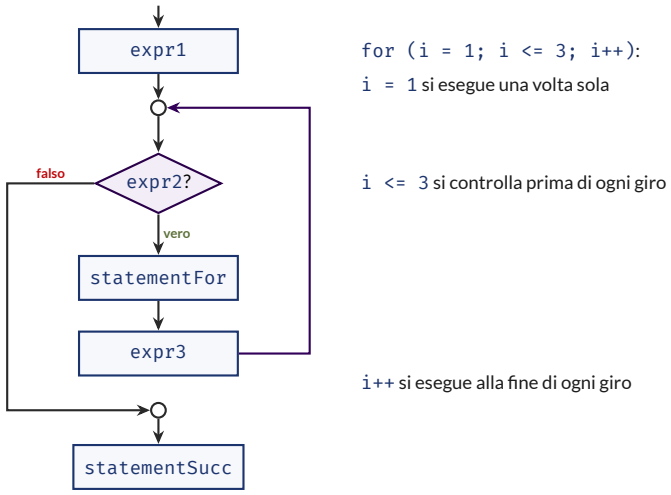


Figura 9.6: *Il* `for`*: inizializzazione, condizione e aggiornamento.*

for_while.c

```c
#include <stdio.h>

int main(void)
{
   int i;

   for (i = 1; i <= 3; i++)
      printf("for: i = %d\n", i);

   i = 1;
   while (i <= 3) {
      printf("while: i = %d\n", i);
      i++;
   }
   return 0;
}
```

Terminale

```
$ ./for_while
for: i = 1
for: i = 2
for: i = 3
while: i = 1
while: i = 2
while: i = 3
```

Ognuna delle tre espressioni si può omettere (i `;` restano): `for (;;)` è un ciclo infinito, come `while (1)`.

Cicli annidati

Il corpo di un ciclo può contenere un altro ciclo. Il programma seguente stampa un triangolo di asterischi: il ciclo esterno scorre le righe, e per ogni riga il ciclo interno stampa tanti asterischi quanto vale `riga`.

triangolo.c

```c
#include <stdio.h>

int main(void)
{
   int riga, col, n = 4;

   for (riga = 1; riga <= n; riga++) {
      for (col = 1; col <= riga; col++)
         printf("*");
      printf("\n");
   }
   return 0;
}
```

riga	col	vale	stampa
1	1		*
2	1, 2		**
3	1, 2, 3		***
4	1, 2, 3, 4		****

Per ogni giro del ciclo esterno, il ciclo interno ricomincia da `col = 1`.

Meglio una relazione che un'uguaglianza

Nelle condizioni dei cicli è spesso preferibile usare un operatore relazionale (`<`, `<=`…) invece di un'uguaglianza (`==`, `!=`): il codice è più robusto. Con i `float` e i `double`, in particolare, un test di uguaglianza può andare oltre la precisione della macchina. Il programma delle lezioni vorrebbe sommare i valori 0.0, 0.1, 0.2, …, 9.8, fermandosi quando `x` arriva a 9.9:

test.c

```c
/* A test that fails. */

#include <stdio.h>

int main(void)
{
   int      cnt = 0;
   double   sum = 0.0, x;

   for (x = 0.0; x != 9.9; x += 0.1) {   /* trouble! */
      sum += x;
      printf("cnt = %5d\n", ++cnt);
   }
   printf("sum = %f\n", sum);
   return 0;
}
```

Terminale

```
$ ./test
cnt =     1
cnt =     2
cnt =     3
…
cnt =    99
cnt =   100
cnt =   101
cnt =   102
…
^C
```

Il ciclo non termina: in un secondo ha stampato quasi 20 milioni di righe. Il motivo è nel capitolo 8: 0.1 non è rappresentabile esattamente, e sommandolo 99 volte si accumula un piccolo errore. `x` non è mai **esattamente** uguale alla costante 9.9: le passa accanto e va oltre (figura 9.7).

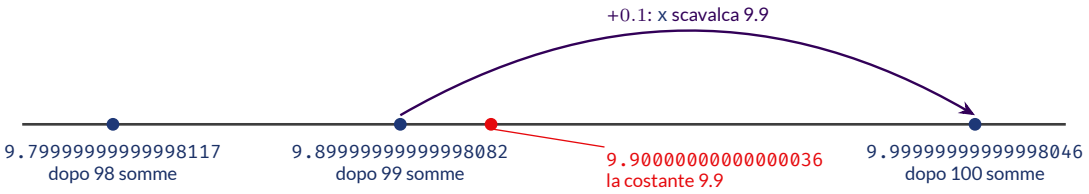


Figura 9.7: *I valori di `x` vicino a 9.9 (la scala non è proporzionale): dopo 99 somme `x` è più piccolo della costante, dopo 100 è più grande. `x != 9.9` è sempre vero.*

Con la condizione relazionale `x < 9.9` il ciclo termina di sicuro. Si noti però che i giri diventano 100, non 99: dopo 99 somme `x` vale 9.89999999999998, che è ancora minore di 9.9. Il modo più sicuro per contare è usare un **intero**, e ricavare da esso il reale: `for (cnt = 0; cnt < 99; cnt++) { x = cnt * 0.1; …}`.

Il do-while

La condizione in coda

Il `do-while` è il ciclo con la condizione **in coda** (figura 4.10): prima si esegue il corpo, poi si controlla la condizione. Il corpo viene quindi eseguito **almeno una volta**.

```
do
    statement
while (expr);
statementSucc
```

1. viene eseguito `statement`;
2. viene valutata `expr`: se è vera si torna al punto 1;
3. se è falsa si prosegue con `statementSucc`.

Il `;` dopo la parentesi della condizione è obbligatorio.

È la struttura naturale per l'**input controllato** visto nel capitolo 4: si chiede un valore, e lo si richiede finché non è accettabile. Chiedere ha senso solo dopo aver chiesto almeno una volta.

positivo.c

```c
#include <stdio.h>

int main(void)
{
   int n;

   do {
      printf("Inserisci un intero positivo: ");
      scanf("%d", &n);
   } while (n <= 0);
   printf("Hai inserito %d\n", n);
   return 0;
}
```

Terminale

```
$ ./positivo
Inserisci un intero positivo: -3
Inserisci un intero positivo: 0
Inserisci un intero positivo: 5
Hai inserito 5
```

Con un `while` la domanda andrebbe scritta due volte, una prima del ciclo e una dentro; il `do-while` la scrive una volta sola.

Lo `switch`

La selezione a *n* vie

Quando una sola domanda decide fra molte strade si può usare una catena di `if-else`, come in `cnt_char.c`; ma se la domanda è "quale valore ha questa espressione intera?", il C offre un costrutto apposito, lo `switch` (la selezione a n vie della figura 4.7). Ecco una piccola calcolatrice:

calcolatrice.c

```c
#include <stdio.h>

int main(void)
{
   double a, b;
   char op;

   printf("Operazione (es. 3 * 4): ");
   scanf("%lf %c %lf", &a, &op, &b);
   switch (op) {
   case '+':
      printf("%g\n", a + b);
      break;
   case '-':
      printf("%g\n", a - b);
      break;
   case '*':
      printf("%g\n", a * b);
      break;
   case '/':
      if (b != 0)
         printf("%g\n", a / b);
      else
         printf("divisione per zero\n");
      break;
   default:
      printf("operatore sconosciuto: %c\n", op);
   }
   return 0;
}
```

Terminale

```
$ ./calcolatrice
Operazione (es. 3 * 4): 3 * 4
12
$ ./calcolatrice
Operazione (es. 3 * 4): 7 / 2
3.5
$ ./calcolatrice
Operazione (es. 3 * 4): 5 / 0
divisione per zero
$ ./calcolatrice
Operazione (es. 3 * 4): 2 ^ 8
operatore sconosciuto: ^
```

La conversione `%g` sceglie da sola fra la notazione normale e quella scientifica, e non scrive gli zeri inutili.

Come funziona

- Si calcola l'espressione fra parentesi, `op`, che deve essere di tipo intero (un `char` va bene).
- Si salta all'etichetta `case` con quel valore; se non ce n'è nessuna, a `default` (che è facoltativa: senza, non si esegue nulla).
- Da lì si eseguono le istruzioni **in sequenza**, finché si incontra un `break`, che fa uscire dallo `switch`, o si arriva alla graffa finale.
- Dopo `case` va una **costante** intera: non una variabile, non un intervallo, non una stringa.

Il terzo punto è il più importante: il `break` non è decorativo. Senza, dopo il `case` giusto l'esecuzione "cade" nei successivi (*fall through*). Togliendo i `break` dalla calcolatrice, con `3 + 4` si ottiene:

Terminale

```
$ ./senza_break
Operazione (es. 3 * 4): 3 + 4
7
-1
12
0.75
operatore sconosciuto: +
```

Dopo il caso `'+'` sono state eseguite tutte le istruzioni seguenti, fino a `default` (figura 9.8). Con `-Wextra` il compilatore avvisa:

Terminale

```
$ gcc -Wall -Wextra -o senza_break senza_break.c
senza_break.c: In function ‘main’:
senza_break.c:12:7: warning: this statement may fall through [-Wimplicit-fallthrough=]
   12 |       printf("%g\n", a + b);
      |       ^~~~~~~~~~~~~~~~~~~~~
senza_break.c:13:4: note: here
   13 |    case '-':
      |    ^~~~
```

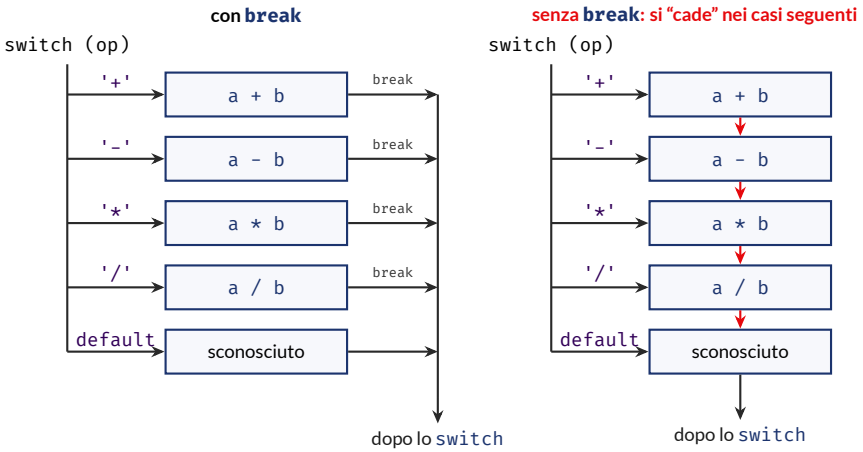


Figura 9.8: *Lo* `switch` *salta al caso giusto; il* `break` *lo fa uscire, altrimenti l'esecuzione prosegue con i casi successivi.*

Più casi, un'azione

La caduta da un caso al successivo è però utile quando più valori devono fare la stessa cosa: basta scrivere più etichette di seguito. Questo programma conta le vocali minuscole, gli spazi e gli altri caratteri:

vocali.c

```c
#include <stdio.h>

int main(void)
{
   int c, vocali = 0, spazi = 0, altri = 0;

   while ((c = getchar()) != EOF) {
      switch (c) {
      case 'a': case 'e': case 'i': case 'o': case 'u':
         vocali++;
         break;
      case ' ':
         spazi++;
         break;
      default:
         altri++;
      }
   }
   printf("vocali: %d  spazi: %d  altri: %d\n",
          vocali, spazi, altri);
   return 0;
}
```

Terminale

```
$ ./vocali
ciao a tutti
Ctrl+D
vocali: 6  spazi: 2  altri: 5
```

Gli "altri" sono `c`, le tre `t` e l'a capo finale.

break E continue

Uscire o saltare un giro

Il `break` non serve solo nello `switch`: dentro un ciclo lo fa terminare subito, e l'esecuzione prosegue dopo il ciclo. Il `continue`, invece, salta il resto del giro corrente e passa al controllo della condizione (in un `for`, prima esegue l'aggiornamento `expr3`). Il programma seguente somma i numeri positivi letti, salta i negativi e si ferma allo 0:

somma_positivi.c

```c
#include <stdio.h>

int main(void)
{
    int x, somma = 0;

    while (1) {
        printf("numero (0 per finire): ");
        scanf("%d", &x);
        if (x == 0)
            break;          /* esce dal ciclo */
        if (x < 0) {
            printf("  negativo, lo salto\n");
            continue;       /* giro successivo */
        }
        somma += x;
    }
    printf("somma dei positivi: %d\n", somma);
    return 0;
}
```

Terminale

```
$ ./somma_positivi
numero (0 per finire): 5
numero (0 per finire): -2
  negativo, lo salto
numero (0 per finire): 10
numero (0 per finire): 0
somma dei positivi: 15
```

La figura 9.9 mostra dove porta ciascuna delle due istruzioni. Se i cicli sono annidati, `break` e `continue` agiscono solo sul ciclo più interno che li contiene.

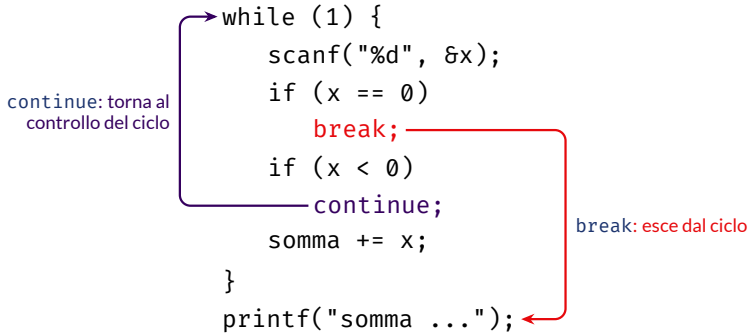


Figura 9.9: *Dove saltano* `break` *e* `continue`.

`break` e `continue` vanno usati con misura: un ciclo con molte uscite è difficile da seguire. Usati bene, però, evitano condizioni complicate e variabili di comodo.

In sintesi

In C una condizione è un numero: **zero** è falso, qualunque altro valore è vero; gli operatori relazionali (`< > <= >=`), di uguaglianza (`== !=`) e logici (`! && ||`) restituiscono 1 o 0. `a < b < c` non ha il significato matematico; `=` non è `==`; `&&` e `||` si fermano appena il risultato è noto (**cortocircuito**). Il corpo di `if`, `while` e `for` è una sola istruzione: per più istruzioni servono le graffe, e conviene metterle sempre; l'`else` si lega all'`if` più vicino. `while` controlla la condizione prima di ogni giro, `do-while` dopo (almeno un giro); `for` equivale a un `while` con inizializzazione e aggiornamento. Nei cicli sono più robuste le condizioni relazionali, mai `==` o `!=` fra reali. Lo `switch` sceglie fra costanti intere e senza `break` prosegue nei casi seguenti; `break` esce da un ciclo, `continue` salta al giro successivo.

Riferimenti

- **A. Kelley, I. Pohl**, *A Book on C*: capitolo 4 (flusso di controllo).
- **Statements** (`if`, `switch`, `while`, `do-while`, `for`, `break`, `continue`) — cppreference.com.
  `https://en.cppreference.com/w/c/language/statements`
- **Logical operators** — cppreference.com.
  `https://en.cppreference.com/w/c/language/operator_logical`

Le funzioni

Scomporre un problema in sotto-problemi più semplici è quasi sempre il modo migliore per risolverlo (lo sviluppo top-down del capitolo 4), e per scrivere programmi grandi è vitale. In C questo approccio si realizza con le **funzioni**. Nel capitolo 5 ne abbiamo scritte alcune e abbiamo incontrato prototipi e passaggio per valore; qui vediamo tutto in dettaglio: come si definisce una funzione, come restituisce un risultato, che cosa succede se il compilatore non la conosce, dove "vivono" le sue variabili.

Programmi fatti di funzioni

Un programma C è costituito da uno o più file (capitolo 7). Ogni file contiene zero funzioni (come gli header) o più funzioni, e uno solo contiene la funzione `main`, da cui comincia l'esecuzione; `main` può chiamare altre funzioni, che a loro volta ne chiamano altre, e tutte operano sulle variabili del programma.

Perché scrivere i programmi usando le funzioni?

- è più facile scrivere **corrette** delle funzioni brevi;
- scrittura e ricerca degli errori (*debugging*) sono più semplici, perché ogni funzione si prova da sola;
- le modifiche sono più semplici: si riscrivono solo le funzioni da cambiare;
- il codice è più comprensibile: funzioni brevi, con un nome che dice che cosa fanno, si leggono e si capiscono più facilmente.

La definizione di una funzione

Intestazione e corpo

Il codice che descrive che cosa fa una funzione è la sua **definizione**. La forma generale è

```
type function_name(parameters)
{
    declarations
    statements
}
```

La parte prima della graffa aperta è l'**intestazione** (*header*); la parte fra le graffe è il **corpo** della funzione. I **parametri** sono una lista di dichiarazioni di variabili separate da virgole: sono detti anche **parametri formali**, perché ricevono i valori (i *parametri attuali*, o argomenti) solo al momento della chiamata. La figura 10.1 mostra le parti nella funzione fattoriale delle slide.

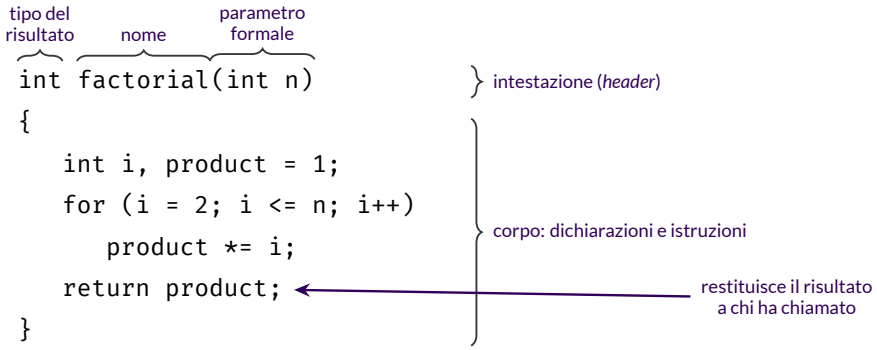


Figura 10.1: *Le parti della definizione di una funzione.*

`factorial` riceve un `int` e restituisce un `int`: il prodotto dei numeri da 2 a `n`, cioè $n!$ (per `n` uguale a 0 o 1 il ciclo non fa nessun giro, e il risultato è 1). Il valore restituito, se necessario, viene convertito nel tipo della funzione, qui `int`; e grazie alla lista dei parametri anche l'argomento, se necessario, viene convertito in `int`.

fattoriale.c

```c
#include <stdio.h>

int factorial(int n)                /* header */
{                                   /* body starts here */
   int i, product = 1;

   for (i = 2; i <= n; i++)
      product *= i;
   return product;
}

int main(void)
{
   int k;

   for (k = 0; k <= 5; k++)
      printf("%d! = %d\n", k, factorial(k));
   return 0;
}
```

Terminale

```
$ ./fattoriale
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
```

Il corpo di una funzione è un blocco, e può contenere dichiarazioni di variabili: `i` e `product` sono visibili **solo** dentro `factorial`.

`void`: niente risultato, niente parametri

Una funzione può non restituire nulla, e può non avere parametri. In entrambi i casi si usa la parola chiave `void` ("vuoto"):

```c
#include <stdio.h>

void wrt_address(void)
{
  printf(" \n%s \n%s \n%s \n%s \n\n",
    "            **********************",
    "            **    SANTA CLAUS     **",
    "            **    NORTH POLE      **",
    "            **    EARTH           **");
}

int main(void)
{
  wrt_address();
  return 0;
}
```

Terminale

```
$ ./santa

            **********************
            **    SANTA CLAUS     **
            **    NORTH POLE      **
            **    EARTH           **
```

Il primo `void` dice al compilatore che la funzione non restituisce alcun valore; il secondo, fra le parentesi, che non ha parametri. La funzione si chiama con l'istruzione `wrt_address();`, con le parentesi vuote. E se si scrivesse `a = wrt_address();`? Non c'è nessun valore da assegnare, e il compilatore si rifiuta:

Terminale

```
$ gcc -Wall -o santa2 santa2.c
santa2.c: In function ‘main’:
santa2.c:12:5: error: void value not ignored as it ought to be
   12 |   a = wrt_address();
      |     ^
```

Una svista nelle slide

In una delle slide la funzione è scritta con `Printf`, con la P maiuscola. Il C distingue maiuscole e minuscole, e `Printf` non è `printf`: il compilatore lo tratta come una funzione sconosciuta (e suggerisce *did you mean 'printf'?*), poi il linker non la trova da nessuna parte e la compilazione fallisce con *undefined reference to 'Printf'*.

Altri esempi

Ecco tre definizioni delle slide: una funzione che non fa nulla, una che raddoppia un reale e una che somma tre interi.

esempi.c

```c
#include <stdio.h>

void nothing(void)
{
}

double twice(double x)
{
    return (2.0 * x);
}

int all_add(int a, int b, int c)
{
    return (a + b + c);
}

int main(void)
{
    nothing();
    printf("%f\n", twice(3.5));
    printf("%d\n", all_add(1, 2, 3));
    return 0;
}
```

Terminale

```
$ ./esempi
7.000000
6
```

`nothing` ha il corpo vuoto: chiamarla non fa nulla, ma è una funzione lecita (può servire come segnaposto, in attesa di scriverla).

Se il tipo della funzione non è indicato, il C tradizionale lo considera `int`: `all_add(int a, int b, int c) { …}` equivale a `int all_add(…)`. Dal C99 questa regola non vale più, e `gcc` protesta (*return type defaults to 'int'*); i compilatori più recenti la considerano un errore. Specificare sempre il tipo della funzione è comunque una buona pratica.

Le definizioni nel C tradizionale

Nel C tradizionale (quello precedente all'ANSI C) la definizione aveva una sintassi diversa: fra le parentesi c'erano solo i nomi dei parametri, e le loro dichiarazioni venivano dopo, prima della graffa aperta. L'ordine delle dichiarazioni non contava, e senza parametri si mettevano solo le parentesi:

```c
void f(a, b, c, x, y)
int     a, b, c;
double  x, y;
{
    ...
}
```

```c
void f()
{
    ...
}
```

Questa forma è obsoleta, e fra poco vedremo perché è pericolosa. Si incontra ancora nei programmi vecchi.

L'ISTRUZIONE `return`

Terminare e restituire

Con l'istruzione `return` la funzione termina e il controllo torna all'ambiente da cui è stata chiamata. `return` non deve necessariamente contenere un'espressione: `return;`, `return ++a;` e `return (a * b);` sono tutte istruzioni lecite (le parentesi attorno all'espressione sono facoltative).

- Se c'è un'espressione, il suo valore viene passato all'ambiente chiamante, convertito se necessario nel tipo della funzione.
- In una funzione ci possono essere zero o più `return`.
- Se non ce n'è nessuno, il controllo torna al chiamante quando si raggiunge la graffa di chiusura.

assoluto.c

```c
#include <stdio.h>

double absolute_value(double x)
{
  if (x >= 0)
    return x;
  else
    return -x;
}

int main(void)
{
  printf("%f %f\n", absolute_value(-2.5),
                    absolute_value(4.0));
  return 0;
}
```

Terminale

```
$ ./assoluto
2.500000 4.000000
```

Due `return`: a seconda del segno di `x`, la funzione termina con l'uno o con l'altro.

Il valore si può ignorare

Non sempre si usa il valore restituito. Nel frammento delle slide

```c
while (...) {
    getchar();        /* il carattere letto viene scartato */
    c = getchar();
    ...
}
```

il primo `getchar()` serve solo a saltare un carattere. Lo facciamo anche con `printf`, senza saperlo: anche `printf` restituisce un valore, il numero di caratteri scritti, che di solito si ignora.

ritorno.c

```c
#include <stdio.h>

int main(void)
{
   int n;

   n = printf("ciao\n");
   printf("caratteri scritti: %d\n", n);
   return 0;
}
```

Terminale

```
$ ./ritorno
ciao
caratteri scritti: 5
```

I caratteri sono 5: `c`, `i`, `a`, `o` e l'a capo.

I PROTOTIPI

Dichiarare prima di usare

Le funzioni devono essere **dichiarate prima di essere usate**. L'ANSI C fornisce per questo il **prototipo**, che informa il compilatore sul numero e sul tipo dei parametri e sul tipo del valore restituito. Il prototipo

```
1 double sqrt(double);
```

dice che `sqrt` è una funzione, che prende un solo argomento di tipo `double` e che restituisce un valore di tipo `double`. La forma generale è

```
1 type function_name(parameter type list);
```

Gli identificatori dei parametri sono facoltativi: `void f(char c, int i);` e `void f(char, int);` sono equivalenti. Il compilatore non li usa, ma servono da documentazione, soprattutto se i nomi sono significativi: conviene metterli sempre.

Nel C tradizionale la dichiarazione non conteneva la lista dei parametri: `double sqrt();`. I prototipi sono preferibili perché, conoscendo i tipi dei parametri, il compilatore **converte** opportunamente i valori passati alla funzione.

Che cosa succede senza prototipo

Consideriamo la chiamata `sqrt(a)`, con `a` intero e uguale a 4. Con il prototipo il compilatore sa che `sqrt` vuole un `double`, e converte 4 in 4.0; senza, non sa nulla, e il risultato non è corretto:

prova.c

```c
#include <stdio.h>

double sqrt();

int main()
{
  int a = 4;

  printf("sqrt(4) = %f \n", sqrt(a));
}
```

Terminale

```
$ gcc -o prova prova.c -lm
$ ./prova
sqrt(4) = 0.000000
```

prova.c

```c
#include <stdio.h>

double sqrt(double);

int main()
{
  int a = 4;

  printf("sqrt(4) = %f \n", sqrt(a));
}
```

Terminale

```
$ gcc -o prova prova.c -lm
$ ./prova
sqrt(4) = 2.000000
```

Perché zero? Sui PC a 64 bit gli argomenti interi e quelli reali viaggiano in due registri diversi del processore. Senza prototipo il valore 4 viene consegnato come intero, nel registro degli interi; `sqrt` va a prendere il suo `double` nell'altro registro, e ci trova un valore che non c'entra (qui, per caso, 0). Con il prototipo il compilatore converte 4 in 4.0 e lo consegna dove `sqrt` lo cerca (figura 10.2).

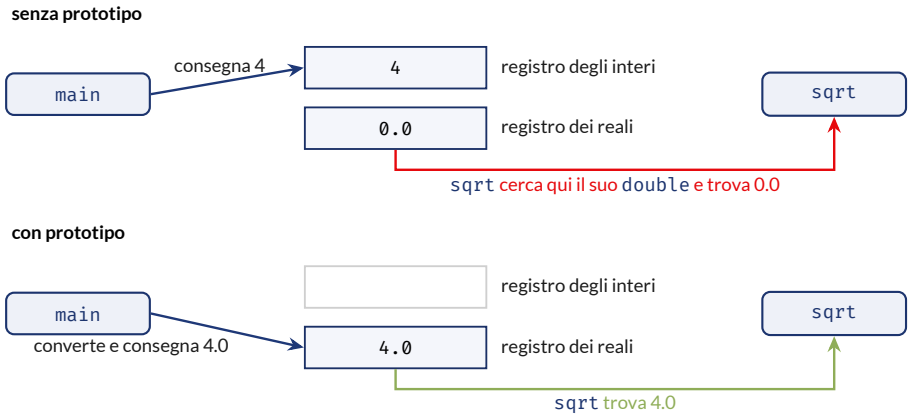


Figura 10.2: *Senza prototipo l'argomento viene consegnato nel posto sbagliato.*

`sqrt`, in realtà, è una funzione che `gcc` conosce bene: anche senza `-Wall` avvisa che l'argomento è un `int` dove ci si aspetta un `double`. Con `clang`, che la conosce altrettanto bene, il primo programma stampa addirittura 2.000000. Con una funzione qualsiasi, però, nessun compilatore può rimediare: senza prototipo il risultato non è garantito.

Come il compilatore scopre le funzioni

Per il compilatore una funzione può essere dichiarata in tre modi: da una **chiamata**, da una **definizione** o da un **prototipo**.

- Se incontra una chiamata `f(x)` prima di qualunque altra dichiarazione, il compilatore assume la dichiarazione `int f();`: un risultato `int` e nessuna ipotesi sui parametri, che il programmatore deve passare del tipo giusto (dal C99 è un errore, che `gcc` segnala con un avviso, come abbiamo visto nel capitolo 5).
- Una definizione in stile tradizionale dichiara solo il nome e il tipo del risultato, non i parametri.
- Una definizione in stile ANSI, come un prototipo, dichiara tutto.

Le slide lo mostrano con tre versioni della stessa funzione, che calcola la radice quadrata del suo parametro, chiamata con un `int`, un `double` e un `long double` che valgono tutti 4. Ecco la prima, con la definizione tradizionale:

kr1.c

```c
#include <stdio.h>
#include <math.h>

float f(x)
long double x;
{
  return sqrt(x);
}

int main()
{
  int a = 4;
  double b = 4;
  long double c = 4;

  printf("parametro intero: %f \n", f(a));
  printf("parametro double: %f \n", f(b));
  printf("parametro long double: %f \n", f(c));
}
```

Terminale

```
$ ./kr1
parametro intero: 0.000000
parametro double: 0.000000
parametro long double: 2.000000
```

Il codice contiene la definizione di `f`, ma solo la riga `float f(x)` vale come dichiarazione: non si assume nulla sui parametri, che vanno passati del tipo giusto, perché il compilatore non può fare nessuna conversione. Così solo la terza chiamata, che passa davvero un `long double`, dà il risultato corretto.

Un parametro senza tipo

Nella seconda versione il tipo di `x` manca del tutto. Il compilatore assume la dichiarazione `int x;` (e avvisa: *type of 'x' defaults to 'int'*); ora funziona solo la chiamata con l'intero:

kr2.c

```c
#include <stdio.h>
#include <math.h>

float f(x)
{
  return sqrt(x);
}
```

Terminale

```
$ ./kr2
parametro intero: 2.000000
parametro double: -nan
parametro long double: -nan
```

`main` è identico a quello di `kr1.c`.

Nelle slide le due chiamate sbagliate stampano 0.000000; sul nostro computer stampano `-nan` (*not a number*: la radice di un numero negativo). Sono valori presi a caso dalla memoria, e possono cambiare da un computer all'altro.

La definizione ANSI

Nella terza versione l'intestazione è `float f(long double x)`: questa volta la riga che fa da dichiarazione è un **prototipo**. Il compilatore conosce la lista dei parametri, e può convertire l'`int` e il `double` in `long double`:

ansi.c

```c
#include <stdio.h>
#include <math.h>

float f(long double x)
{
  return sqrt(x);
}
```

Terminale

```
$ ./ansi
parametro intero: 2.000000
parametro double: 2.000000
parametro long double: 2.000000
```

definizione di `f`	con un `int`	con un `double`	con un `long double`
`float f(x) long double x;`	0.000000	0.000000	2.000000
`float f(x)`	2.000000	-nan	-nan
`float f(long double x)`	2.000000	2.000000	2.000000

In definitiva: la definizione della funzione in stile ANSI, oppure il suo prototipo, deve comparire **prima** che la funzione sia usata. È anche uno dei motivi per cui si includono gli header: contengono i prototipi delle funzioni di libreria.

IL PASSAGGIO PER VALORE

Una copia per ogni argomento

Gli argomenti delle funzioni sono passati **per valore**: ogni argomento viene calcolato, e il suo valore viene usato localmente al posto del parametro corrispondente. Se a una funzione si passa una variabile, il suo valore nell'ambiente chiamante **non cambia**, anche se la funzione modifica il valore che riceve (lo abbiamo visto nella figura 5.9). L'esempio delle slide:

compute_sum.c

```c
#include <stdio.h>

int compute_sum(int n)       /* sum the integers from 1 to n */
{
   int   sum = 0;

   for ( ; n > 0; --n)       /* stored value of n is changed */
      sum += n;
   return sum;
}

int main(void)
{
   int   n = 3, sum, compute_sum(int);

   printf("%d\n", n);
   sum = compute_sum(n);
   printf("%d\n", n);
   printf("%d\n", sum);
   return 0;
}
```

Terminale

```
$ ./compute_sum
3
3
6
```

`compute_sum(int)`, fra le dichiarazioni di `main`, è un prototipo: lecito anche lì.

La pila delle chiamate

Dove si trovano le variabili di una funzione? Ogni chiamata riceve una propria zona di memoria (un *record di attivazione*), che contiene i parametri e le variabili locali; le zone si impilano una sull'altra, come piatti, e ognuna viene tolta quando la sua funzione termina: è la **pila delle chiamate** (*call stack*). La figura 10.3 la segue durante l'esecuzione di `compute_sum.c`.

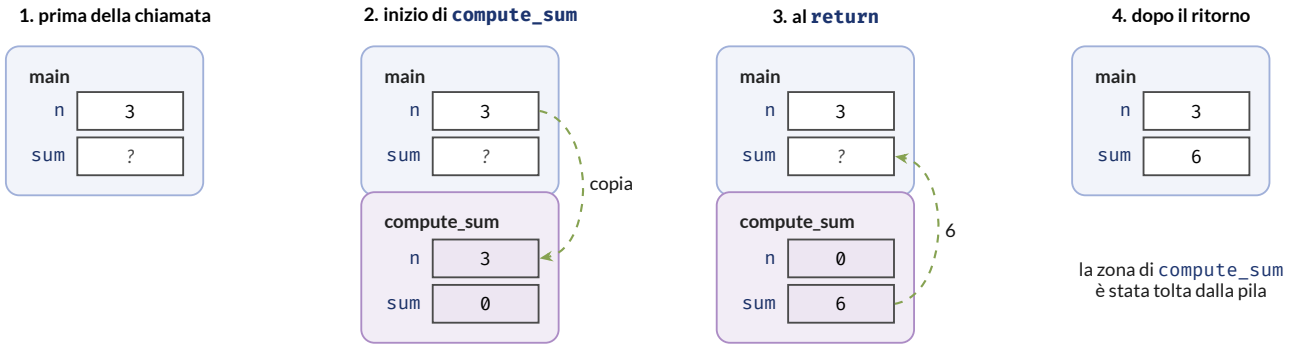


Figura 10.3: *La pila delle chiamate:* `compute_sum` *ha la sua* `n` *(una copia) e la sua* `sum`*; modifica la propria* `n`*, e quando termina la sua zona scompare.*

`compute_sum` porta la **sua** `n` da 3 a 0 mentre somma; la `n` di `main` è un'altra variabile, e resta 3. Se si vuole che una funzione modifichi una variabile del chiamante, ci sono due strade: restituire il nuovo valore come risultato (e assegnarlo nel chiamante), oppure passare l'**indirizzo** della variabile, cioè un puntatore (capitolo 11).

VISIBILITÀ: VARIABILI LOCALI E GLOBALI

Locali e globali

Le variabili dichiarate nel corpo di una funzione sono dette **locali**: esistono e sono visibili solo lì. Le variabili dichiarate fuori da ogni funzione sono dette **globali** (o esterne): sono visibili da tutte le funzioni del file che vengono dopo la dichiarazione. Che cosa stampa questo programma?

globale.c

```c
#include <stdio.h>

int a = 33;

int all_add(int a, int b, int c)
{
  return (a+b+c);
}

int main(void)
{
  int b = 15, c = 34;
  printf("%d \n", all_add(a,b,c));
}
```

Terminale

```
$ ./globale
82
```

In `main` non c'è una `a` locale: `a` è quella globale, che vale 33, e $33 + 15 + 34 = 82$.

Ora aggiungiamo una `a` locale in `main`, con valore 7:

locale.c

```c
int main(void)
{
  int a=7, b = 15, c = 34;
  printf("%d \n", all_add(a,b,c));
}
```

Terminale

```
$ ./locale
56
```

La `a` locale **nasconde** quella globale: $7 + 15 + 34 = 56$.

Dentro `main`, il nome `a` indica la variabile locale, che nasconde la globale; dentro `all_add`, `a` è il parametro, che a sua volta nasconde la globale (figura 10.4). In ogni punto del programma vale la dichiarazione **più vicina**.

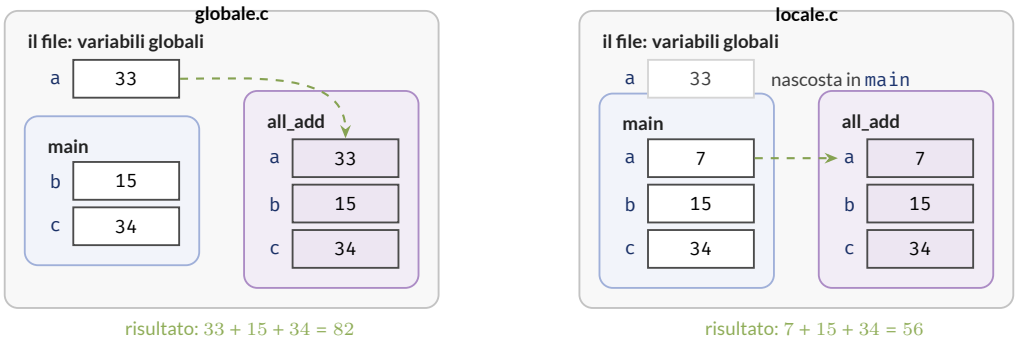


Figura 10.4: *Da dove viene la* `a` *passata ad* `all_add`*: a sinistra è la globale, a destra quella locale di* `main`*.*

Più in generale, un **blocco** è una parte di codice delimitata da graffe, e può contenere dichiarazioni di variabili, visibili solo all'interno del blocco. Il corpo di una funzione è un blocco, ma un blocco può stare anche dentro un altro:

blocchi.c

```c
#include <stdio.h>

int main(void)
{
    int a = 1;

    {
        int a = 2;              /* nasconde la a esterna */
        int b = 3;

        printf("dentro il blocco: a = %d, b = %d\n", a, b);
    }
    printf("fuori dal blocco: a = %d\n", a);
    return 0;
}
```

Terminale

```
$ ./blocchi
dentro il blocco: a = 2, b = 3
fuori dal blocco: a = 1
```

Fuori dal blocco `b` non esiste più: usarla sarebbe un errore. L'opzione `-Wshadow` di `gcc` segnala le variabili che ne nascondono altre.

LE CLASSI DI MEMORIZZAZIONE

Ogni variabile e ogni funzione del C ha due attributi: il **tipo** e la **classe di memorizzazione** (*storage class*), che stabilisce **dove** e **per quanto tempo** esiste e **da dove** è visibile. Le classi sono quattro, corrispondenti alle parole chiave `auto`, `extern`, `register` e `static`.

`auto`

Le variabili dichiarate dentro un blocco sono `auto` per default. Quando si entra nel blocco, il sistema alloca la memoria per le sue variabili locali; quando se ne esce, la libera (è la pila della figura 10.3). Poiché è la classe più comune, la parola chiave `auto` non si scrive quasi mai.

extern

Un modo semplice, ma sconsigliato, di passare informazioni fra blocchi e funzioni sono le variabili esterne. Una variabile è esterna se è definita fuori da ogni blocco, e riceve automaticamente la classe `extern`. La parola chiave `extern`, invece, dice al compilatore di cercare la variabile **da un'altra parte**. Un esempio su due file:

file1.c

```c
#include <stdio.h>

int a = 1, b = 2, c = 3;        /* variabili esterne */
int f(void);

int main(void)
{
  printf("%3d\n", f());
  printf("%3d %3d %3d\n", a, b, c);
  return 0;
}
```

file2.c

```c
extern int a;               /* definita altrove */

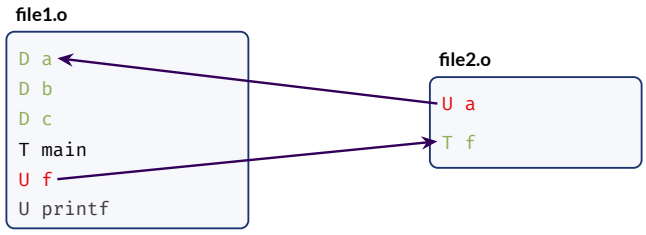
int f(void)
{
  int b, c;                   /* variabili locali */

  a = b = c = 4;
  return (a + b + c);
}
```

Terminale

```
$ gcc -Wall -c file1.c
$ gcc -Wall -c file2.c
$ gcc -o esterne file1.o file2.o
$ ./esterne
 12
  4   2   3
```

- I due file sono compilati separatamente (capitolo 7). In `file1.c`, `a`, `b` e `c` sono variabili esterne.
- Compilando `file2.c`, la riga `extern int a;` dice al compilatore che `a` è globale e definita in un altro file.
- Il linker mette insieme le informazioni e fa sì che `f` usi la cella di memoria giusta (figura 10.5).
- `f` assegna 4 alla `a` globale e alle sue `b` e `c` locali, e restituisce 12; le `b` e `c` globali restano 2 e 3.



nm: D è una variabile definita nel file, T una funzione definita,
U un nome usato ma definito altrove: il linker collega ogni U alla sua definizione

Figura 10.5: *Che cosa vede il linker:* `file2.o` *usa la* `a` *definita in* `file1.o`*, e* `file1.o` *usa la* `f` *definita in* `file2.o`*.*

Le informazioni si possono passare anche come parametri delle funzioni, ed è la strada da preferire: bisogna cercare di evitare le variabili globali. Con i parametri il codice è più chiaro e si capisce meglio dove i valori vengono modificati; una variabile globale, invece, può essere cambiata da qualunque funzione di qualunque file.

Anche le funzioni hanno per default la classe `extern`: sono visibili ovunque nel programma, anche se è scritto su più file. Si può aggiungere `extern` alla definizione o al prototipo (`extern double sin(double);`), ma in generale non serve.

register

La classe `register` chiede al compilatore di tenere la variabile, se possibile, in un **registro** del processore, la memoria più veloce (capitolo 2). I registri sono pochi e servono anche al sistema, quindi la richiesta non sempre è soddisfatta: in quel caso la variabile diventa `auto`. Le variabili globali e le funzioni non possono essere `register`.

`register` è un tentativo di velocizzare il programma, e le variabili vanno scelte con cura: poche, usate molto spesso. I candidati tipici sono le variabili di controllo dei cicli. Dichiarata all'inizio di `main`, però, la variabile occupa il registro per tutta l'esecuzione della funzione; è più efficiente racchiudere dichiarazione e ciclo in un blocco:

```c
int main(void)
{
    register int i;
    ...
    for (i = 0; i < LIMIT; i++) {
        ...
    }
}
```

```c
int main(void)
{
    ...
    {
        register int i;
        for (i = 0; i < LIMIT; i++) {
            ...
        }
    }
}
```

Una variabile `register` non ha un indirizzo in memoria, quindi non le si può applicare `&`: *address of register variable 'i' requested*, risponde `gcc`. Oggi i compilatori scelgono da soli, e molto bene, quali variabili tenere nei registri: la parola chiave `register` è di fatto superata.

`static` per le variabili locali

Le dichiarazioni `static` hanno due usi distinti. Il primo è **conservare il valore** di una variabile locale anche dopo l'uscita dal blocco: una variabile locale `static` viene inizializzata una sola volta, e non sta nella pila ma in una zona di memoria che esiste per tutta l'esecuzione del programma.

statica.c

```c
#include <stdio.h>

void f(void)
{
    static int cnt = 0;
    int volte = 0;

    ++cnt;
    ++volte;
    printf("cnt = %d   volte = %d\n", cnt, volte);
}

int main(void)
{
    f();
    f();
    f();
    return 0;
}
```

Terminale

```
$ ./statica
cnt = 1   volte = 1
cnt = 2   volte = 1
cnt = 3   volte = 1
```

La prima volta `cnt` viene inizializzata a 0 e incrementata a 1. La seconda volta che `f` viene chiamata, `cnt` **non** viene inizializzata di nuovo: conserva il valore 1 e lo porta a 2. `volte`, che è `auto`, nasce e muore a ogni chiamata, e vale sempre 1 (figura 10.6). Le slide usano questa idea per fare una cosa sì e una no: `if (cnt % 2 == 0)` distingue le chiamate pari da quelle dispari.

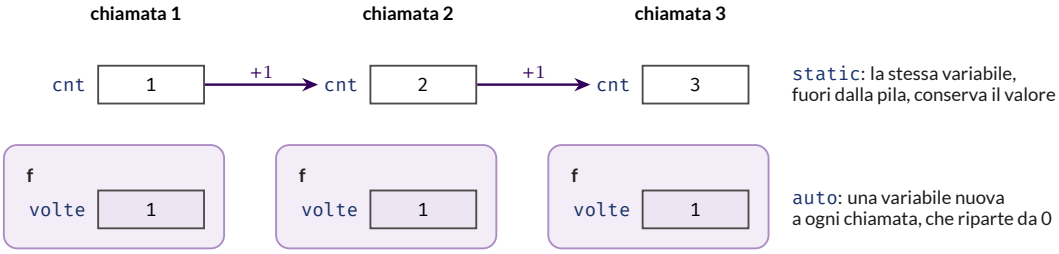


Figura 10.6: `cnt` *attraversa le chiamate;* `volte` *viene creata e distrutta ogni volta.*

`static` per le variabili globali

Il secondo uso riguarda le variabili esterne. Una variabile globale `static` ha una visibilità più ristretta: è visibile solo dalle funzioni del suo file definite dopo di lei, e non da quelle di altri file. Tentare di raggiungerla con `extern` non serve. L'esempio delle slide è una famiglia di generatori di numeri pseudo-casuali:

random.c

```c
/* A family of pseudo random number generators. */

#define    INITIAL_SEED         17
#define    MULTIPLIER           25173
#define    INCREMENT            13849
#define    MODULUS              65536
#define    FLOATING_MODULUS     65536.0

static unsigned seed = INITIAL_SEED;   /* esterna, ma privata di questo file */

unsigned random(void)
{
   seed = (MULTIPLIER * seed + INCREMENT) % MODULUS;
   return seed;
}

double probability(void)
{
   seed = (MULTIPLIER * seed + INCREMENT) % MODULUS;
   return (seed / FLOATING_MODULUS);
}
```

- `random()` produce un intero fra 0 e `MODULUS` (escluso), `probability()` un reale fra 0 e 1.
- Entrambe usano il vecchio valore di `seed` (il *seme*) per calcolare il nuovo, e lo aggiornano: ogni numero dipende dal precedente.
- Se `seed` venisse modificata per errore da un'altra parte del programma, la sequenza cambierebbe. Siccome è `static`, è privata di questo file e nessun altro modulo può toccarla.

Lo verifichiamo con un programma di prova, e poi con uno che prova a "rubare" il seme:

prova.c

```c
#include <stdio.h>

unsigned random(void);
double probability(void);

int main(void)
{
   int i;

   for (i = 0; i < 5; i++)
      printf("%u ", random());
   printf("\n%f\n", probability());
   return 0;
}
```

Terminale

```
$ gcc -c random.c
$ gcc -o prova prova.c random.o
$ ./prova
48574 61999 40372 31453 39802
0.537521
$ nm random.o
000000000000002a T probability
0000000000000000 T random
0000000000000000 d seed
```

Il primo numero è proprio $(25173 \cdot 17 + 13849) \bmod 65536 = 48574$. In `nm` il seme compare con la `d` **minuscola**: la minuscola indica un simbolo locale al file, che il linker non mostra agli altri. Infatti un programma che dichiara `extern unsigned seed;` e prova ad assegnargli 0 non viene collegato:

ruba.c

```c
#include <stdio.h>

extern unsigned seed;
unsigned random(void);

int main(void)
{
    seed = 0;
    printf("%u\n", random());
    return 0;
}
```

Il compilatore accetta `ruba.c`: `extern` promette che `seed` è definita da qualche parte. È il linker a non trovarla, perché in `random.o` il seme non è visibile da fuori (le righe con *warning* sono avvisi secondari, conseguenza dello stesso problema).

Terminale

```
$ gcc -o ruba ruba.c random.o
/usr/bin/ld: /tmp/ccEe4NI7.o: warning: relocation against `seed' in read-only section `.text'
/usr/bin/ld: /tmp/ccEe4NI7.o: in function `main':
ruba.c:(.text+0xa): undefined reference to `seed'
/usr/bin/ld: warning: creating DT_TEXTREL in a PIE
collect2: error: ld returned 1 exit status
```

`static` per le funzioni

Anche una funzione può essere `static`: allora è visibile solo nel file in cui è definita, e solo le funzioni dello stesso file possono chiamarla. Serve a definire moduli con funzioni **private**. Nell'esempio delle slide due file definiscono ciascuno la propria `sf`, `static`, e non c'è alcun conflitto:

main.c

```c
#include <stdio.h>

void a(void);
void f() { puts("main f"); }
static void sf() { puts("main sf"); }

void m() {
    f();
    sf();
}

int main() {
    m();
    a();
    return 0;
}
```

file.c

```c
#include <stdio.h>

void f(void);
static void sf() { puts("a sf"); }

void a() {
    f();
    sf();
}
```

Terminale

```
$ gcc -c file.c -o file.o
$ gcc -c main.c -o main.o
$ gcc -o main main.o file.o
$ ./main
main f
main sf
main f
a sf
```

`puts` scrive una stringa e va a capo. `main` chiama `m`, che chiama `f` e la `sf` del suo file; poi chiama `a`, che sta in `file.c`: `a` chiama `f`, che esiste solo in `main.c` ed è visibile ovunque, e la `sf` del **proprio** file (figura 10.7).

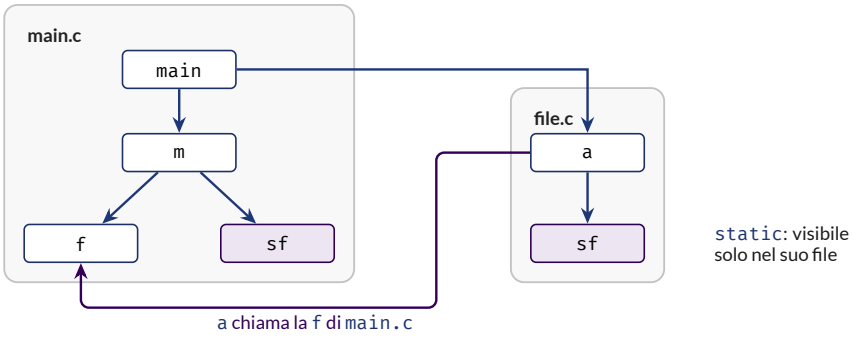


Figura 10.7: *Chi chiama chi: `f` è visibile da entrambi i file, ciascuna `sf` solo dal proprio.*

classe	dove si dichiara	da dove è visibile	quanto vive
`auto`	in un blocco (è il default)	nel blocco	dall'ingresso all'uscita dal blocco
`register`	in un blocco	nel blocco	come `auto`
`static`	in un blocco	nel blocco	per tutto il programma
esterna (`extern`)	fuori dalle funzioni	nel file; negli altri file con `extern`	per tutto il programma
`static`	fuori dalle funzioni	solo nel suo file	per tutto il programma

In sintesi

Una funzione ha un'**intestazione** (tipo del risultato, nome, parametri formali) e un **corpo**; `void` indica che non restituisce nulla o che non ha parametri. `return` termina la funzione e restituisce un valore, convertito nel tipo della funzione; il valore si può ignorare. Una funzione va dichiarata **prima** dell'uso, con la definizione in stile ANSI o con un **prototipo**: senza, il compilatore non converte gli argomenti e i risultati sono sbagliati. Gli argomenti sono passati **per valore**: ogni chiamata ha la sua zona nella pila, con copie dei valori. Le variabili locali sono visibili solo nel loro blocco e nascondono quelle esterne con lo stesso nome. Le classi di memorizzazione stabiliscono durata e visibilità: `auto` (il default), `extern` (variabili definite altrove), `register` (superata), `static` (variabili locali che conservano il valore, variabili e funzioni private di un file).

RIFERIMENTI



- **A. Kelley, I. Pohl**, *A Book on C*: capitolo 5 (funzioni e classi di memorizzazione).
- **Function declarations** e **Storage-class specifiers** — cppreference.com.
  `https://en.cppreference.com/w/c/language/function_declaration`
  `https://en.cppreference.com/w/c/language/storage_duration`

Array e puntatori

I programmi lavorano spesso su molti dati dello stesso tipo: i voti di una classe, le misure di un esperimento, le parole di un testo. Gli **array** sono il tipo di dato che li raccoglie in variabili **indicizzate**. In C gli array sono strettamente legati ai **puntatori**: il nome di un array è trattato come un puntatore costante, i puntatori si possono indicizzare come gli array, e il C ha un'**aritmetica dei puntatori**. I puntatori servono anche a ottenere l'effetto del passaggio "per riferimento" dei parametri, e a chiedere memoria mentre il programma è in esecuzione. Le stringhe, che in C sono array di caratteri, e gli array a più dimensioni sono l'argomento del capitolo 12.

Gli array

Tante variabili, un solo nome

Un programma che gestisce dei voti potrebbe dichiarare `int grade0, grade1, grade2;`. Ma se le variabili necessarie sono cento, o mille, dichiararle tutte una per una diventa impossibile. Una variabile **indicizzata** è molto più comoda:

```
1 int grade[3];                    /* un array di tre interi */
2
3 grade[0] = grade[1] = grade[2] = 0;
```

`grade` è un **array** di tre interi, che si chiamano `grade[0]`, `grade[1]` e `grade[2]`: un solo nome, e un **indice** fra parentesi quadre che seleziona l'elemento. Il primo indice è **sempre 0**. In memoria gli elementi occupano celle **consecutive** (figura 11.1).

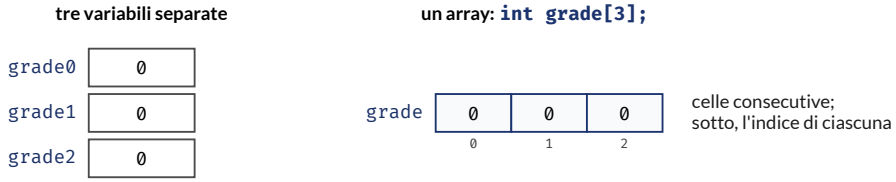


Figura 11.1: *Tre variabili indipendenti e un array di tre elementi.*

La dichiarazione

La dichiarazione di un array a una dimensione indica il **tipo** degli elementi, l'**identificatore** e, fra parentesi quadre, un'espressione **costante** intera: la dimensione. La dichiarazione `int a[size];` riserva lo spazio per gli elementi da `a[0]` a `a[size-1]`. È buona norma definire la dimensione come costante simbolica, così che per cambiarla basti modificare una riga:

somma.c

```c
#include <stdio.h>

#define N 4

int main(void)
{
  int a[N] = {3, 7, 1, 9}, i;
  int sum = 0;

  for (i=0; i<N; i++)
    sum += a[i];
  printf("Sum is %d \n", sum);
  return 0;
}
```

Terminale

```
$ ./somma
Sum is 20
```

Il ciclo `for` con `i` da 0 a `N-1` è il modo tipico di visitare tutti gli elementi.

Come per ogni variabile, ogni elemento occupa i byte del suo tipo: gli elementi di un array di `int` stanno a 4 byte di distanza l'uno dall'altro (figura 11.2).

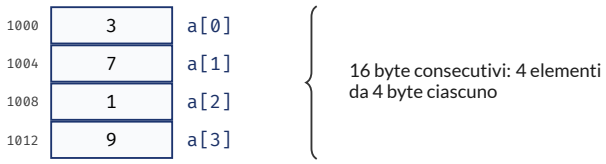


Figura 11.2: `int a[4] = {3, 7, 1, 9};` *in memoria (indirizzi di esempio).*

L'inizializzazione

Gli array possono avere classe di memorizzazione automatica, esterna o statica, ma non `register` (capitolo 10). Nel C tradizionale solo gli array esterni e statici potevano essere inizializzati esplicitamente; in ANSI C lo possono essere anche quelli automatici, con una lista di valori fra graffe:

- `float v[5] = {0.0, 1.0, 2.0, 3.0, 4.0};` assegna `v[0] = 0.0`, `v[1] = 1.0`, e così via;
- `int v[5] = {0};` azzera l'intero vettore;
- `int v[5] = {1};` assegna `v[0] = 1` e pone a 0 i rimanenti: se i valori sono meno degli elementi, gli altri valgono 0;
- se la dimensione manca e c'è l'inizializzazione, la dimensione è il numero dei valori: `int a[] = {1, 2, 3, 4, 5};` equivale a `int a[5] = {1, 2, 3, 4, 5};`.

Se invece un array **non** viene inizializzato, dipende dalla sua classe: gli array esterni e statici vengono azzerati dal sistema, quelli automatici no, e contengono ciò che c'era in memoria.

init.c

```c
#include <stdio.h>

int v[5];

int main(void)
{
   static int a[5];
   int b[5];
   int i;

   for (i=0; i<5; i++)
     printf("%d ",a[i]);
   printf("\n");

   for (i=0; i<5; i++)
     printf("%d ",v[i]);
   printf("\n");

   for (i=0; i<5; i++)
     printf("%d ",b[i]);
   printf("\n");
}
```

Terminale

```
$ ./init
0 0 0 0 0
0 0 0 0 0
0 0 -2037142800 32652 179344864
$ ./init
0 0 0 0 0
0 0 0 0 0
0 0 744688368 32742 1091613072
```

a (statico) e v (esterno) sono azzerati; b (automatico) contiene valori a caso, diversi a ogni esecuzione. `gcc` con `-Wall` avvisa: *'b' may be used uninitialized*.

Gli indici e l'errore più comune

Se `a` è un array, `a[expr]`, con `expr` espressione intera, è un suo elemento. Con la dichiarazione `int a[N];` gli indici validi vanno da 0 a `N-1`. Accedere ad `a[i]` con `i` fuori da questo intervallo è un errore di programmazione molto comune, e il C **non lo controlla**: si legge o si scrive memoria che non è stata riservata per l'array. L'effetto dipende dal sistema: il programma può fermarsi con un errore, oppure proseguire con risultati sbagliati.

fuori.c

```c
#include <stdio.h>

int main(void)
{
   int a[5] = {10, 20, 30, 40, 50};
   int i;

   for (i = 0; i <= 5; i++)      /* un giro di troppo */
      printf("a[%d] = %d\n", i, a[i]);
   return 0;
}
```

Terminale

```
$ ./fuori
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 40
a[4] = 50
a[5] = 32764
```

`a[5]` non esiste: il programma legge senza protestare ciò che c'è dopo l'array.

Errori come questo sono difficili da trovare, perché spesso il programma sembra funzionare. Un aiuto prezioso è l'opzione `-fsanitize=address` di `gcc`, che aggiunge al programma dei controlli sugli accessi alla memoria e lo ferma al primo accesso sbagliato, dicendo dove (qui il rapporto è abbreviato, e al posto del percorso della cartella c'è un segnaposto):

Terminale

```
$ gcc -g -fsanitize=address -o fuori fuori.c
$ ./fuori
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 40
a[4] = 50
=================================================================
==2607==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7f364ce00034 at pc 0x5575a6d3d467 bp 0
    x7fff693a7980 sp 0x7fff693a7970
READ of size 4 at 0x7f364ce00034 thread T0
    #0 0x5575a6d3d466 in main [percorso del file]/fuori.c:9
…
    [32, 52) 'a' (line 5) <== Memory access at offset 52 overflows this variable
…
```

Il rapporto dice che alla riga 9 si è letto (*READ*) un `int` (4 byte) appena oltre la fine dell'array `a`, dichiarato alla riga 5.

I PUNTATORI

Indirizzi

Una variabile è memorizzata in un certo numero di byte, in una particolare posizione della memoria identificata da un **indirizzo** (capitolo 2). L'indirizzo di una variabile `v` si ottiene con l'operatore `&`: `&v`. Gli indirizzi sono valori, e come tali si possono memorizzare e manipolare: le variabili che contengono indirizzi si chiamano **puntatori**, e servono ad accedere alla memoria e a manipolare gli indirizzi.

Dichiarare e usare un puntatore

La dichiarazione

```
int *p;
```

dichiara la variabile `p` come **puntatore a `int`**: può contenere l'indirizzo di una variabile `int`. I suoi valori possibili sono lo zero e gli interi positivi che sul sistema possono essere interpretati come indirizzi. Alcuni esempi di assegnazione:

```
p = 0;
p = NULL;              /* equivalente a p = 0 */
p = &i;
p = (int *) 1776;      /* un indirizzo assoluto in memoria */
```

`NULL`, definita negli header standard, vale 0 e indica un puntatore che **non punta a niente**. L'ultima assegnazione impone un indirizzo preciso: non si fa mai nei programmi normali, perché quella cella probabilmente non appartiene al programma.

L'operatore `*`, applicato a un puntatore, dà il **valore della variabile puntata**: `*p` è il valore della variabile di cui `p` contiene l'indirizzo (si dice che `p` viene *dereferenziato*). `*` e `&` sono uno l'inverso dell'altro: `i` e `*&i` sono equivalenti. Seguiamo le istruzioni delle slide (figura 11.3):

```
int a, b, *p;

a = 1;
b = 2;
p = &a;
b = *p;               /* equivale a b = a */
```

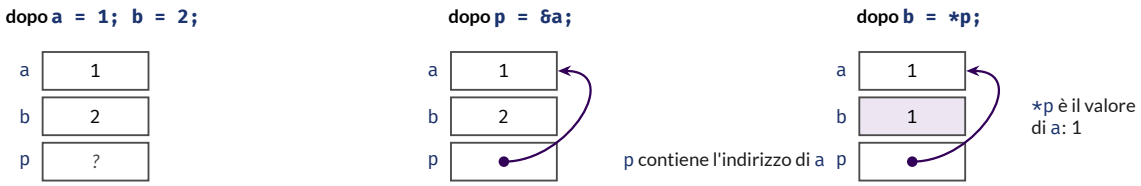


Figura 11.3: *Un puntatore si disegna come una freccia verso la variabile di cui contiene l'indirizzo.*

Che cosa contiene un puntatore

Un indirizzo si stampa con la conversione `%p`, in esadecimale:

indirizzo.c

```c
#include <stdio.h>

int main(void)
{
   int   i = 7, *p;

   p = &i;
   printf("%s%d\n%s%p\n",
      "   Value of i: ", *p,
      "Location of i: ",  p);
   return 0;
}
```

Terminale

```
$ ./indirizzo
   Value of i: 7
Location of i: 0x7ffda49df4ac
$ ./indirizzo
   Value of i: 7
Location of i: 0x7ffe989ec3bc
```

L'indirizzo cambia a ogni esecuzione: per sicurezza, il sistema operativo colloca la memoria del programma in posizioni diverse ogni volta. Nelle slide le due stringhe contengono anche un `%d` (`" Value of i: %d"`): così, però, `%d` non è una conversione (sta in un argomento, non nella stringa di controllo) e viene stampato tale e quale, `Value of i: %d7`. La versione qui sopra è quella corretta.

Espressioni con i puntatori

Con le dichiarazioni

```c
int i = 3, j = 5, *p = &i, *q = &j, *r;
double x;
```

ecco alcune espressioni, con l'espressione equivalente e il valore (verificato con il compilatore, calcolando ciascuna partendo da questi valori):

espressione	equivalente	valore
`p == &i`	`p == (&i)`	1
`**&p`	`*(*(&p))`	3
`*(r = &j) *= *p`	`(*(r = (&j))) *= *p`	15
`7 * *p / *q + 7`	`(((7 * (*p))) / (*q)) + 7`	11
`r = &x`	`r = &x`	illegale

- `**&p`: `&p` è l'indirizzo di `p`; `*` lo riporta a `p`, e un altro `*` dà il valore puntato da `p`, cioè `i`: 3.
- `*(r = &j) *= *p`: `r` riceve l'indirizzo di `j`, e `*r *= *p` moltiplica `j` per `i`: `j` diventa 15.
- `7 * *p / *q + 7`: $(7 \cdot 3)/5 + 7 = 21/5 + 7 = 4 + 7 = 11$ (divisione intera). Nelle slide è scritta `7**p / *q+7`: gli spazi non contano, ma rendono leggibile.
- `r = &x` è illegale: `r` punta a `int`, `&x` è l'indirizzo di un `double`. `gcc` lo segnala (e i compilatori più recenti lo considerano un errore):

Terminale

```
$ gcc -Wall -o rx rx.c
rx.c: In function ‘main’:
rx.c:9:5: warning: assignment to ‘int *’ from incompatible pointer type ‘double *’ [-Wincompatible-pointer-
    types]
    9 |    r = &x;
      |      ^
```

Che cosa non si può puntare

Non tutto ha un indirizzo. Non si può puntare a una **costante** (`&3`), a un'**espressione** (`&(k + 9)`) né a una variabile `register` (`&v`). Le prime due non sono celle di memoria con un nome (sono *lvalue* mancati); la terza, come abbiamo visto, sta in un registro:

Terminale

```
$ gcc -Wall -o nonpuntabili nonpuntabili.c
nonpuntabili.c: In function ‘main’:
nonpuntabili.c:6:8: error: lvalue required as unary ‘&’ operand
nonpuntabili.c:7:8: error: lvalue required as unary ‘&’ operand
nonpuntabili.c:8:4: error: address of register variable ‘v’ requested
```

Lo stesso vale per un cast: nella cartella degli esempi c'è un tentativo, `r = &((int) x);`, di far puntare un `int *` a un `double` "convertito". Il risultato di un cast è un valore, non una variabile, e il compilatore risponde ancora *lvalue required as unary '&' operand*.

Il passaggio per indirizzo

Modificare le variabili del chiamante

Una variabile passata come parametro a una funzione non cambia valore dopo la chiamata: è il passaggio **per valore** (capitolo 10). Altri linguaggi hanno anche il passaggio **per riferimento**, in cui la funzione può modificare la variabile passata. In C si ottiene un effetto simile con i puntatori: si passa l'**indirizzo** della variabile, e la funzione lavora sulla variabile attraverso `*`.

valore.c

```c
int f(int a, int b)
{
  a = a+1;
  b = b+1;
  return (a*b);
}

int main()
{
  int a = 1, b = 2, c;

  c = f(a,b);
  printf("a = %d\nb = %d\nc = %d \n", a, b, c);
}
```

Terminale

```
$ ./valore
a = 1
b = 2
c = 6
```

indirizzi.c

```c
int f(int *a, int *b)
{
  *a = *a + 1;
  *b = *b + 1;
  return (*a * *b);
}

int main()
{
  int a = 1, b = 2, c;

  c = f(&a, &b);
  printf("a = %d\nb = %d\nc = %d \n", a, b, c);
}
```

Terminale

```
$ ./indirizzi
a = 2
b = 3
c = 6
```

A sinistra `f` incrementa le proprie copie; a destra riceve gli indirizzi delle variabili di `main` e incrementa **quelle** (figura 11.4). Si noti che `a` e `b`, dentro `f`, ora sono puntatori: `*a` è l'intero puntato.

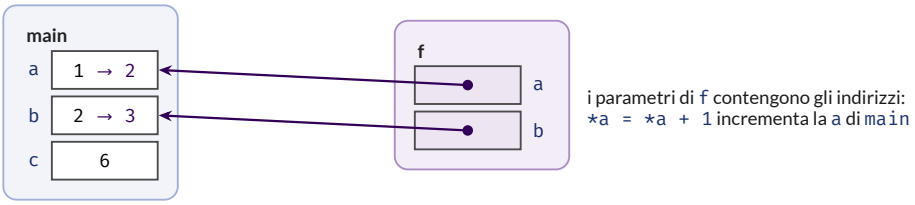


Figura 11.4: *Il passaggio per indirizzo:* `f` *arriva alle variabili di* `main` *attraverso i puntatori.*

Una svista nelle slide

Nella versione delle slide il corpo di `f` è rimasto quello di prima: `a = a+1; b = b+1; return (a*b);`. Ma ora `a` e `b` sono puntatori: `a = a+1` sposterebbe il puntatore, non incrementerebbe il valore, e moltiplicare due puntatori non ha senso. Il compilatore infatti si ferma: *invalid operands to binary * (have 'int *' and 'int *')*. Bisogna usare `*` ovunque si intenda il valore puntato, come nella versione qui sopra.

Lo scambio di due variabili

Lo scambio (*swap*) dei valori di due variabili si fa di solito con una variabile di appoggio: `tmp = a; a = b; b = tmp;`. Una funzione che scambi due variabili del chiamante deve riceverne gli indirizzi:

swap.c

```c
#include <stdio.h>

void   swap(int *, int *);

int main(void)
{
   int    i = 3, j = 5;

   swap(&i, &j);
   printf("%d  %d\n", i, j);       /* 5  3 is printed */
   return 0;
}

void swap(int *p, int *q)
{
   int   tmp;

   tmp = *p;
   *p = *q;
   *q = tmp;
}
```

Terminale

```
$ ./swap
5  3
```

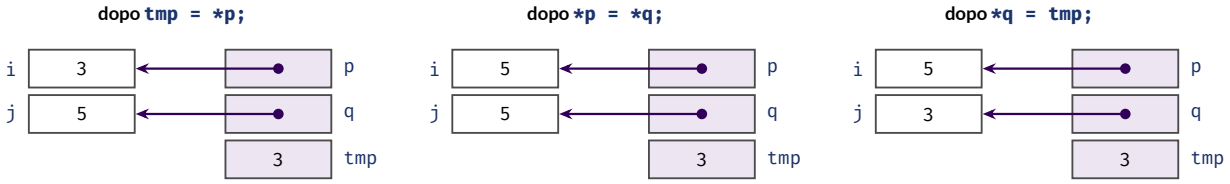


Figura 11.5: `swap` *passo per passo:* `p` *e* `q` *puntano a* `i` *e* `j` *di* `main`.

È anche il motivo per cui `scanf` riceve `&x`: per scrivere in `x`, deve sapere dove si trova (capitolo 5).

Il nome di un array è un indirizzo

Il nome di un array è un indirizzo, cioè il valore di un puntatore: quello del suo primo elemento. Da qui le relazioni fondamentali:

- se `a` è un array e `i` un intero, `a[i]` (l'elemento di indice `i`) è equivalente a `*(a + i)` (il contenuto dell'indirizzo `a + i`);
- se `p` è un puntatore, anche `p[i]` è equivalente a `*(p + i)`: i puntatori si possono indicizzare come gli array;
- con `int a[N], *p;`, l'istruzione `p = a;` equivale a `p = &a[0];`, e `p = a + 1;` equivale a `p = &a[1];`.

Il punto chiave è l'**aritmetica dei puntatori**: sommare 1 a un puntatore non lo sposta di un byte, ma di **un elemento**, cioè di tanti byte quanti ne occupa il tipo puntato (4 per un `int`).

aritmetica.c

```c
#include <stdio.h>

int main(void)
{
    int a[3] = {10, 20, 30};
    int *p = a;

    printf("a     = %p\n", (void *) a);
    printf("&a[0] = %p\n", (void *) &a[0]);
    printf("a + 1 = %p\n", (void *) (a + 1));
    printf("&a[1] = %p\n", (void *) &a[1]);
    printf("a + 2 = %p\n", (void *) (a + 2));
    printf("*(a + 2) = %d   p[2] = %d   *(p + 2) = %d\n",
           *(a + 2), p[2], *(p + 2));
    return 0;
}
```

Terminale

```
$ ./aritmetica
a     = 0x7ffc5944463c
&a[0] = 0x7ffc5944463c
a + 1 = 0x7ffc59444640
&a[1] = 0x7ffc59444640
a + 2 = 0x7ffc59444644
*(a + 2) = 30   p[2] = 30   *(p + 2) = 30
```

Da `a` ad `a + 1` l'indirizzo cresce di 4 (`...63c`, `...640`): un `int`.

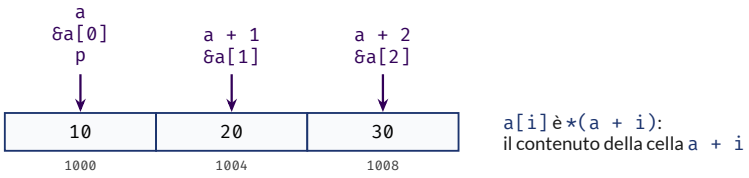


Figura 11.6: `a + i` *è l'indirizzo dell'elemento* `i` *(indirizzi di esempio): ogni passo vale 4 byte.*

Quattro modi di sommare un array

Grazie a queste equivalenze, la somma degli elementi di un array si può scrivere in molti modi, tutti equivalenti:

quattro.c

```c
#include <stdio.h>

#define N 5

int main(void)
{
   int a[N] = {1, 2, 3, 4, 5}, i, *p, sum;

   sum = 0;
   for (i = 0; i < N; i++)
      sum += a[i];
   printf("a[i]:      %d\n", sum);

   sum = 0;
   for (p = a, i = 0; i < N; i++)
      sum += p[i];
   printf("p[i]:      %d\n", sum);

   sum = 0;
   for (i = 0; i < N; i++)
      sum += *(a + i);
   printf("*(a + i):  %d\n", sum);

   sum = 0;
   for (p = a; p < &a[N]; p++)
      sum += *p;
   printf("*p, p++:   %d\n", sum);
   return 0;
}
```

Terminale

```
$ ./quattro
a[i]:       15
p[i]:       15
*(a + i):   15
*p, p++:    15
```

Nell'ultimo ciclo è il puntatore a scorrere l'array: parte da `a` e avanza di un elemento a ogni giro, finché arriva a `&a[N]`, l'indirizzo subito dopo l'ultimo elemento.

Un array non si può spostare

C'è però una differenza fra un array e un puntatore: un puntatore può assumere indirizzi diversi, mentre il nome di un array è un indirizzo **costante**, che non si può cambiare. Le istruzioni `a = p;`, `++a;` e `a += 2;` sono illegali:

Terminale

```
$ gcc -Wall -o costante costante.c
costante.c: In function ‘main’:
costante.c:5:6: error: assignment to expression with array type
costante.c:6:4: error: lvalue required as increment operand
costante.c:7:6: error: assignment to expression with array type
```

GLI ARRAY COME PARAMETRI

Si passa l'indirizzo

Nella definizione di una funzione, un parametro dichiarato come array è in realtà un **puntatore**. Quando si passa un array come argomento, viene passato per valore il suo **indirizzo base**: gli elementi **non vengono copiati**. Come esempio, una funzione che somma gli elementi di un array di `double`: le due intestazioni seguenti sono equivalenti.

```c
double sum_array(double a[], int n)
```

```c
double sum_array(double *a, int n)
```

Siccome la funzione riceve solo un indirizzo, non sa quanto è lungo l'array: la dimensione va passata come parametro (qui `n`). E siccome riceve un indirizzo qualsiasi, può lavorare anche su una **parte** dell'array. Con `double v[100];` e `v[i]` uguale a `i`:

sum_array.c

```c
#include <stdio.h>

double sum_array(double a[], int n)   /* n e' la dimensione di a[] */
{
   int    i;
   double sum = 0.0;

   for (i = 0; i < n; ++i)
      sum += a[i];
   return sum;
}

int main(void)
{
   double v[100];
   int    i, k = 10;

   for (i = 0; i < 100; i++)
      v[i] = i;                        /* v[i] vale i */
   printf("%.0f\n", sum_array(v, 100));        /* v[0] + ... + v[99]   */
   printf("%.0f\n", sum_array(v, 88));         /* v[0] + ... + v[87]   */
   printf("%.0f\n", sum_array(&v[7], k - 7));  /* v[7] + ... + v[k-1]  */
   printf("%.0f\n", sum_array(v + 7, 2 * k));  /* v[7] + ... + v[2k+6] */
   return 0;
}
```

Terminale

```
$ ./sum_array
4950
3828
24
330
```

Con k uguale a 10: $7 + 8 + 9 = 24$, e la somma da 7 a 26 vale 330.

Nella chiamata `sum_array(v + 7, 2 * k)` il parametro `a` punta a `v[7]`: per la funzione `a[0]` è `v[7]`, `a[1]` è `v[8]`, e così via (figura 11.7).

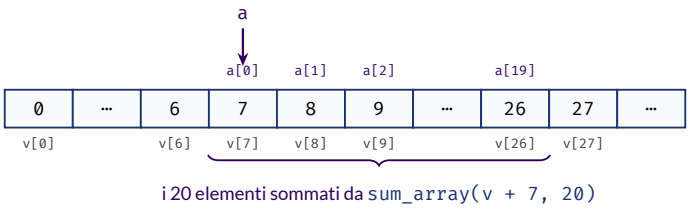


Figura 11.7: *La funzione vede l'array a partire dall'indirizzo che riceve.*

`sizeof` su un parametro

Una conseguenza che sorprende: dentro la funzione, `sizeof` applicato al parametro dà la dimensione di un **puntatore**, non dell'array.

dimensione.c

```c
#include <stdio.h>

void stampa_dimensione(int a[])
{
    printf("nella funzione: sizeof(a) = %zu\n", sizeof(a));
}

int main(void)
{
    int v[10];

    printf("in main:        sizeof(v) = %zu\n", sizeof(v));
    stampa_dimensione(v);
    return 0;
}
```

Terminale

```
$ ./dimensione
in main:        sizeof(v) = 40
nella funzione: sizeof(a) = 8
```

40 byte sono i dieci `int`; 8 byte sono un indirizzo. `gcc` avvisa: *'sizeof' on array function parameter 'a' will return size of 'int *'*.

Ordinare un array

Un'applicazione classica degli array (e dei puntatori, per gli scambi) è l'**ordinamento**: disporre gli elementi in ordine crescente. Le slide presentano due algoritmi semplici. Entrambi usano la `swap` vista sopra, a cui passano gli indirizzi di due elementi, `a + j - 1` e `a + j`, cioè `&a[j-1]` e `&a[j]`.

Il bubble sort

bubblesort.c

```c
void bubble(int *a, int n)      /* n is the size of a */
{
    int    i, j;

    for (i = 0; i < n - 1; ++i)
        for (j = n - 1; j > i; --j)
            if (a[j-1] > a[j])
                swap(a+j-1, a+j);
}
```

Terminale

```
$ ./bubblesort
7 3 66 3 -5 22 -77 2
-77 -5 2 3 3 7 22 66
```

`main` stampa l'array prima e dopo la chiamata `bubble(v, 8)`.

A ogni passo (un giro del ciclo esterno), il ciclo interno scorre l'array **dal fondo verso l'inizio** confrontando ogni coppia di elementi vicini, e li scambia se sono in ordine sbagliato. Così il più piccolo degli elementi rimasti "risale" come una bolla fino alla posizione `i`: dopo il passo k, i primi k elementi sono definitivamente al loro posto (figura 11.8).

Dati	7	3	66	3	-5	22	-77	2
Passo 1	-77	7	3	66	3	-5	22	2
Passo 2	-77	-5	7	3	66	3	2	22
Passo 3	-77	-5	2	7	3	66	3	22
Passo 4	-77	-5	2	3	7	3	66	22
Passo 5	-77	-5	2	3	3	7	22	66
Passo 6	-77	-5	2	3	3	7	22	66
Passo 7	-77	-5	2	3	3	7	22	66

Figura 11.8: *Il bubble sort passo per passo (verificato stampando l'array dopo ogni giro del ciclo esterno): in verde la parte già ordinata.*

Gli ultimi passi non cambiano più nulla: l'array è già ordinato dal passo 5, ma l'algoritmo non se ne accorge e fa comunque tutti gli $n - 1$ passi.

selectionsort.c

```c
void selection_sort(int *a, int n)     /* n is the size of a */
{
   int    i, j;

   for (i = 0; i < n - 1; ++i)
      for (j = i + 1; j < n ; j++)
         if (a[j] < a[i])
            swap(a+j, a+i);
}
```

Terminale

```
$ ./selectionsort
7 3 66 3 -5 22 -77 2
-77 -5 2 3 3 7 22 66
```

Qui al passo `i` si confronta l'elemento in posizione `i` con tutti quelli che seguono, scambiandolo ogni volta che se ne trova uno più piccolo: alla fine del passo, in posizione `i` c'è il più piccolo degli elementi rimasti, cioè quello che va lì (figura 11.9).

Dati	7	3	66	3	-5	22	-77	2
Passo 1	-77	7	66	3	3	22	-5	2
Passo 2	-77	-5	66	7	3	22	3	2
Passo 3	-77	-5	2	66	7	22	3	3
Passo 4	-77	-5	2	3	66	22	7	3
Passo 5	-77	-5	2	3	3	66	22	7
Passo 6	-77	-5	2	3	3	7	66	22
Passo 7	-77	-5	2	3	3	7	22	66

Figura 11.9: *Il selection sort passo per passo: a ogni passo il minimo dei rimanenti va nella sua posizione definitiva.*

L'ALLOCAZIONE DINAMICA

`malloc`, `calloc` e `free`

Finora la dimensione di un array era una costante, decisa scrivendo il programma. Spesso però la si conosce solo durante l'esecuzione (quanti numeri inserirà l'utente?). Per questo il C fornisce le funzioni `malloc` (*memory allocation*) e `calloc` (*contiguous allocation*), con i prototipi in `stdlib.h`: creano spazio **dinamicamente**, mentre il programma è in esecuzione, per array, strutture e unioni.

- `calloc(n, el_size)` riserva spazio contiguo per un array di `n` elementi da `el_size` byte ciascuno (la dimensione si ottiene con `sizeof`), e lo inizializza con tutti i bit a zero. I due parametri sono di tipo `size_t`.
- `malloc(size)` riserva `size` byte e non li inizializza: può essere un po' più veloce.
- Se riescono, entrambe restituiscono un puntatore di tipo `void *` (un indirizzo "generico") alla base della memoria; altrimenti restituiscono `NULL`.

```c
a = (int *) calloc(n, sizeof(int));
a = (int *) malloc(n * sizeof(int));
```

Il cast `(int *)` si può anche omettere, perché in C un `void *` si converte da solo in qualsiasi altro puntatore. Lo spazio allocato dinamicamente **non viene liberato** automaticamente all'uscita dalle funzioni: va restituito esplicitamente con `free(ptr)`. Se `ptr` è `NULL`, `free` non fa nulla; altrimenti `ptr` deve essere esattamente l'indirizzo base restituito da `malloc` o `calloc`, o il comportamento è indefinito (e l'effetto dipende dal sistema).

Un esempio

Il programma seguente chiede ripetutamente un numero `n`, alloca un array di `n` interi, lo riempie di numeri casuali fra -9 e 9, lo stampa con la somma e lo libera. `rand()` (in `stdlib.h`) restituisce un intero pseudo-casuale, e `srand(time(NULL))` sceglie il seme in base all'ora, così che i numeri cambino a ogni esecuzione.

dinamico.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void   fill_array(int *a, int n);
int    sum_array(int *a, int n);
void   wrt_array(int *a, int n);

int main(void)
{
   int   *a, n;

   srand(time(NULL));     /* seed the random number generator */
   printf("Input n: ");
   while (scanf("%d", &n) == 1 && n >= 1) {
      putchar('\n');
      a = calloc(n, sizeof(int));     /* allocate space for a[] */
      fill_array(a, n);
      wrt_array(a, n);
      printf("sum = %d\n\n", sum_array(a, n));
      free(a);
      printf("Input n: ");
   }
   printf("\nBye!\n\n");
   return 0;
}

void fill_array(int *a, int n)
{
   int   i;

   for (i = 0; i < n; ++i)
      a[i] = rand() % 19 - 9;
}

int sum_array(int *a, int n)
{
   int   i, sum = 0;

   for (i = 0; i < n; ++i)
      sum += a[i];
   return sum;
}

void wrt_array(int *a, int n)
{
   int   i;

   printf("a = [");
   for (i = 0; i < n; ++i)
      printf("%d%s", a[i], ((i < n - 1) ? ", " : "]\n"));
}
```

Il programma in esecuzione

Ecco un'esecuzione (i numeri, naturalmente, cambiano ogni volta):

Terminale

```
$ ./dinamico
Input n: 5

a = [8, -1, -5, -9, -3]
sum = -10

Input n: 3

a = [-4, -1, 7]
sum = 2

Input n: 0

Bye!
```

- `rand() % 19` vale da 0 a 18; togliendo 9 si va da −9 a 9.
- In `wrt_array`, `(i < n - 1) ? ", " : "]\n"` è l'**operatore condizionale**: vale `", "` se la condizione è vera, `"]\n"` altrimenti. Così dopo l'ultimo elemento si chiude la parentesi invece di mettere la virgola.
- Il ciclo continua finché `scanf` legge un numero **e** il numero è almeno 1.

Una svista nelle slide

Nelle slide la condizione è `scanf("%d", &n) == 1 || n >= 1`, con l'*or*. Così, se si inserisce 0, la prima parte è vera e il ciclo prosegue con un array vuoto; e se l'input finisce (Ctrl + D) dopo un numero valido, la prima parte è falsa ma `n` è rimasto quello di prima, la seconda è vera, e il ciclo non termina più (in una prova ha stampato oltre 10 milioni di righe in un secondo). La condizione giusta, quella del libro da cui viene l'esempio, è con `&&`.

Lo stack e lo heap

Dove finisce la memoria allocata dinamicamente? La memoria di un programma in esecuzione è divisa in zone (figura 11.10):

- lo **stack** (la pila) è lo spazio di lavoro delle funzioni: a ogni chiamata viene riservato un blocco in cima allo stack per le variabili locali e alcuni dati, e viene tolto al ritorno (figura 10.3);
- lo **heap** è la memoria riservata all'allocazione dinamica: `malloc` e `calloc` prendono spazio da lì, e `free` lo restituisce;
- le variabili globali e `static` hanno una loro zona, che esiste per tutta l'esecuzione, e il codice del programma un'altra ancora.

Lo stack è tipicamente molto più piccolo dello heap (su Linux, di solito, 8 MB): un array enorme dichiarato come variabile locale può esaurirlo (*stack overflow*), mentre lo stesso array allocato con `malloc` non dà problemi.

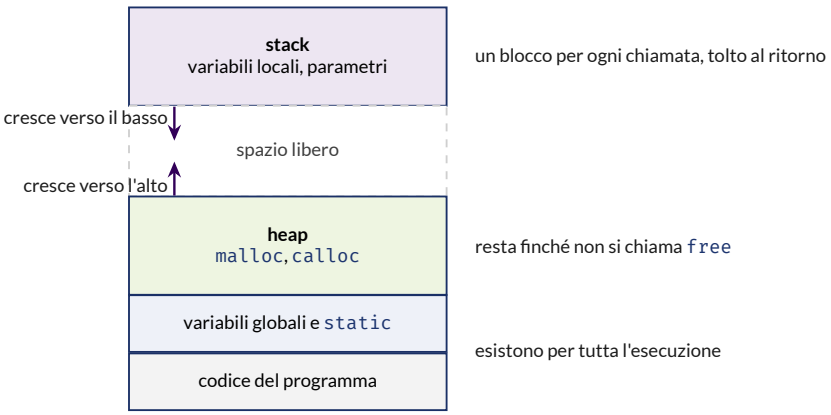


Figura 11.10: *Le zone della memoria di un programma (lo schema è semplificato).*

In sintesi

Un **array** raccoglie elementi dello stesso tipo in celle consecutive; gli indici vanno da 0 a $N-1$, e il C non controlla gli accessi fuori dai limiti (`-fsanitize=address` aiuta a trovarli). Gli array automatici non inizializzati contengono valori casuali. Un **puntatore** contiene un indirizzo: `&v` è l'indirizzo di `v`, `*p` il valore puntato, `NULL` il puntatore che non punta a niente. Passando gli indirizzi una funzione può modificare le variabili del chiamante (`swap`). Il nome di un array è l'indirizzo costante del primo elemento: `a[i]` equivale a `*(a + i)`, e `p + 1` avanza di un elemento. Un array passato a una funzione è passato come indirizzo (la dimensione va passata a parte). Bubble sort e selection sort ordinano con confronti e scambi. `malloc` e `calloc` allocano memoria nello heap durante l'esecuzione, e `free` la restituisce.

RIFERIMENTI



- **A. Kelley, I. Pohl**, *A Book on C*: capitolo 6 (array, puntatori e stringhe).
- **AddressSanitizer** — documentazione di GCC (`-fsanitize=address`).
  `https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html`
- **Dynamic memory management** — cppreference.com.
  `https://en.cppreference.com/w/c/memory`

Stringhe e array a più dimensioni

Questo capitolo completa la lezione sugli array e i puntatori con due applicazioni importantissime. Le **stringhe**, cioè i testi, in C sono array di caratteri: vedremo come sono fatte, le funzioni della libreria standard che le trattano e come si ordinano le parole di un testo con un array di puntatori. Gli **array a più dimensioni**, come le matrici, sono array di array: vedremo come stanno in memoria, come si passano alle funzioni e come si allocano dinamicamente.

Le stringhe

Un array di caratteri con un terminatore

Le stringhe sono array a una dimensione di caratteri. In C una stringa è **terminata dal carattere nullo** `'\0'`, un byte con tutti i bit a zero, il cui valore decimale è 0: è così che le funzioni riconoscono dove finisce il testo. Per questo la dimensione di una stringa deve includere lo spazio per il terminatore: la stringa `"a"` ha **due** elementi, `'a'` e `'\0'`. La dichiarazione

```c
char s[] = "abc";
```

è trattata dal compilatore come `char s[] = {'a', 'b', 'c', '\0'};`: un array di 4 caratteri (figura 12.1). Il programmatore deve stare attento a non superare la dimensione dell'array, perché il C non lo controlla.

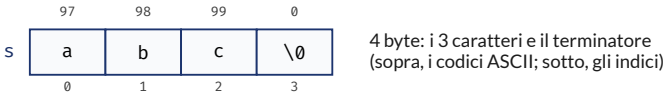


Figura 12.1: *La stringa* `"abc"` *in memoria.*

Le stringhe costanti sono puntatori

Una stringa costante, come `"abc"`, è trattata dal compilatore come un **puntatore** al suo primo carattere. Così si può scrivere

```
char *p = "abc";

printf("%s %s\n", p, p + 1);
```

che stampa `abc bc`: la conversione `%s` riceve un indirizzo e stampa i caratteri da lì fino al `'\0'`, e `p + 1` punta alla `b` (figura 12.2). Per lo stesso motivo `"abc"[1]` vale `'b'` e `*("abc" + 2)` vale `'c'`: espressioni corrette, che però non vanno mai usate nel codice vero.

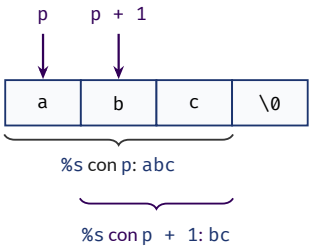


Figura 12.2: `%s` *stampa dall'indirizzo ricevuto fino al terminatore.*

Array o puntatore?

`char s[] = "abc";` e `char *p = "abc";` sembrano uguali, ma non lo sono. Il primo crea un array di 4 caratteri, una copia della stringa che il programma può modificare. Il secondo crea solo un puntatore alla stringa costante, che sta in una zona di memoria **di sola lettura**: tentare di modificarla è un comportamento indefinito, e su Linux il programma viene fermato.

modifica.c

```c
#include <stdio.h>

int main(void)
{
    char s[] = "abc";
    char *p = "abc";

    s[0] = 'x';
    printf("%s\n", s);
    p[0] = 'x';
    printf("%s\n", p);
    return 0;
}
```

Terminale

```
$ ./modifica
xbc
Segmentation fault
```

Segmentation fault (a volte seguito da *core dumped*) è il messaggio della shell quando il sistema ferma un programma che ha toccato memoria non sua.

La stringa vuota

Il carattere nullo non va scritto in output: è un segnale per i programmi, non un carattere da mostrare, e scriverlo in un file potrebbe confondere i programmi che lo elaborano. Scrivere stringhe **vuote** (`""`, che contiene solo il terminatore), invece, può essere utile. Per esempio, per mettere il plurale solo quando serve:

```
for (nfrogs = 1; nfrogs <= 3; nfrogs++)
    printf("%d frog%s\n", nfrogs, (nfrogs == 1) ? "" : "s");
```

Terminale

```
1 frog
2 frogs
3 frogs
```

Contare le parole

La funzione seguente conta le parole di una stringa, intese come sequenze di caratteri separate da spazi. Il parametro `s` è un puntatore che **scorre** la stringa: `++s` lo sposta sul carattere successivo, e `*s` è il carattere su cui si trova. `isspace` (in `ctype.h`) è vera per gli spazi, le tabulazioni e gli a capo.

word_cnt.c

```c
/* Count the number of words in a string. */

#include <stdio.h>
#include <ctype.h>

int word_cnt(const char *s)
{
   int    cnt = 0;

   while (*s != '\0') {
      while (isspace(*s))         /* skip white space */
         ++s;
      if (*s != '\0') {           /* found a word */
         ++cnt;
         while (!isspace(*s) && *s != '\0')
            ++s;                  /* skip the word */
      }
   }
   return cnt;
}

int main(void)
{
   printf("%d\n", word_cnt("  Il linguaggio   C  "));
   printf("%d\n", word_cnt(""));
   printf("%d\n", word_cnt("una"));
   return 0;
}
```

Terminale

```
$ ./word_cnt
3
0
1
```

A ogni giro del ciclo esterno si saltano gli spazi; se non si è arrivati alla fine, si è trovata una parola: si incrementa il contatore e si salta la parola fino al prossimo spazio (figura 12.3).

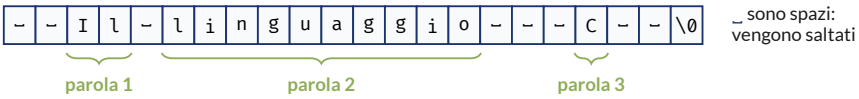


Figura 12.3: `word_cnt` *alterna due fasi: salta gli spazi, poi salta una parola contandola.*

LA PAROLA CHIAVE `const`

Variabili che non cambiano

La parola chiave `const`, introdotta dall'ANSI C, indica che una variabile può essere inizializzata ma non modificata (capitolo 5). Con `const int k = 3;` l'istruzione `k = 1;` è un errore: *assignment of read-only variable 'k'*.

Con i puntatori, `const` può riguardare il dato puntato o il puntatore stesso:

- `const int *p;` dichiara un puntatore a un `const int`: il **dato puntato** non si può modificare attraverso `p` (`*p = 1;` è un errore), ma il puntatore sì (`p = &j;` va bene);
- `int *const p = &i;` è il contrario: un puntatore **costante**, che punterà sempre a `i`, attraverso cui però `i` si può modificare.

Una promessa verificata dal compilatore

Il parametro di `word_cnt` è `const char *s`: la funzione **promette** di non modificare la stringa che riceve (sposta `s`, ma non tocca i caratteri). È un'informazione preziosa per chi la usa, e il compilatore la fa rispettare. Se nella funzione si prova a scrivere nella stringa:

word_cnt2.c

```c
/* Count the number of words in a string. */

#include <ctype.h>

int word_cnt(const char *s)
{
   int    cnt = 0;

   while (*s != '\0') {
      while (isspace(*s))         /* skip white space */
         ++s;
      if (*s != '\0') {           /* found a word */
         ++cnt;
         while (!isspace(*s) && *s != '\0') {
            *s = 'a';             /* the compiler will complain */
            ++s;                  /* skip the word */
         }
      }
   }
   return cnt;
}
```

Terminale

```
$ gcc -Wall -c word_cnt2.c
word_cnt2.c: In function 'word_cnt':
word_cnt2.c:15:16: error: assignment of read-only location '*s'
   15 |             *s = 'a';             /* the compiler will complain */
      |                ^
```

FUNZIONI PER CARATTERI E STRINGHE

I caratteri: `ctype.h`

La libreria standard ha molte funzioni per classificare e trasformare i caratteri, con i prototipi in `ctype.h`. Ricevono un `int` (come `getchar`, capitolo 8) e le funzioni di verifica restituiscono un valore diverso da zero (vero) o zero (falso).

funzione	che cosa fa
`int isblank(int c)`	vero se `c` è uno spazio o una tabulazione (non c'era nei vecchi compilatori Microsoft)
`int isdigit(int c)`	vero se `c` è una cifra
`int isalpha(int c)`	vero se `c` è una lettera
`int isalnum(int c)`	vero se `c` è una lettera o una cifra
`int islower(int c)`	vero se `c` è una lettera minuscola
`int isupper(int c)`	vero se `c` è una lettera maiuscola
`int tolower(int c)`	se `c` è maiuscola restituisce la minuscola, altrimenti `c`
`int toupper(int c)`	se `c` è minuscola restituisce la maiuscola, altrimenti `c`

ctipi.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void)
{
   const char *s = "aZ 7!";
   int i;

   printf("car  isalpha isdigit isalnum islower isupper isblank toupper\n");
   for (i = 0; s[i] != '\0'; i++)
      printf(" '%c' %5d %7d %7d %7d %7d %7d %7c\n", s[i],
             isalpha(s[i]) != 0, isdigit(s[i]) != 0, isalnum(s[i]) != 0,
             islower(s[i]) != 0, isupper(s[i]) != 0, isblank(s[i]) != 0,
             toupper(s[i]));
   return 0;
}
```

Terminale

```
$ ./ctipi
car  isalpha isdigit isalnum islower isupper isblank toupper
 'a'     1       0       1       1       0       0       A
 'Z'     1       0       1       0       1       0       Z
 ' '     0       0       0       0       0       1
 '7'     0       1       1       0       0       0       7
 '!'     0       0       0       0       0       0       !
```

Nel programma, `!= 0` trasforma qualunque "vero" in 1, per avere una tabella leggibile.

Le stringhe: `string.h`

Le funzioni per le stringhe hanno i prototipi in `string.h`. Il parametro `const char *` indica una stringa che la funzione non modifica; il `char *` senza `const` è l'array in cui la funzione scrive, che **deve avere spazio sufficiente**: nessuna di queste funzioni lo controlla.

prototipo	che cosa fa
`size_t strlen(const char *s)`	la lunghezza di `s`, senza contare il terminatore
`char *strcpy(char *s1, const char *s2)`	copia `s2` nell'array `s1`; restituisce `s1`
`char *strncpy(char *s1, const char *s2, size_t n)`	copia al massimo `n` caratteri di `s2` in `s1`
`char *strcat(char *s1, const char *s2)`	aggiunge `s2` in coda a `s1`; restituisce `s1`
`char *strncat(char *s1, const char *s2, size_t n)`	aggiunge in coda al massimo `n` caratteri di `s2`
`int strcmp(const char *s1, const char *s2)`	confronta: restituisce 0 se sono uguali, un negativo se `s1` viene prima, un positivo se viene dopo
`int strncmp(const char *s1, const char *s2, size_t n)`	come `strcmp`, ma solo sui primi `n` caratteri
`char *strchr(const char *s, int c)`	un puntatore alla prima `c` in `s`, o `NULL`
`char *strrchr(const char *s, int c)`	un puntatore all'ultima `c` in `s`, o `NULL`
`char *strstr(const char *s1, const char *s2)`	un puntatore alla prima occorrenza di `s2` in `s1`, o `NULL`
`char *strpbrk(const char *s1, const char *s2)`	un puntatore al primo carattere di `s1` che compare in `s2`, o `NULL`
`size_t strspn(const char *s1, const char *s2)`	la lunghezza del tratto iniziale di `s1` fatto **solo** di caratteri di `s2`
`size_t strcspn(const char *s1, const char *s2)`	la lunghezza del tratto iniziale di `s1` fatto di caratteri **non** in `s2`

`strcmp` confronta i codici dei caratteri uno alla volta, finché trova una differenza: per questo l'ordine è quello alfabetico per le minuscole (o per le maiuscole), ma tutte le maiuscole vengono prima delle minuscole (capitolo 8). Ecco gli esempi delle slide per le ultime due funzioni:

spn.c

```c
printf("%zu\n", strcspn("abc", "abc"));      /* 0 */
printf("%zu\n", strcspn("abc", "abcde"));    /* 0 */
printf("%zu\n", strcspn("djpc", "lqrabc"));  /* 3 */
printf("%zu\n", strspn("abc", "abc"));       /* 3 */
printf("%zu\n", strspn("abc", "abcde"));     /* 3 */
printf("%zu\n", strspn("djpc", "lqrabc"));   /* 0 */
```

- `strcspn("djpc", "lqrabc")` vale 3: `d`, `j` e `p` non sono in `"lqrabc"`, la `c` sì.
- `strspn("abc", "abcde")` vale 3: tutti i caratteri di `"abc"` sono in `"abcde"`.
- I valori sono di tipo `size_t`, che si stampa con `%zu` (nelle slide c'è `%d`, e `gcc` avvisa).

Come sono fatte

Non c'è niente di speciale in queste funzioni: si possono scrivere in C, in poche righe. Ecco le versioni delle slide (con il prefisso `my_`: i nomi della libreria standard vanno trattati come riservati, capitolo 6):

mystr.c

```c
size_t my_strlen(const char *s)
{
    size_t n;

    for (n = 0; *s != '\0'; ++s)
        ++n;
    return n;
}

char *my_strcpy(char *s1, const char *s2)
{
    char *p = s1;

    while ((*p++ = *s2++))
        ;
    return s1;
}

char *my_strcat(char *s1, const char *s2)
{
    char *p = s1;

    while (*p)
        ++p;
    while ((*p++ = *s2++))
        ;
    return s1;
}
```

```c
int main(void)
{
    char *s1;

    s1 = malloc(10 * sizeof(char));
    my_strcpy(s1, "abc");
    printf("%s %zu\n", s1, my_strlen(s1));
    my_strcat(s1, "efg");
    printf("%s %zu\n", s1, my_strlen(s1));
    free(s1);
    return 0;
}
```

Terminale

```
$ ./mystr
abc 3
abcefg 6
```

Il cuore di `my_strcpy` è il ciclo `while ((*p++ = *s2++))`, che fa tutto nella condizione (figura 12.4): copia il carattere puntato da `s2` nella cella puntata da `p`, sposta entrambi i puntatori in avanti (`++` postfisso: si usa il valore e poi si incrementa), e il valore dell'assegnazione, cioè il carattere copiato, fa da condizione. Quando si copia il `'\0'`, che vale 0, il ciclo si ferma: il terminatore è stato copiato anche lui. Le doppie parentesi dicono al compilatore (e al lettore) che l'assegnazione nella condizione è voluta: le slide hanno le parentesi singole, e `gcc -Wall` suggerirebbe di aggiungerle. `my_strcat` prima porta `p` sul terminatore di `s1`, poi copia da lì.

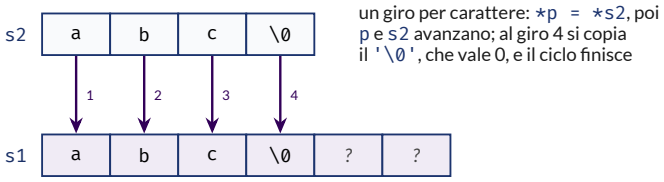


Figura 12.4: `my_strcpy(s1, s2)`*: quattro giri, l'ultimo copia il terminatore.*

Lo spazio per la copia

Nelle slide, il programma di prova di `strcpy` riserva per `s1` `malloc(3*sizeof(char))`, cioè 3 byte, e poi ci copia `"abc"`, che ne occupa 4 con il terminatore. È proprio l'errore da cui mettono in guardia le funzioni di `string.h`: si scrive un byte oltre la memoria riservata. Il programma sembra funzionare, ma `gcc` se ne accorge già in compilazione, e `-fsanitize=address` lo conferma in esecuzione:

Terminale

```
$ gcc -Wall -o strcpy3 strcpy3.c
strcpy3.c: In function ‘main’:
strcpy3.c:11:8: warning: ‘strcpy’ writing 4 bytes into a region of size 3 overflows the destination [-Wstringop
    -overflow=]
   11 |   s1 = strcpy(s1, s2);
      |        ^~~~~~~~~~~~~~
$ ./strcpy3
abc abc
$ gcc -g -fsanitize=address -o strcpy3 strcpy3.c
$ ./strcpy3
=================================================================
==3088==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x502000000013 at pc 0x7f5d792a7923 bp 0
    x7ffec73255a0 sp 0x7ffec7324d48
WRITE of size 4 at 0x502000000013 thread T0
…
```

La regola: per copiare una stringa `s2` servono `strlen(s2) + 1` byte, cioè `malloc(strlen(s2) + 1)`.

GLI ARRAY DI PUNTATORI

Un array di stringhe

La dichiarazione `int *a[20];` definisce `a` come un array di 20 elementi, ognuno dei quali è un **puntatore a intero**: può essere pensato come un array di vettori, ciascuno con la sua lunghezza. Il caso più comune è l'array di stringhe, `char *w[N];`: ogni `w[i]` punta a una stringa, e le stringhe possono essere lunghe quanto serve (figura 12.5).

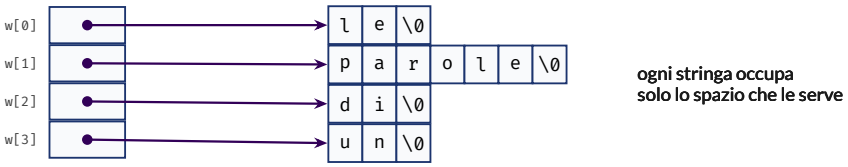


Figura 12.5: *Un array di puntatori a stringhe di lunghezza diversa.*

Ordinare le parole di un testo

L'esempio delle slide è un programma, diviso in più file (capitolo 7), che legge le parole dallo standard input e le stampa in ordine lessicografico (alfabetico, secondo i codici ASCII).

sort.h

```h
#define   MAXWORD   50        /* max word size */
#define   N         300       /* array size of w[] */

void   error_exit_calloc_failed(void);
void   error_exit_too_many_words(void);
void   error_exit_word_too_long(void);
void   sort_words(char *w[], int n);
void   swap(char **p, char **q);
void   wrt_words(char *w[], int n);
```

sort_words.c

```c
#include <string.h>
#include "sort.h"

void sort_words(char *w[], int n) /* n elements are to be sorted
     */
{
   int   i, j;

   for (i = 0; i < n; ++i)
      for (j = i + 1; j < n; ++j)
         if (strcmp(w[i], w[j]) > 0)
            swap(&w[i], &w[j]);
}
```

swap.c

```c
#include "sort.h"

void swap(char **p, char **q)
{
   char   *tmp;

   tmp = *p;
   *p = *q;
   *q = tmp;
}
```

main.c

```c
/* Sort words lexicographically. */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "sort.h"

int main(void)
{
   char   word[MAXWORD];      /* work space */
   char   *w[N];              /* an array of pointers
        */
   int    n;                  /* number of words to be
         sorted */
   int    i;

   for (i = 0; scanf("%s", word) == 1; ++i) {
      if (i >= N)
         error_exit_too_many_words();
      if (strlen(word) >= MAXWORD)
         error_exit_word_too_long();
      w[i] = calloc(strlen(word) + 1, sizeof(char));
      if (w[i] == NULL)
         error_exit_calloc_failed();
      strcpy(w[i], word);
   }
   n = i;
   sort_words(w, n);          /* sort the words */
   wrt_words(w, n);           /* write sorted list of
        words */
   return 0;
}
```

wrt.c

```c
#include <stdio.h>
#include "sort.h"

void wrt_words(char *w[], int n)
{
   int   i;

   for (i = 0; i < n; ++i)
      printf("%s\n", w[i]);
}
```

Come funziona

- `scanf("%s", word)` legge una parola (una sequenza di caratteri senza spazi) nell'array `word`, aggiungendo il terminatore. Il ciclo continua finché `scanf` riesce a leggere.
- Per ogni parola si controlla che non ce ne siano troppe e che non sia troppo lunga; poi `calloc` riserva esattamente `strlen(word) + 1` caratteri, e `strcpy` ci copia la parola. `w[i]` punta alla copia: `word` verrà riusato per la parola successiva.
- `sort_words` è un ordinamento per selezione con `strcmp`: se `w[i]` viene dopo `w[j]`, si scambiano.
- `swap` riceve gli **indirizzi** di due elementi di `w`, cioè dei `char **`, e scambia i due **puntatori**: le stringhe non si spostano, cambia solo l'ordine delle frecce (figura 12.6).
- Il file `error.c` contiene le tre funzioni di errore, che stampano un messaggio e terminano il programma con `exit(1)`.

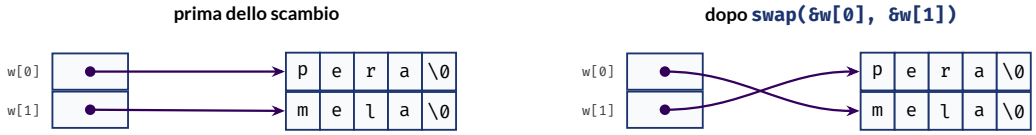


Figura 12.6: `swap` *scambia i puntatori, non i caratteri: ora* `w[0]` *punta a* `"mela"`.

Il programma in esecuzione

Si compilano insieme i cinque file sorgente e si prova con una frase:

Terminale

```
$ gcc -Wall -o sort_words main.c error.c wrt.c swap.c sort_words.c
$ echo "le parole di un Testo da Ordinare" | ./sort_words
Ordinare
Testo
da
di
le
parole
un
```

`echo` scrive la frase sullo standard output, e il simbolo `|` (*pipe*) la collega allo standard input di `sort_words`: è un altro modo di fornire l'input, oltre alla tastiera e alla redirezione `<`. Le parole con la maiuscola vengono prima di tutte le altre, perché in ASCII le maiuscole precedono le minuscole.

Nella cartella degli esempi c'è anche un file `input.txt` con una frase di 50 parole (una citazione sugli array a lunghezza variabile, che vedremo alla fine del capitolo); `./sort_words < input.txt` le stampa ordinate, da `According` e `Arrays` fino a `world`.

GLI ARRAY A PIÙ DIMENSIONI

Array di array

In C si possono definire array di array:

```c
int a[100];          /* array a una dimensione */
int b[2][7];         /* array a due dimensioni */
int c[5][3][2];      /* array a tre dimensioni */
```

Tutti gli elementi di un array sono memorizzati in memoria **contigua**, a partire dal primo: `a[0]`, `b[0][0]`, `c[0][0][0]`. Un array a due dimensioni si può pensare come una **matrice**: con `int a[3][5];`, 3 è il numero di righe e 5 il numero di colonne. In memoria, però, la matrice è "srotolata": prima tutta la riga 0, poi la riga 1, e così via (figura 12.7).

matrice.c

```c
#include <stdio.h>

int main(void)
{
   int a[3][5], i, j;

   for (i = 0; i < 3; i++)
      for (j = 0; j < 5; j++)
         a[i][j] = 10 * i + j;

   for (i = 0; i < 3; i++) {
      for (j = 0; j < 5; j++)
         printf("%4d", a[i][j]);
      printf("\n");
   }
   printf("&a[0][0] = %p\n", (void *) &a[0][0]);
   printf("&a[0][4] = %p\n", (void *) &a[0][4]);
   printf("&a[1][0] = %p\n", (void *) &a[1][0]);
   i = 2; j = 3;
   printf("%d %d %d %d %d\n", a[i][j], *(a[i] + j), (*(a + i))[
       j],
          *((*(a + i)) + j), *(&a[0][0] + 5 * i + j));
   return 0;
}
```

Terminale

```
$ ./matrice
   0   1   2   3   4
  10  11  12  13  14
  20  21  22  23  24
&a[0][0] = 0x7ffff4d0aaa0
&a[0][4] = 0x7ffff4d0aab0
&a[1][0] = 0x7ffff4d0aab4
23 23 23 23 23
```

`a[1][0]` sta 4 byte dopo `a[0][4]`: subito dopo l'ultimo elemento della riga 0.

come la pensiamo

riga 0	0	1	2	3	4
riga 1	10	11	12	13	14
riga 2	20	21	22	23	24

come sta in memoria

0	1	2	3	4	10	11	12	13	14	20	21	22	23	24

`a[i][j]` sta a $5 \cdot i + j$ posti da `a[0][0]`

Figura 12.7: *La matrice* `int a[3][5]`*: righe una dopo l'altra in memoria.*

La storage mapping function

La funzione che, per ogni array, associa agli indici la posizione dell'elemento si chiama **storage mapping function**. Per un array a una dimensione è `a[i]` → `*(&a[0] + i)`. Per un array a due dimensioni, con N colonne,

$$\texttt{a[i][j]} \rightarrow \texttt{*(\&a[0][0] +} \; N \cdot i + j \texttt{)}$$

perché per arrivare alla riga i bisogna saltare i righe intere da N elementi. Il numero di righe (la prima dimensione) **non viene usato**. Per lo stesso motivo `a[i][j]` si può scrivere anche `*(a[i] + j)`, `(*(a + i))[j]` o `*((*(a + i)) + j)`: l'ultima riga di `matrice.c` li stampa tutti, e valgono tutti 23.

Lo stesso vale con più dimensioni. Con `int A[7][9][2];` il compilatore riserva spazio contiguo per $7 \cdot 9 \cdot 2 = 126$ interi, l'indirizzo base è `&A[0][0][0]`, e

$$\texttt{A[i][j][k]} \rightarrow \texttt{*(\&A[0][0][0] +} \; 9 \cdot 2 \cdot i + 2 \cdot j + k \texttt{)}$$

Anche qui la prima dimensione, 7, non serve.

L'inizializzazione

Un array a più dimensioni si può inizializzare in molti modi. Queste inizializzazioni sono equivalenti:

```c
int A[2][3] = {1, 2, 3, 4, 5, 6};
int A[2][3] = {{1, 2, 3}, {4, 5, 6}};
int A[][3] = {{1, 2, 3}, {4, 5, 6}};
int A[][3] = {1, 2, 3, 4, 5, 6};
```

- Senza graffe interne, gli elementi vengono riempiti per riga: `A[0][0]`, `A[0][1]`, `A[0][2]`, `A[1][0]`, … (`gcc -Wall` suggerisce comunque le graffe: *missing braces around initializer*).
- Se i valori sono meno degli elementi, i rimanenti valgono 0: `{{1}, {4, 5}}` dà le righe `1 0 0` e `4 5 0`.
- Se la prima dimensione è vuota, il compilatore la ricava dal numero di graffe interne (o dei valori); tutte le altre dimensioni vanno indicate.
- `int A[2][2][3] = {0};` azzera tutto l'array; qui vanno dichiarate tutte le dimensioni. È il modo più semplice di inizializzare un array automatico, che altrimenti conterrebbe valori a caso.

Anche queste due inizializzazioni di un array a tre dimensioni sono equivalenti (verificato stampando gli elementi: `1 1 0 2 0 0 3 0 0 4 4 0`):

```c
int A[2][2][3] = {
    {{1, 1, 0}, {2, 0, 0}},
    {{3, 0, 0}, {4, 4, 0}}
};
int A[][2][3] = {{{1, 1}, {2}}, {{3}, {4, 4}}};
```

Le matrici come parametri

Una matrice si può passare a una funzione, ma per generare il codice il compilatore ha bisogno della storage mapping function: nella dichiarazione del parametro vanno quindi indicate **tutte le dimensioni tranne la prima**. Queste tre intestazioni sono equivalenti:

```c
int sum(int a[3][5])
int sum(int a[][5])
int sum(int (*a)[5])        /* a punta ad array di 5 int */
```

ma, se il corpo usa `i < 3` e `j < 5`, vanno bene solo per le matrici 3×5. Passando il numero di righe come parametro, la funzione va bene per tutte le matrici $m \times 5$:

sum2d.c

```c
int sum(int a[][5], int m)
{
   int   i, j, sum = 0;

   for (i = 0; i < m; ++i)
      for (j = 0; j < 5; ++j)
         sum += a[i][j];
   return sum;
}
```

Il numero di colonne, però, resta fisso. Come scrivere una funzione per matrici di qualunque dimensione? La prima idea, `int **a`, è sbagliata, come mostra la prossima sezione.

`int **` non è una matrice

Si potrebbe pensare di generalizzare quanto succede per gli array a una dimensione, e dichiarare il parametro come puntatore a puntatori:

sumpp.c

```c
#include <stdio.h>

int sum(int **a, int m, int n)
{
   int   i, j, sum = 0;

   for (i = 0; i < m; ++i)
      for (j = 0; j < n; ++j)
         sum += a[i][j];
   return sum;
}

int main()
{
   int A[3][5] = {{1, 2, 3, 4, 5}}, s;

   s = sum(A, 3, 5);
   printf("%d\n", s);
   return 0;
}
```

Terminale

```
$ ./sumpp
Segmentation fault
```

Terminale

```
$ gcc -Wall -o sumpp sumpp.c
sumpp.c: In function ‘main’:
sumpp.c:17:12: warning: passing argument 1 of ‘sum’ from incompatible pointer type [-Wincompatible-pointer-
    types]
   17 |    s = sum(A, 3, 5);
      |            ^
      |            |
      |            int (*)[5]
sumpp.c:3:15: note: expected ‘int **’ but argument is of type ‘int (*)[5]’
```

Il compilatore avvisa, e il programma si blocca. Il motivo è che `A`, dichiarato `int A[3][5]`, è un puntatore ad **array** di 5 interi (`int (*)[5]`), mentre un `int **` è un puntatore a **puntatori**. Dentro `sum`, `a[i]` viene letto come un indirizzo: ma nella memoria di `A` non ci sono indirizzi, ci sono gli interi 1, 2, 3…, e usarli come indirizzi porta in zone di memoria proibite (figura 12.8). Con un `int **`, invece, `a[i]` è un puntatore che si può anche assegnare (`a[i] = …;`): le due cose non sono equivalenti.

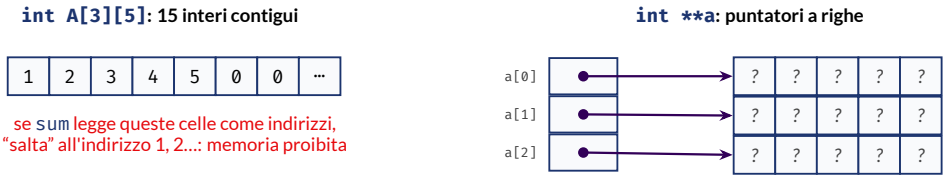


Figura 12.8: *Un array a due dimensioni non contiene puntatori; un* `int **` *sì.*

Passare l'indirizzo base

Un modo per scrivere una funzione valida per qualunque dimensione è passarle l'**indirizzo base** e le dimensioni, e applicare a mano la storage mapping function:

```c
int sum_base(int *a, int m, int n)
{
   int   i, j, sum = 0;

   for (i = 0; i < m; ++i)
      for (j = 0; j < n; ++j)
         sum += *(a + n*i + j);
   return sum;
}
```

Si chiama con `sum_base(&A[0][0], 3, 5)`. Con una matrice `A[3][5]` tutta di 1, sia `sum(A, 3)` sia `sum_base(&A[0][0], 3, 5)` restituiscono 15 (verificato). Funziona, ma si perde la comoda scrittura `a[i][j]`.

LE MATRICI DINAMICHE



Un array di righe

Gli array a più dimensioni, insomma, non sono molto flessibili: non si possono passare alle funzioni senza perdere in generalità (dimensioni fisse) o in semplicità (indirizzo base). Sarebbe più comodo poter scrivere funzioni come `int f(int **a, int m, int n);`, a cui si passa la matrice e le sue dimensioni. Si può, allocando la matrice dinamicamente come un **array di puntatori alle righe**:

matrici.c

```c
double **matrix_alloc(int m, int n)
{
   int i;
   double **p;

   p = malloc(m * sizeof(double *));
   for (i = 0; i < m; i++)
      p[i] = malloc(n * sizeof(double));
   return p;
}

void free_matrix(double **a, int m)
{
   int i;

   for (i = 0; i < m; i++)
      free(a[i]);
   free(a);
}
```

- Prima si alloca l'array di `m` puntatori (ognuno occupa `sizeof(double *)`), poi ogni riga, di `n double`.
- All'elemento si accede con la comoda scrittura `a[i][j]`: `a[i]` è il puntatore alla riga i, e `[j]` sceglie l'elemento nella riga.
- La memoria della matrice **non è contigua**: ogni riga sta dove `malloc` l'ha messa.
- Si libera con una `free` per ogni riga, e infine quella dell'array dei puntatori.

Memoria contigua

Una variante alloca tutti gli elementi in un'unica zona contigua, e fa puntare le righe nei punti giusti: `a[0]` all'inizio, e ogni riga n elementi dopo la precedente (figura 12.9).

```c
double **matrix_alloc2(int m, int n)
{
   int i;
   double **a;

   a = malloc(m * sizeof(double *));
   a[0] = malloc(m * n * sizeof(double));
   for (i = 1; i < m; i++)
      a[i] = a[i-1] + n;
   return a;
}

void free_matrix2(double **a)
{
   free(a[0]);
   free(a);
}
```

Terminale

```
$ ./matrici
a[2][3] = 23   b[2][3] = 23
b: riga 1 inizia 4 double dopo la riga 0
```

a è allocata riga per riga, b in modo contiguo; entrambe sono 3×4, con `[i][j]` uguale a $10i + j$. Compilato con `-fsanitize=address`, il programma non dà errori.

Nelle slide la seconda funzione di liberazione è dichiarata `double free_matrix(double **a)`: non restituisce nulla, quindi il tipo giusto è `void` (con `double`, `gcc -Wall` avvisa *control reaches end of non-void function*).

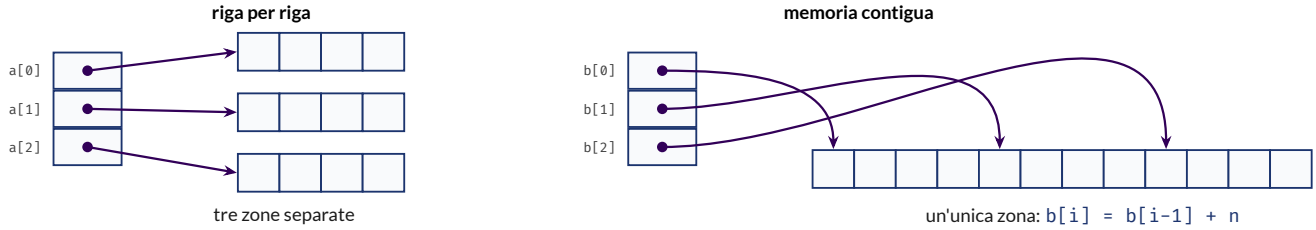


Figura 12.9: *Due modi di allocare una matrice* 3×4.

GLI ARRAY A LUNGHEZZA VARIABILE

Le dimensioni decise durante l'esecuzione

Il C99 ha introdotto gli **array a lunghezza variabile** (*variable length array*, VLA): array automatici la cui dimensione è un'espressione calcolata durante l'esecuzione, per esempio un valore letto da tastiera. Ecco una versione ridotta dell'esempio delle slide (che a sua volta viene dal libro di Deitel):

vla.c

```c
#include <stdio.h>

void print2DArray(int row, int col, int array[row][col]);

int main(void)
{
   int row, col;

   printf("Righe e colonne: ");
   scanf("%d %d", &row, &col);

   int m[row][col];                /* un VLA */

   for (int i = 0; i < row; ++i)
      for (int j = 0; j < col; ++j)
         m[i][j] = i + j;
   printf("sizeof(m) = %zu byte\n", sizeof(m));
   print2DArray(row, col, m);
   return 0;
}

void print2DArray(int row, int col, int array[row][col])
{
   for (int i = 0; i < row; ++i) {
      for (int j = 0; j < col; ++j)
         printf("%5d", array[i][j]);
      puts("");
   }
}
```

Terminale

```
$ ./vla
Righe e colonne: 2 5
sizeof(m) = 40 byte
    0    1    2    3    4
    1    2    3    4    5
$ ./vla
Righe e colonne: 4 3
sizeof(m) = 48 byte
    0    1    2
    1    2    3
    2    3    4
    3    4    5
```

- La dichiarazione `int m[row][col];` viene dopo la lettura delle dimensioni: in C99 le dichiarazioni si possono mettere in mezzo alle istruzioni, e così le variabili dei `for` (`for (int i = 0; …)`).
- `sizeof(m)` viene calcolato durante l'esecuzione: $2 \cdot 5 \cdot 4 = 40$ byte la prima volta, $4 \cdot 3 \cdot 4 = 48$ la seconda.
- Nel parametro `int array[row][col]` le dimensioni sono altri parametri, dichiarati prima: la funzione va bene per matrici di qualunque dimensione.

Pro e contro

I VLA sono comodi, ma hanno un difetto: sono allocati nello **stack** (capitolo 11), che è piccolo rispetto allo heap, e la loro dimensione non è nota in compilazione. Con dimensioni grandi c'è il rischio concreto di esaurire lo stack: con `2000 2000` la matrice occupa 16 MB, il doppio degli 8 MB che Linux assegna di solito allo stack, e il programma viene fermato con *Segmentation fault*. I VLA sono quindi utilizzabili per piccole dimensioni; per le grandi si usa `malloc`. È anche il parere di Linus Torvalds, il creatore di Linux, riportato nel file `input.txt` dell'ordinamento delle parole: i VLA *bring nothing of positive value*, meglio un array fisso sufficientemente grande o l'allocazione nello heap.

In sintesi

Una **stringa** è un array di caratteri terminato da `'\0'`, che va contato nella dimensione. Una stringa costante è un puntatore a memoria di sola lettura; `%s` stampa fino al terminatore. `const` impedisce di modificare una variabile o il dato puntato, e il compilatore lo verifica. `ctype.h` classifica e trasforma i caratteri, `string.h` copia, concatena, confronta e cerca nelle stringhe, senza controllare lo spazio: per una copia servono `strlen + 1` byte. Un **array di puntatori** raccoglie stringhe di lunghezza diversa, e si ordina scambiando i puntatori. Gli array a **più dimensioni** stanno in memoria riga dopo riga (storage mapping function `&a[0][0] + N*i + j`); come parametri vanno dichiarate tutte le dimensioni tranne la prima, e un `int **` non è una matrice. Le matrici dinamiche sono array di puntatori alle righe; i VLA hanno dimensioni decise in esecuzione, ma stanno nello stack.

RIFERIMENTI



- **A. Kelley, I. Pohl**, *A Book on C*: capitolo 6 (stringhe, array di puntatori, array a più dimensioni).
- **H. M. Deitel, P. J. Deitel**, *C: How to Program*: array a lunghezza variabile.
- **Null-terminated byte strings** — cppreference.com.
  `https://en.cppreference.com/w/c/string/byte`